

f1-data-platform

Informe de situación del proyecto

Datos, arquitectura, código y visualizaciones · antes de la fase 7

Autor del proyecto	Antonio Sánchez de la Blanca Romero
Origen	TFG «Formula 1 Dashboard» (UCLM, 2024)
Fecha	29–30 de septiembre de 2026
Versión de datos	F1DB v2026.15.1 · FastF1 2018–2026 (ronda 15)
Repositorio	github.com/AntonioSchez32/f1-data-platform
Web	f1-data-platform.vercel.app
API	f1-data-api-h7c7.onrender.com/docs

Documento elaborado a partir de nueve investigaciones independientes (datos que faltan, fuentes externas, carreras al sprint, arquitectura de datos, arquitectura de aplicación, alternativas de arquitectura, revisión de código, gráficos de otras webs y evaluación de nuestros gráficos). Las cifras proceden de consultas de solo lectura a `data/gold/f1.duckdb`, a la capa bronze y al código del repositorio en el commit `cceddd3`. No se ha modificado código del proyecto.

Índice general

Cómo leer este informe	1
1. Resumen ejecutivo	2
1.1. El proyecto en una página	2
1.2. Doce hallazgos que importan	2
1.3. Decisiones que conviene tomar	3
1.4. Hoja de ruta consolidada	4
 I Datos: qué falta, dónde conseguirlo y sprints	 5
Síntesis de la parte I	6
 2. Inventario de los datos que faltan	 8
2.1. Resumen ejecutivo	8
2.2. Metodología	9
2.2.1. Fuentes y qué cubre cada una (según ingestion/ y transform/)	9
2.2.2. Consultas principales	9
2.3. Tablas de cobertura	10
2.3.1. Matriz de cobertura por tipo de dato	10
2.3.2. Vueltas oficiales cubiertas por década	12
2.3.3. Rellenado de campos de fact_laptimes (% de vueltas con valor)	12
2.3.4. Tiempo por vuelta: carreras sin tiempos o con tiempos parciales desde 1990 . . .	13
2.3.5. Estado de validación de los tiempos (validation_status)	13
2.3.6. Paradas y entradas al pit lane por temporada	14
2.3.7. Clasificación	14
2.3.8. Carreras al sprint	14
2.3.9. Telemetría (fact_quali_telemetry, 518 699 muestras cada 10 m)	15
2.3.10. Banderas rojas (vueltas con is_red_flag)	15
2.3.11. Otras sesiones y datos de F1DB que no se exponen	16
2.4. Lista priorizada de huecos	16
2.4.1. Prioridad alta	16
2.4.2. Prioridad media	17
2.4.3. Prioridad baja	19
2.5. Incoherencias abiertas conocidas	19
2.6. Carreras concretas con huecos de vueltas (missing_laps, no_laps, extra_laps)	20
2.7. Resumen del impacto en la web (web/src/app/[lang]/races/[id]/...)	21
 3. Fuentes externas para cubrir y contrastar huecos	 23
3.1. Resumen ejecutivo	23
3.2. Punto de partida: qué tenemos (bronze medido el 29-09-2026)	23
3.3. Pruebas reales realizadas (29-09-2026)	24

3.4.	Fichas por fuente	25
3.4.1.	APIs y bases abiertas (candidatas a ingesta)	25
3.4.2.	Datasets derivados (GitHub, Kaggle, Hugging Face)	27
3.4.3.	Fuentes oficiales (evidencia, no redistribución)	28
3.4.4.	Sitios estadísticos (consulta manual; no scraping)	29
3.4.5.	Complementarias	30
3.5.	Matriz hueco × fuente	30
3.6.	Propuesta priorizada	32
3.6.1.	Correspondencia con los huecos de la nota 01	32
3.7.	Notas de cumplimiento	33
4.	Vueltas de las carreras al sprint	34
4.1.	Conclusiones	34
4.2.	Situación actual (revisión de código)	34
4.3.	F1DB (release v2026.15.1) — medido en bronze	35
4.4.	FastF1 3.8.3 — prueba real (caché propia en el scratchpad)	35
4.4.1.	Compleitud FastF1 (29 sprints disputados, 2021-R10 → 2026-R12; medido)	35
4.4.2.	Vuelta 1 del sprint: reconstruible con exactitud	36
4.5.	OpenF1 (api.openf1.org, sin clave) — prueba real	37
4.6.	Jolpica-F1 / Ergast	37
4.7.	FIA, formula1.com y formula1db.com	37
4.8.	Clasificación sprint (Shootout 2023 / Sprint Qualifying 2024+)	38
4.9.	Catálogo de sprints y cobertura esperada por fuente	38
4.10.	Diseño propuesto	39
4.10.1.	Ingesta	39
4.10.2.	dbt	39
4.10.3.	API	40
4.10.4.	Web	40
4.10.5.	Esfuerzo estimado	40
4.11.	Riesgos	40
4.12.	Prototipo (scratchpad .../730e2b1e-.../scratchpad/sprint/)	41
II	Arquitectura y estructura del proyecto	42
	Síntesis de la parte II	43
5.	Arquitectura de aplicación y estructura del repositorio	45
5.1.	Estructura del repositorio	45
5.1.1.	Visión general	45
5.1.2.	Árbol comentado	46
5.1.3.	pyproject.toml y uv.lock	46
5.1.4.	Ficheros de configuración de la raíz	47
5.1.5.	scripts/ y tests/	47
5.2.	API (api/app/)	48
5.2.1.	Configuración (config.py)	48
5.2.2.	Acceso a datos (database.py)	48
5.2.3.	Descarga de datos (release.py)	49
5.2.4.	Ciclo de vida y refresco (main.py)	49
5.2.5.	Caché HTTP (cache.py)	50
5.2.6.	Dependencias y esquemas	50
5.2.7.	Catálogo de <i>endpoints</i>	50
5.2.8.	Imagen Docker (api/Dockerfile)	53

5.2.9.	Pruebas de la API	53
5.3.	Web (web/)	54
5.3.1.	Pila y convenciones de Next.js 16	54
5.3.2.	Cliente API tipado (src/lib/api/)	54
5.3.3.	Mapa de rutas	55
5.3.4.	Componentes (src/components/)	56
5.3.5.	Estrategia de caché y renderizado	57
5.3.6.	Estilos	57
5.3.7.	Accesibilidad	57
5.3.8.	Pruebas de la web	58
5.4.	CI/CD	58
5.4.1.	ci.yml (push a main y <i>pull requests</i>)	58
5.4.2.	pipeline.yml (datos)	59
5.4.3.	Despliegue continuo	59
5.4.4.	Flujo de refresco de datos extremo a extremo	59
5.5.	Diagramas	60
5.5.1.	Arquitectura de despliegue	60
5.5.2.	Secuencia de una petición (/es/races/1102/pace)	60
5.6.	Medidas reales	60
5.6.1.	Tamaño del código y superficie	61
5.6.2.	Tiempos de respuesta de la API desplegada	61
5.6.3.	Tiempos de la web desplegada	61
5.7.	Debilidades y propuestas de mejora	62
5.7.1.	API	62
5.7.2.	Web	63
5.7.3.	CI/CD	64
5.7.4.	Infraestructura	64
5.8.	Resumen	65
6.	Arquitecturas y estructuras alternativas	66
6.1.	Resumen ejecutivo	66
6.2.	Punto de partida medido en el repositorio (28/09/2026)	67
6.3.	Estructura del repositorio	67
6.3.1.	Monorepo frente a multirrepo	67
6.3.2.	Layout de Python y uv	68
6.3.3.	Estructura del proyecto dbt frente a la guía de dbt Labs	68
6.3.4.	Estructura del proyecto Next.js	69
6.3.5.	Herramientas de monorepo y ejecutores de tareas	70
6.4.	Arquitectura de datos	70
6.4.1.	Medallion frente a otras organizaciones y modelos	70
6.4.2.	Formatos de almacenamiento y de tabla	71
6.4.3.	Dónde publicar los artefactos de datos	71
6.4.4.	Versionado de datos	72
6.5.	Orquestación y observabilidad	73
6.5.1.	GitHub Actions frente a orquestadores	73
6.5.2.	Disparar el pipeline según el calendario de F1	74
6.5.3.	Observabilidad	74
6.6.	Serving: cómo llegan los datos a la web	74
6.6.1.	Situación y problema	75
6.6.2.	Alojar la misma API en otro sitio	75
6.6.3.	Web estática o «API estática» (sin servidor en el camino crítico)	75
6.6.4.	DuckDB-WASM en el navegador	76

6.6.5.	BI-as-code como alternativa o complemento a Next.js	76
6.7.	Frontend	77
6.7.1.	Librería de gráficos	77
6.7.2.	Internacionalización	78
6.8.	Proyectos de referencia	78
6.9.	Arquitecturas objetivo	79
6.9.1.	A · Evolución mínima (endurecer lo que hay)	79
6.9.2.	B · API estática + web con ISR (recomendada)	79
6.9.3.	C · Orquestador + almacenamiento de objetos (lakehouse ligero)	79
6.9.4.	Comparación	79
6.9.5.	Hoja de ruta sugerida	80
6.10.	Fuentes principales (agrupadas)	80
III	Arquitectura de datos y código	82
	Síntesis de la parte III	83
7.	Arquitectura de datos: capas, modelo y linaje	85
7.1.	Visión general	85
7.1.1.	Inventario del proyecto dbt (según transform/target/manifest.json)	85
7.2.	Datos en bruto y capa BRONZE (ingestion/)	86
7.2.1.	config.py — rutas y constantes	86
7.2.2.	io.py — utilidades de escritura	87
7.2.3.	f1db_loader.py — F1DB (fuente principal de resultados)	87
7.2.4.	fastf1_loader.py — FastF1 (cronometraje 2018+ y telemetría)	88
7.2.5.	legacy_loader.py — formula1db.com (scraping del TFG, carga única)	89
7.2.6.	ergast_loader.py — Ergast (volcado de octubre de 2022, solo validación)	90
7.2.7.	snapshot.py — empaquetado y restauración	90
7.2.8.	cli.py — la CLI f1-ingest	91
7.2.9.	Resumen de fuentes	91
7.3.	Proyecto dbt (transform/) y capa SILVER	91
7.3.1.	Configuración	91
7.3.2.	Sources (models/staging/f1db/_f1db__sources.yml)	92
7.3.3.	Macros (transform/macros/)	92
7.3.4.	Seeds (transform/seeds/, esquema silver)	93
7.3.5.	Staging (models/staging/<fuente>/stg_<fuente>__<entidad>.sql, vistas en silver)	93
7.3.6.	Intermediate (models/intermediate/int_*.sql, tablas en silver)	95
7.4.	Capa GOLD: modelo dimensional	98
7.4.1.	Enfoque	98
7.4.2.	Inventario de tablas gold (DuckDB, 28/09/2026)	99
7.4.3.	Dimensiones	99
7.4.4.	Hechos	100
7.4.5.	Métricas (marts/metrics)	101
7.4.6.	Diagramas del modelo (para TikZ)	101
7.4.7.	Comparación con el modelo del TFG (Fig. 5.7, secciones 5.1–5.3)	102
7.5.	Capa de CALIDAD	103
7.5.1.	Modelos qa_* (esquema quality, tablas)	103
7.5.2.	qa_summary (40 comprobaciones)	104
7.5.3.	Tests de dbt	105
7.6.	Linaje completo y pipeline	106

7.6.1.	Linaje de extremo a extremo	106
7.6.2.	Pipeline <code>.github/workflows/pipeline.yml</code> paso a paso	107
7.7.	Diccionario de datos de la capa gold (resumen)	108
7.7.1.	Dimensiones	108
7.7.2.	Hechos	109
7.7.3.	Agregados	110
7.8.	Propuestas de mejora	110
7.8.1.	Higiene y robustez inmediata (esfuerzo bajo)	110
7.8.2.	Gobierno del modelo (esfuerzo medio)	111
7.8.3.	Evolución del modelo dimensional (esfuerzo medio-alto)	112
7.8.4.	Rendimiento y operación	112
7.8.5.	Resumen de prioridades	113
8.	Revisión de código e infraestructura	114
8.1.	Método y estado de partida	114
8.2.	Hallazgos por área	114
8.2.1.	Ingesta (<code>ingestion/</code>)	114
8.2.2.	dbt y modelo de datos (<code>transform/</code>)	115
8.2.3.	API (<code>api/</code>)	117
8.2.4.	Web (<code>web/</code>)	119
8.2.5.	CI/CD e infraestructura	120
8.2.6.	Pruebas	121
8.2.7.	Documentación	121
8.3.	Dependencias	121
8.4.	Mediciones de la API (<code>TestClient</code> , <code>dist/f1.duckdb</code> , mediana de 5)	122
8.5.	Top 15 de mejoras (ordenadas por valor/esfuerzo)	122
IV	Visualizaciones y récords	124
	Síntesis de la parte IV	125
9.	Evaluación de nuestros gráficos y propuestas nuevas	128
9.1.	Resumen	128
9.2.	Método	129
9.3.	Inventario completo	129
9.3.1.	Gráficos y mapas	129
9.3.2.	Tarjetas KPI	131
9.3.3.	Tablas	131
9.4.	Evaluación crítica	131
9.4.1.	Problemas transversales	132
9.4.2.	Evaluación de cada elemento	133
9.4.3.	Repeticiones y solapamientos (qué fusionar o eliminar)	136
9.4.4.	Huecos de relato (preguntas típicas de un aficionado sin respuesta)	136
9.5.	Propuestas nuevas	137
9.5.1.	Qué se puede dibujar y desde cuándo (comprobado en <code>data/gold/f1.duckdb</code>)	137
9.5.2.	Carrera (pestañas de <code>/races/[id]</code>)	138
9.5.3.	Temporada (<code>/seasons/[year]</code>)	141
9.5.4.	Piloto y constructor	142
9.5.5.	Historia (vistas agregadas nuevas)	143
9.5.6.	Entidades nuevas	145
9.5.7.	Prioridad sugerida	146
9.6.	Reorganización de la navegación	146

9.7. Notas para el informe LaTeX	147
10. Gráficos y récords de otras webs y proyectos	149
10.1. Metodología y limitaciones	149
10.2. Qué datos tenemos (gold)	149
10.3. Catálogo por fuente	150
10.3.1. formula1db.com (solo estructura indexada)	150
10.3.2. statsf1.com	151
10.3.3. FIA: documentos de cronometraje (PDF)	151
10.3.4. formula1.com y F1 Insights powered by AWS	152
10.3.5. F1 TV / Live Timing, MultiViewer y apps de cronometraje (Undercut, F1Dash, Formula Live Pulse, formula1dashboard)	152
10.3.6. f1-tempo.com y GP Tempo	153
10.3.7. TracingInsights (web, GitHub, redes y Substack)	153
10.3.8. Galería de ejemplos de FastF1	153
10.3.9. OpenF1	154
10.3.10. formula1points.com	154
10.3.11. RaceFans (datos interactivos de cada GP)	154
10.3.12. The Race / F1MATHS / laptime.me (análisis de ritmo)	154
10.3.13. Autosport Forix / Motorsport Stats	155
10.3.14. fermanpre/f1-stats (estilo Atlas F1) y otros «gap charts»	155
10.3.15. ChicaneF1 y grandprix.com (enciclopedias)	155
10.3.16. Wikipedia	155
10.3.17. Kaggle, Tableau Public, comunidad de Power BI, Observable	155
10.3.18. Redes (F1Visualized, F1_charts, r/formula1, r/F1Technical)	156
10.3.19. Modelos de valoración de pilotos (f1metrics, F1 Mathematical Model, F1-analysis)	156
10.3.20. Proyectos de GitHub parecidos	156
10.3.21. Prensa (FT, NYT, Guardian, Datawrapper)	156
10.4. Catálogo de récords y estadísticas curiosas	156
10.5. Lista consolidada y deduplicada de ideas (ordenada por valor/esfuerzo)	158
10.6. Fuentes consultadas	160
11. Hoja de ruta y preparación de la fase 7	162
11.1. Iteraciones propuestas	162
11.2. Qué necesita la fase 7 de los datos	162
11.3. Pendientes que siguen abiertos	163
A. Metodología del informe	164

Cómo leer este informe

El informe responde a los cuatro encargos de la noche del 29 de septiembre y está pensado para leerse en dos niveles:

- **Nivel de decisión (unas 20 páginas).** El capítulo 1 resume el estado del proyecto, los hallazgos más importantes, las decisiones que conviene tomar y una hoja de ruta consolidada. Cada parte empieza con una síntesis de dos o tres páginas con diagramas propios.
- **Nivel de detalle (el resto).** Tras cada síntesis van los capítulos de detalle, uno por investigación, con las consultas SQL, las tablas de cobertura, las fichas de cada fuente, el recorrido por el código y las URL consultadas. Son la base documental de las síntesis.

Encargo	Dónde está
1. Datos que faltan, cómo obtenerlos y contrastarlos; vueltas de sprint	Parte I: capítulos 2, 3 y 4
2. Arquitectura y estructura del proyecto; alternativas	Parte II: capítulos 5 y 6
3. Arquitectura de datos, qué hace cada parte del código, mejoras	Parte III: capítulos 7 y 8
4. Gráficos y récords: evaluación de los nuestros e ideas nuevas	Parte IV: capítulos 9 y 10
Hoja de ruta y preparación de la fase 7	Capítulo 11

Convenciones: ✓ disponible o correcto, ● parcial, ✗ no disponible, ○ existe en la fuente pero no se expone. Esfuerzo: S horas, M uno o dos días, L más de tres días. Los identificadores H1–H7, M1–M11 y B1–B8 son los huecos de datos del capítulo 2; P1–P26 son las propuestas de gráficos del capítulo 9.

CAPÍTULO 1

Resumen ejecutivo

1.1 El proyecto en una página

La plataforma está **completa, desplegada y en verde**: el TFG de Power BI se ha convertido en un sistema de datos automatizado que ingiere cuatro fuentes, las concilia con dbt en capas bronze/silver/gold, publica cada semana una base DuckDB en una release de GitHub, la sirve con una API FastAPI y la muestra en una web Next.js accesible y bilingüe que replica las once páginas del informe original y añade telemetría, constructores y calidad de datos.

Componente	Qué hay	Estado
Ingesta (bronze)	F1DB v2026.15.1 (31 tablas), FastF1 (187 carreras 2018–2026 y telemetría de 63), Ergast 2022 (535 748 vueltas), CSV de formula1db.com del TFG (1 252 550 vueltas); CLI <code>f1-ingest</code> idempotente	✓ funciona; solo sesiones R y Q de FastF1
Transformación (dbt)	54 modelos (20 staging, 10 intermediate, 17 marts, 7 de calidad), 3 seeds, 3 macros, 50 tests; <code>dbt build</code> en 22 s	✓ todo pasa; 40 comprobaciones QA: 23 PASS, 17 INFO, 0 FAIL
Modelo gold	5 dimensiones, 8 hechos, 4 agregados (TFG: 5 dimensiones y 6 hechos)	✓
API	FastAPI, 28 endpoints GET, DuckDB de solo lectura con cambio de versión en caliente, ETag por versión de datos; 30 pruebas	✓ < 60 ms por endpoint; ● arranque en frío de Render
Web	Next.js 16, 16 páginas × 2 idiomas, 17 gráficos, 23 componentes; Playwright + axe (WCAG 2.2 AA), 38 pruebas	✓ todas pasan también en producción
CI/CD	<code>ci.yml</code> (Python, dbt, imagen Docker, web) y <code>pipeline.yml</code> (lunes 06:00 UTC)	✓ verde; ● acciones en Node 20
Despliegue	Render (API, Frankfurt, free) + Vercel (web) + GitHub Releases (datos); coste 0 €	✓; ● Vercel ejecuta en Washington

1.2 Doce hallazgos que importan

1. **Lo esencial está completo de 1950 a 2026**: resultados, parrillas, clasificación y campeonatos de las 1 164 carreras. Los huecos están *dentro* de cada carrera y crecen hacia atrás en el tiempo: hay posiciones vuelta a vuelta en todas, pero tiempos por vuelta solo desde 1990 (cap. 2).
2. **Las vueltas de sprint se pueden recuperar ya**. FastF1 tiene las 11 319 vueltas oficiales de los 29 sprints (2021–2026) sin huecos; la vuelta 1, que FastF1 no cronometra, se reconstruye exactamente con el tiempo total de F1DB. Versión mínima: unos 4 días (cap. 4).
3. **Hay un error visible en los sprints**: `dim_race.has_sprint` es falso en los 12 sprints de 2021–2023, y la web los oculta aunque sus resultados están cargados. Se arregla en minutos derivándolo de `SPRINT_RACE_RESULT`.
4. **Las banderas rojas caen una vuelta antes**. En 22 carreras la vuelta que contiene la suspensión (20–130 minutos) no está marcada; es el caso de São Paulo 2024 que obligó a filtrar vueltas lentas

en la web. No hay ninguna bandera roja marcada en 2025.

5. **2025–2026 dependen de una sola fuente** (FastF1): 734 vueltas sin tiempo (570 recuperables sumando sectores), 436 con compuesto NAN/NONE y ningún compuesto Pirelli real (C1–C6) desde 2024. **Jolpica**, sucesor de Ergast y con el mismo esquema que ya cargamos, daría la segunda fuente con 1–2 días de trabajo (cap. 3).
6. **Tenemos más datos de los que mostramos**: F1DB trae libres desde 1986, warm-up, preclasificación, las clasificaciones de 1980–2005, la vuelta rápida de cada piloto y el piloto del día; FastF1 ofrece meteorología y mensajes de dirección de carrera con solo activar dos parámetros.
7. **Antes de 1996 no existe ninguna fuente abierta de tiempos por vuelta** con la que contrastar el CSV de formula1db.com. Queda como limitación documentada; la única vía de validación interna es `f1resultsdatabase` (con riesgo de licencia).
8. **La arquitectura es adecuada y no hay que reescribirla**. Las guías de referencia (dbt Labs, uv, Next.js) respaldan las decisiones tomadas. El punto débil es el *servicing*: una API que se duerme a los 15 minutos para servir datos que solo cambian los lunes (cap. 6).
9. **La web no aprovecha ninguna caché de página**: todas se generan en cada visita, las funciones de Vercel se ejecutan en Washington mientras la API está en Frankfurt, y la portada tarda 1,3–1,5 s en empezar a responder (cap. 5).
10. **Hay dos errores de datos que se ven en la web**: 85 vueltas duplicadas de coches compartidos en los años 50–60 (la pestaña de neumáticos las superpone) y abreviaturas de piloto que chocan en la misma carrera (Michael y Ralf Schumacher, ambos «SCH») (cap. 8).
11. **La operación es frágil en tres puntos**: la `release data-latest` se sobrescribe cada semana y es la única copia en la nube del bronze de formula1db.com y Ergast; no hay prueba de humo antes de publicar; y el repositorio no tiene `LICENSE` ni atribuciones, que F1DB exige.
12. **Las visualizaciones replican bien el TFG, pero se solapan y dejan preguntas sin responder**. Faltan el *race trace* (diferencia con el líder vuelta a vuelta), la página de circuito, un comparador libre de pilotos y el momento en que se decidió el título. Hay tres gráficos que dicen lo mismo sobre los compañeros, y varios errores de lectura concretos (cap. 9).

1.3 Decisiones que conviene tomar

Decisión	Opciones	Recomendación
Licencia de los datos y del código	(a) MIT/Apache para el código y CC BY 4.0 para los datos derivados de F1DB; (b) además integrar Jolpica/OpenF1, que obliga a publicar lo derivado como CC BY-NC-SA	(a) ya. Integrar Jolpica y OpenF1 solo si se acepta la cláusula no comercial para los datos (el proyecto no es comercial, así que es asumible)
Vueltas de sprint	(a) versión mínima solo FastF1 (~4 días); (b) completa con validación Jolpica/OpenF1 y selector en la web (6–8,5 días); (c) aplazar	(b), empezando por (a). Antes, el arreglo de <code>has_sprint</code> (minutos)
Segunda fuente para 2023–2026	Jolpica (volcado CSV semanal) u OpenF1 (API, 30 peticiones/min)	Jolpica: mismo esquema que Ergast, 1–2 días
Cómo servir los datos a la web	A: endurecer lo actual; B: «API estática» (JSON precalculados en GitHub Pages, la API queda como servicio público); C: Dagster + R2 + DuckLake	A ahora (2–3 días) y B después (1–2 semanas); C como línea futura

Decisión	Opciones	Recomendación
Región de Vercel	Mantener Washington (<code>iad1</code>) o fijar Frankfurt (<code>fra1</code>)	<code>fra1</code> : una línea en <code>vercel.json</code>
Navegación de la web	Mantener 7 pestañas de carrera o pasar a 5 (Resultado, Clasificación, Carrera, Ritmo, Estrategia) y añadir Circuitos, Comparar e Historia	Pasar a 5 pestañas; añadir Circuitos y Comparar por fases
Casos de numeración aplazados (1994, 2001)	Resolverlos ahora o mantenerlos como pendientes documentados	Mantenerlos aplazados; no bloquean nada
Orden relativo con la fase 7	Empezar la analítica ya o preparar antes los datos que necesita	Preparar antes meteorología, compuesto Cx y vueltas limpias (1–2 semanas), porque la degradación los necesita

1.4 Hoja de ruta consolidada

La tabla combina las propuestas de las nueve investigaciones, ordenadas por valor y esfuerzo. El capítulo 11 la desarrolla por iteraciones.

Iter.	Trabajo	Esfuerzo	Origen
1	Arreglos rápidos de datos: <code>has_sprint</code> , bandera roja en la vuelta correcta, tiempo desde sectores en 2025–26, <code>NAN/NONE</code> como nulo, tipos <code>HUGEINT</code> , tabla huérfana	1 día	cap. 2, 7
1	Higiene: <code>LICENSE</code> y atribuciones, Dependabot, acciones fijadas a SHA y fuera de Node 20, cabeceras de seguridad, <code>fetch</code> con timeout, <code>ruff</code> alineado, región <code>fra1</code>	1–2 días	cap. 8, 5
1	Robustez de datos: releases fechadas e inmutables, copia del <code>bronze</code> estático, prueba de humo antes de publicar, arranque de la API con la última copia	1–2 días	cap. 8, 6
2	Correcciones visibles: coches compartidos (<code>driver_number</code> en la API), abreviaturas únicas, errores de lectura de los gráficos (referencia de clasificación, ejes de temporadas, Ferrari 1950–57, etiquetas solapadas, formato de miles)	2–3 días	cap. 8, 9
2	Pruebas: unit tests de dbt para <code>int_laptimes</code> e <code>int_lap_numbering</code> ; tests de clave en paradas, standings, agregados y telemetría	2 días	cap. 7, 8
3	Datos nuevos: FastF1 ampliado (sprint, meteo, dirección de carrera, libres), Jolpica como segunda fuente, eventos históricos SC/VSC/rojas como seed	4–6 días	cap. 3, 4
3	Vueltas de sprint extremo a extremo (dbt, API <code>?session=sprint</code> , selector en la web)	3–4 días	cap. 4
4	Gráficos de mayor valor: race trace, franjas SC/VSC, matriz de resultados de temporada, récords ampliados, eliminación matemática, pestañas 7 → 5	1–2 semanas	cap. 9, 10
5	Arquitectura B: exportación estática de la API a GitHub Pages y revalidación bajo demanda	1–2 semanas	cap. 6
6	Fase 7: degradación de neumáticos, predicción de resultados, estilos de pilotaje	—	plan original

Parte I

Datos: qué falta, dónde conseguirlo y sprints

Síntesis de la parte I

Qué cubrimos hoy

La figura 1 resume la cobertura temporal de cada tipo de dato en la capa gold. El patrón es claro: lo agregado por carrera está completo desde 1950, y el detalle por vuelta aparece por capas a medida que avanzan las fuentes: posiciones desde 1950 (formula1db.com), tiempos desde 1990, paradas desde 1994, compuesto desde 2011 y todo lo demás desde 2018 con FastF1.

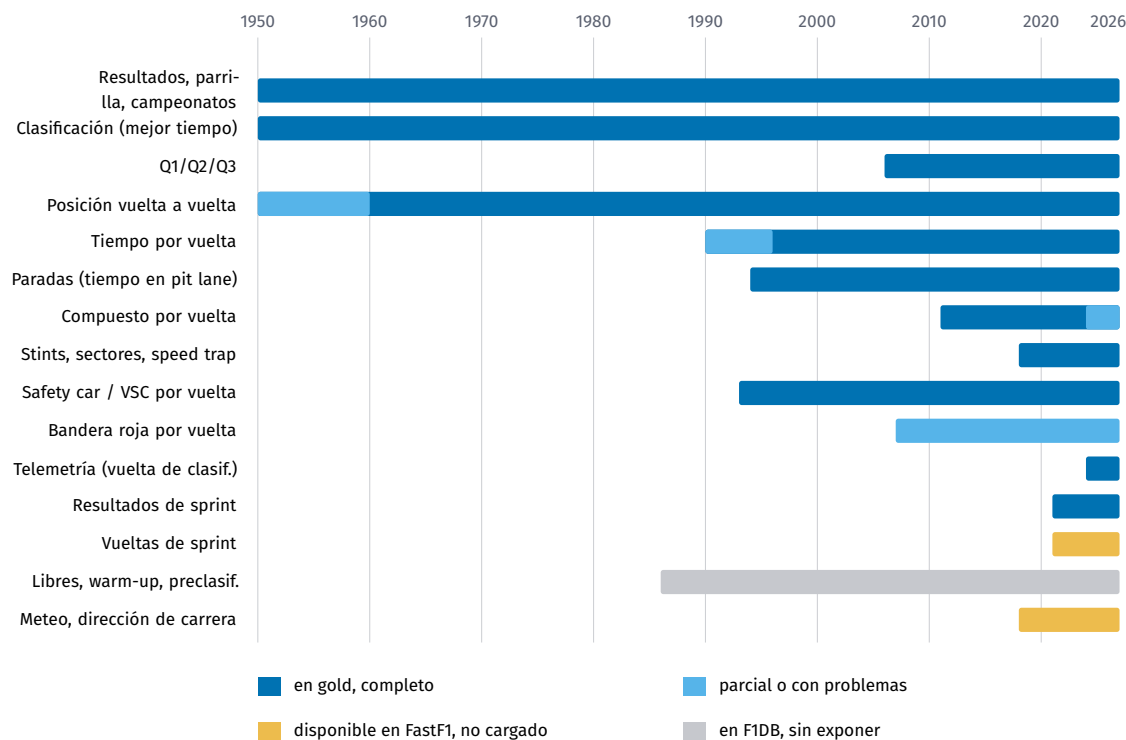


Figura 1: Cobertura temporal de la capa gold por tipo de dato (datos del 28/09/2026). Tramos claros: posiciones de los años 50 con un 43,6 % de «?», tiempos 1990–1995 sin segunda fuente, compuesto sin C1–C6 desde 2024 y banderas rojas mal situadas.

Qué falta, por orden de importancia

Los 26 huecos del inventario se agrupan en tres bloques con tratamiento distinto:

- **Arreglos internos, sin fuentes nuevas** (H2, H3 en parte, H5, M11, B7, B8). La bandera roja se puede recolocar en la vuelta cuyo tiempo contiene la suspensión; los tiempos que faltan en 2025–26 salen sumando sectores; `has_sprint` se deriva del resultado; los speed traps intermedios, el piloto del día y varias dimensiones de F1DB ya están en bronze.
- **Activar lo que ya tenemos a mano** (H1, M1–M4, M6, M8). FastF1 da vueltas de sprint, meteorolo-

gía, mensajes de dirección de carrera, libres y telemetría de 2018–2023 cambiando parámetros del cargador existente. F1DB ya trae libres y las clasificaciones antiguas.

- **Fuentes nuevas** (H3, H4, H6, M5, M7, M9, M10 y los B1–B6 históricos). Aquí entran Jolpica, OpenF1, las nominaciones de Pirelli, los eventos históricos de Wikipedia/TracingInsights, Open-Meteo y, como último recurso para validar, `f1resultsdatabase`.

Dónde conseguirlo y cómo contrastarlo

Fuente	Qué aporta (comprobado con peticiones reales)	Licencia	Esfuerzo
FastF1 ampliado	Sprint 2021+, meteo y dirección de carrera 2018+, libres, telemetría 2018–2023	MIT (datos © F1)	1–2 días
Jolpica-F1	Tercera fuente de vueltas 1996+ y paradas con duración 2011+, al día en 2026; Q1–Q3 1994+	CC BY-NC-SA 4.0	1–2 días
OpenF1	2023+: dirección de carrera, meteo, stints, adelantamientos, tiempo parado (2025+)	CC BY-NC-SA 4.0	2–3 días
TracingInsights RaceData	Safety car 1973+, banderas rojas 1950+, VSC 2015+ (de Wikipedia)	CC BY-SA	1 día
Pirelli (prensa)	Compuestos nominados por GP desde 2011: validar neumáticos y obtener C1–C6	© Pirelli (hechos)	2 días
Open-Meteo ERA5	Meteorología estimada 1940+ por coordenadas del circuito	CC BY 4.0	1 día
f1resultsdatabase	Posiciones por vuelta 1950+ y paradas 1983+ para validación interna	ODbL (origen dudoso)	2–3 días
Documentos FIA	Evidencia para casos concretos (lap chart, history chart, pit stop summary)	no redistribuible	puntual

Restricciones respetadas

formula1db.com no se ha consultado: su `robots.txt` veta a los rastreadores y está tras Cloudflare. statsf1.com rechaza clientes automáticos. No se ha descargado ningún volcado completo (Jolpica, f1resultsdatabase) porque requiere tu permiso. Los PDF de la FIA se tratan solo como evidencia.

Carreras al sprint: sí se puede

La prueba con FastF1 cargó los 29 sprints disputados: 568 de los 590 pilotos cuadran vuelta a vuelta con F1DB y el resto son casos explicables (vuelta de abandono, un DNS, una descalificación). El diseño propuesto no toca el modelo de carreras largas:

- `bronze` nuevo `fastf1/sprint_laps` particionado igual que las vueltas de carrera;
- modelo `int_sprint_laptimes` separado, con un estado de validación `derived` para la vuelta 1 reconstruida;
- columna `session_type` en `fact_laptimes` y `fact_pit_lane_passes`. Es el mayor riesgo: toda consulta que no filtre por sesión mezclaría sprint y carrera;
- parámetro `?session=sprint` en `/laps`, `/stints` y `/pit-lane-passes`, y un selector `carrera/sprint` en las pestañas de vueltas, neumáticos y ritmo.

Recomendación

Primero la iteración 1 de arreglos internos (un día). Después, FastF1 ampliado y Jolpica, con lo que en una semana quedan cubiertos los sprints, la dirección de carrera, la meteorología 2018+ y la segunda fuente para 2023–2026. Los tiempos anteriores a 1990 se documentan como limitación.

CAPÍTULO 2

Inventario de los datos que faltan

Nota de trabajo para el informe de situación de `f1-data-platform` (29-09-2026). Base medida: `data/gold/f1.duckdb` construido el 28-09-2026 (F1DB v2026.15.1, publicada el 27-09-2026; 1 172 carreras en el calendario, 1 164 disputadas hasta Azerbaiyán 2026, ronda 15). Todas las cifras salen de consultas de solo lectura sobre esa base y sobre la capa `bronze`.

2.1 Resumen ejecutivo

1. **Resultados, parrillas, clasificaciones y campeonatos están completos 1950→2026** (1 164 de 1 164 carreras). Lo que falta es el **detalle dentro de cada carrera**, y crece hacia atrás en el tiempo.
2. **Vueltas: todas las carreras tienen lap chart, pero no todas tienen tiempos.** Hay tiempo por vuelta solo desde 1990 (y no completo: faltan 4 carreras de 1990 y Hungría 1993). De 1950 a 1989 solo hay posiciones (y en los años 50 el 43,6 % de ellas son desconocidas, «?» en `formula1db.com`). Excepción: Mónaco 1984 y Monza 1988 tienen tiempos.
3. **Faltan 3 223 vueltas de 116 pilotos** frente a las vueltas oficiales, más 39 de 11 pilotos sin ninguna vuelta; el 96 % son de 1950–1959 (Indianápolis 1953–1955, relevos y coches compartidos). Desde 1970 solo faltan 28 vueltas sueltas (10 de ellas en Francia 1981).
4. **Validación de tiempos:** el 53,7 % de las vueltas (682 058) son `single_source`. Esto afecta a todo lo anterior a 1996 (Ergast empieza en 1996) y a todo 2025–2026, que solo tiene FastF1 (`formula1db` termina en Abu Dabi 2024).
5. **2025–2026 (FastF1 como única fuente):** 734 vueltas sin tiempo (570 de ellas tienen los tres sectores, así que el tiempo se puede calcular), 436 vueltas con compuesto `NAN/NONE` (354 en Miami 2025, 15 pilotos), y ninguna vuelta con compuesto Pirelli real (C1–C6).
6. **Banderas rojas mal marcadas.** Solo hay marcas desde 2007 (31 carreras). En 22 carreras (24 vueltas), la vuelta cuyo tiempo contiene la suspensión (20–130 min) no lleva `is_red_flag`. La marca, tanto de `formula1db` como de FastF1, cae en la vuelta en que se muestra la bandera, que es la anterior (São Paulo 2024, v. 33, es uno de estos casos). Además faltan todas las rojas anteriores a 2007 y Bélgica 2021.
7. **Paradas:** hay duración (tiempo en el pit lane) desde 1994, en 612 carreras. Faltan Brasil y Mónaco 1994 e Italia 1997. Desde 1990 hay entradas a boxes sin tipificar, y antes de 1990 no hay nada. En ninguna época está el tiempo parado.
8. **Neumáticos:** el compuesto está desde 2011 y los stints desde 2018 (con FastF1). No hay nada antes de 2011, y falta el compuesto Pirelli real en 2024–2026.
9. **Sesiones que F1DB ya tiene y el modelo no expone:** libres FP1–FP4 (1986+), warm-up (1984–2003), precalificación (1977–1992), clasificaciones de viernes y sábado (1980–2005), porcentaje del piloto del día y tiempo de parrilla.

10. **Sesiones que no carga ninguna fuente:** vueltas y paradas de las **carreras al sprint** (hay 29 sprints con resultado y 0 vueltas), vueltas de clasificación y libres, meteorología, mensajes de dirección de carrera y sanciones estructuradas. FastF1 las tiene todas desde 2018 (y los sprints desde 2021), pero el cargador las desactiva (`weather=False`, `messages=False`, solo sesiones R y Q).
11. **Telemetría:** solo la vuelta rápida de clasificación de cada piloto en 2024–2026 (63 carreras). No hay telemetría de 2018–2023 ni de carrera.
12. **Incoherencias de modelo:** `dim_race.has_sprint` vale falso en los 12 sprints de 2021–2023. FastF1 no tiene Italia 2018 en bronze, así que esa carrera no tiene stint, speed trap ni estado de pista. Queda una tabla huérfana, `silver.int_formula1db_laps_validated`. Siguen abiertos los 3 casos de numeración de vueltas (San Marino y Japón 1994, Bélgica 2001).

2.2 Metodología

2.2.1 Fuentes y qué cubre cada una (según ingestion/ y transform/)

Fuente	Cargador	Qué aporta	Periodo	Estado
F1DB (SQLite, release GitHub)	<code>f1db_loader.py</code> (todas las tablas)	Resultados, clasificación, parrillas, vuelta rápida, paradas, libres, campeonatos, dimensiones	1950→actual	Semanal
formula1db.com (scraping del TFG)	<code>legacy_loader.py</code> (solo <code>lap_times</code> y <code>drivers</code> ; <code>race_entries</code> no está en bronze)	Vuelta a vuelta: posición, tiempo, diferencias, sectores, neumático, estado	1950 → Abu Dabi 2024	Congelado (web bloquea bots)
FastF1	<code>fastf1_loader.py</code> : sesión R (laps, results) y Q (results; telemetría opcional)	Vueltas 2025+; stint, vida del neumático, speed trap, estado de pista 2018+; Q1–Q3; telemetría de la vuelta rápida de clasificación	2018→actual (telemetría 2024+)	Semanal (temporada en curso)
Ergast (volcado CSV oct-2022)	<code>ergast_loader.py</code>	Solo validación de tiempos (1996–2022) y relleno N8 (0 vueltas)	1996–2022 (vueltas), 2011–2022 (paradas)	Congelado

Columnas de FastF1 que llegan a bronze y no se usan: `SpeedI1`, `SpeedI2`, `SpeedFL`, `FreshTyre` (está en staging pero no en `fact_laptimes`), `DeletedReason`, `PitInTime/PitOutTime`, `LapStartDate`, `TeamColor` y `HeadshotUrl`. De F1DB no se usan `driver_of_the_day_percentage`, `starting_grid_position_time`, `practice_*` ni las tablas de chasis, motor, inscritos, trazados y relaciones familiares.

2.2.2 Consultas principales

Todas se lanzan desde `transform/` (las vistas de staging leen `../data/bronze/...`) con `duckdb`. `connect('../data/gold/f1.duckdb', read_only=True)`.

Cobertura de vueltas frente a las oficiales por temporada. Las vueltas oficiales salen de `RACE_RESULT`. `race_laps` (con correcciones).

```
with off as (
  select race_id, driver_id, sum(race_laps) official_laps, bool_or(race_shared_car) shared
  from silver.int_race_data_corrected where session_type = 'RACE_RESULT' group by all),
lp as (select race_id, driver_id, count(distinct lap_number) rec from gold.fact_laptimes group
↪ by all)
select r.season, count(distinct r.race_id) races,
```

```
sum(official_laps) oficiales, sum(least(coalesce(rec,0), official_laps)) cubiertas
from gold.dim_race r left join off using (race_id) left join lp using (race_id, driver_id)
where r.is_completed group by 1 order by 1;
```

Estado por piloto y carrera. Se usa el modelo `silver.int_lap_completeness`, que ya concilia las vueltas registradas con las oficiales:

```
select r.season, r.round, r.grand_prix_name, c.status, count(*) pilotos, sum(missing_laps)
↪ faltan
from silver.int_lap_completeness c join gold.dim_race r using (race_id)
where status in ('missing_laps', 'no_laps', 'extra_laps') group by all order by 1, 2;
```

Rellenado de cada campo de `fact_laptimes` por temporada (resultado en la §3.3):

```
select r.season, count(*) vueltas,
round(100.0*count(lap_time_ms)/count(*),1) tiempo, round(100.0*count(position)/count(*),1)
↪ pos,
round(100.0*count(sector_1_ms)/count(*),1) s1,
↪ round(100.0*count(nullif(tyre_compound,''))/count(*),1) comp,
round(100.0*count(tyre_compound_pirelli)/count(*),1) pirelli,
↪ round(100.0*count(stint)/count(*),1) stint,
sum(is_pit_in_lap::int) boxes, sum(is_safety_car::int) sc, sum(is_red_flag::int) roja,
round(100.0*count(speed_trap_kmh)/count(*),1) speed_trap
from gold.fact_laptimes join gold.dim_race r using (race_id) group by 1 order by 1;
```

Suspensiones sin marca de bandera roja. Una vuelta de más de 400 s que registran al menos 3 pilotos contiene la suspensión:

```
select r.season, r.round, r.grand_prix_name, lap_number, count(*) n,
median(lap_time_ms)/60000 min_mediana, bool_or(is_red_flag) marcada
from gold.fact_laptimes join gold.dim_race r using (race_id)
where lap_time_ms > 400000 group by all having count(*) >= 3 order by 1, 2, 4;
```

Sesiones de F1DB por década (incluidas las que no se exponen):

```
select r.season//10*10 decada,
count(distinct d.race_id) filter (where type = 'FREE_PRACTICE_1_RESULT') fp1, ...
from gold.dim_race r left join read_parquet('../data/bronze/f1db/race_data.parquet') d
on r.race_id = d.race_id group by 1;
```

Además se consultaron `quality.qa_summary`, `silver.int_lap_numbering`, `silver.int_formula1db_race_alignment`, `gold.fact_pit_stops`, `gold.fact_pit_lane_passes`, `gold.fact_qualifying_result`, `gold.fact_race_result`, `gold.fact_quali_telemetry` y el `TrackStatus` de `bronze` de `FastF1`.

2.3 Tablas de cobertura

2.3.1 Matriz de cobertura por tipo de dato

✓ completo · ● parcial · ✗ no hay · ○ la fuente existe pero no se expone en gold

Dato	1950–59	60–69	70–79	80–89	90–95	96–2005	06–10	11–17	18–24	25–26	Origen
Resultado de carrera	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	F1DB
Parrilla	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	F1DB
Penalización de parrilla (texto)	✗	✗	✗	✗	●	●	●	✓	✓	✓	F1DB (○ solo en silver)

Dato	1950–59	60–69	70–79	80–89	90–95	96–2005	06–10	11–17	18–24	25–26	Origen
Clasificación: mejor tiempo	✓	✓	✓	✓	✓	✓	–	–	–	–	F1DB
Clasificación: Q1/Q2/Q3	–	–	–	–	–	–	✓	✓	✓	✓	F1DB (+FastF1 QA)
Clasificaciones de viernes y sábado	✗	✗	✗	○	○	○ (hasta 2002)	–	–	–	–	F1DB QUALIFYING_1/2_RESULT
Nº de vueltas de clasificación	✗	✗	✗	✗	● (1994+)	✓	✓	✓	✓	✓	F1DB
Precalificación	–	–	○	○	○	–	–	–	–	–	F1DB (1977–1992)
Libres (mejor tiempo y vueltas)	✗	✗	✗	○	○ (1986+)	○	○	○	○	○	F1DB
Warm-up	✗	✗	✗	○	○ (1984+)	○ (hasta 2003)	–	–	–	–	F1DB
Vuelta rápida de cada piloto	● (solo la absoluta)	●	●	●	✓ (74 %)	✓	✓	✓	✓	✓	F1DB FASTEST_LAP (○ solo QA)
Posición vuelta a vuelta	● (56 %)	● (95 %)	✓	✓	✓	✓	✓	✓	✓	✓	formula1db / FastF1
Tiempo por vuelta	✗	✗	✗	✗*	● (4 carreras sin tiempos)	✓	✓	✓	✓	● (97,8–98,7 %)	formula1db / FastF1
Validación con otra fuente	✗	✗	✗	✗	✗	✓ Ergast	✓	✓	✓ Er-gast+FastF1	✗	–
Sectores	✗	✗	✗	✗	✗	✗	✗	✗	✓ (~98 %)	✓	formula1db 2018+ / FastF1
Entradas y salidas de boxes por vuelta	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	formula1db / FastF1
Paradas (vuelta y duración)	✗	✗	✗	✗	● (1994–95)	✓	✓	✓	✓	✓	F1DB
Tiempo parado	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	–
Compuesto por vuelta	✗	✗	✗	✗	✗	✗	✗	✓	✓ (C1–C5 solo 2019–23)	● (sin Cx; Miami 2025 NAN)	formula1db / FastF1
Vida del neumático	✗	✗	✗	✗	✗	✗	✗	● (2014–15)	✓	✓	formula1db / FastF1
Stint	✗	✗	✗	✗	✗	✗	✗	✗	✓ (menos Italia 2018)	✓	FastF1
Speed trap	✗	✗	✗	✗	✗	✗	✗	✗	✓ (menos Italia 2018)	✓	FastF1 (I1, I2 y FL sin usar)
Amarilla / SC / VSC por vuelta	✗	✗	✗	✗	● (SC 1993+)	✓ (amarilla 1996+)	✓	✓ (VSC 2015+)	✓	✓	formula1db / FastF1
Bandera roja por vuelta	✗	✗	✗	✗	✗	✗	● (2007+)	●	● (desfase de 1 vuelta)	●	formula1db / FastF1
Telemetría	✗	✗	✗	✗	✗	✗	✗	✗	● (Q 2024)	● (Q)	FastF1
Meteorología	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	(FastF1 2018+, sin cargar)
Dirección de carrera y sanciones	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	(FastF1 2018+, sin cargar)
Sprint: resultado y parrilla	–	–	–	–	–	–	–	–	✓ (2021+)	✓	F1DB
Sprint: clasificación SQ1–SQ3	–	–	–	–	–	–	–	–	✓ (2023+)	✓	F1DB
Sprint: vueltas, paradas y vuelta rápida	–	–	–	–	–	–	–	–	✗	✗	(FastF1 sesión S, sin cargar)

Dato	1950–59	60–69	70–79	80–89	90–95	96–2005	06–10	11–17	18–24	25–26	Origen
Piloto del día	–	–	–	–	–	–	–	● (2016+)	✓	✓	F1DB (sin el porcentaje)
Hora de salida (UTC)	✗	✗	✗	✗	✗	✗	✗	✗	●	✓	F1DB (71 carreras)

* De 1950 a 1989 solo tienen tiempos Mónaco 1984 (396 vueltas), Monza 1988 (927) y 33 vueltas de Mónaco 1989.

2.3.2 Vueltas oficiales cubiertas por década

Vueltas oficiales = suma de race_laps por piloto. Cubiertas = vueltas registradas dentro del rango oficial (int_lap_completeness.recorded_in_range, sin contar las que pasan de las oficiales). Parcial = al menos un piloto con missing_laps o no_laps. La última columna excluye los coches compartidos y los descalificados.

Década	Carreras	Completas	Parciales	Sin vueltas	Vueltas oficiales	Cubiertas	%	% sin compartidos ni DSQ
1950	84	44	40	0	108 257	104 650	96,67	97,01
1960	100	89	11	0	92 875	92 700	99,81	99,89
1970	144	142	2	0	157 646	157 641	100,00	100,00
1980	156	151	5	0	166 476	166 446	99,98	99,98
1990	162	161	1	0	174 640	174 639	100,00	100,00
2000	174	174	0	0	182 660	182 660	100,00	100,00
2010	198	198	0	0	226 213	226 213	100,00	100,00
2020	146	146	0	0	160 245	160 244	100,00	100,00

Hay además vueltas registradas por encima de las oficiales: 1 027 de los Tyrrell descalificados en 1984 (F1DB les pone 0 vueltas), 7 716 de coches compartidos y vueltas de abandono de FastF1.

Estado por piloto (int_lap_completeness, 25 752 pares carrera-piloto):

Estado	Pilotos	Vueltas que faltan	Comentario
ok	25 339	0	
missing_laps	116	3 223	95 de ellos en los años 50 (3 097 vueltas)
no_laps	11	39	Abandonos en la vuelta 1 no registrados (años 50–80) y Behra, Italia 1951 (29)
extra_laps	9	7	Francia 1981 (numeración tras el relanzamiento), Marimón 1954, Daywalt 1954, Russo 1960, Trintignant 1951, Gasly GB 2024 (1 vuelta sin estar en el resultado)
shared_car	117	531	Coche compartido: la fuente no separa las vueltas de cada piloto
disqualified	160	19	F1DB no computa sus vueltas

2.3.3 Rellenado de campos de fact_laptimes (% de vueltas con valor)

Periodo	Vueltas	Tiempo	Posición	Sectores	Compuesto	Pirelli (Cx)	Relativo	Vida	Stint	Speed trap	Boxes marcados
1950–1959	108 201	0 %	56 %	0	0	0	0	0	0	0	0
1960–1969	92 874	0 %	95 %	0	0	0	0	0	0	0	0
1970–1989	325 148	0,4 %	100 %	0	0	0	0	0	0	0	5
1990	19 165	75,3 %	100 %	0	0	0	0	0	0	0	163
1991–1995	91 880	98,5 %	100 %	0	0	0	0	0	0	0	2 109

Periodo	Vueltas	Tiempo	Posición	Sectores	Compuesto	Pirelli (Cx)	Relativo	Vida	Stint	Speed trap	Boxes marcados
1996–2010	268 832	100 %	100 %	0	0	0	0	0	0	0	8 850
2011–2017	157 786	100 %	100 %	0	100 %	100 %†	0–9 %	85–100 %	0	0	6 410
2018	22 246	100 %	100 %	98,2 %	100 %	100 %†	0,1 %	100 %	94,5 %	95,8 %	557
2019–2023	113 629	100 %	100 %	~98 %	100 %	100 %	100 %	100 %	~100 %	~100 %	3 867
2024	26 574	100 %	100 %	98,1 %	100 %	8,3 %	100 %	100 %	100 %	100 %	844
2025	26 689	98,7 %	99,9 %	98,1 %	100 %‡	6,0 %	98,5 %	98,4 %	98,7 %	99,9 %	841
2026	17 236	97,8 %	99,8 %	97,5 %	100 %‡	0,1 %	99,9 %	99,7 %	100 %	99,8 %	625

† Hasta 2018 `tyre_compound_pirelli` es el nombre comercial (SOFT, SUPERSOFT...), no Cx. ‡ Incluye 354 vueltas NAN (Miami 2025, 15 pilotos), 57 NONE (Bélgica 2025) y 25 NONE (Hungría 2026). El % de Pirelli en 2024–2026 corresponde solo a INTERMEDIATE/WET: **no hay compuesto Cx desde 2024** (`formula1db` 2024 y `FastF1` dan HARD/MEDIUM/SOFT, y `int_tyre_compound_mapping` solo cubre 2019–2023).

2.3.4 Tiempo por vuelta: carreras sin tiempos o con tiempos parciales desde 1990

Carrera	Vueltas	Con tiempo	Comentario
EE. UU. 1990 (R1)	1 341	0	Solo lap chart
Brasil 1990 (R2)	1 271	0	Solo lap chart
San Marino 1990 (R3)	1 026	0	Solo lap chart
Mónaco 1990 (R4)	1 084	0	Solo lap chart
Alemania 1990 (R9)	744	728	97,8 %
Hungría 1993 (R11)	1 333	0	Solo lap chart
San Marino 1994 (R3)	1 000	957	95,7 %; además, numeración inconsistent
2025: 18 carreras	—	–352	<code>FastF1</code> sin <code>LapTime</code> : 341 bajo SC/VSC, 83 de boxes. 319 con los tres sectores. Destacan Gran Bretaña (–91), Bélgica (–60) y Azerbaiyán (–58)
2026: 15 carreras	—	–382	Ídem (251 con sectores). Italia –62, Azerbaiyán –91, Bélgica –62, Países Bajos –47, Mónaco –37

2.3.5 Estado de validación de los tiempos (`validation_status`)

Periodo	Vueltas	confirmedsingle_source(de ellas con tiempo)			timing_convention	disputed	corrected	Fuentes
1950–1989	526 223	0	526 223	1 356	0	0	0	<code>formula1db</code>
1990–1995	111 045	0	111 045	104 931	0	0	0	<code>formula1db</code>
1996–2017	426 618	425 731	204	204	380	303	0	<code>formula1db</code> (+Ergast)
2018–2022	111 488	111 450	0	0	38	0	0	<code>formula1db</code> (+Ergast, <code>FastF1</code>)
2023–2024	50 961	50 272	661	661	25	2	1	<code>formula1db</code> (+ <code>FastF1</code>); Norris Imola 2024 solo <code>FastF1</code>
2025–2026	43 925	0	43 925	43 191	0	0	0	<code>FastF1</code>

Hay 106 287 tiempos anteriores a 1996 sin contrastar. En 2023–2024 `FastF1` no es independiente de `formula1db` (las dos salen del cronometraje en directo de la F1), y `FastF1` toma Q1/Q2/Q3 de Ergast.

2.3.6 Paradas y entradas al pit lane por temporada

Temporada	Carreras	Con paradas F1DB	Paradas (todas con vuelta y duración)	Entradas al pit lane (vueltas)
1950–1989	481	0	0	3 (Mónaco 1984, Monza 1988)
1990–1993	64	0	0	1 038 (58 carreras, sin tipificar)
1994	16	14 (sin Brasil y Mónaco)	476 (475 con duración)	552
1995–2020	471	470 (sin Italia 1997)	17 481	17 787
2021	22	21 (Bélgica 2021 no tuvo paradas)	798	823
2022–2026	107	107	3 780	4 066

Tipos de paso (fact_pit_lane_passes, 24 271): 22 485 paradas, 1 118 sin tipificar, 351 retiradas, 141 penalización u otra, 114 Safety Car y 62 bandera roja. La **duración** de F1DB es el tiempo en el pit lane de la FIA. En ninguna época hay **tiempo parado**.

2.3.7 Clasificación

Periodo	Formato F1DB	Tiempo expuesto	Q1 / Q2 / Q3	Vueltas Sin exponer	
1950–1960	FOUR_LAPS / TWO_SESSION	Mejor tiempo (86–100 %)	—	—	—
1961–1979	TWO_SESSION	Mejor tiempo (99–100 %)	—	—	Precalificación 1977+ (9 carreras)
1980–1995	TWO_SESSION	Mejor tiempo (99–100 %)	—	1994+	Clasificaciones de viernes y sábado (QUALIFYING_1/2_RESULT, 15 248 filas en 292–293 carreras hasta 2005) y precalificación hasta 1992
1996–2002	ONE_SESSION	99–100 %	—	✓	—
2003–2005	ONE_LAP / AGGREGATE	94–97 %	—	✓	Tandas 1 y 2 de 2003–2005
2006–2026	KNOCKOUT	—	98–100 % / ~72 % / ~47 % (lo esperable por las eliminaciones)	✓	—

Lo que no hay: vuelta a vuelta de clasificación (FastF1 lo tiene desde 2018, sin cargar), sectores y speed trap de clasificación, y telemetría anterior a 2024.

2.3.8 Carreras al sprint

Temporada	Sprints con resultado	has_sprint	Clasificación sprint	Vueltas	Paradas	Vuelta rápida
2021	3	falso	(Q del viernes)	0	0	0
2022	3	falso	(Q del viernes)	0	0	0
2023	6	falso	6 (SQ1–SQ3)	0	0	0
2024	6	verdadero	6	0	0	0
2025	6	verdadero	6	0	0	0
2026	5 (+Singapur, pendiente)	verdadero	5	0	0	0

Causa del error de has_sprint: stg_f1db__races lo deriva de sprint_race_date, que F1DB solo rellena desde 2024. Debería derivarse de la existencia de SPRINT_RACE_RESULT o de qualifying_format / sprint_qualifying_format.

2.3.9 Telemetría (fact_quali_telemetry, 518 699 muestras cada 10 m)

Temporada	Carreras	Pilotos-carrera	Muestras
2018–2023	0	0	0 (FastF1 la tiene y no se carga)
2024	24	475	195 334
2025	24	475	191 381
2026	15	325	131 984

Solo hay la vuelta más rápida de clasificación de cada piloto: nada de carrera, libres ni sprint. Se guardan X e Y, pero no Z.

2.3.10 Banderas rojas (vueltas con is_red_flag)

Hay marcas en 31 carreras de 2007–2026. Entre 1990 y 2006 no hay ninguna carrera marcada, aunque hubo varias suspendidas (San Marino y Japón 1994, Bélgica 1998, Brasil 2003...).

Vueltas que contienen la suspensión (más de 400 s, al menos 3 pilotos) y **no** llevan la marca:

Carrera	Vuelta con la suspensión	Minutos (mediana)	Vueltas marcadas
Europa 2007	5	23,1	3–4
Corea 2010	4	48,9	3
Gran Bretaña 2014	2	61,5	1
Japón 2014	3	21,5	2
Australia 2016	19	20,2	17–18 (la 18 sí)
Bélgica 2016	10	19,2	9
Brasil 2016	21 y 29	35,3 / 26,6	20, 28
Azerbaiyán 2017	23	24,5	21–22
Italia 2020	27	28,0	26
Baréin 2020	2	82,5	1
Emilia-Romaña 2021	34	28,6	31–33
Azerbaiyán 2021	49	36,4	47–48
Gran Bretaña 2021	3	37,3	2
Hungría 2021	3	26,4	2
Arabia Saudí 2021	14 y 16	19,9 / 21,8	13, 15
Gran Bretaña 2022	2	53,3	1
Japón 2022	3	129,1	2
Países Bajos 2023	65	43,0	63–64
México 2023	35	22,3	34
São Paulo 2023	3	25,6	2
Japón 2024	2	28,5	1
São Paulo 2024	33	25,6	30–32

El patrón es sistemático. La marca de roja (el estado «Red Flag» de formula1db, que se usa hasta 2017, y el código 5 del `TrackStatus` de FastF1, desde 2018) se asigna a la vuelta en la que se muestra la bandera, mientras que el tiempo de la suspensión se suma a la vuelta siguiente, la del relanzamiento. Los datos sí marcan bien Mónaco 2011, Canadá 2011, Malasia 2012, Mónaco 2013 y 2022, Toscana 2020 y Australia 2023. Faltan además:

- **Bélgica 2021.** FastF1 marca la vuelta 3, pero en gold solo queda la vuelta 1 de formula1db, sin marca.
- **Japón y Mónaco 2024.** Faltan las vueltas de los pilotos que abandonaron en la vuelta 1 (FastF1 tiene 20 filas y el modelo 16–18).
- **2025.** No hay ninguna bandera roja marcada, ni en FastF1 ni en el modelo. Hay que comprobarlo.

En 2025–2026 la vuelta de la suspensión no tiene tiempo en FastF1 (Italia 2026: 62 vueltas sin tiempo).

2.3.11 Otras sesiones y datos de F1DB que no se exponen

Sesión o dato	Carreras	Periodo	Filas
FREE_PRACTICE_1_RESULT	713	1986–2026	16 164 (97 % con tiempo)
FREE_PRACTICE_2_RESULT	687	1986–2026	15 575
FREE_PRACTICE_3_RESULT	424	2003–2026	8 889
FREE_PRACTICE_4_RESULT	36	2004–2005	706
WARMING_UP_RESULT	324	1984–2003	7 683
PRE_QUALIFYING_RESULT	79	1977–1992	647 (636 con tiempo)
QUALIFYING_1/2_RESULT	292/293	1980–2005	15 248
DRIVER_OF_THE_DAY (porcentaje)	228	2016–2026	832 porcentajes
FASTEST_LAP por piloto	1 163	1950–2026	17 170 (solo la absoluta hasta 1979)
SPRINT_STARTING_GRID_POSITION	29	2021–2026	589 (se usa dentro del resultado)
Tablas chassis, engine, entrant, season_entrant_*, circuit_layout, driver_family_relationship	—	—	Sin modelo gold

2.4 Lista priorizada de huecos

Prioridad: **alta** = error visible en la web o dato básico que falta en una época cubierta; **media** = amplía análisis relevantes; **baja** = histórico marginal o complemento.

2.4.1 Prioridad alta

#	Hueco	Magnitud	Impacto en la web o el análisis	Fuentes posibles (hipótesis)
H1	Vueltas de las carreras al sprint	29 sprints (2021–2026), 0 vueltas	La página de la carrera no tiene lap chart, ritmo ni estrategia del sprint. <code>agg_driver_career.sprint_wins</code> sí existe	FastF1 sesión S (2021+) desde la misma API; Jolpica /sprint (resultados); formula1db.com (si tiene páginas de sprint; lo estudia la nota 03)
H2	Banderas rojas mal situadas o ausentes	24 vueltas de suspensión sin marca en 22 carreras (2007–2024); 0 marcas antes de 2007; Bélgica 2021	Los gráficos de ritmo muestran vueltas de 20–130 min como si fueran normales, y el lap chart no sombrea la suspensión. Los tipos <code>red_flag</code> de pit lane están infracontados	Regla derivada: marcar la vuelta siguiente a la del código 5 si su tiempo supera X; mensajes de dirección de carrera de FastF1 (<code>messages=True</code> , 2018+); lista de carreras con bandera roja de Wikipedia; Stats F1 e informes FIA para 1990–2006

#	Hueco	Magnitud	Impacto en la web o el análisis	Fuentes posibles (hipótesis)
H3	2025–2026 sin segunda fuente y con huecos	43 925 vueltas <code>single_source</code> ; 734 sin tiempo (570 recuperables sumando sectores); 436 con compuesto <code>NAN/NONE</code>	Ritmo con huecos; la estrategia de Miami 2025 sale vacía; el semáforo de calidad no cubre la temporada en curso	Calcular el tiempo con <code>S1+S2+S3</code> o con la diferencia de <code>Time</code> ; <code>Jolpica/laps</code> y <code>/pitstops</code> (2023+, sucesor de <code>Ergast</code>); <code>formula1db.com</code> en la sesión del usuario (ver restricciones); el <code>livetiming</code> de F1 para el compuesto (<code>FastF1 session.laps</code> con <code>livedata</code>)
H4	Compuesto <code>Pirelli real</code> (C1–C6) en 2024–2026	0 % de vueltas secas con Cx	La web solo puede mostrar <code>SOFT/MEDIUM/HARD</code> ; no se puede comparar el desgaste real entre carreras	Notas de prensa de <code>Pirelli</code> con los compuestos elegidos para cada GP (una fila por carrera; se extiende el seed de <code>int_tyre_compound_mapping</code>); Wikipedia (« <code>Pirelli tyre allocation</code> »); <code>racingnews365</code> y <code>f1technical</code>
H5	<code>dim_race.has_sprint</code> falso en 2021–2023	12 carreras	Filtros y etiquetas «fin de semana sprint» incorrectos; afecta a la API <code>/seasons</code> y a los agregados	Corrección interna (derivar de <code>SPRINT_RACE_RESULT</code>)
H6	Tiempos por vuelta 1990 (4 carreras) y Hungría 1993	5 carreras sin tiempos; Alemania 1990 y San Marino 1994 parciales	Página de ritmo vacía justo en la primera temporada con tiempos	Scraping de <code>formula1db.com</code> en la sesión del usuario (quizás ahora tenga esas carreras); <code>lap charts</code> de la FIA/FOCA; <code>Forix/Autosport</code> ; revistas de la época (hipótesis)
H7	Numeración inconsistente pendiente	San Marino 1994 (31 paradas desfasadas), Japón 1994 (14 paradas +13), Bélgica 2001 (15 vueltas rápidas –4) y Alboreto/-Brundle/Hill/Wendlinger 1994	Paradas y vuelta rápida en una vuelta equivocada en la web; <code>fact_pit_lane_passes</code> las cuenta como <code>penalty_or_other</code>	Ya documentado en <code>INFORME.md</code> («Pendiente para otra iteración»): seed de correcciones con los tiempos por vuelta como evidencia y propuesta a F1DB

2.4.2 Prioridad media

#	Hueco	Magnitud	Impacto	Fuentes posibles
M1	Datos de sesión que F1DB ya tiene y no se exponen: <code>libres</code> , <code>warm-up</code> , <code>precalificación</code> , <code>clasificaciones de viernes</code> y <code>sábado</code>	~64 000 filas en 713 carreras	Faltan las pestañas de <code>libres</code> y de <code>clasificación histórica</code> (antes de 2006 solo hay el mejor tiempo, cuando F1DB tiene las dos sesiones de 1980–2005)	F1DB (ya en <code>bronze</code>): basta con modelos <code>gold</code> nuevos
M2	Vuelta rápida de cada piloto (<code>FASTEST_LAP</code>)	17 170 filas; solo en QA	Sin <code>ranking</code> de vueltas rápidas por carrera ni <code>evolución histórica</code> fuera de las vueltas; antes de 1990 es el único dato de ritmo	F1DB (ya en <code>bronze</code>)

#	Hueco	Magnitud	Impacto	Fuentes posibles
M3	Meteorología	0 carreras	No se puede explicar el ritmo, el uso de intermedios o el desgaste; es clave para la fase 7 (degradación)	FastF1 <code>weather=True</code> (2018+, temperatura de aire y pista, lluvia por minuto); Open-Meteo histórico (reanálisis desde 1940) con coordenadas del circuito y hora de salida para 1950–2017 (la hora UTC solo está en 71 carreras); Wikipedia (condiciones de carrera)
M4	Mensajes de dirección de carrera, sanciones e investigaciones	0	No hay línea de tiempo de incidentes ni tipificación fiable de drive-through o stop-and-go (141 <code>penalty_or_other</code> sin detalle)	FastF1 <code>messages=True</code> (2018+); documentos de decisión de la FIA (PDF, 2019+); Wikipedia (notas de las clasificaciones); <code>time_penalty_ms</code> de F1DB (solo el total: 12 casos en 2000–09, 103 en 2010–19 y 162 en 2020–26)
M5	Stints y neumáticos antes de 2011	0 % de compuesto entre 1950 y 2010; vida del neumático parcial en 2014–15 (1 038 y 2 864 vueltas sin dato)	La estrategia de neumáticos solo funciona desde 2011; antes solo se puede deducir de las paradas	Stints derivados de las paradas de F1DB (1994+) y de las entradas a boxes (1990+); Bridgestone/Michelin 2001–2010 (prime/option en notas de prensa de 2007–2010); Forix
M6	Telemetría 2018–2023 y de carrera	6 temporadas sin telemetría	La comparación de telemetría solo funciona desde 2024	FastF1 (<code>--telemetry</code> para 2018–2023; la carga cuesta muchas peticiones, con un límite de 500/h)
M7	Validación de 1990–1995	106 287 tiempos <code>single_source</code>	El semáforo de calidad no cubre esas temporadas	No hay tiempos por vuelta en Ergast antes de 1996. Hipótesis: lap charts FIA/FOCA, Forix y la vuelta rápida de F1DB como control parcial (ya en <code>qa_fastest_laps</code>)
M8	FastF1 Italia 2018 ausente	925 vueltas sin stint, speed trap ni estado de pista; sin QA de clasificación	Estrategia sin tramos de FastF1 (las pausas se deducen de las entradas a boxes)	Volver a cargar con <code>f1-ingest fastf1 --season 2018</code> (quizás fue un fallo puntual de la API)
M9	Paradas de 1994 (Brasil, Mónaco) e Italia 1997	3 carreras	Página de paradas vacía	Documentos de la FIA (pit stop summary), formula1.com (archivo de resultados; hipótesis), las entradas a boxes de formula1db (ya en <code>fact_pit_lane_passes</code>)
M10	Tiempo parado	0 en todas las épocas	Solo existe el tiempo en el pit lane: no hay ranking de equipos por parada	DHL Fastest Pit Stop Award (tiempos parados de las paradas más rápidas, 2015+); derivar <code>PitInTime/PitOutTime</code> de FastF1 (tiempo en el pit lane, no parado)
M11	Speed traps intermedios y de meta	<code>SpeedI1</code> , <code>SpeedI2</code> y <code>SpeedFL</code> en bronze, no expuestos	Menos variables para comparar velocidad punta	FastF1 (ya en bronze)

2.4.3 Prioridad baja

#	Hueco	Magnitud	Impacto	Fuentes posibles
B1	Tiempos por vuelta 1950–1989	524 867 vueltas (1950–1989) sin tiempo, solo con posición	No hay gráfico de ritmo antes de 1990 (solo lap chart)	Prácticamente inexistentes en abierto. Hipótesis: hojas de cronometraje de la época (Longines, Heuer, TAG), anuarios (<i>Autocourse</i>), Forix/Autosport, <i>Motor Sport</i> archive
B2	Posiciones desconocidas («?») en los años 50–60	43,6 % de las vueltas de los 50; 5,4 % de los 60	Lap chart con huecos	Stats F1 (<i>tour-par-tour</i>), Forix, lap charts de revistas; muy costoso
B3	Vueltas que faltan en los años 50	3 097 vueltas en 95 pilotos (Indianápolis 1953–1955, relevos)	Lap chart incompleto en Indianápolis y en los relevos	Stats F1; resultados de la USAC para Indianápolis (hipótesis)
B4	Coches compartidos	117 pilotos, 531 vueltas sin separar	Vueltas de piloto aproximadas en los años 50	Stats F1 reparte las vueltas por piloto (visto al revisar el V7)
B5	Vuelta rápida por piloto antes de 1980	Solo la absoluta (94 + 104 + 147 filas en 1950–79); 74 % de los pilotos en los 80	Rankings históricos de vuelta rápida incompletos	Stats F1 (<i>meilleur-tour</i>)
B6	Hora de salida histórica	71 de 1 172 carreras con hora UTC	Limita el cruce con la meteorología horaria	Wikipedia, formula1.com
B7	Piloto del día (porcentaje)	832 porcentajes sin exponer	Detalle menor	F1DB (ya en bronze)
B8	Dimensiones de F1DB sin modelo (chasis, motor, inscritos, trazado, familias)	—	Fichas de coche y trazado, «coche de cada victoria»	F1DB (ya en bronze)

2.5 Incoherencias abiertas conocidas

1. **Numeración de vueltas** (*int_lap_numbering*): 3 inconsistent (San Marino 1994, Japón 1994, Bélgica 2001) y 1 *pit_lap_convention* (España 1994). 487 carreras quedan con *no_evidence*; 486 son anteriores a 1994, porque sin paradas en F1DB (y casi sin vueltas rápidas por piloto) no hay con qué comprobar la numeración.
2. **Bandera roja en la vuelta equivocada** (§3.10): 24 vueltas en 22 carreras, entre ellas el caso de São Paulo 2024, v. 33.
3. ***dim_race.has_sprint*** falso en 12 sprints de 2021–2023.
4. **Paradas de F1DB sin entrada a boxes**: 41 de 21 297 (desfases de ± 1 en 1994–95 y São Paulo 2023, donde el cambio de neumáticos con bandera roja cuenta como parada en la vuelta 1).
5. **Vida del neumático**: FastF1 y formula1db coinciden en el 95–98 % de las vueltas desde 2019 y solo en el 67 % en 2018. Se usa FastF1, pero en 2025–2026 no hay con qué contrastarla.
6. ***disputed***: 305 vueltas (303 en 1996–2017 y 2 en 2023–24) conservan el valor de formula1db sin mayoría.
7. **Realineación de rondas de formula1db**: 17 carreras de 2023 venían etiquetadas con la ronda siguiente; se corrigen por contenido. En Mónaco 2023 hay un piloto sin emparejar (95 %).
8. **Vueltas sin resultado oficial**: Trintignant, Italia 1951 (29 vueltas; no figura en el resultado con vueltas) y Gasly, Gran Bretaña 2024 (1 vuelta; no tomó la salida).

9. **Francia 1981:** formula1db numera desde el relanzamiento: 10 pilotos con -1 y 4 con vueltas de más.
10. **Descalificados:** F1DB les pone 0 vueltas (Tyrrell 1984 y otros), así que la cobertura supera el 100 % en los años 80.
11. **Tabla huérfana `silver.int_formula1db_laps_validated`** (1 226 299 filas) en `f1.duckdb`, sin modelo dbt: quedó de una versión anterior. Conviene borrarla o activar la limpieza de relaciones huérfanas.
12. **Compuestos `NAN/NONE`** guardados como texto en 2025–2026: deberían ser NULL (los cuenta el test de rellenado).
13. **QA de FastF1 frente a F1DB:** 30 parrillas (99,18 %), 8 tiempos de clasificación y 7 vueltas rápidas no coinciden. Dentro del umbral, pero sin revisar una por una.

2.6 Carreras concretas con huecos de vueltas (`missing_laps`, `no_laps`, `extra_laps`)

Formato `piloto(registradas/oficiales)`. Se omiten los coches compartidos y los descalificados.

Carrera	Estado	Faltan	Detalle
Gran Bretaña 1950	missing	22	joe-fry(45/64), peter-walker(2/5)
Indianápolis 1950	missing	98	chitwood(82/136), banks(71/112), levrett(105/108)
Francia 1950	missing	84	11 pilotos: pozzi(14/56), etancelin(26/59)...
Italia 1950	missing	42	serafini(47/80), taruffi(25/34)
Indianápolis 1951	missing	107	mcgrath(100/200), scarborough(93/100)
Francia 1951	missing	42	gonzález(35/77)
Italia 1951	missing / no_laps / extra	79	bonetto(29/79), behra(0/29); trintignant con 29 vueltas y sin vueltas oficiales
Suiza 1952	missing	33	andre-simon(18/51)
Francia 1952	missing	47	fischer(33/66), de-graffenried(20/34)
Gran Bretaña 1952	no_laps	1	cantoni(0/1)
Países Bajos 1952	missing	40	landi(43/83)
Argentina 1953	missing	41	trintignant(50/91)
Indianápolis 1953	missing	543	9 pilotos
Países Bajos 1953	missing	64	bonetto(25/89)
Bélgica 1953	missing	22	claes(13/35)
Italia 1953	missing	38	mantovani(38/76)
Argentina 1954	missing / extra	1	rosier(1/2); marimón con 48 vueltas registradas y 5 oficiales
Indianápolis 1954	missing / extra	620	11 pilotos; daywalt con 165 registradas y 111 oficiales
Bélgica 1954	missing	15	hawthorn(20/35)
Gran Bretaña 1954	missing	12	villoresi(30/40), bira(42/44)
Alemania 1954	missing	6	gonzález(16/22)
Italia 1954	missing	48	maglioli(30/78)
España 1954	missing	27	de-graffenried(30/57)
Argentina 1955	missing	139	herrmann, gonzález, bucci, castellotti
Mónaco 1955	missing	36	taruffi(50/86)
Indianápolis 1955	missing	101	bettenhausen(123/200), faulkner(176/200)
Bélgica 1955	missing	25	mieres(10/35)
Gran Bretaña 1955	missing	58	hawthorn, wharton, rolt
Argentina 1956	missing	158	musso, landi, uria
Mónaco 1956	missing	90	collins(54/100), bayol(44/88)

Carrera	Estado	Faltan	Detalle
Indianápolis 1956	missing	37	elisian(123/160)
Bélgica 1956	missing	23	perdisa(13/36)
Francia 1956	missing	85	hawthorn(10/56), perdisa(20/59)
Gran Bretaña 1956	missing	12	castellotti(80/92)
Alemania 1956	missing	7	de-portago, musso
Italia 1956	missing	29	collins, maglioli, villoresi
Argentina 1957	missing	117	perdisa(30/98), de-portago(49/98)
Mónaco 1957	missing	25	scarlatti(42/64), von-trips(92/95)
Francia 1957	missing	38	macdowel(30/68)
Gran Bretaña 1957	missing	3	trintignant(85/88)
Pescara 1957	no_laps	2	piotti(0/1), gould(0/1)
Italia 1957	missing	66	scarlatti(50/84), simon(40/72)
Francia 1958	missing	15	lewis-evans(20/35)
Italia 1958	missing	24	gregory(45/69)
Marruecos 1958	missing	7	bridger(26/30), picard(28/31)
Argentina 1960	missing	34	trintignant(46/80)
Indianápolis 1960	extra	0	russo (90 registradas / 84 oficiales)
Francia 1960	missing	1	von-trips(30/31)
EE. UU. 1961	missing	62	gendebien(30/92)
Mónaco 1965	no_laps	1	ginther(0/1)
México 1966	missing	1	solana(8/9)
Mónaco 1967	missing	3	servoz-gavin(1/4)
México 1967	no_laps	1	fisher(0/1)
Bélgica 1968	no_laps	1	bonnier(0/1)
España 1969	no_laps	1	oliver(0/1)
Brasil 1973	missing	1	pace(8/9)
Suecia 1974	missing	2	reutemann(29/30), regazzoni(23/24)
Francia 1981	missing / extra	17	10 pilotos con -1 y 4 con vueltas de más (numeración tras el relanzamiento)
Canadá 1983	no_laps	1	surer(0/1)
Alemania 1986	missing	10	brundle(24/34)
San Marino 1988	no_laps	1	de-cesaris(0/1)
Mónaco 1988	no_laps	1	piquet(0/1)
España 1997	missing	1	hill(17/18)
Gran Bretaña 2024	extra	0	gasly, 1 vuelta sin estar en el resultado
São Paulo 2024	missing	1	colapinto(29/30)

2.7 Resumen del impacto en la web (web/src/app/[lang]/races/[id]/...)

Página	Depende de	Épocas en que sale vacía o incompleta
lap-chart	position	Huecos en 1950–1969 («?» y relevos)
pace	lap_time_ms	Vacía antes de 1990 (salvo 3 carreras), en 1990 (4 carreras) y en Hungría 1993; picos de suspensión sin marcar desde 2007
tyres (API / stints)	tyre_compound, is_pit_in_lap	Vacía antes de 2011; Miami 2025 sin compuesto; sin Cx desde 2024
pitstops	fact_pit_stops, fact_pit_lane_passes	Vacía antes de 1994 (entradas sin tipificar en 1990–93); faltan 3 carreras

Página	Depende de	Épocas en que sale vacía o incompleta
qualifying	fact_qualifying_result	Antes de 2006 solo el mejor tiempo; faltan las sesiones de viernes y sábado de 1980–2005
telemetry	fact_quali_telemetry	Solo 2024–2026
Resultado del sprint	fact_race_result (SPRINT)	Hay resultado, pero no vueltas, paradas ni vuelta rápida
quality	qa_summary	No mide 2025–2026 ni 1950–1995 (todo single_source)

CAPÍTULO 3

Fuentes externas para cubrir y contrastar huecos

Nota de investigación (29-09-2026). Consultas web + peticiones reales pequeñas a las APIs públicas. No se ha accedido a formula1db.com (su robots.txt veta explícitamente a ClaudeBot), no se ha intentado saltar ninguna protección y no se ha descargado ningún volcado completo.

3.1 Resumen ejecutivo

- Lo que ya tenemos es más de lo que ofrece casi cualquier fuente abierta: posiciones vuelta a vuelta de las 1 125 carreras 1950–2024 (CSV de formula1db.com), tiempos por vuelta desde 1990, neumático por vuelta desde 2011 y paradas de F1DB desde 1994. El problema no es tanto *cubrir* como *contrastar*: antes de 1996 formula1db es fuente única.
- Jolpica-F1 es la integración más rentable: mismo esquema que el volcado Ergast que ya cargamos, pero vivo (2023–2026 incluidos), con volcados CSV/SQL de ~14,5 MB. Da una tercera fuente independiente de vueltas (1996+) y paradas con duración (2011+) también para 2023–2026.
- OpenF1 (2023+) y F1 Live Timing vía FastF1 (2018+) cubren lo que falta en la era moderna: mensajes de dirección de carrera (banderas, SC/VSC, bandera roja, sanciones), meteorología, sectores, speed traps, vueltas de sprint y libres, y el tiempo parado en boxes (solo ~2024+).
- Pre-1996 no existe ninguna fuente abierta y descargable de tiempos por vuelta. Las únicas alternativas son Forix/Motorsport Stats (comercial, lap-by-lap desde 1981), StatsF1 (posiciones vuelta a vuelta, solo consulta manual) y el proyecto f1resultsdatabase (SQLite ODbL construido a partir de StatsF1, Motorsport Stats y formula1.com: útil para validar, con riesgo de licencia).
- Eventos históricos (SC 1973+, bandera roja 1950+, VSC 2015+) están en listas de Wikipedia y su derivado CSV de TracingInsights/Kaggle; la meteorología histórica puede aproximarse con el reanálisis ERA5 de Open-Meteo (1940+, CC BY 4.0).
- Licencias: F1DB es CC BY 4.0; Jolpica y OpenF1 son CC BY-NC-SA 4.0 (no comercial y «compartir igual»). El repositorio no tiene fichero LICENSE ni atribuciones: hay que añadirlos antes de integrar nada NC-SA.

3.2 Punto de partida: qué tenemos (bronze medido el 29-09-2026)

Dato	Cobertura propia	Fuente actual	Observación
Posición vuelta a vuelta (lap chart)	1950–2024, 1 125/1 125 carreras (1 252 550 filas)	formula1db (CSV TFG)	Fuente única antes de 1996
Tiempo por vuelta	1950–1989: < 1 % de vueltas con tiempo (505/110 056 en los 50; 3 531/171 428 en los 80); 1990+ casi completo	formula1db; Ergast 1996–2022; FastF1 2018+	1990–1995 solo formula1db

Dato	Cobertura propia	Fuente actual	Observación
Sectores por vuelta	1990+ (formula1db); 2018+ FastF1	formula1db, FastF1	—
Neumático por vuelta	2011+ (formula1db, etiquetas S/M/H/SS/US/I/W); 2018+ FastF1 (compuesto, stint, vida)	formula1db, FastF1	Nada antes de 2011
Paradas	F1DB 1994+ (vuelta + tiempo en pit lane, 22 535 filas); Ergast 2011–2022 (duración)	F1DB, Ergast	Nada antes de 1994; sin tiempo parado
Clasificación	F1DB: tiempo 1950+; Q1/Q2/Q3 2006+; Q1/Q2 de días separados 1980–2005	F1DB	FastF1 toma Q1–Q3 de Ergast (no independiente)
Libres / warm-up / precalificación	F1DB: FP1–FP2 1986+, FP3 2003+, warm-up 1984–2003, preclasif. 1977–1992 (solo mejor tiempo)	F1DB	Sin vueltas
Sprint	F1DB: resultados 2021+, parrilla, sprint quali 2023+	F1DB	Sin vueltas de sprint
SC/VSC/banderas	FastF1 TrackStatus por vuelta 2018+	FastF1	Sin mensajes de dirección de carrera ni histórico
Meteorología	—	—	FastF1 se carga con <code>weather=False</code>
Speed traps	FastF1 SpeedI1/I2/FL/ST 2018+	FastF1	—

3.3 Pruebas reales realizadas (29-09-2026)

Peticiones pequeñas con requests desde `.venv/Scripts/python.exe`, cabecera User-Agent propia.
OpenF1 — <https://api.openf1.org/v1/>

- `meetings?year=2023` → 24 eventos; `sessions?year=2022` → 404 «No results found»: el histórico empieza en 2023. `sessions?year=2026&session_name=Race` → 25 carreras (ya incluye 2026).
- Imola 2024 (`session_key=9515`): `race_control` 100 mensajes (categorías Flag, Drs, SessionStatus, Other; banderas GREEN, YELLOW, DOUBLE YELLOW, CLEAR, BLUE ×68, BLACK AND WHITE, CHEQUERED; con `lap_number`, `sector`, `scope`); pit 27 filas con `lane_duration` pero `stop_duration` nulo; `stints` con compuesto y edad del neumático; `weather` 143 muestras (aire, pista, lluvia, viento, humedad, presión); `laps` con sectores, mini-sectores e `i1/i2/st_speed`; `session_result`; `overtakes` 133 filas; `starting_grid` → 404.
- Japón 2025 (`session_key=10006`): pit 21/21 con `stop_duration` (p. ej. 2,6 s) → el tiempo parado existe desde ~2024–2025. Bahrein 2023 (`session_key=7953`): pit → 404.
- China 2024 sprint (`session_key=9672`): 19 vueltas de Verstappen → vueltas de sprint disponibles.
- Límite: HTTP 429 con `retry-after: 1` al encadenar ~4 peticiones sin pausa (límite publicado 3 req/s y 30 req/min sin cuenta).

Jolpica-F1 — <https://api.jolpi.ca/ergast/f1/>

- `laps` exige temporada y ronda (400 si no). 1995 → 0 vueltas; 1996 R1 → 812.
- `pitstops` 2010 → 0; 2011 R1 → 45 con `duration`.
- `qualifying` 1993 → 0; 1994 R1 → 26. 2024 R6 (Miami): Q1/Q2/Q3 y sprint presentes.
- 2026 al día: R5 con 1 206 vueltas y 34 paradas; `2026/results` 330 filas.
- GET <https://api.jolpi.ca/data/dumps/download/> (JSON, sin autenticación): volcados csv y sql de ~14,5 MB; «latest» del 26-09-2026 (solo patrocinadores) y «delayed» (14 días, gratuito no comercial). No se descargó.

F1 Live Timing — <https://livetiming.formula1.com/static/>

- 2017/Index.json → 403; 2018/Index.json → 200 con 20 eventos (falta Australia 2018).
- Feeds de una carrera 2025: TimingData, TimingAppData, RaceControlMessages, TrackStatus, WeatherData, PitLaneTimeCollection, PitStopSeries, TyreStintSeries, OvertakeSeries, LapSeries, TeamRadio, CarData.z, Position.z, etc.
- PitStopSeries (tiempo parado + tiempo en pit lane): 200 en Japón 2025 y Las Vegas 2024; 403 en China 2019, Hungría 2020, Portugal 2021 y São Paulo 2023.
- Bahrein 2018: RaceControlMessages, TimingAppData, TrackStatus y WeatherData disponibles.

Otras

- statsf1.com: robots.txt permite *, pero a peticiones no navegador devuelve una página «Erreur» → solo consulta manual (como ya se hizo en *revision_divergencias*).
- formula1.com: robots.txt no bloquea /en/results; hay «Pit stop summary» desde 1994 (Pacífico, Francia 1994), el mismo inicio que F1DB.
- TracingInsights/RaceData (GitHub): *safety_cars.csv* 375 filas (1973–2026), *red_flags.csv* 101 (1950–2026), *virtual_safety_cars.csv* 113 (2015–2026).
- TracingInsights-Archive/PitStops (MIT): un JSON por GP 2018–2026 con el ranking DHL (posición, equipo, piloto, tiempo parado, vuelta, puntos).
- Wikidata (Q171864, GP de Francia 1975): solo ganador, pole, vuelta rápida, distancia, fecha e identificadores externos (Freebase, Racing-Reference).
- Open-Meteo Archive (Monza, 07-09-1975): temperatura, precipitación y viento horarios devueltos.
- f1resultsdatabase: release v2.10.1+20260921 (*sessionresults.zip*, 74 MB), no descargada.

3.4 Fichas por fuente

Para cada una: **datos y cobertura · acceso/formato · licencia/ToS/robots · fiabilidad · integración (bronze → dbt) · uso para contraste.**

3.4.1 APIs y bases abiertas (candidatas a ingesta)**Jolpica-F1 (sucesor de Ergast)**

- **Datos:** temporadas, carreras, resultados, clasificación (Q1–Q3; desde 1994), sprint, paradas con duración (2011+), vueltas con posición y tiempo (1996+), clasificaciones del campeonato, estados. Actualizado en la semana del GP. Se nutre en parte de los PDF de la FIA con el parser *fia-doc* (2025+).
- **Acceso:** API REST compatible con Ergast (*/ergast/f1/{season}/{round}/laps/*); volcados CSV y SQL en */data/dumps/download/*. Límite: 4 req/s y 500 req/h sin token; se exige un User-Agent identificativo.
- **Licencia:** datos CC BY-NC-SA 4.0 («freely available for non-commercial use»); código Apache-2.0; volcado inmediato solo para patrocinadores; gratuito con 14 días de retraso.
- **Fiabilidad:** alta desde 1996 para vueltas (heredada de Ergast); en *revision_divergencias* Ergast nunca discrepó en solitario de formula1db en 2018–2022. Riesgo: proyecto de voluntarios sin SLA.
- **Integración:** *ingestion/jolpica_loader.py* que descargue el volcado *delayed* (una petición semanal) → *bronze/jolpica/*.parquet* (sustituye a *bronze/ergast*, congelado en 2022). En dbt, *stg_jolpica_** con el mismo mapeo de pilotos que Ergast (fecha de nacimiento + apellido).

- **Contraste:** 3.^a fuente de vueltas 2023–2026 (hoy solo formula1db + FastF1 hasta 2024 y FastF1 en solitario después); paradas F1DB vs Jolpica (vuelta y duración) 2011+; clasificación Q1–Q3 (ojo: FastF1 la toma de aquí, no son independientes); resultados de sprint vs F1DB.
- **URLs:** <https://github.com/jolpica/jolpica-f1> · <https://github.com/jolpica/jolpica-f1/blob/main/TERMS.md> · https://github.com/jolpica/jolpica-f1/blob/main/docs/rate_limits.md · <https://github.com/jolpica/jolpica-f1/discussions/261>

OpenF1

- **Datos (2023+):** sessions/meetings, laps (sectores, mini-sectores, speed traps), stints, pit (pit lane y, desde ~2024–2025, tiempo parado), race_control (banderas por sector, SC/VSC, bandera roja, investigaciones y sanciones, DRS), weather, position, intervals, overtakes, session_result, starting_grid, car_data (telemetría ~3,7 Hz), location, team_radio. Incluye libres, clasificación sprint y sprint.
- **Acceso:** REST JSON/CSV sin clave (3 req/s, 30 req/min); tiempo real solo con patrocinio (9,90 €/mes).
- **Licencia:** CC BY-NC-SA 4.0; no oficial ni asociado a F1/FIA/FOM.
- **Fiabilidad:** buena (procede del mismo live timing que FastF1); huecos puntuales (pit 404 en Bahrein 2023; starting_grid 404 en Imola 2024).
- **Integración:** loader por sesión (sessions?year=, luego endpoints por session_key) → bronze/openf1/{endpoint}/season=/session=.parquet. Primero race_control, weather, pit, stints y laps de sprint. Modelos fact_race_control_message, fact_weather_sample, fact_pit_stop (con stop_duration).
- **Contraste:** vueltas y stints 2023+ contra FastF1 (misma raíz, pero distinto procesado: útil para detectar errores de FastF1 como los 35 neumáticos que violaban reglas); pit contra F1DB y Jolpica; resultados de sprint contra F1DB.
- **URLs:** <https://openf1.org/> · <https://openf1.org/docs/> · <https://api.openf1.org/v1/sessions>

F1 Live Timing vía FastF1 (ya integrado parcialmente)

- **Datos (2018+; falta Australia 2018):** vueltas, sectores, speed traps, compuesto/stint/vida, Track Status por vuelta, RaceControlMessages, WeatherData, telemetría y posición, todas las sesiones (FP1–FP3, Q, sprint shootout/quali, sprint, R). PitStopSeries (tiempo parado) solo desde ~2024. Antes de 2018 FastF1 solo ofrece lo que da Jolpica.
- **Acceso:** ficheros estáticos JSON/jsonStream sin autenticación; FastF1 (MIT) los cachea. Límite propio de FastF1: 500 peticiones/h.
- **Licencia/ToS:** datos propiedad de Formula One World Championship Ltd.; FastF1 se declara no oficial. No hay licencia explícita para el feed: uso razonable para análisis, sin redistribuir en bruto (la web y la API del proyecto publican derivados).
- **Fiabilidad:** alta en tiempos, pero revision_divergencias detectó que FastF1 se desvía en solitario en 1 085 tiempos y 1 144 posiciones (2018–2022) y viola reglas de neumáticos en 35 casos.
- **Integración (bajo esfuerzo, ya hay loader):** activar messages=True, weather=True y añadir sesiones S, SQ/SS, FP1–FP3 en ingestion/fastf1_loader.py; guardar race_control_messages y weather_data en bronze.
- **Contraste:** SC/VSC de TrackStatus contra la lista de Wikipedia/TracingInsights; paradas contra F1DB; sanciones contra documentos FIA.

- URLs: <https://docs.fastf1.dev/> · <https://github.com/the0ehrly/Fast-F1> · <https://livetiming.formula1.com/static/2018/Index.json>

F1DB (ya integrado)

- **Datos:** 1950–hoy; resultados, libres (1986+), warm-up, (pre)clasificación, sprint, parrillas, vueltas rápidas, paradas (1994+), piloto del día. Sin vueltas ni compuestos.
- **Formatos:** CSV, JSON, SQL (MySQL/PostgreSQL/SQLite) y SQLite; release tras cada GP.
- **Licencia:** CC BY 4.0 → exige atribución visible en web/API.
- **Contraste:** no es independiente de formula1.com para paradas (ambas empiezan en 1994).
- URL: <https://github.com/f1db/f1db>

3.4.2 Datasets derivados (GitHub, Kaggle, Hugging Face)

TracingInsights

- **RaceData (CC0):** tablas estilo Ergast 1950–hoy + `safety_cars` (1973+, causa, vuelta de despliegue y retirada), `red_flags` (1950+, vuelta, reanudación, incidente, excluidos), `virtual_safety_cars` (2015+), `virtual_safety_car_estimates.json`, accidentes mortales. Actualizado por GitHub Actions ≤ 3 h tras la carrera. Origen: Kaggle «Formula 1 Race Data» y «Formula 1 Race Events» (jtrotman), que a su vez derivan de Ergast y de listas de Wikipedia (CC BY-SA): el CC0 declarado no limpia esa cadena → citar Wikipedia.
- **Repos por temporada 2018–2026 (Apache-2.0):** por sesión `session_laptimes.json`, `rcm.json` (mensajes de dirección de carrera), `weather.json`, `corners.json`, telemetría. Origen FastF1/MultiViewer/Jolpica: es un espejo, no una fuente independiente.
- **PitStops (MIT):** ranking DHL por GP 2018–2026 con tiempo parado y vuelta.
- **Integración:** `safety_cars`, `red_flags`, `virtual_safety_cars` como seeds o bronze (≈ 600 filas); resolver Race («1973 Canadian Grand Prix») a `race_id` de F1DB. PitStops DHL → `fact_pit_stop_stationary_top` para 2018–2023 (donde no hay PitStopSeries).
- **Contraste:** validar `TrackStatus` de FastF1 y las vueltas lentas del lap chart de formula1db (una SC debería comprimir los intervalos); validar vueltas y paradas en carreras con bandera roja.
- URLs: <https://github.com/TracingInsights/RaceData> · <https://huggingface.co/datasets/tracinginsights/RaceData> · <https://github.com/TracingInsights/2026> · <https://github.com/TracingInsights-Archive/PitStops> · <https://tracinginsights.com/data/>

f1resultsdatabase (mclarenmp4-22)

- **Datos:** SQLite de 26 tablas; posiciones vuelta a vuelta 1950+, tiempos 1996+, sectores y compuestos 2018+, **paradas desde 1983** (tiempo parado 2018+), `TrackStatus` con SC desde 1973, sanciones estructuradas, Q1/Q2 de 1983–2005, warm-up 1984–2003, libres 2000–2003 (newsonf1.com), meteo (Open-Meteo). Release v2.10.1+20260921 (74 MB).
- **Origen:** scraping de StatsF1, APIs internas de motorsportstats.com, formula1.com, pitwall.app, TracingInsights y Wikipedia.
- **Licencia:** código GPL-3.0, datos ODbL 1.0, pero el origen (StatsF1, Motorsport Stats) no es abierto y el propio autor advierte de bloqueos de IP de StatsF1. **Riesgo alto para redistribuir.**
- **Uso recomendado:** solo validación interna (no publicar sus datos): segunda fuente de lap chart 1950–1995 (hoy solo formula1db) y de paradas 1983–1993 (hoy ninguna).
- URL: <https://github.com/mclarenmp4-22/f1resultsdatabase>

Kaggle

- «Formula 1 World Championship (1950–2024)» de Rohan Rao: 14 CSV de Ergast, CC0. Redundante con Jolpica. <https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020>
- «Formula 1 Race Data» y «Formula 1 Race Events» (jtrotman): ver TracingInsights. <https://www.kaggle.com/datasets/jtrotman/formula-1-race-events>
- Hay decenas de conjuntos derivados (pit stops, etc.) sin valor añadido y con licencias dudosas.

Otros proyectos GitHub revisados

- `ekschallenberg/formula1project`: 589 081 vueltas 1996–2024 desde Ergast (redundante).
- `toUpperCase78/formula1-datasets`: CSV por temporada 2019–2026 (redundante).
- `harningle/fia-doc` (Apache-2.0): parsers de PDF FIA (clasificación, lap chart, history chart, lap analysis, paradas; probados en 2025+). Base para automatizar el arbitraje con la FIA.
- `f1dataR`, `LiveF1`: envoltorios de `FastF1`/`Jolpica`/live timing (no fuentes nuevas).
- Índice útil: <https://github.com/subinium/awesome-f1>

3.4.3 Fuentes oficiales (evidencia, no redistribución)

FIA — documentos y «Event & Timing Information»

- **Datos:** por GP, PDF de clasificación final/provisional, parrilla, vueltas rápidas, **Lap Chart**, **History Chart**, **Lap Analysis**, **Pit Stop Summary**, tiempos de libres y clasificación, **mejores sectores**, **speed trap**, **velocidades máximas**, decisiones de los comisarios (sanciones), notas del director de carrera. Páginas de timing verificadas para 2015; el archivo de clasificaciones cubre 2012–2025; el portal de documentos filtra 2019–2026 y 2015.
- **Acceso:** PDF (algunos escaneados → OCR). Sin API.
- **Licencia:** © FIA; términos genéricos. Ya acordado: **solo evidencia, no redistribuir**.
- **Fiabilidad:** máxima (árbitro en `revision_divergencias`).
- **Integración:** no en bronze público. Proceso interno de verificación por muestreo con `fia-doc`, guardando solo el veredicto (como `B*_*.csv`).
- **URLs:** <https://www.fia.com/documents/championships/fia-formula-one-world-championship-14> · <https://www.fia.com/events/fia-formula-one-world-championship/season-2015/event-timing-information-3> · <https://api.fia.com/f1-archives>

formula1.com — resultados

- **Datos:** resultados 1950+, vuelta rápida, parrilla, libres y clasificación; **Pit stop summary desde 1994** (parada, vuelta, hora, duración en pit lane, total). Premio DHL de vuelta rápida y de parada más rápida por temporada.
- **ToS:** © Formula One World Championship Ltd.; `robots.txt` no bloquea `/en/results`, pero los términos prohíben el uso automatizado que sobrecargue el servicio. Solo consulta puntual.
- **Contraste:** es el probable origen de las paradas de F1DB → no aporta independencia.
- **URL:** <https://www.formula1.com/en/results/1994/races/606/pacific/pit-stop-summary>

DHL Fastest Pit Stop Award

- Premio desde 2015; por GP, ranking de tiempos parados (equipo, piloto, vuelta). Sin API ni descarga (el espejo reutilizable es TracingInsights-Archive/PitStops 2018+).
- <https://inmotion.dhl/en/formula-1/fastest-pit-stop-award> · https://en.wikipedia.org/wiki/DHL_Fastest_Pit_Stop_Award

Pirelli — press area

- Notas de prensa por GP desde 2011 con los compuestos nominados (C1–C6 desde 2019; nombres comerciales 2011–2018). HTML, © Pirelli; se usa como referencia (tabla pequeña hecha a mano).
- **Contraste:** regla «el compuesto usado debe estar entre los nominados» para formula1db (2011+) y FastF1 (2018+); completa `int_tyre_compound_mapping` fuera de 2019–2023.
- Antes de 2011 (Bridgestone 1997–2010, Michelin 2001–2006, Goodyear hasta 1998) solo hay notas de prensa y artículos dispersos (RaceFans, Autosport); no hay neumático por vuelta en ninguna fuente abierta.
- <https://press.pirelli.com/?h=1&t=2024+Tyre+Compound+Choices> · <https://www.racefans.net/2009/02/26/tyre-compounds-for-first-five-races-of-2009/>

3.4.4 Sitios estadísticos (consulta manual; no scraping)

Fuente	Qué ofrece	Cobertura	Acceso / condiciones	Uso propuesto
StatsF1 (statsf1.com)	Inscritos, clasificación, parrilla, resultado, vueltas rápidas, «Tours en tête», «Tour par tour» (posición por vuelta)	1950+	HTML; robots permite *, pero bloquea clientes no navegador (página «Erreur»); datos ©	Árbitro manual (ya usado); comprobar lap charts pre-1996 por muestreo
formula1db.com	Resultados, lap chart, tiempos por vuelta, vueltas rápidas, estadísticas de paradas (1994+)	1950+	robots prohíbe todo salvo Google/Seobility y veta ClaudeBot; Cloudflare y login	Ya tenemos el CSV del TFG; no volver a scrapear
Forix / 8W (Autosport)	Resultados con diferencias por vuelta desde 1981 y tiempos completos desde 1982 (según usuarios del foro de Autosport)	1981+	Suscripción; hoy parte de Motorsport Stats	Solo si se obtiene licencia
Motorsport Stats	Resultados, timing points, datos en vivo; 14 000 eventos desde 1911	1911+	Comercial (API bajo contrato, info@motorsportstats.com)	Opción de pago para pre-1996
ChicaneF1	Resultados de campeonato 1950+ y 600 carreras no puntuables 1945–1983	1945+	HTML, © Jonathan Davies	Contraste de resultados y de carreras no puntuables
GP Encyclopedia (grandprix.com)	Crónicas y resultados históricos	1950+	HTML, ©	Contexto narrativo
F1-Fansite	Resultados, clasificación, parrillas, libres	1950+	HTML, «All rights reserved»	Nada nuevo
Motor Sport Magazine Database	Carreras desde 1894, crónicas y archivo de la revista	1894+	HTML, ©	Contexto; lap charts dispersos en crónicas
Racing-Reference	Resultados, vueltas lideradas; ID enlazado desde Wikidata (P6806)	1950+	403 a clientes automáticos	Contraste de vueltas lideradas
OldRacingCars	Inscripciones y resultados 1966–1985, archivo McKinney 1949–1953	1949–1985	HTML, «All rights reserved»	Coches compartidos y chasis (A1)

Fuente	Qué ofrece	Cobertura	Acceso / condiciones	Uso propuesto
teamdan.com	Referenciado como web de lap charts; hoy no responde (error TLS)	—	—	Descartado
Pitwall, LapF1, BigDataF1, F1TheData, F1 Tempo/GP Tempo	Visores sobre Ergast/Jolpica/FastF1 (vueltas 1996+, telemetría 2018+)	—	HTML	Sin datos nuevos
F1 Fantasy / F1 TV	Puntuaciones de juego / vídeo y timing con suscripción	—	Login; ToS restrictivos	Descartado
Sportradar, API-Sports	APIs comerciales (vueltas, paradas)	Modern	De pago	Descartado salvo presupuesto

URLs: <https://www.statsf1.com/fr/1962/france.aspx> · <https://www.statsf1.com/robots.txt> · <https://www.formula1db.com/robots.txt> · <https://8w.forix.com/> · <https://www.forix.com/> · <https://www.motorsportstats.com/> · <https://www.chicanef1.com/> · <https://www.grandprix.com/> · <https://www.f1-fansite.com/f1-results/> · <https://www.motorsportmagazine.com/database/series/f1/> · <https://www.racing-reference.info/> · <https://www.olderacingcars.com/f1/> · <https://pitwall.app/> · <https://www.lapf1.com/stats> · <https://www.bigdataf1.com/> · <https://www.f1-tempo.com/> · <https://developer.sportradar.com/racing/reference/f1-overview> · <https://forums.autosport.com/topic/191857-historical-pitstop-laptime-information-needed-for-exciting-project/>

3.4.5 Complementarias

Wikipedia / Wikidata

- Wikipedia: artículos por GP (clasificación, parrilla, vueltas lideradas, incidentes), listas «List of red-flagged Formula One races» y artículo «Safety car» (despliegues). CC BY-SA 4.0 → atribución y «compartir igual» en lo derivado.
- Wikidata: por GP solo ganador (P1346), pole (P3764), vuelta rápida (P5053), distancia (P3157), fecha e IDs (Racing-Reference P6806). CC0. Útil como tabla puente de identificadores.
- https://en.wikipedia.org/wiki/List_of_red-flagged_Formula_One_races · https://en.wikipedia.org/wiki/Safety_car · <https://www.wikidata.org/wiki/Q171864> · <https://query.wikidata.org/>

Open-Meteo Historical Weather (reanálisis ERA5)

- Temperatura, precipitación, viento... horarios desde 1940 en cualquier coordenada. API gratuita no comercial; datos CC BY 4.0. Resolución ~9–25 km: indica «lluvia probable», no estado de pista.
- Integración: `bronze/openmeteo/race_weather.parquet` por carrera (coordenadas de `circuit` de F1DB + hora de salida) → columna `weather_reanalysis_*` en `dim_race`, marcada como estimación. Contraste con `WeatherData` de FastF1/OpenF1 (2018+) y con el neumático «W/I» de formula1db (2011+).
- <https://open-meteo.com/en/docs/historical-weather-api>

3.5 Matriz hueco × fuente

Leyenda: C = cubre (integrable), V = sirve para validar/contrastar, M = solo consulta manual o evidencia, \$ = de pago, — = no aporta. Entre paréntesis, años.

Huevo	Jolpica	OpenF1	FastF1/LiveF1DB Timing	F1DB	FIA PDF	Tracing	Infra	Results	Stats	Forix/MO Stats	Otras
Tiempos por vuelta < 1990	—	—	—	—	—	—	—	—	—	\$ (1981/82)	Crónicas de época (M)
Tiempos 1990–1995 (solo formula1db)	—	—	—	—	—	—	—	—	—	\$ V	—
Tiempos 1996–2022 (3.ª fuente)	V (ya con Ergast)	—	V (2018+)	—	M	—	V	—	—	\$	—
Tiempos 2023–2026	C/V	C/V	C (ya)	—	M	V (espejo)	V	—	—	\$	—
Lap chart 1950–1995 (solo formula1db)	—	—	—	—	—	—	V	M (tour par tour)	—	\$	—
Paradas < 1994	—	—	—	—	—	—	V (1983+)	M	—	\$	—
Paradas: duración 2011+	C/V	C (2023+)	C (2018+)	V (pit lane 1994+)	M	—	V	—	—	—	formula1.com (M)
Tiempo parado en boxes	—	C (~2024+)	C (Pit Stop Series ~2024+)	—	M	C (DHL top, 2018+)	V (2018+)	—	—	—	DHL (M)
Neumáticos < 2011	—	—	—	—	—	—	—	—	—	—	Notas de prensa Bridgestone (M, solo nominación)
Neumáticos 2011–2017 (solo formula1db)	—	—	—	—	M	—	—	—	—	—	Pirelli nominaciones (V)
Stints 2018+	—	C/V (2023+)	C (ya)	—	M	V	V	—	—	—	Pirelli (V)
Clasificación Q1–Q3	V (1994+; no independiente de FastF1)	C/V (2023+)	C (vía Jolpica)	C (ya)	M	—	V	M	—	—	—
Libres (vueltas)	—	C (2023+)	C (2018+)	resultado 1986+	M	C (espejo)	V	M	—	—	—
Race control (banderas, SC/VSC, rojas)	—	C (2023+)	C (2018+)	—	M	C (rcm. json)	V	—	—	—	—
SC/VSC/bandera-roja histórico	—	—	—	—	—	C (1950/1973)	V (SC 2015+)	M	—	—	Wikipedia (V)
Sanciones	—	C (mensajes 2023+)	C (mensajes 2018+)	parcial (penalización de tiempo/parrilla)	M (decisiones)	—	V	—	—	—	—
Meteorología	—	C (2023+)	C (2018+)	—	M	C (espejo)	V	—	—	—	Open-Meteo C (1940+, estimada)
Sectores y speed traps	—	C (2023+)	C (ya 2018+)	—	M (mejores sectores, speed trap)	—	V	—	—	—	—
Sprint: resultados	V (2021+)	V (2023+)	C	C (ya)	M	—	V	M	—	—	—
Sprint: vueltas	—	C (2023+)	C (2021+)	—	M	C (espejo)	V	—	—	—	—
Telemetría	—	C (2023+)	C (ya 2024–2026 quali)	—	—	C (espejo)	—	—	—	—	—

3.6 Propuesta priorizada

#	Acción	Cubre / contrasta	Esfuerzo	Riesgos
0	Añadir LICENSE + sección de atribuciones (F1DB CC BY 4.0, Jolpica/OpenF1 CC BY-NC-SA, FastF1 MIT, datos © F1/FIA) en README, web y API	Requisito legal	0,5 días	Integrar NC-SA obliga a publicar los derivados como no comerciales y con la misma licencia
1	FastF1 ampliado: <code>messages=True</code> , <code>weather=True</code> , sesiones S/SQ/SS/FP1-FP3 en el loader existente	Race control, meteo, sprint (vueltas), libres 2018+	1-2 días (+ tiempo de backfill por el límite de 500 peticiones/h)	Más volumen en bronze; Australia 2018 sin live timing
2	Jolpica sustituye al volcado Ergast congelado: descarga semanal del volcado <i>delayed</i>	3.ª fuente de vueltas y paradas 2023-2026; Q1-Q3; sprint	1-2 días (mismo esquema que Ergast)	NC-SA; retraso de 14 días; sin SLA
3	Eventos históricos (SC/VSC/rojas) desde TracingInsights RaceData como seed	SC 1973+, rojas 1950+, VSC 2015+; validar Track Status y vueltas lentas	1 día (resolver nombres de carrera → <code>race_id</code>)	Cadena de licencias desde Wikipedia (CC BY-SA): atribuir
4	OpenF1 para 2023+: <code>race_control</code> , <code>pit</code> (tiempo parado), <code>stints</code> , <code>weather</code> , <code>overtakes</code>	Tiempo parado 2024+, adelantamientos, contraste con FastF1	2-3 días (paginación por sesión, 30 req/min)	NC-SA; huecos puntuales (404)
5	Tiempo parado DHL 2018-2023 desde TracingInsights-Archive/PitStops	Tiempo parado (solo mejores paradas por GP)	0,5-1 día	Cobertura parcial (ranking, no todas las paradas)
6	Nominaciones Pirelli 2011+ como seed hecho a mano + test dbt «compuesto € nominados»	Validar neumáticos formula1db 2011-2017 y FastF1	2 días (~300 GP)	Trabajo manual; nomenclatura pre-2019
7	Meteo estimada con Open-Meteo para 1950-2017	Condición de carrera histórica	1 día	Resolución gruesa: etiquetar como estimación
8	Validación interna del lap chart 1950-1995 y paradas 1983-1993 contra <code>f1resultsdatabase</code> (sin publicar sus datos)	Primer contraste pre-1996	2-3 días	Licencia dudosa (StatsF1/Motorsport Stats): solo informes de diferencias; requiere permiso para descargar 74 MB
9	Verificación por muestreo con PDF FIA + <code>fia-doc</code> (2012+)	Árbitro automatizado de discrepancias	3-5 días	PDF no redistribuibles; OCR en escaneados
10	Tiempos por vuelta pre-1990	Sin fuente abierta: solo Forix/Motorsport Stats bajo licencia	—	Coste; fuera del alcance de un TFG

Orden recomendado: 0 → 1 → 2 → 3 (una semana de trabajo cubre race control, meteo, sprint, eventos históricos y la 3.ª fuente 2023+), después 4-7 y, si interesa la historia anterior a 1996, 8-9. El punto 10 se documenta como limitación conocida.

3.6.1 Correspondencia con los huecos de la nota 01

Qué fuente verificada aquí resuelve cada hueco de `01_inventario_huecos.md` (sustituye las «hipótesis» de aquella nota por lo comprobado).

Hueco (nota 01)	Fuente recomendada (verificada)	Acción de la tabla 6
H1 Vueltas de sprint	FastF1 sesión S (2021+); OpenF1 laps (2023+, probado en China 2024: 19 vueltas)	1 (y 4 para contraste)
H2 Banderas rojas	TracingInsights red_flags.csv (1950+, vuelta y reanudación); FastF1/OpenF1 race control (2018+/2023+); Wikipedia	3 y 1
H3 2025–2026 fuente única	Jolpica /laps y /pitstops (2026 R5 ya disponible); OpenF1 laps/stints	2 y 4
H4 Compuesto C1–C6 2024–2026	Pirelli press area (nominación por GP)	6
H6 Tiempos 1990 y Hungría 1993	Sin fuente abierta (Jolpica empieza en 1996); solo Forix/Motorsport Stats bajo licencia o PDF FOCA/FIA de época	10 (limitación)
H7 Numeración 1994/2001	Arbitraje manual FIA/StatsF1; f1resultsdatabase como segunda opinión interna	8
M3 Meteorología	FastF1 weather=True (2018+); OpenF1 weather (2023+); Open-Meteo ERA5 (1940+, probado)	1 y 7
M4 Race control y sanciones	FastF1 messages=True (2018+); OpenF1 race_control (2023+); decisiones FIA (evidencia)	1, 4 y 9
M5 Neumáticos antes de 2011	Ninguna fuente abierta con neumático por vuelta; solo nominaciones Bridgestone en prensa	—
M7 Validación 1990–1995	Ninguna fuente abierta de tiempos; posiciones contra f1resultsdatabase/StatsF1	8
M9 Paradas Brasil/Mónaco 1994, Italia 1997	formula1.com «Pit stop summary» existe para 1994 (consulta manual puntual)	manual
M10 Tiempo parado	OpenF1 stop_duration y PitStop Series (~2024+); DHL vía TracingInsights-Archive/PitStops (2018+, solo mejores)	4 y 5
B1 Tiempos 1950–1989	Sin fuente abierta	10
B2–B5 Lap chart, coches compartidos y vuelta rápida en los 50–70	StatsF1 (tour-par-tour, meilleur-tour) manual; f1resultsdatabase para validar	8
B6 Hora de salida	Wikipedia; sessions de Jolpica (hora en carreras recientes)	manual

3.7 Notas de cumplimiento

- formula1db.com: no se ha accedido (robots.txt veta ClaudeBot; Cloudflare/login). El CSV del TFG sigue siendo la única copia y no puede regenerarse: documentarlo como fuente congelada.
- StatsF1, Racing-Reference y los foros de Autosport bloquean clientes automáticos: solo consulta manual puntual; no se ha intentado sortear el bloqueo.
- PDF de la FIA: solo como evidencia; no se suben al repositorio ni a releases.
- Volcados (Jolpica 14,5 MB, f1resultsdatabase 74 MB) no descargados: requieren permiso explícito.

CAPÍTULO 4

Vueltas de las carreras al sprint

Estudio de viabilidad con pruebas reales (29 sprints con FastF1, 7 con OpenF1, Jolpica, FIA). Fecha: 2026-09-29.
No se ha modificado código del repo; prototipos en el scratchpad.

4.1 Conclusiones

1. **Viable y barato:** FastF1 (`get_session(año, ronda, 'Sprint')`) tiene el 100 % de las vueltas oficiales de los 29 sprints disputados (11 319 vueltas; 568/590 pilotos exactos y el resto son la vuelta de abandono, 1 DNS y 1 DSQ), con posición, compuesto, stint, boxes y estado de pista (SC/VSC/roja). Cubre los tres formatos (2021-22, 2023 Shootout, 2024+).
2. **Hueco sistemático: la vuelta 1** no tiene tiempo en FastF1 para los sprints (sí en las carreras largas). Se reconstruye con exactitud (verificada contra la FIA) desde el total oficial de F1DB, o con un desfase de salida constante por sesión para el resto de pilotos.
3. **OpenF1** (solo 2023+) coincide al milisegundo con FastF1 y rellena la v1 y las vueltas bajo SC sin tiempo, pero no es validación independiente (misma fuente Live Timing). **Jolpica** da la vuelta rápida por piloto de los 29 sprints: es la evidencia independiente (F1DB no tiene ni vuelta rápida ni paradas de sprint). formula1db.com no sirve (el TFG solo extrajo carreras largas y el sitio bloquea).
4. **Defecto actual** (independiente de las vueltas): `dim_race.has_sprint` es falso en 2021-2023 porque F1DB no trae fechas de sesión; la web oculta 12 sprints ya cargados (6 de 2021-2022 y 6 de 2023) en el selector.
5. **Diseño recomendado:** tipo `bronze sprint_laps`, `int_sprint_laptimes` separado de `int_laptimes` y `session_type` en `fact_laptimes/fact_pit_lane_passes`; parámetro `?session=sprint` en `/laps`, `/stints`, `/pit-lane-passes`, y `SessionSwitch` en las pestañas vuelta a vuelta, neumáticos y ritmo. Esfuerzo: 6-8,5 días (mínimo viable ~4).

4.2 Situación actual (revisión de código)

- F1DB ya aporta resultados de sprint (`SPRINT_RACE_RESULT`) y de clasificación sprint (`SPRINT_QUALIFYING_RESULT`): `fact_race_result.session_type in ('RACE', 'SPRINT')`, `fact_qualifying_result` con `SPRINT_QUALIFYING`, `dim_race.has_sprint`, `dim_race.sprint_qualifying_format`, `agg_driver_career.sprint_wins`.
- API: `/races/{id}/results?session=race|sprint` y `/races/{id}/qualifying?session=qualifying|sprint_qualifying`. Web: `session-switch.tsx` en resultados y clasificación.
- Vueltas: `ingestion/fastf1_loader.py` solo carga `get_session("R")` y "Q"; ningún sprint en bronze ni en la caché `data/cache/fastf1` (0 carpetas de sesión Sprint).
- `ergast_loader.py` usa el volcado CSV de oct-2022 (incluye `sprint_results.csv`, sin vueltas de sprint); no se ingiere `sprint_results`.

- `int_laptimes`, `int_lap_completeness`, `fact_laptimes`, `fact_pit_lane_passes` filtran `session_type = 'RACE_RESULT'` y claves (`race_id`, `driver`, `lap`): sin dimensión de sesión.

4.3 F1DB (release v2026.15.1) — medido en bronze

- 29 fines de semana con `SPRINT_RACE_RESULT` (2021: 3, 2022: 3, 2023: 6, 2024: 6, 2025: 6, 2026: 5 disputados + Singapur 10-oct pendiente, ya en el calendario sin resultados).
- `SPRINT_QUALIFYING_RESULT` solo 2023+ (23 fines de semana): en 2021-2022 la parrilla del sprint salía de la clasificación del viernes (`QUALIFYING_RESULT`).
- `SPRINT_STARTING_GRID_POSITION` 29. NO hay `FASTEST_LAP` ni `PIT_STOP` de sprint: la validación de numeración (`int_lap_numbering`) y el tipado de pasos por boxes no tienen referencia F1DB.
- `race_laps` del sprint = vueltas oficiales por piloto (referencia de completitud).
- **Defecto detectado:** `stg_f1db__races.has_sprint = sprint_race_date is not null`, pero F1DB no trae fechas de sesión en 2021-2023 → `dim_race.has_sprint = 0` en 2021-2023 (12 sprints con resultados en `fact_race_result` que la web no ofrece en el selector). Arreglo: derivar `has_sprint` de la existencia de `SPRINT_RACE_RESULT`.

4.4 FastF1 3.8.3 — prueba real (caché propia en el scratchpad)

Calendario (`get_event_schedule`, `EventFormat`): 30 fines de semana con sprint 2021-2026.

- 2021-2022 sprint: sesiones FP1 | Qualifying | FP2 | **Sprint** | Race (FastF1 normaliza el nombre oficial de 2021 «Sprint Qualifying» a **Sprint**).
- 2023 sprint_shootout: FP1 | Qualifying | **Sprint Shootout** | Sprint | Race.
- 2024+ sprint_qualifying: FP1 | **Sprint Qualifying** | Sprint | Qualifying | Race.
- `get_session(año, ronda, 'Sprint')` funciona para todos; la carrera sigue en `Session5`.

Primeras mediciones (vueltas FastF1 vs `race_laps` de `SPRINT_RACE_RESULT`):

4.4.1 Completitud FastF1 (29 sprints disputados, 2021-R10 → 2026-R12; medido)

Sprint	F1DB vueltas	FastF1 vueltas	Pilotos OK/total	Vuelta abandono (+1)	Tiempos nulos (v1 / resto)	SC / VSC / roja (vueltas-piloto)	Pit-in
2021 Gran Bretaña	339	339	20/20	0	20 / 0	0/0/0	1
2021 Italia	342	343	19/20	1	20 / 21	58/0/0	1
2021 São Paulo	480	480	20/20	0	20 / 0	0/0/0	0
2022 Emilia-Romaña	399	400	19/20	1	20 / 0	77/0/0	1
2022 Austria	435	436	19/20	0 (Alonso DNS con 1 vuelta en FastF1)	20 / 0	0/0/0	1
2022 São Paulo	468	469	19/20	1	20 / 1	0/0/0	1
2023 Azerbaiyán	308	308	20/20	0	19 / 54	55/38/0	5
2023 Austria	480	480	20/20	0	20 / 0	0/0/0	11
2023 Bélgica	208	209	19/20	1	20 / 39	58/0/0	11
2023 Qatar	318	322	16/20	4	20 / 16	175/0/0	4
2023 EE. UU.	377	377	20/20	0	20 / 0	0/0/0	1
2023 São Paulo	480	480	20/20	0	20 / 0	0/0/0	0
2024 China	378	378	20/20	0	20 / 0	0/0/0	2

Sprint	F1DB vueltas	FastF1 vueltas	Pilotos OK/total	Vuelta abandono (+1)	Tiempos nulos (v1 / resto)	SC / VSC / roja (vueltas-piloto)	Pit-in
2024 Miami	343	344	19/20	1	20 / 23	56/0/0	38
2024 Austria	460	460	20/20	0	20 / 0	0/0/0	0
2024 EE. UU.	380	380	20/20	0	20 / 0	0/0/0	0
2024 São Paulo	475	476	19/20	1	20 / 1	0/46/0	0
2024 Qatar	380	380	20/20	0	20 / 0 (12 vueltas sin compuesto)	0/0/0	2
2025 China	380	380	20/20	0	20 / 0	0/0/0	1
2025 Miami	331	331	20/20	0	19 / 32	84/0/0	21
2025 Bélgica	297	297	20/20	0	20 / 0	0/0/0	1
2025 EE. UU.	315	320	15/20	5	20 / 40	150/0/0	3
2025 São Paulo	441	444	17/20	3	20 / 23	37/0/19	20
2025 Qatar	380	380	20/20	0	20 / 0	0/0/0	3
2026 China	396	397	21/22	1	22 / 13	78/0/0	14
2026 Miami	361	380	21/22	0 (Bortoleto DSQ: 19 vueltas, F1DB race_laps nulo)	20 / 0	0/0/0	1
2026 Canadá	490	491	21/22	1	22 / 2	0/0/0	7
2026 Gran Bretaña	372	372	22/22	0	22 / 0	0/0/0	2
2026 Países Bajos	506	506	22/22	0	22 / 1	0/0/0	2
Total	11 319	11 359	568/590 exactos	22			

Lectura:

- **Cobertura 100 %:** en los 29 sprints cada piloto tiene todas sus vueltas oficiales, sin huecos (`count = max(lap)`). Las 22 diferencias son la vuelta de abandono (+1, mismo criterio que `int_lap_completeness`), un DNS con vuelta registrada y un DSQ sin `race_laps` en F1DB.
- **Vuelta 1 sin tiempo SIEMPRE** en sprint (FastF1 no la calcula en sprints; en carreras largas 2023-26 solo 31 de ~95 000 vueltas tienen la v1 nula). Tampoco sectores de la v1.
- Tiempos nulos adicionales bajo SC/VSC/bandera roja (igual que en carreras largas: 1 013 de las 1 363 nulas de 2023-26 son con SC).
- Compuestos: nombres relativos (SOFT/MEDIUM/HARD/INTERMEDIATE/WET); 1 sprint con 12 vueltas sin compuesto (2024 Qatar). Posición nula solo en la vuelta de abandono. `TrackStatus` completo.
- F1DB no trae paradas del sprint, pero FastF1 sí registra entradas a boxes (p. ej. 2023 Austria: 11 cambios de intermedios a secos; 2024 Miami: 38 pasos bajo SC).

4.4.2 Vuelta 1 del sprint: reconstruible con exactitud

FastF1 no calcula la v1 en sprints, pero conserva `LapStartTime` y `Time` de la v1. Medido:

- `race_time_millis` (F1DB) – Σ vueltas 2..n (FastF1) reproduce **exactamente** la v1 oficial de la FIA *Sprint History Chart* (Imola 2022: LEC 1:31.400, VER 1:34.500, NOR 1:38.060).
- $(\text{Time} - \text{LapStartTime})$ de la v1 excede la oficial en un **desfase constante dentro de cada sesión** (160-286 ms según el sprint). Restando `race_time_penalty_millis` (las penalizaciones de 5/10 s van incluidas en `race_time_millis`), el rango intra-sesión es 0 ms en 22 de los 24 sprints calibrables; 38 ms en 2026 China y 5 000 ms en 2026 Miami (una penalización de 5 s que F1DB no refleja en `race_time_penalty_millis`: la mediana lo absorbe, y el caso sirve de alerta). Así se reconstruye la v1 de todos los pilotos, incluidos doblados y abandonos. En 5 sprints con SC y tiempos nulos (2021 Italia, 2023 Bakú y Bélgica, 2025 Miami y São Paulo) no hay pilotos calibradores: usar la v1 de OpenF1/FIA o el desfase típico (~260 ms) marcando `timing_convention`.

4.5 OpenF1 (api.openf1.org, sin clave) — prueba real

- Sesiones: `session_name` = Sprint (type Race) y Sprint Qualifying (type Qualifying; normaliza el «Sprint Shootout» de 2023). **Solo 2023+:** 6 sprints/año 2023-2026 (24 sesiones, Singapur 2026 incluida con `session_key` 11383 aún sin datos). No cubre 2021-2022.
- Endpoints probados por sesión: `laps`, `stints`, `pit`, `race_control`, `position`, `session_result` (todos responden; ~6 peticiones/sprint, limitar a ~2 req/s).

Validación cruzada FastF1 ↔ OpenF1 (7 sprints: 2023 Austria y Qatar, 2024 China, 2025 Bélgica y EE. UU., 2026 China y Gran Bretaña):

Sprint	Vueltas FastF1	Emparejadas	Con tiempo en ambas	Iguales ±1 ms	Distintas	Solo OpenF1 tiene tiempo (de ellas v1)	Compuesto igual*
2023 Austria	480	480	460	460	0	20 (20)	480/480
2023 Qatar	322	322	286	286	0	32 (19)	318/318
2024 China	378	378	358	358	0	20 (20)	378/378
2025 Bélgica	297	297	277	277	0	19 (19)	297/297
2025 EE. UU.	320	320	260	260	0	55 (17)	317/318
2026 China	397	397	362	362	0	34 (22)	240/240
2026 Gran Bretaña	372	372	350	350	0	22 (22)	356/356

* sobre las vueltas cubiertas por algún stint de OpenF1: **0 contradicciones de compuesto**, pero OpenF1 omite stints (2026 China: le faltan los primeros stints de 157 vueltas) y numera algún stint distinto (2025 Bélgica, 11 vueltas).

Conclusiones OpenF1: mismos datos de cronometraje que FastF1 (100 % idénticos cuando ambos tienen tiempo) → sirve para **rellenar** la v1 y las vueltas bajo SC que FastF1 deja sin tiempo, no como validación independiente (misma fuente F1 Live Timing). Su v1 es ~0,35-0,44 s más larga que la oficial (mide desde otra referencia): marcarla `timing_convention`, preferir la reconstrucción.

4.6 Jolpica-F1 / Ergast

- <https://api.jolpi.ca/ergast/f1/{año}/sprint/> → resultados de los 29 sprints (2021-2026), y **por piloto FastestLap (vuelta y tiempo)**. No existen `/año/{ronda}/sprint/laps` ni `/sprint/pitstops` (404); `/año/{ronda}/laps` son solo de la carrera larga.
- Volcado CSV Ergast (oct-2022) del repo: `sprint_results.csv` (100 filas = 5 sprints 2021-22) con `fastestLap/fastestLapTime`; sin vueltas de sprint.
- Uso propuesto: la vuelta rápida por piloto de Jolpica es la **evidencia independiente** para validar tiempo y numeración (sustituye a `FASTEST_LAP` de F1DB, que no existe para sprints).

4.7 FIA, formula1.com y formula1db.com

- FIA publica por sprint PDFs con patrón `fia.com/sites/default/files/{año}_{ronda}_{gp}_f1_s0_timing_sprint{historychart|lapchart|...}_v01.pdf` (comprobados: 2022_04_ita *Sprint History Chart*, 5 páginas con vuelta, piloto, gap, tiempo y PIT; 2026_05_can *Sprint Lap Chart*). Es la **referencia oficial** (incluye la v1), pero son PDF de maquetación variable y con aviso de copyright restrictivo: útil para verificar casos puntuales (seed de correcciones con evidencia), no como fuente masiva.
- formula1.com: resultados, parrilla y clasificación sprint; sin vuelta a vuelta del sprint.
- formula1db.com: el scraping del TFG solo recorrió `/results/race` (no hay vueltas de sprint en `f1_lap_times.csv`); la web responde 403 y robots.txt lo prohíbe → **no es fuente para sprints**.

4.8 Clasificación sprint (Shootout 2023 / Sprint Qualifying 2024+)

- FastF1: `get_session(a, r, 'Sprint Shootout')` (2023) y `'Sprint Qualifying'` (2024+) cargan vueltas (2023 Austria 355, 2024 China 218, 2025 Bélgica 185), pero **results** llega **sin posición ni SQ1/SQ2/SQ3** (FastF1 los toma de Ergast/Jolpica, que no los tiene) → los resultados siguen viniendo de F1DB (`SPRINT_QUALIFYING_RESULT`, ya en `fact_qualifying_result`).
- Trampa de alias:** 'SQ' en 2021-2022 devuelve la *carrera* sprint (se llamaba «Sprint Qualifying») y en 2023 falla (allí es 'SS'). Usar el nombre completo según `EventFormat`.
- Aporte útil: telemetría de la vuelta rápida de SQ (como `quali_telemetry`), no vueltas de carrera.

4.9 Catálogo de sprints y cobertura esperada por fuente

Formatos: **A** = 2021-2022 (la clasificación del viernes fija la parrilla del sprint; el sprint se llamó «Sprint Qualifying» en 2021 y «Sprint» en 2022); **B** = 2023 (Qualifying del viernes para el GP + *Sprint Shootout* el sábado); **C** = 2024+ (*Sprint Qualifying* el viernes; sprint y Qualifying el sábado). En 2021-2026 no se canceló ningún sprint ya disputable (Imola 2023, que iba a tener sprint, se canceló entero por las inundaciones y no figura en F1DB).

Año	Ronda / GP (F1DB race_id)	Fmt	F1DB res.	FastF1 vueltas	OpenF1	Jolpica res. + VR	Ergast CSV	FIA PDF
2021	10 Gran Bretaña (1045), 14 Italia (1049), 19 São Paulo (1054)	A	✓	✓ medido	✗	✓	✓	✓
2022	4 Emilia-Romaña (1061), 11 Austria (1068), 21 São Paulo (1078)	A	✓	✓ medido	✗	✓	✓ (Imola, Austria)	✓ (Imola comprobado)
2023	4 Azerbaiyán (1083), 9 Austria (1088), 12 Bélgica (1091), 17 Qatar (1096), 18 EE. UU. (1097), 20 São Paulo (1099)	B	✓ + SQ	✓ medido	✓	✓	✗	✓
2024	5 China (1106), 6 Miami (1107), 11 Austria (1112), 19 EE. UU. (1120), 21 São Paulo (1122), 23 Qatar (1124)	C	✓ + SQ	✓ medido	✓	✓	✗	✓
2025	2 China (1127), 6 Miami (1131), 13 Bélgica (1138), 19 EE. UU. (1144), 21 São Paulo (1146), 23 Qatar (1148)	C	✓ + SQ	✓ medido	✓	✓	✗	✓
2026	2 China (1151), 4 Miami (1153), 5 Canadá (1154), 9 Gran Bretaña (1158), 12 Países Bajos (1161)	C	✓ + SQ	✓ medido	✓	✓	✗	✓ (Canadá comprobado)
2026	17 Singapur (1166), 10-oct	C	pendiente	pendiente	sesión creada	pendiente	✗	pendiente

Totales: 29 sprints disputados = **11 319 vueltas oficiales** (F1DB) / 11 359 registros FastF1 ($\approx +1\%$ de filas sobre `fact_laptimes`). OpenF1 medido en 7 de sus 23 sprints disputados; Jolpica, resultados de los 29.

Problemas identificados:

- Nombres de sesión cambiantes: resolver siempre por `EventFormat` + nombre completo, nunca por alias (SQ es ambiguo entre años).
- v1 sin tiempo en FastF1 (reconstruible) y tiempos nulos bajo SC (rellenables con OpenF1 2023+).
- F1DB no tiene paradas ni vuelta rápida del sprint → `int_lap_numbering`, `fact_pit_stops`, `fact_pit_lane_passes` (`pass_type='pit_stop'`) y `qa_fastest_laps` no tienen referencia; usar la VR de Jolpica y tipificar los pasos por boxes por SC/retirada/otros.
- `dim_race.has_sprint` falso en 2021-2023 (defecto actual, independiente de las vueltas).
- DSQ con `race_laps` nulo (Bortoleto, 2026 Miami) y DNS con 1 vuelta en FastF1 (Alonso, 2022 Austria): conciliarlos como hace `int_lap_completeness` (`disqualified`, DNS excluidos).

6. Compuesto Pirelli (C1-C6): `int_tyre_compound_mapping` depende de `formula1db` (carreras largas hasta 2023); para el sprint se reutiliza el mapeo de la carrera del mismo fin de semana (misma asignación de neumáticos). En 2024+ tampoco existe para la carrera: sin cambio.
7. Carga incremental: `completed_races` mira `Session5DateUtc` (carrera); el sprint está disponible un día antes (Singapur 2026 el 10-oct). Usar la fecha de la sesión «Sprint».

4.10 Diseño propuesto

4.10.1 Ingesta

- `fastf1_loader.py`: nuevo tipo `sprint_laps` (`bronze/fastf1/sprint_laps/season=YYYY/round=RR.parquet`, con columna `session='SPRINT'`) para los eventos con 'sprint' in `EventFormat`, sesión 'Sprint'. Se prefiere un tipo propio a `laps/season=YYYY/session=S/...` porque la fuente dbt actual lee `fastf1/laps/*/*.parquet`: hoy no lo recogería, pero un cambio futuro del patrón a ** mezclaría sprint y carrera sin avisar. Opcional: `sprint_quali_telemetry` (sesiones «Sprint Shoo-tout»/«Sprint Qualifying», mismo patrón que `quali_telemetry`).
- Nuevo `openf1_loader.py` (2023+): `laps`, `stints`, `pit`, `race_control` de las sesiones Sprint → `bronze/openf1/{tipo}/season=YYYY/round=RR.parquet`; `session_key` resuelto con `/sessions?year=...` (filtrar `session_name == 'Sprint'`) y casado con F1DB por fecha y país.
- Jolpica: `jolpica_loader.py` con `/{{año}}/sprint/` (resultados + `FastestLap`), paginando (`limit=100`, `offset`). Sirve además para contrastar los resultados de sprint de F1DB.

4.10.2 dbt

- Staging: `stg_fastf1__sprint_laps` (igual que `stg_fastf1__laps` + `lap_start_time_ms`), `stg_openf1__laps`, `stg_jolpica__sprint_results`.
- `int_fastf1_sprint_laps` (prototipo en el scratchpad): claves F1DB + reconstrucción de la v1.
- `int_sprint_laptimes`: FastF1 principal; OpenF1 rellena tiempos nulos (`source='openf1'`); `validation_status` con la misma semántica que `int_laptimes`, más un valor nuevo:
 - `confirmed`: FastF1 = OpenF1 ±1 ms (documentar que comparten la fuente primaria) o = VR de Jolpica;
 - `derived (nuevo)`: v1 reconstruida desde el total oficial de F1DB;
 - `timing_convention`: v1 de OpenF1 o por desfase de sesión;
 - `disputed`: la VR de Jolpica no coincide con ninguna vuelta del piloto;
 - `single_source`: resto. Numeración: evidencia = VR de Jolpica (±1 ms) en lugar de `FASTEST_LAP` de F1DB.
- **Hecho**: añadir `session_type` (RACE/SPRINT) a `fact_laptimes` (union de `int_laptimes`, sin tocar, + `int_sprint_laptimes`) y a `fact_pit_lane_passes`; clave única (`race_id`, `session_type`, `driver_id`, `driver_number`, `lap_number`). Se elige columna en vez de hecho separado porque la API y la web ya tratan la sesión como parámetro (`fact_race_result` hace lo mismo) y las consultas de vueltas, stints y ritmo se reutilizan tal cual. Contrapartida: todo consumidor debe filtrar la sesión: 3 consultas de `api/app/routers/races.py` (`laps`, `stints`, `pit-lane-passes`), `qa_fastest_laps`, `qa_pit_stops`, `qa_summary` e `int_lap_completeness`. Alternativa de menor riesgo: `fact_sprint_laptimes` separado.
- `int_lap_completeness` parametrizado por sesión (`SPRINT_RACE_RESULT`) → `quality_status`.
- `stg_f1db__races.has_sprint`: derivarlo de la existencia de `SPRINT_RACE_RESULT` (arreglo inmediato, útil aunque no se haga lo demás).

- **Tests:** `unique_combination` con `session_type`; `accepted_values` de `session_type`; singular `assert_sprint_laps_match_f1db` (máx. vuelta = `race_laps` salvo vuelta de abandono/DSQ/DNS); `assert_sprint_lap_sum_matches_total` (Σ vueltas = `race_time` - penalización ± 1 ms para los clasificados en la vuelta del líder); umbral de sprint en `assert_quality_thresholds`; `qa_sprint_fastest_laps` (VR Jolpica frente a vueltas).

4.10.3 API

- GET `/races/{id}/laps?session=race|sprint, /stints?session=..., /pit-lane-passes?session=...` (Literal["race", "sprint"] = "race", como `/results`); lista vacía si no hay sprint. `/pitstops` sigue siendo solo de carrera (F1DB no tiene paradas de sprint). Regenerar `web/src/lib/api/openapi.json` y `schema.d.ts`; fixtures de `api/tests` con un sprint.

4.10.4 Web

- Pestañas `lap-chart`, `tyres`, `pace` (y `pitstops` para los pasos por boxes): `SessionSwitch` `carra/sprint` cuando `race.has_sprint`, con `?session=sprint` (sin JS, como en resultados). `getDriverNames(raceId, session)` debe usar `getResults(raceId, "sprint")` para el orden. Textos en `dictionaries/{es,en}.json` y aviso de «vuelta 1 derivada».

4.10.5 Esfuerzo estimado

Bloque	Días
Loader FastF1 sprint + backfill de 29 sprints (~1-2 min/sprint con caché fría)	0,5-1
Loaders OpenF1 (2023+) y Jolpica sprint	1
dbt: staging, <code>int_fastf1_sprint_laps</code> (v1), <code>int_sprint_laptimes</code> , <code>session_type</code> en hechos, <code>has_sprint</code>	2-3
Tests dbt + QA	1
API (3 endpoints + tests + OpenAPI)	0,5-1
Web (selector en 3-4 pestañas, i18n, e2e Playwright)	1-1,5
Total	6-8,5 (≈ 1,5 semanas)

Versión mínima (solo FastF1 + v1 derivada + `session_type` + API/web, sin OpenF1/Jolpica): ~4 días.

4.11 Riesgos

- **Regresión por cambio de grano** de `fact_laptimes`: una consulta sin filtro de sesión mezcla sprint y carrera (vueltas 1..24 duplicadas, ritmo sesgado). Mitigar con test de grano y revisando los consumidores listados.
- **Validación poco independiente:** FastF1 y OpenF1 salen del mismo F1 Live Timing; lo independiente es la VR de Jolpica y el total de F1DB. La FIA (PDF), solo para casos puntuales.
- Dependencia de APIs de terceros (Live Timing vía FastF1, límites de OpenF1, Jolpica ~200 peticiones/hora). Mitigan la caché y el bronze por sprint.
- v1 no calibrable en 5 sprints con SC y tiempos nulos → `timing_convention`.
- F1DB puede no reflejar la penalización en `race_time_penalty_millis` (2026 Miami): la v1 derivada saldría 5 s desplazada. Mitigar con la mediana del desfase y un test de rango (v1 entre $0,9\times$ y $1,6\times$ la v2 del piloto) que dispare el uso del desfase.
- Licencia: los PDF de la FIA prohíben la reproducción sin permiso; no redistribuirlos.

4.12 Prototipo (scratchpad ../730e2b1e-.../scratchpad/sprint/)

- `schedule.py` → `sprint_schedule_fastf1.csv` (30 fines de semana con sprint 2021-2026).
- `ff1_sprint_probe.py` → carga `get_session(a, r, 'Sprint')`, escribe `bronze_proto/fastf1/laps/season=YYYY/session=S/round=RR.parquet` y mide frente a F1DB.
- `measure_all.py` → `coverage_fastf1_sprint.csv` (tabla de completitud); `anomalies.py`.
- `openf1_sessions.py`, `openf1_probe.py` → `openf1_sprint_sessions.csv` y `bronze OpenF1`.
- `compare_ff1_openf1.py` (validación cruzada), `jolpica_probe.py`, `sq_probe.py`, `lap1_derivation.py` (calibración de la v1), `fia_2022_04_ita_sprinthistorychart.txt`.
- `proposal/fastf1_loader_sprint.py` y `proposal/int_fastf1_sprint_laps.sql` (código propuesto).

Código clave (reconstrucción de la v1, del prototipo dbt):

```
session_offset as (
  select race_id, median(lap1_from_start_ms - (race_time_ms - rest_ms)) as start_offset_ms
  from per_driver
  -- race_time_ms = race_time_millis -
  ↪ race_time_penalty_millis
  where race_time_ms is not null and rest_laps = race_laps - 1
  group by race_id
)
-- v1 = race_time_ms - suma(v2..n) si el piloto tiene total oficial y vueltas 2..n completas
-- = (Time - LapStartTime de la v1) - start_offset_ms en otro caso
```

Carga de la sesión (loader propuesto):

```
def _load_sprint(event, season):
    session = event.get_session("Sprint") # válido en los tres formatos 2021+
    session.load(laps=True, telemetry=False, weather=False, messages=False)
    laps = _with_keys(timedeltas_to_millis(pd.DataFrame(session.laps)), event, season)
    laps.insert(3, "session", "SPRINT")
    write_parquet(laps, race_path("sprint_laps", season, int(event["RoundNumber"])))
```

Parte II

Arquitectura y estructura del proyecto

Síntesis de la parte II

Arquitectura desplegada

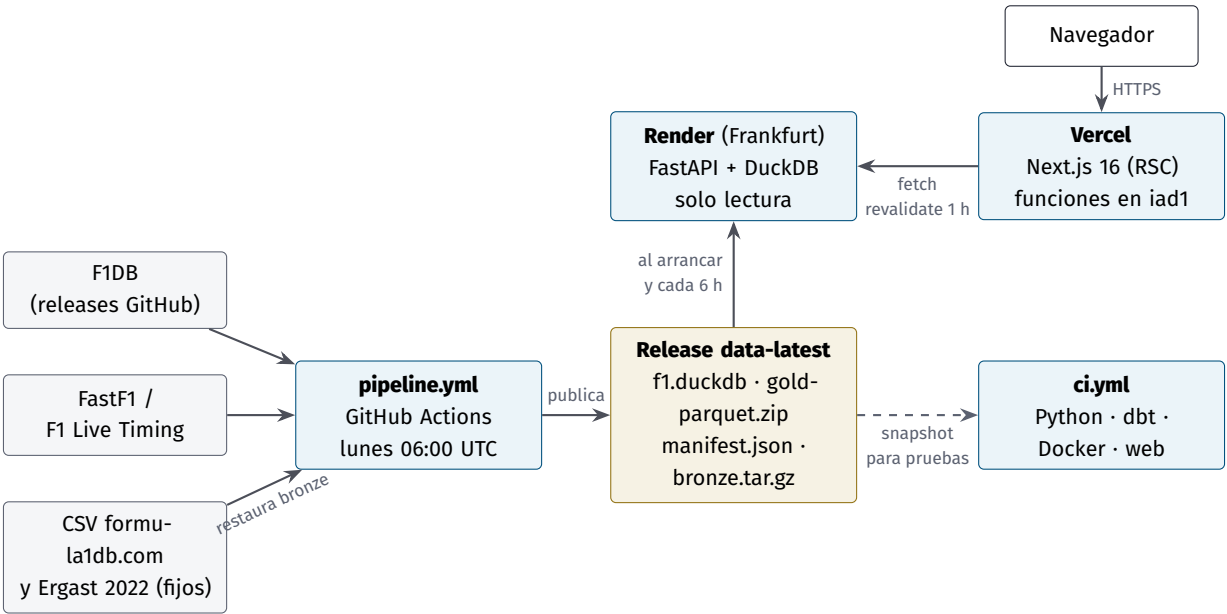


Figura 2: Arquitectura de despliegue actual. Las flechas continuas son el flujo de datos y las discontinuas, lecturas para pruebas. Cada push a main despliega además la API en Render (solo si cambian api/**, pyproject.toml o uv.lock) y la web en Vercel, aunque la CI falle.

Cómo está estructurado

El repositorio es un **monorepo** de 218 ficheros versionados con cuatro piezas independientes y un contrato explícito entre ellas:

Carpeta	Responsabilidad	Contrato con la siguiente
ingestion/	Paquete Python con la CLI <code>f1-ingest</code> : descarga, normaliza a Parquet y empaqueta el snapshot	Parquet en <code>data/bronze/<fuente>/</code>
transform/	Proyecto <code>dbt-duckdb</code> : staging, intermediate, marts, quality, seeds, macros y tests	Esquema gold de <code>f1.duckdb</code>
api/	Paquete Python FastAPI: routers por entidad, esquemas Pydantic, caché y descarga de la release	OpenAPI exportado a <code>web/src/lib/api/openapi.json</code>
web/	Next.js 16 con App Router, segmento <code>[lang]</code> , componentes de gráficos y pruebas e2e	Tipos generados <code>schema.d.ts</code>
.github/	<code>ci.yml</code> y <code>pipeline.yml</code>	Release data-latest

Las decisiones que la investigación respalda son el monorepo (una persona, un ciclo de vida), uv con grupos de dependencias, dbt con capas medallion y modelo Kimball, y las releases de GitHub como almacén de artefactos. Las que conviene revisar son pocas y concretas.

Alternativas evaluadas

Alternativa	Motivo	Veredicto
justfile con las órdenes habituales	Una sola forma de ejecutar ingesta, build, pruebas y web	adoptar
Sources dbt para todo el bronze	Hoy solo F1DB está declarado; se gana linaje y <i>freshness</i>	adoptar
Releases fechadas e inmutables	Versionado de datos y recuperación ante errores	adoptar
Disparo diario por calendario de F1	Datos el mismo día de la carrera y <i>keepalive</i> del cron	adoptar
«API estática» en GitHub Pages	Elimina el arranque en frío del camino crítico sin perder la API	adoptar después
Koyeb en lugar de Render	Arranque de 1–5 s frente a unos 60 s	considerar
DuckDB-WASM (página «Laboratorio»)	Consultas libres en el navegador; GitHub y Hugging Face no permiten las lecturas parciales necesarias	considerar como complemento
uv workspaces	Útil si la fase 7 trae dependencias pesadas de ML	considerar
Dagster + R2 + DuckLake	Buen ejercicio de portfolio, pero no lo pide la necesidad real	línea futura
Exportación estática completa de Next.js	Unos 16 400 HTML y los filtros sin JavaScript dejarían de funcionar	descartar
Fly.io, Cloudflare Workers/D1	Sin plan gratuito; DuckDB no cabe en los límites de Workers	descartar
Nx, Turborepo	Sobredimensionados para un solo paquete web	descartar

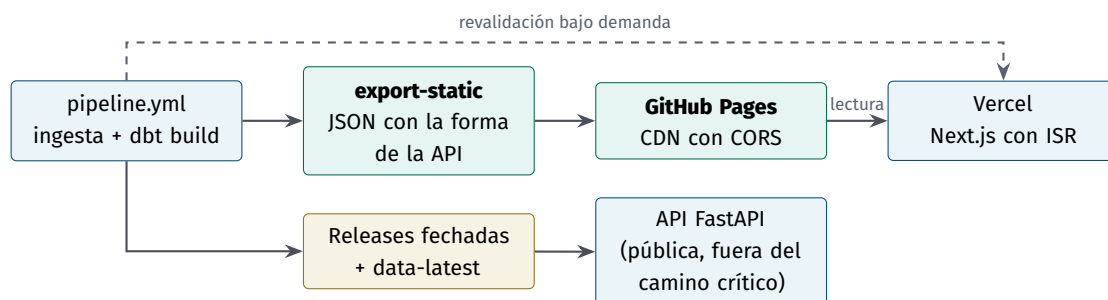


Figura 3: Arquitectura objetivo B («API estática»). En verde, las piezas nuevas.

Recomendación

Arquitectura A en la próxima iteración (2–3 días): *justfile*, sources completos, releases fechadas, disparo por calendario, región *fra1* y caché de página. Arquitectura B después (1–2 semanas), que además da el mejor argumento de diseño para la memoria: datos que cambian una vez por semana se sirven de forma estática. La C queda como línea futura.

CAPÍTULO 5

Arquitectura de aplicación y estructura del repositorio

Nota de trabajo para el informe de situación de `f1-data-platform`. Estado del repositorio en el commit `cceddd3` («Direcciones públicas en el README y pruebas contra la web desplegada»), rama `main`. Medidas tomadas el 28-09-2026 hacia las 23:25 UTC. Todas las rutas son relativas a la raíz del repositorio salvo que se indique lo contrario.

5.1 Estructura del repositorio

5.1.1 Visión general

El repositorio es un *monorepo* con cuatro componentes desplegables o ejecutables de forma independiente: la ingesta (`ingestion/`, paquete Python con CLI `f1-ingest`), la transformación (`transform/`, proyecto dbt sobre DuckDB), la API (`api/`, FastAPI) y la web (`web/`, Next.js). Los datos **no** se versionan en Git: viven en la GitHub Release `data-latest`, que el pipeline regenera cada semana y de la que se alimentan tanto la CI como la API desplegada.

`git ls-files` devuelve 218 ficheros versionados. Reparto por carpeta de primer nivel:

Carpeta / fichero	Ficheros	Líneas (aprox.)	Papel
<code>api/ (app)</code>	16	1 799	API FastAPI de solo lectura
<code>api/tests/</code>	4 + 18 Parquet	361 (+504 KB de fixture)	Pruebas de la API
<code>ingestion/</code>	9	740	Cargadores bronze y snapshots
<code>transform/</code>	74	3 217	Proyecto dbt (staging, intermediate, marts, quality, seeds, tests)
<code>web/src/app/</code>	22	2 661	Rutas (App Router)
<code>web/src/components/</code>	23	1 821	Componentes React
<code>web/src/lib/</code>	8	164 (sin contar <code>openapi.json</code> 4 404 y <code>schema.d.ts</code> 2 125, generados)	Cliente API, formato, i18n
<code>web/src/dictionaries/</code>	2	676	Textos es/en
<code>web/tests/</code>	2	129	Playwright + axe
<code>tests/</code>	2	125	Pruebas de la ingesta
<code>scripts/</code>	3	237	Utilidades de mantenimiento
<code>.github/</code>	2	311	CI y pipeline
<code>workflows/</code>			
<code>docs/</code>	9	—	Revisión de divergencias entre fuentes (CSV + Markdown)
Raíz	10	—	Configuración (<code>pyproject.toml</code> , <code>uv.lock</code> de 1 563 líneas, <code>render.yaml</code> , README de 299 líneas...)

5.1.2 Árbol comentado

```

f1-data-platform/
├── .github/workflows/
│   ├── ci.yml                # CI: jobs python, dbt, api-image, web
│   └── pipeline.yml          # Pipeline de datos semanal (lunes 06:00 UTC) + docs dbt
├── opcionales
├── api/
│   ├── Dockerfile            # Imagen python:3.12-slim + uv; solo grupo «api»
│   ├── __init__.py
│   └── app/
│       ├── config.py         # Settings (dataclass inmutable leída de variables F1_*)
│       ├── database.py       # Database: DuckDB read_only, cursor por consulta, swap()
│       ├── release.py        # Descarga de f1.duckdb desde GitHub Releases (stdlib)
│       ├── cache.py          # DataCacheMiddleware: ETag débil + Cache-Control + 304
│       ├── deps.py           # get_database, DB (Annotated), not_found, split_ids
│       ├── schemas.py        # 33 modelos Pydantic de respuesta (382 líneas)
│       ├── main.py           # create_app(), lifespan, middlewares, /health, /quality
│       └── routers/          # seasons, races, drivers, constructors, records, rankings,
├── circuits
│   └── tests/
│       └── conftest.py        # Construye un DuckDB temporal a partir de los Parquet de
├── ejemplo
│   ├── test_endpoints.py     # 20 pruebas funcionales de endpoints
│   ├── test_infrastructure.py # 10 pruebas: caché, CORS, swap, descarga verificada
│   └── fixtures/sample/      # 18 Parquet (gold.* y quality.qa_summary), 504 KB
├── docs/revision_divergencias/ # INFORME.md, PROPUESTA_F1DB.md y CSV de evidencias
├── ingestion/                # cli.py (f1-ingest), f1db/fastf1/legacy/ergast loaders, io.py,
├── snapshot.py
├── scripts/
│   ├── export_openapi.py     # Vuelca create_app().openapi() a web/src/lib/api/openapi.json
│   ├── make_api_fixture.py   # Genera api/tests/fixtures/sample desde dist/f1.duckdb
│   └── generate_override_seed.py # Genera transform/seeds/race_data_overrides.csv
├── tests/                    # test_ingestion.py (5), test_snapshot.py (5)
├── transform/                # dbt: dbt_project.yml, profiles.yml, macros, models, seeds,
├── tests
├── web/
│   ├── src/
│   │   ├── proxy.ts          # Redirección / -> /{es|en} según Accept-Language
│   │   └── app/
│   │       ├── globals.css    # Tailwind 4 + tokens de diseño (claro/oscuro)
│   │       ├── favicon.ico
│   │       └── [lang]/        # Segmento de idioma: layout raíz, páginas, error, not-found
│   ├── components/           # ui.tsx, formularios GET, mapas SVG, charts/ (ECharts)
│   ├── dictionaries/         # es.json, en.json
│   └── lib/                   # api/{client.ts, openapi.json, schema.d.ts}, format, i18n,
├── race, text, tyres
├── tests/                    # accesibilidad.spec.ts, navegacion.spec.ts
├── playwright.config.ts, next.config.ts (vacío), eslint.config.mjs, postcss.config.mjs
├── package.json, package-lock.json (7 354 líneas), tsconfig.json, .env.example
├── AGENTS.md / CLAUDE.md     # Aviso generado por `next dev` sobre la versión de Next
├── .dockerignore .gitattributes .gitignore .pre-commit-config.yaml .python-version (3.12)
├── pyproject.toml uv.lock render.yaml README.md

```

5.1.3 pyproject.toml y uv.lock

- Proyecto f1-data-platform 0.1.0, requires-python = ">=3.12,<3.13", backend hatchling; el *wheel* empaqueta ingestion y api.
- Dependencia base única: duckdb>=1.1 (común a ingesta, dbt y API).

- **Grupos de dependencias** ([`dependency-groups`], PEP 735) para que cada consumidor instale solo lo suyo: | Grupo | Paquetes | Quién lo usa | |—|—|—| | ingest | fastf1>=3.4, pandas>=2.2, pyarrow>=17, requests>=2.32 | CLI de ingesta, pipeline | | api | fastapi>=0.115, uvicorn>=0.30 | Imagen Docker (`--no-default-groups --group api`) | | transform | dbt-core>=1.9, dbt-duckdb>=1.9 | dbt en local y en CI | | dev | httpx2>=2.13.1, pre-commit>=4, pytest>=8, ruff>=0.8 | Desarrollo y CI | [`tool.uv`] `default-groups` activa los cuatro en desarrollo. `httpx2` es la dependencia que exige el `TestClient` de `Starlette` 1.7 (se comprobó en `starlette/testclient.py`).
- Script de consola: `f1-ingest = "ingestion.cli:main"` (subcomandos `f1db`, `fastf1`, `legacy`, `ergast` y `snapshot {pack,restore,notes}`).
- Ruff: `line-length = 100`, `target-version = "py312"`, reglas E, F, I, UP, B, SIM.
- Pytest: `testpaths = ["tests", "api/tests"]`.
- `uv.lock` fija las versiones reales resueltas (p. ej. `duckdb 1.5.5`, `fastapi 0.141.1`, `starlette 1.7.0`, `ruff 0.16.9`); CI, pipeline y Dockerfile usan `uv sync --locked`, por lo que una desincronización entre `pyproject.toml` y el lock hace fallar la construcción.

5.1.4 Ficheros de configuración de la raíz

- **.pre-commit-config.yaml**: `pre-commit-hooks` v5.0.0 (`trailing-whitespace`, `end-of-file-fixer`, `check-yaml`, `check-added-large-files --maxkb=1024`) y `ruff-pre-commit` v0.8.4 (`ruff --fix` y `ruff-format`). Nota: el hook fija `ruff` 0.8.4 mientras que el lock resuelve `ruff` 0.16.9, que es el que ejecuta la CI (posible deriva de reglas/formato).
- **.gitignore**: excluye `data/`, `*.duckdb`, `*.parquet` (salvo `!api/tests/fixtures/sample/*.parquet`), entornos y cachés de Python, artefactos de dbt (`transform/target/`, `logs/`), Node/Next (`node_modules/`, `web/.next/`), `.env*` (salvo `.env.example`), `dist/`, `snapshot/`, `site/` (salidas del pipeline) y `.claude/`.
- **.dockerignore**: el contexto de Docker es la raíz, pero se excluye todo lo que no es la API: `.git`, `.github`, `.venv`, `data`, `dist`, `snapshot`, `site`, `docs`, `transform`, `tests`, `api/tests`, `web`, `scripts`, `*.duckdb`, `*.parquet`.
- **.gitattributes**: `* text=auto eol=lf` (finales de línea LF aunque se desarrolle en Windows) y `*.parquet/*.duckdb` como binarios.
- **.python-version**: 3.12.
- **render.yaml**: Blueprint de Render (ver §4.3).
- **README.md** (299 líneas): arquitectura, ingesta, pipeline (tabla de ficheros de la release y puesta en marcha), API (tabla de endpoints y variables de entorno), web (correspondencia con las figuras del TFG) y despliegue. La tabla de endpoints del README no menciona `/rankings/`.

5.1.5 scripts/ y tests/

Script	Qué hace
<code>scripts/export_openapi.py</code>	Importa <code>create_app()</code> y escribe <code>web/src/lib/api/openapi.json</code> (JSON indentado, UTF-8). Es el primer eslabón del contrato API→web.

Script	Qué hace
<code>scripts/make_api_fixture.py</code>	Adjunta <code>dist/f1.duckdb</code> como warehouse (solo lectura) y exporta cada tabla a <code>api/tests/fixtures/sample/<esquema>.<tabla>.parquet (zstd)</code> . Dimensiones y agregados completos; resultados y clasificaciones 2021-2024; vueltas, paradas y pasos por boxes solo del GP de Baréin 2024; telemetría solo de Verstappen, Leclerc y Hamilton en esa carrera.
<code>scripts/generate_override_seed.py</code>	Genera el <code>seed_race_data_overrides.csv</code> a partir de las evidencias de <code>docs/revision_divergencias</code> y falla si la corrección ya no encaja con F1DB.

`tests/test_ingestion.py` (5 pruebas: conversión de `timedelta` a `ms`, submuestreo de telemetría, particionado, idempotencia de escritura, limpieza de duplicados del CSV legado) y `tests/test_snapshot.py` (5 pruebas: `pack/restore` ida y vuelta, que la base de la API solo contiene `gold` y `quality.qa_summary`, requisitos de `pack`, rechazo de archivos ajenos en `restore` y notas de la release).

5.2 API (api/app/)

5.2.1 Configuración (config.py)

`Settings` es un `dataclass(frozen=True)` cuyos campos se leen de variables de entorno mediante `default_factory` (así se pueden crear instancias en pruebas con valores explícitos):

Campo	Variable	Por defecto	Uso
<code>db_path</code>	<code>F1_API_DB_PATH</code>	None	Fichero DuckDB fijo, sin refresco
<code>data_repo</code>	<code>F1_DATA_REPO</code>	None	<code>owner/repo</code> de la release de datos
<code>data_tag</code>	<code>F1_DATA_TAG</code>	<code>data-latest</code>	Etiqueta de la release
<code>github_token</code>	<code>F1_GITHUB_TOKEN</code>	None	Solo para repositorio privado
<code>data_dir</code>	<code>F1_API_DATA_DIR</code>	<code>data/api</code> (en Docker / <code>data</code>)	Dónde guardar las versiones descargadas
<code>refresh_hours</code>	<code>F1_API_REFRESH_HOURS</code>	6	Periodo de comprobación (0 = nunca)
<code>cors_origins</code>	<code>F1_API_CORS_ORIGINS</code>	<code>http://localhost:3000</code>	Lista separada por comas
<code>cache_max_age</code>	<code>F1_API_CACHE_MAX_AGE</code>	600	<code>max-age</code> de <code>Cache-Control</code>

`local_candidates()` devuelve `dist/f1.duckdb` y `data/gold/f1.duckdb` para desarrollo.

5.2.2 Acceso a datos (database.py)

- **Resolución del origen** (`resolve_database(settings)`), por prioridad: (1) `F1_API_DB_PATH` (debe existir; se intenta leer un `manifest.json` hermano con `_local_manifest`), (2) `F1_DATA_REPO` → `release.ensure_database(...)`, (3) candidatos locales; si no hay nada, `FileNotFoundError` con instrucciones.
- **DataVersion** (`id`, `generated_at`, `f1db_release`, `last_completed_race`): `id` son los 12 primeros caracteres del SHA-256 de `f1.duckdb` según el manifiesto; sin manifiesto se deriva de ruta + tamaño + `mtime_ns`. Alimenta las ETag y `/health`.
- **Clase Database:**

- `_open(path, manifest): duckdb.connect(path, read_only=True)`, comprueba que existe al menos una tabla en el esquema `gold` (si no, `ValueError`), y bajo un `threading.Lock` sustituye la conexión activa; la anterior pasa a la lista `_retired`.
 - `query(sql, params):` toma el *lock* solo para crear un **cursor por consulta** (`connection.cursor()`); DuckDB permite cursores concurrentes de una misma conexión desde varios hilos), ejecuta con parámetros posicionales `?`, construye `list[dict]` con los nombres de columna y cierra el cursor en `finally`. `query_one` devuelve la primera fila o `None`.
 - `swap(path, manifest):` **intercambio atómico de versión:** abre la nueva conexión y la publica; las consultas en curso siguen con su cursor de la conexión antigua.
 - `close_retired()` cierra las conexiones retiradas; `close()` cierra todo.
- Los *endpoints* son funciones síncronas (`def`), así que FastAPI las ejecuta en su *threadpool*; el paralelismo real lo aporta DuckDB por cursor.

5.2.3 Descarga de datos (release.py)

Solo biblioteca estándar (`urllib`, `hashlib`, `json`), para no añadir dependencias a la imagen.

- Con repositorio **público** (sin token) usa enlaces directos `https://github.com/{repo}/releases/download/{tag}/{name}`, que no consumen el límite de 60 peticiones/h de la API de GitHub (compartido por IP en proveedores *cloud*).
- Con **token** consulta `GET /repos/{repo}/releases/tags/{tag}` y descarga el *asset* con `Accept: application/octet-stream`; un `_NoRedirect` evita reenviar `Authorization` al almacén externo al que redirige GitHub (se sigue la redirección a mano en `_open`).
- `fetch_manifest()` descarga `manifest.json` (fecha, versión de F1DB, última carrera, calidad y SHA-256 de cada fichero).
- `ensure_database(repo, tag, token, data_dir):` calcula `versioned_path = f1-<sha256[:12]>`. `duckdb`; si existe y su SHA coincide, lo reutiliza; si no, descarga en bloques de 1 MiB a `*.download` (timeout 600 s), **verifica el SHA-256** contra el manifiesto (si no coincide, borra y lanza `ValueError`) y renombra con `Path.replace`. El nombre versionado permite abrir la versión nueva mientras la antigua sigue abierta (necesario en Windows).
- `remove_old_versions(data_dir, keep)` borra las otras `f1-*.duckdb` ignorando `OSError`.

5.2.4 Ciclo de vida y refresco (main.py)

- `create_app(settings=None, database=None)` es una **factoría** (las pruebas inyectan una `Database` sobre el fixture). El módulo expone `app = create_app()` para Uvicorn y configura `logging.basic` `Config(level=INFO)`.
- **lifespan:** si no se inyecta base de datos, ejecuta `resolve_database` en un hilo (`asyncio.to_thread`, para no bloquear el bucle con la descarga de ~52 MB) y crea `Database`. Si hay `data_repo`, `refresh_hours > 0` y no hay `db_path`, lanza la tarea `refresh_periodically`. Al cerrar, cancela la tarea y cierra la base de datos.
- **refresh_periodically(app, settings):** bucle infinito que *primero duerme* `refresh_hours * 3600` s, descarga el manifiesto, compara `sha[:12]` con `database.version.id`, y si difiere llama a `ensure_database`, `database.swap(...)`, espera 120 s de margen, `close_retired()` y `remove_old_versions()`. Cualquier excepción se registra (`log.exception`) y se sigue sirviendo la versión actual.
- **Middlewares** (Starlette aplica como más externo el último añadido): | Orden (petición) | Middleware | Configuración | |—|—| 1 | `CORSMiddleware` | `allow_origins=settings.cors_origins`,

```
allow_methods=["GET"], allow_headers=["If-None-Match"], expose_headers=["ETag"] | | 2 |
GZipMiddleware | minimum_size=1024 | | 3 | DataCacheMiddleware | max_age=settings.cache_
max_age |
```

- Metadatos OpenAPI: título «F1 Data Platform API», version="1.0.0"; documentación interactiva en /docs (Swagger) y /redoc, contrato en /openapi.json.

5.2.5 Caché HTTP (cache.py)

DataCacheMiddleware (subclase de BaseHTTPMiddleware) actúa solo sobre GET cuando ya hay base de datos y la ruta no está en UNCAHED_PATHS = {"/health"}:

1. Calcula etag_for(version_id, request) = W/"<version>-<sha1(path?query)[:16]>".
2. Si If-None-Match contiene esa ETag, responde 304 sin ejecutar la consulta.
3. En caso contrario ejecuta el *endpoint* y, solo si el estado es 200, añade ETag y Cache-Control: public, max-age=600, stale-while-revalidate=86400. Los errores (404, 422) no se marcan como cacheables.

Al publicarse datos nuevos cambia *version.id* y, con él, todas las ETag (invalidación implícita).

5.2.6 Dependencias y esquemas

- *deps.py*: *get_database(request)* devuelve *request.app.state.database*; alias DB = *Annotated[Database, Depends(get_database)]*; *not_found(what)* → *HTTPException(404, "No existe ...")*; *split_ids(value, limit)* divide listas separadas por comas y responde 422 si se supera el límite.
- *schemas.py*: 33 modelos Pydantic agrupados en estado (Ref, RaceRef, DataInfo, Health, QualityCheck), temporadas (SeasonSummary, RaceSummary, SeasonDetail, DriverStanding, ConstructorStanding, StandingsProgression), carreras (RaceDetail hereda de RaceSummary, RaceResult, QualifyingResult, Lap, Stint, PitStop, PitLanePass, TelemetryLap), pilotos y constructores (DriverSummary, DriverDetail, DriverSeason, TeammateComparison, DriverRaceResult, ConstructorSummary, ConstructorDetail, ConstructorSeason) y récords (RecordEntry, ConstructorRanking, DriverRanking hereda de ConstructorRanking, Circuit). Convenciones: tiempos en milisegundos, identificadores de F1DB (lewis-hamilton, ferrari), *race_id* entero.

5.2.7 Catálogo de endpoints

28 operaciones GET (26 en 7 *routers* + /health y /quality en *main.py*). Parámetros con * son obligatorios. Las consultas usan siempre parámetros enlazados; cuando se interpola texto en el SQL (nombre de columna o tabla) procede de un Literal o de un diccionario cerrado (DRIVER_METRICS, CONSTRUCTOR_METRICS, ORDER), nunca de la entrada del usuario.

Estado (main.py)

Endpoint	Parámetros	Respuesta	Tablas
GET /health	—	Health (status, data.version, generated_at, f1db_release, last_completed_race)	ninguna (versión en memoria); sin caché
GET /quality	—	list[QualityCheck] ordenada FAIL → PASS → resto	quality.qa_summary

Temporadas (routers/seasons.py, prefijo /seasons)

Endpoint	Parámetros	Respuesta	Tablas gold
GET /seasons	—	list[SeasonSummary] (carreras, disputadas, campeones de pilotos y constructores), de la más reciente a la más antigua	dim_race, agg_driver_season, fact_constructor_standing, dim_constructor
GET /seasons/{season}	season*	SeasonDetail (calendario con ganador)	dim_race, fact_race_result, dim_driver (SQL compartido RACE_SUMMARY_SQL)
GET /seasons/{season}/standings/drivers	season*, round (≥1)	list[DriverStanding] tras la ronda pedida o la última disputada	dim_race, fact_driver_standing, fact_race_result, dim_driver, dim_constructor
GET /seasons/{season}/standings/constructors	season*, round	list[ConstructorStanding] (con motor)	dim_race, fact_constructor_standing, fact_race_result, dim_constructor, dim_engine_manufacturer
GET /seasons/{season}/standings/progression	season*, type (drivers\constructors), top (1-50, 10)		list[Standings Progression] (puntos acumulados por ronda de los N primeros)

RACE_SUMMARY_SQL resuelve el caso de coches compartidos de los años 50 escogiendo un único ganador por carrera (mínimo position_display_order). _standings_race() elige la carrera de referencia de la clasificación.

Carreras (routers/races.py, prefijo /races)

Todas validan antes la existencia con _require_race() (404 si no existe).

Endpoint	Parámetros	Respuesta	Tablas gold
GET /races/{race_id}	race_id*	RaceDetail (resumen + hora UTC, vueltas, distancia, tipo de circuito, sentido, longitud, curvas, coordenadas, is_season_final_race)	dim_race, fact_race_result, dim_driver
GET /races/{race_id}/results	session (race\)	sprint)	list[RaceResult] con corrected_fields (columnas corregidas por overrides)
GET /races/{race_id}/qualifying	session (qualifying\)	sprint_qualifying)	list[QualifyingResult] (Q1-Q3, mejor tiempo, % respecto a la pole)
GET /races/{race_id}/laps	drivers (≤30, por comas)	list[Lap]: posición, tiempo, gap, sectores, compuesto relativo/Pirelli, edad, pit in/out, banderas, SC/VSC, fuente, estado de validación	fact_laptimes (excluye is_incomplete_lap)
GET /races/{race_id}/stints	—	list[Stint]: tramos calculados con funciones ventana (nuevo tramo tras entrar en boxes o cambio de compuesto)	fact_laptimes
GET /races/{race_id}/pitstops	—	list[PitStop] (vuelta, número, duración)	fact_pit_stops, dim_driver
GET /races/{race_id}/pit-lane-passes	—	list[PitLanePass] tipificados (parada, SC, sanción...)	fact_pit_lane_passes

Endpoint	Parámetros	Respuesta	Tablas gold
GET /races/{race_id}/telemetry	drivers* (1-4)	list[TelemetryLap]: listas de distancia, velocidad, acelerador, freno, marcha y x/y de la vuelta más rápida de clasificación, en el orden pedido	fact_quali_telemetry (2024+)

Pilotos (routers/drivers.py, prefijo /drivers)

Endpoint	Parámetros	Respuesta	Tablas gold
GET /drivers	search (≥2, insensible a acentos con strip_accents(lower()), season, limit (1-1000, 50), offset	list[DriverSummary] ordenada por victorias, podios, salidas	agg_driver_career, agg_driver_season
GET /drivers/{driver_id}	—	DriverDetail (biografía + estadísticas de carrera, tasas de victoria/podio)	agg_driver_career, dim_driver
GET /drivers/{driver_id}/seasons	—	list[DriverSeason]	agg_driver_season
GET /drivers/{driver_id}/results	season	list[DriverRaceResult] (carrera y sprint)	fact_race_result, dim_race
GET /drivers/{driver_id}/teammates	—	list[Teammate Comparison] (carreras/clasificaciones por delante, puntos)	agg_teammate_h2h, dim_constructor, dim_driver

Constructores (routers/constructors.py, prefijo /constructors)

Endpoint	Parámetros	Respuesta	Tablas gold
GET /constructors	search, season, limit (1-500, 50), offset	list[Constructor Summary]	agg_constructor_career, fact_race_result, dim_race
GET /constructors/{constructor_id}	—	ConstructorDetail (podios, dobles, poles, vueltas rápidas, puntos, mejor posición)	agg_constructor_career, dim_constructor
GET /constructors/{constructor_id}/seasons	—	list[ConstructorSeason] (posición final vía qualify row_number(), victorias, pilotos)	fact_race_result, dim_race, dim_driver, fact_constructor_standing, dim_constructor

Récords y rankings (routers/records.py y routers/rankings.py)

Endpoint	Parámetros	Respuesta	Tablas gold
GET /records/drivers	metric (championships, wins, podiums, pole_positions, fastest_laps, race_starts, points, grand_slams; por defecto wins), limit (1-100, 20)	list[RecordEntry] (rank() SQL, id, name, country_alpha2, value)	agg_driver_career

Endpoint	Parámetros	Respuesta	Tablas gold
GET / records/ constructors	metric (championships, wins, podiums, pole_positions, fastest_laps, race_entries, one_two_finishes, points→race_points), limit	list[RecordEntry]	agg_constructor_career
GET / rankings/ drivers	season_from, season_to, order_by (wins, championships, podiums, pole_positions, fastest_laps, points, entries), limit (1-500, 50)	list[DriverRanking] recalculado desde resultados carrera a carrera en el rango (inscripciones, salidas —excluye DNQ/DNPQ/DNP/EX/DNS—, victorias, podios, poles, VR, puntos, títulos)	fact_race_result, dim_race, fact_driver_standing, dim_driver
GET / rankings/ constructors	ídem	list[ConstructorRanking] (podios contados por coche: race_id '-' driver_number)	fact_race_result, dim_race, fact_constructor_standing, dim_constructor

/records lee los agregados precalculados; /rankings los recalcula para un rango de temporadas (con el rango completo coinciden con los totales oficiales de F1DB, lo que prueba test_rankings_match_season_totals).

Circuitos (routers/circuits.py)

Endpoint	Parámetros	Respuesta	Tablas gold
GET / circuits	season_from, season_to	list[Circuit] (coordenadas, nº de carreras, primera y última temporada, GP disputados)	dim_race (solo is_completed)

5.2.8 Imagen Docker (api/Dockerfile)

1. FROM python:3.12-slim; copia el binario de uv desde ghcr.io/astral-sh/uv:0.12. Variables UV_COMPILE_BYTECODE=1, UV_LINK_MODE=copy, UV_PYTHON_DOWNLOADS=never, PYTHONUNBUFFERED=1.
2. Capa de dependencias cacheable: copia pyproject.toml uv.lock README.md y ejecuta uv sync --locked --no-default-groups --group api --no-install-project (solo DuckDB, FastAPI, Uvicorn: sin pandas ni FastF1).
3. Copia ingestion/__init__.py (necesario porque el *wheel* declara el paquete), api/__init__.py y api/app, y repite uv sync para instalar el proyecto.
4. Usuario sin privilegios app (UID 1000), carpeta /data propia; F1_API_DATA_DIR=/data, PORT=8000.
5. HEALTHCHECK --interval=30s --timeout=5s --start-period=120s con un urllib.request.urlopen a /health (sin curl en la imagen).
6. CMD uvicorn api.app.main:app --host 0.0.0.0 --port \${PORT} --proxy-headers --forwarded-allow-ips='*'.

5.2.9 Pruebas de la API

- **Fixture:** api/tests/conftest.py crea en una carpeta temporal sample.duckdb a partir de los 18 Parquet de api/tests/fixtures/sample/ (nombre esquema.tabla.parquet → create schema + create table ... as select * from '<parquet>'), y el *fixture* client monta create_app(Settings(cors_origins=["https://f1.example.org"], refresh_hours=0), Database(sample_db_path)) con TestClient. Las pruebas no necesitan red ni la release.

- `test_endpoints.py` (20 pruebas): salud, calidad ordenada, temporadas con campeones, calendario, clasificaciones final y tras ronda, progresión (un punto por ronda y piloto), detalle de carrera, resultados y clasificación, vueltas filtradas, *stints* coherentes con paradas, pasos por boxes tipificados, telemetría (orden y límite de 4), búsqueda de pilotos (con y sin acentos), constructores, récords, circuitos por rango, 404, y rankings que cuadran con los totales.
- `test_infrastructure.py` (10 pruebas): ETag + 304, /health sin caché, errores sin caché, CORS solo para el origen configurado, swap cambia la versión, prioridad de `F1_API_DB_PATH`, descarga verificada/versionada/reutilizada (con una release falsa), descarte por SHA incorrecto, borrado de versiones antiguas y uso de enlaces directos para repos públicos.
- Resultado local: `pytest` recoge **40 pruebas** (30 de la API + 10 de ingesta) y pasan todas en 2,4 s.

5.3 Web (web/)

5.3.1 Pila y convenciones de Next.js 16

- next 16.3.6, react/react-dom 19.2.8, TypeScript 5 (strict), Tailwind CSS 4 (@tailwindcss/postcss), ESLint 9 (eslint-config-next *core-web-vitals* + *typescript*). Dependencias de ejecución mínimas: echarts ^6.1, d3-geo, topojson-client, world-atlas, server-only.
- **App Router** con todo bajo el segmento dinámico `src/app/[lang]/`; no hay `app/layout.tsx` global: el *layout* raíz es `[lang]/layout.tsx`, que fija `<html lang={locale}>`.
- **src/proxy.ts** (en Next 16 sustituye a `middleware.ts`): si la ruta no empieza por /es o /en, calcula el idioma preferido a partir de Accept-Language (ordenado por q) y redirige (307) a /{locale}{pathname}. `matcher: ["/(?!_next|.*\\.\\.\\.)*"]` excluye internos y estáticos.
- **i18n** (`src/lib/i18n.ts`, `server-only`): diccionarios `es.json/en.json` cargados con `import()` dinámico; `getDictionary(lang)` devuelve `{locale, t}` o llama a `notFound()`. `src/lib/text.ts::fill()` sustituye marcadores `{clave}`. El cambio de idioma (`LanguageSwitch` en `nav-links.tsx`) reescribe el prefijo de la ruta actual y marca `hrefLang/lang`.
- **params y searchParams son Promise** (Next 15+): todas las páginas hacen `await params / await searchParams`. Los tipos `PageProps<"/[lang]/...">` y `LayoutProps<...>` los genera **next typegen** (el script `typecheck` es `next typegen && tsc --noEmit`; el commit 66c61e0 lo añadió a la CI porque sin él faltaban los tipos de rutas).
- **Scripts npm**: `dev`, `build`, `start`, `lint`, `gen:api` (`openapi-typescript src/lib/api/openapi.json -o src/lib/api/schema.d.ts`), `test:e2e`, `typecheck`.
- `next.config.ts` está vacío (sin cabeceras, sin images, sin output).

5.3.2 Cliente API tipado (src/lib/api/)

- `openapi.json` (exportado por `scripts/export_openapi.py`) → `schema.d.ts` (generado por `openapi-typescript`). `client.ts` exporta `Schemas = components["schemas"]`, de modo que las páginas escriben `apiGetRequired<Schemas["RaceResult"][]>(...)`.
- `client.ts` es `server-only`: la URL de la API (`F1_API_URL`, por defecto `http://127.0.0.1:8000`) nunca llega al navegador y **el navegador no llama a la API** (el CORS de la API no interviene en la web).
- `apiGet<T>(path, query)`: construye la URL omitiendo parámetros vacíos, `fetch` con `next: { revalidate: 3600 }` y `Accept: application/json; 404 → null`; otro error → `ApiError(status, path)`. `apiGetRequired<T>` convierte `null` en `ApiError(404)`.

- `src/lib/race.ts`: `getRace` (valida `^\d+$/, 404` si no existe), `getResults` y `getDriverNames`, memorizados por petición con `React.cache` (los usan el *layout* de carrera y cada pestaña sin duplicar llamadas).
- `src/lib/format.ts`: `lapTime`, `raceTime`, `gap`, `number` e `date` con `Intl`, `driverCode`.
- `src/lib/tyres.ts`: colores Pirelli y **letra** de cada compuesto (S, M, H, I, W, HS, US, SS, SH) para no depender solo del color; `compoundStyle()`.

5.3.3 Mapa de rutas

20 ficheros de ruta bajo `[lang]` (16 `page.tsx`, 2 `layout.tsx`, `error.tsx`, `not-found.tsx`) → **16 páginas por idioma, 32 URL-patrón en total (es/en)**.

Ruta	Qué muestra	Endpoints	Figura del TFG
<code>/</code> (proxy)	Redirección 307 a <code>/es</code> o <code>/en</code>	—	—
<code>/[lang]</code>	Inicio: temporada en curso (calendario, clasificaciones, último resultado) y mapa mundial de GP por país o circuito (<code>?view=</code>), filtrable por rango (<code>?from=&to=</code>)	<code>/seasons</code> , <code>/seasons/{y}</code> , <code>/seasons/{y}/standings/drivers</code> , <code>.../constructors</code> , <code>/circuits?season_from&season_to</code> , <code>/races/{id}/results</code>	5.23
<code>/[lang]/seasons</code>	Selector de temporada (formulario GET; con <code>?year=</code> hace <code>redirect</code>)	<code>/seasons</code>	—
<code>/[lang]/seasons/[year]</code>	Campeones, clasificación de pilotos y constructores, gráfico de evolución y calendario	<code>/seasons/{y}</code> , <code>/seasons</code> , <code>.../standings/drivers</code> , <code>.../standings/constructors</code> , <code>.../standings/progression?top=10</code>	5.26
<code>/[lang]/races/[id]</code> (<i>layout</i>)	Cabecera de la carrera, navegación anterior/siguiente y pestañas (TabNav); telemetría solo si <code>season ≥ 2024</code> ; carreras futuras sin pestañas	<code>/races/{id}</code> , <code>/seasons/{y}</code>	—
<code>/[lang]/races/[id]</code>	Resultado (carrera o sprint, <code>?session=</code>)	<code>/races/{id}/results</code>	—
<code>/[lang]/races/[id]/qualifying</code>	Clasificación y gráfico de barras de <i>gaps</i> (pilotos o constructores; clasificación normal o sprint)	<code>/races/{id}/qualifying?session=</code>	5.27
<code>/[lang]/races/[id]/lap-chart</code>	Posiciones vuelta a vuelta	<code>/races/{id}/laps</code> , <code>/races/{id}/results</code>	5.28
<code>/[lang]/races/[id]/tyres</code>	Estrategia (StintChart) y matriz de compuestos vuelta a vuelta (Compound Matrix)	<code>/races/{id}/stints</code> , <code>/races/{id}/laps</code> , <code>/results</code>	5.29
<code>/[lang]/races/[id]/pace</code>	Tiempos por vuelta (líneas) y ritmo (violín)	<code>/races/{id}/laps</code> , <code>/results</code>	5.30, 5.31
<code>/[lang]/races/[id]/pitstops</code>	Paradas, media por constructor (barras) y todos los pasos por el <i>pit lane</i>	<code>/pitstops</code> , <code>/pit-lane-passes</code> , <code>/results</code>	5.32
<code>/[lang]/races/[id]/telemetry</code>	Comparación de dos pilotos (<code>?a=&b=</code> , validados contra el resultado): velocidad/acelerador/freno/marcha y trazado coloreado por velocidad	<code>/races/{id}/telemetry?drivers=a,b</code> , <code>/results</code>	nueva
<code>/[lang]/drivers</code>	Buscador (<code>?q=</code> , ≥ 2 caracteres) y listado	<code>/drivers?search&limit</code>	—
<code>/[lang]/drivers/[id]</code>	Ficha, cifras, puntos por temporada, comparativas con compañeros (puntos y % por delante)	<code>/drivers/{id}</code> , <code>.../seasons</code> , <code>.../teammates</code>	5.33
<code>/[lang]/constructors</code>	Buscador y listado	<code>/constructors?search&limit</code>	—
<code>/[lang]/constructors/[id]</code>	Ficha y puntos por temporada	<code>/constructors/{id}</code> , <code>.../seasons</code>	nueva

Ruta	Qué muestra	Endpoints	Figura del TFG
/[lang]/records	Títulos y «más laureados» de pilotos o constructores (?entity=&order=&from=&to=) con gráfico de barras	/seasons, /rankings/{entity} (×2)	5.24, 5.25
/[lang]/quality	Controles de calidad (bloqueantes e informativos) y fuentes	/quality	nueva
error.tsx	<i>Error boundary</i> cliente con textos es/en embebidos y botón de reintento	—	—
not-found.tsx	404 del sitio	—	—

Además, **todas las páginas** llaman a GET /health desde SiteFooter (componente de servidor asíncrono) para mostrar la fecha de los datos y la versión de F1DB; el error se captura (.catch(() => null)) para que el pie no rompa la página si la API no responde.

No se usan generateStaticParams (ni siquiera para [lang]); las páginas de detalle validan el identificador con una expresión regular antes de llamar a la API y usan generateMetadata para el <title>.

5.3.4 Componentes (src/components/)

Gráficos (charts/, todos "use client")

- echarts.tsx: núcleo de ECharts **tree-shaken** (echarts/core + solo BarChart, BoxplotChart, Line Chart, ScatterChart, DataZoom, Grid, Legend, MarkLine, Tooltip, VisualMap y **SVGRenderer**). El componente EChart({build, height, label}) lee los *tokens* CSS del documento (readTheme()), inicializa con renderer: "svg", desactiva la animación si prefers-reduced-motion: reduce, redimensiona con ResizeObserver y se redibuja al cambiar prefers-color-scheme. El contenedor es role="img" con aria-label. Paletas **Okabe-Ito** clara y oscura (LIGHT_SERIES, DARK_SERIES) y seriesStyle() que combina color + tipo de línea + símbolo para no depender solo del color. Utilidades baseOption, axisStyle, tooltipStyle.
- chart-figure.tsx: ChartFigure = <figure> con <figcaption> (título + **resumen textual**) y la **tabla de datos completa en <details>**.
- Gráficos concretos: bar-chart.tsx, lap-chart.tsx, lap-times-chart.tsx, progression-chart.tsx, season-points-chart.tsx, stint-chart.tsx, teammate-charts.tsx (TeammatePointsChart, TeammateShareChart), telemetry-chart.tsx, violin-chart.tsx (violín con SVG propio, colores por variables CSS .violin).

Servidor (sin JavaScript en el cliente)

- world-map.tsx: mapa mundial con d3-geo (geoEqualEarth) y world-atlas/countries-110m (TopoJSON → GeoJSON con topojson-client), **calculado en el servidor** como SVG estático 960×470; los contornos se calculan una vez a nivel de módulo; círculos con radio $\propto \sqrt{\text{carreras}}$; <title>/<desc> accesibles.
- track-map.tsx: trazado del circuito a partir de x/y de telemetría, coloreado por velocidad (escala de 4 paradas), SVG con proporciones reales.
- compound-matrix.tsx: matriz piloto × vuelta con color y letra del compuesto.
- ui.tsx: PageHeader, Section, DataTable<T> (tabla accesible con <caption>, th scope="col", celdas th scope="row", aria-sort, contenedor role="region" desplazable y enfocable con tab Index=0), Pill, EmptyState, Stat, Highlight.
- Formularios **GET sin JavaScript**: search-form.tsx (buscador), season-picker.tsx (selector de temporada), season-range-form.tsx (desde/hasta + parseYear() que acota valores), session-switch.tsx (conmutador como enlaces).

- Navegación: `site-header.tsx`, `site-footer.tsx`, `nav-links.tsx` (NavLinks con `aria-current`, `LanguageSwitch`), `tab-nav.tsx` (pestañas como enlaces con URL propia y `aria-current="page"`); estos dos últimos son componentes cliente solo por `usePathname()`.

5.3.5 Estrategia de caché y renderizado

- Cada `fetch` a la API lleva `next: { revalidate: 3600 }` (Data Cache de Next/Vercel con revalidación incremental: tras 1 h la siguiente petición recibe el dato antiguo y dispara la regeneración en segundo plano). El `layout` raíz declara además `export const revalidate = 3600`.
- En la práctica **todas las páginas medidas se sirven dinámicamente**: Vercel devuelve `Cache-Control: private, no-cache, no-store, max-age=0, must-revalidate` y `X-Vercel-Cache: MISS` en `/es`, `/es/drivers/lewis-hamilton`, `/es/seasons/2021` y `/es/races/1102`. El HTML se renderiza en una función en cada visita; lo que se reutiliza es la respuesta de la API guardada en la Data Cache. Las páginas con filtros leen `searchParams` (API dinámica, renderizado por petición inevitable); en las que no los leen (p. ej. la ficha de piloto) el código no usa `headers()`, `cookies()` ni `force-dynamic`, así que la causa más probable es que ningún segmento (empezando por `[lang]`) declare `generateStaticParams`, de modo que Next no genera ni conserva HTML estático para esas rutas. Conviene confirmarlo con la salida de `next build` (columna de tipo de ruta).
- La cabecera `X-Vercel-Id: cdg1::iad1::...` indica que la petición entra por el *edge* de París pero la función se ejecuta en **iad1 (Washington)**, mientras que la API está en **Frankfurt**: cada fallo de la Data Cache cruza el Atlántico.

5.3.6 Estilos

- Tailwind 4 con `@import "tailwindcss"` y `@theme inline` que mapea los *tokens* a utilidades (`bg-surface`, `text-muted`, `border-line`, `fill-red...`).
- *Tokens* en `:root` de `globals.css` (`--bg`, `--surface`, `--surface-2`, `--ink`, `--muted`, `--line`, `--red`, `--purple` = «lo más rápido», `--green` = mejora personal, `--amber`, `--focus`, `--on-accent`) redefinidos en `@media (prefers-color-scheme: dark)`; `color-scheme` acorde. Modo oscuro automático según el sistema (no hay conmutador manual).
- Tipografías con `next/font/google` autoalojadas: Titillium Web (titulares), IBM Plex Sans (texto), IBM Plex Mono (tiempos; clase `.tabular` con `tabular-nums`).

5.3.7 Accesibilidad

- Enlace «Saltar al contenido» como primer elemento enfocable; `<main id="contenido" tabIndex=-1>`.
- `:focus-visible` con contorno de 3 px en `--focus`.
- Cada gráfico: `figcaption` con resumen en texto + tabla alternativa en `<details>`; el lienzo ECharts es `role="img"` con `aria-label` y `aria.enabled: false` para no duplicar lectura.
- Codificación redundante: color + forma/línea en series, color + letra en neumáticos.
- `prefers-reduced-motion`: regla global que anula animaciones y transiciones y desactiva la animación de ECharts.
- Formularios y pestañas funcionan sin JavaScript (enlaces y GET).
- Tablas con semántica completa (`caption`, `scope`, `aria-sort`) y desplazamiento horizontal accesible por teclado.

5.3.8 Pruebas de la web

`playwright.config.ts`: `testDir: ./tests`, `timeout 60 s`, `fullyParallel`, 1 reintento en CI, `reporter github+list` en CI. Proyectos escritorio (Desktop Chrome) y móvil (Pixel 7, solo navegacion). En local usa el Edge del sistema (`channel: "msedge"`), en CI el Chromium de Playwright. `webServer: npx next start --port 3100` salvo que se defina `BASE_URL`, en cuyo caso se prueba contra la web desplegada (commit cceddd3).

Fichero	Pruebas	Contenido
<code>accesibilidad.spec.ts</code>	26	axe WCAG 2.0/2.1/2.2 A y AA en 21 URL (incl. /en, filtros de récords, búsqueda); 1 prueba de que cada <code><figure></code> tiene <code>figcaption</code> y <code>details summary</code> ; 4 de contraste en tema oscuro (<code>colorScheme: "dark"</code>)
<code>navegacion.spec.ts</code>	6 × 2 proyectos = 12	Redirección por idioma, <i>skip link</i> , del inicio a una carrera y sus pestañas (<code>aria-current</code>), buscador sin JavaScript (<code>javascriptEnabled: false</code>), cambio de idioma que conserva la ruta y <code>lang</code> , ausencia de desbordamiento horizontal en 9 páginas

Total: 38 ejecuciones de prueba e2e por pasada. La carrera de referencia es Baréin 2024 (`race_id 1102`).

5.4 CI/CD

5.4.1 ci.yml (push a main y pull requests)

Permisos `contents: read`; `concurrency: ci-${{ github.ref }}` con cancelación de ejecuciones anteriores. Cuatro *jobs* en paralelo:

- python** (15 min): `actions/checkout@v5` → `astral-sh/setup-uv@v6` (caché) → `uv sync --locked` → `ruff check .` → `ruff format --check .` → `pytest`.
- dbt** (30 min): `checkout`, `uv`, `uv sync --locked`; descarga `bronze.tar.gz` de `data-latest` con `gh release download` y `f1-ingest snapshot restore` (si no existe, aviso y `available=false`); `dbt parse`; `dbt build` solo si hay datos; en fallo sube `transform/logs/` y `run_results.json` con `actions/upload-artifact@v4`.
- api-image** (20 min): `docker build -f api/Dockerfile -t f1-api .`; descarga `f1.duckdb` y `manifest.json`; **prueba de humo**: `docker run` con `F1_API_DB_PATH=/snapshot/f1.duckdb` (volumen de solo lectura), espera hasta 60 s a `/health`, pide `/seasons` y comprueba que `/races/999999` da 404; en fallo muestra `docker logs`.
- web** (30 min, `F1_API_URL=http://127.0.0.1:8000`): `checkout`, `uv`, `actions/setup-node@v5` (Node 24, caché npm), `uv sync`, `npm ci`; **comprobación de contrato**: `export_openapi.py` + `npm run gen:api` + `git diff --exit-code web/src/lib/api` (falla si los tipos TS no están al día con la API); `npm run lint`, `npm run typecheck`, `npm run build`; descarga la base publicada, arranca `uvicorn` con `F1_API_DB_PATH`, instala Chromium y ejecuta `npx playwright test`; en fallo sube `playwright-report/`.

Duración observada (API pública de GitHub Actions): entre 30 s y 2,5 min por ejecución de CI. De las 6 ejecuciones de CI del 28-09-2026, 4 terminaron bien y 2 fallaron (22:37 y 22:45 UTC); las siguientes, tras los commits de corrección (66c61e0, tipos de rutas; d0d9bd5, desbordamiento de la matriz de neumáticos), volvieron a pasar.

5.4.2 pipeline.yml (datos)

- Disparadores: cron: "0 6 * * 1" (lunes 06:00 UTC) y workflow_dispatch con entradas seasons (temporadas FastF1) y force_f1db. Permisos contents: write; concurrency: data-pipeline sin cancelación.
- Pasos del job pipeline (120 min): checkout → setup-uv → uv sync --locked → **restaurar snapshot** (bronze.tar.gz; error si no existe) → f1-ingest f1db [--force] → f1-ingest fastf1 --season <año> --telemetry (continue-on-error: true; en enero incluye también el año anterior; valida el formato de temporadas con expresión regular) → **dbt build** (si falla una prueba de calidad, no se publica nada) → snapshot pack --out dist + snapshot notes (al *step summary*) → **publicación**: crea data-latest si no existe (--latest=false), sube f1.duckdb, gold-parquet.zip, manifest.json y, al final, bronze.tar.gz (--clobber), y actualiza las notas → aviso si FastF1 falló → opcional: dbt docs generate --static y upload-pages-artifact@v3 si vars. PUBLISH_DOCS == 'true' → logs de dbt si falla.
- Job docs (needs: pipeline, condicionado a PUBLISH_DOCS): actions/deploy-pages@v4.
- Última ejecución: manual el 28-09-2026 19:45 UTC, ~1 min, correcta. Release actual: f1.duckdb 52,2 MB, bronze.tar.gz 51,7 MB, gold-parquet.zip 28,1 MB, manifest.json 1,5 KB; versión servida 35677c2df776, F1DB v2026.15.1, última carrera: GP de Azerbaiyán 2026 (ronda 15).

5.4.3 Despliegue continuo

- **Render** (render.yaml): servicio web f1-data-api, runtime: docker, plan: free, region: frankfurt, dockerfilePath: ./api/Dockerfile, dockerContext: ., healthCheckPath: /health, autoDeploy: true con **buildFilter.paths**: api/**, pyproject.toml, uv.lock (un cambio en la web o en dbt no reconstruye la API). Variables: F1_DATA_REPO=AntonioSchez32/f1-data-platform, F1_API_REFRESH_HOURS=6, F1_API_CORS_ORIGINS (sync: false, se rellena en el panel). Render construye su propia imagen: la imagen de api-image en CI no se publica ni se reutiliza.
- **Vercel**: proyecto con *Root Directory* web, variable F1_API_URL apuntando a Render; despliegue automático en cada push a main (producción) y *previews* por rama/PR. No hay vercel.json.
- Ninguno de los dos despliegues espera a que la CI pase: se disparan por el push en paralelo.

5.4.4 Flujo de refresco de datos extremo a extremo

¿Cuándo aparece en la web una carrera disputada el domingo?

1. **Lunes 06:00 UTC** (+ cola de GitHub, típicamente minutos): el pipeline ingiere F1DB/FastF1, dbt build, y publica f1.duckdb + manifest.json en data-latest.
2. **API**: el bucle refresh_periodically comprueba el manifiesto cada 6 h **contadas desde el arranque del proceso**; al detectar un SHA distinto descarga, verifica y hace swap. En el plan gratuito la instancia se duerme tras ~15 min sin tráfico y al despertar vuelve a ejecutar resolve_database, que descarga siempre la última versión: en la práctica, la API sirve datos nuevos en **0-6 h** tras la publicación (antes si ha habido un *cold start*).
3. **Web**: las respuestas de la API quedan en la Data Cache de Next hasta **1 h**; al caducar, la primera visita recibe el dato antiguo y dispara la revalidación. Además, las ETag de la API cambian con la versión, así que las revalidaciones posteriores no reutilizan nada antiguo.
4. **Peor caso**: ~7 h tras la publicación (~ lunes 13:00 UTC); caso habitual con la instancia dormida: pocos minutos tras la primera visita posterior a la publicación.

5.5 Diagramas

5.5.1 Arquitectura de despliegue

```

NODOS
dev      [actor]      "Desarrollador (Windows, uv, npm)"
gh       [repo]       "GitHub: AntonioSchez32/f1-data-platform (main)"
ci       [proceso]    "GitHub Actions · ci.yml (python, dbt, api-image, web)"
pipe     [proceso]    "GitHub Actions · pipeline.yml (lunes 06:00 UTC)"
rel      [almacén]    "GitHub Release data-latest (f1.duckdb, manifest.json, bronze.tar.gz,
↳ gold-parquet.zip)"
f1db     [externo]    "F1DB (GitHub releases)"
ff1      [externo]    "FastF1 / F1 Live Timing"
pages    [externo]    "GitHub Pages (docs dbt, opcional)"
render   [servicio]   "Render · f1-data-api (Docker, free, Frankfurt) · uvicorn + FastAPI"
duck     [almacén]    "DuckDB f1-<sha>.duckdb (solo lectura, /data efímero)"
cf       [red]        "Cloudflare (delante de Render, cf-cache-status: DYNAMIC)"
vercel   [servicio]   "Vercel · Next.js 16 (edge cdg1, funciones iad1) + Data Cache"
user     [actor]      "Navegador"

ARISTAS
dev  -> gh      "git push"
gh   -> ci      "push / pull_request"
gh   -> render  "autoDeploy (buildFilter: api/**, pyproject.toml, uv.lock)"
gh   -> vercel  "deploy (root: web)"
pipe -> rel     "gh release download bronze.tar.gz / upload --clobber"
f1db -> pipe    "f1-ingest f1db"
ff1  -> pipe    "f1-ingest fastf1 --telemetry"
pipe -> pages   "dbt docs (si PUBLISH_DOCS)"
ci   -> rel     "descarga snapshot para dbt build, humo y e2e"
rel  -> render  "arranque + cada 6 h: manifest.json, f1.duckdb (SHA-256)"
render-> duck   "cursor por consulta"
user -> vercel  "HTTPS (HTML + RSC)"
vercel-> cf     "fetch server-side (F1_API_URL), revalidate 3600"
cf   -> render  "proxy"

```

5.5.2 Secuencia de una petición (/es/races/1102/pace)

```

PARTICIPANTES: Navegador, VercelEdge(cdg1), FuncionNext(iad1), DataCache, APIRender(FRA),
↳ DuckDB

1. Navegador      -> VercelEdge      GET /es/races/1102/pace
2. VercelEdge     -> FuncionNext      (X-Vercel-Cache: MISS; página dinámica)
3. FuncionNext    : proxy.ts ya tiene prefijo /es -> sin redirección
4. FuncionNext    : [lang]/layout -> getDictionary("es")
5. FuncionNext    : races/[id]/layout -> getRace("1102") (React.cache)
6. FuncionNext    -> DataCache        GET /races/1102 (revalidate 3600)
7a. DataCache     --> FuncionNext      HIT (dato < 1 h) [camino rápido]
7b. DataCache     -> APIRender         MISS/caducado: GET /races/1102
8. APIRender      : CORS -> GZip -> DataCacheMiddleware (ETag W/"35677c2df776-...")
9. APIRender      -> DuckDB           cursor.execute(sql, params) [threadpool]
10. DuckDB        --> APIRender        filas -> list[dict] -> Pydantic RaceDetail
11. APIRender     --> DataCache        200 JSON gzip + ETag + Cache-Control
12. FuncionNext   : en paralelo (Promise.all) /seasons/2024, /races/1102/laps,
↳ /races/1102/results;
SiteFooter pide /health (mismo camino 6-11, sin ETag)
13. FuncionNext   : render RSC (servidor) + props serializadas para LapTimesChart/ViolinChart
14. FuncionNext   --> VercelEdge      HTML (~497 KB) no-store
15. VercelEdge    --> Navegador        HTML; hidratación; ECharts (SVG) dibuja en el cliente

```

5.6 Medidas reales

5.6.1 Tamaño del código y superficie

Métrica	Valor
Ficheros versionados	218
Líneas API (api/app)	1 799 en 16 ficheros (7 <i>routers</i>)
Líneas web (src/app + components + lib + diccionarios)	2 661 + 1 821 + 164 + 676 = 5 322 (más 6 529 generadas: openapi.json, schema.d.ts)
Líneas ingesta / dbt / scripts / workflows	740 / 3 217 / 237 / 311
Endpoints	28 GET (33 esquemas en OpenAPI)
Rutas web	16 páginas × 2 idiomas + error/not-found + 2 <i>layouts</i>
Componentes	23 ficheros (12 generales + 11 de gráficos)
Pruebas Python	40 (30 API + 10 ingesta), todas correctas en 2,4 s
Pruebas e2e	38 ejecuciones (32 de escritorio + 6 móvil)

5.6.2 Tiempos de respuesta de la API desplegada

Medidos con `curl` desde España (vía Cloudflare MAD) el 28-09-2026 23:25 UTC. No hubo **cold start**: la instancia estaba despierta (la primera petición a `/health` tardó 171 ms). Tres rondas:

Petición	TTFB (s) r1 / r2 / r3	Tamaño JSON	Tamaño gzip
<code>/health</code>	0,102 / 0,108 / 0,094	—	205 B
<code>/seasons</code>	0,171 / 0,118 / 0,116	12 952 B	1 367 B
<code>/races/1102/laps</code>	0,304 / 0,348 / 0,317	465 464 B	27 401 B
<code>/rankings/drivers?order_by=wins&season_from=2000</code>	0,130 / 0,299 / 0,133	—	2 461 B*
<code>/drivers/lewis-hamilton</code>	0,095 / 0,119 / 0,100	—	—
<code>/races/1102/telemetry?drivers=max-verstappen,charles-leclerc</code>	0,118	59 141 B	23 478 B
<code>/seasons</code> con If-None-Match vigente	0,096 → 304	0	0

* sin filtro de temporada. Cabeceras observadas en `/seasons`: `Cache-Control: public, max-age=600, stale-while-revalidate=86400, Content-Encoding: gzip, etag: W/"35677c2df776-7b5df41fd06336e7", vary: Accept-Encoding, Origin, Server: cloudflare, cf-cache-status: DYNAMIC (Cloudflare no cachea), x-render-origin-server: uvicorn. CORS: con Origin: https://f1-data-platform.vercel.app responde access-control-allow-origin; con un origen ajeno no. /docs público (200). /races/999999 → 404.`

El *cold start* no se pudo observar en esta sesión; por diseño incluye arrancar el contenedor, descargar 52 MB de GitHub, calcular su SHA-256 y abrir DuckDB (el `HEALTHCHECK` concede 120 s de `start-period`), por lo que cabe esperar decenas de segundos.

5.6.3 Tiempos de la web desplegada

URL	TTFB (s)	Total (s)	HTML
<code>/</code>	0,679	0,679	307 → <code>/es</code>
<code>/es</code>	1,470 / 1,308	1,799 / 1,485	413 731 B
<code>/es/seasons/2021</code>	0,421	0,524	181 345 B
<code>/es/races/1102/pace</code>	0,559	0,825	497 443 B
<code>/es/drivers/lewis-hamilton</code>	0,323	0,347	136 511 B
<code>/es/records</code>	0,298	0,322	144 764 B

Todas X-Vercel-Cache: MISS, Cache-Control: private, no-store, función en iad1.

5.7 Debilidades y propuestas de mejora

5.7.1 API

#	Debilidad observada	Propuesta concreta	Justificación
A1	Cold start del plan gratuito de Render: tras ~15 min sin tráfico la instancia se detiene; al volver hay que arrancar el contenedor y descargar/verificar 52 MB antes de responder. Si Next recibe un error, la página muestra <code>error.tsx</code> .	(a) Plan de pago mínimo (instancia siempre activa) o (b) hornear la base en la imagen/usar un disco persistente para no descargarla en cada arranque; (c) un <i>ping</i> de disponibilidad (p. ej. cada 10 min) que además sirva de monitorización; (d) en Next, <code>timeout</code> + reintento y servir la última respuesta en caché.	La web depende en tiempo real de la API; con la Data Cache de Next el impacto se limita a las páginas no cacheadas, pero la primera visita tras un periodo inactivo es lenta o falla.
A2	Refresco con retraso de hasta 6 h y reloj que se reinicia en cada arranque (el bucle duerme antes de la primera comprobación).	Endpoint POST <code>/admin/refresh</code> protegido con un secreto (o un <i>deploy hook</i> de Render) que el <code>pipeline.yml</code> invoque tras publicar la release; mantener el sondeo como respaldo.	Reduce la latencia de datos a minutos y hace el comportamiento determinista.
A3	Sin versionado de la API (rutas en la raíz; <code>version="1.0.0"</code> solo informativo).	Prefijo <code>/v1</code> (con <code>APIRouter(prefix="/v1")</code> y redirección de las rutas actuales) y política de <i>deprecación</i> con cabecera <code>Deprecation/Sunset</code> .	Permite evolucionar esquemas sin romper a terceros; hoy la única garantía es el tipado de la propia web.
A4	Observabilidad mínima: <code>logging</code> , <code>basicConfig</code> en texto, sin identificador de petición, sin métricas ni trazas; los logs del plan gratuito tienen retención corta.	Logs JSON con <code>request_id</code> , ruta, estado, duración y versión de datos; métricas Prometheus (<code>/metrics</code>) o OpenTelemetry → Grafana Cloud/Honeycomb gratuitos; Sentry para excepciones; cabecera <code>Server-Timing</code> con el tiempo de consulta.	Sin ello no se puede medir el <i>cold start</i> , ni detectar consultas lentas (p. ej. <code>/rankings</code> con rangos amplios) ni fallos del refresco (que solo se registran).
A5	Sin limitación de tasa en una API pública con consultas costosas (<code>/races/{id}/laps</code> devuelve 465 KB; <code>/rankings</code> agrega todos los resultados).	<code>slowapi</code> (limitador por IP) o reglas de Cloudflare/Render; límites más estrictos en <code>/rankings</code> ; <code>timeout</code> de consulta en DuckDB.	Una instancia gratuita (0,1 CPU, 512 MB) se satura con pocas peticiones concurrentes.
A6	La caché CDN no se aprovecha: la respuesta lleva <code>Cache-Control: public</code> pero Cloudflare de Render responde <code>cf-cache-status: DYNAMIC</code> .	Poner un dominio propio en Cloudflare con <i>Cache Rules</i> para GET (respetando Vary) o servir la API detrás de Vercel/Cloudflare Workers; añadir <code>s-maxage</code> .	Con datos que cambian una vez a la semana, casi el 100 % de las peticiones serían aciertos de caché y la API apenas trabajaría (mitiga A1 y A5).
A7	Middleware de caché basado en <code>BaseHTTPMiddleware</code> (coste por petición y problemas conocidos con <i>streaming</i>); ETag calculada sobre la <i>query</i> sin normalizar (<code>?a=1&b=2 ≠ ?b=2&a=1</code>).	Middleware ASGI puro; normalizar los parámetros ordenándolos antes del hash.	Más rendimiento y mayor tasa de 304.
A8	Pruebas sobre un <i>fixture</i> de 2021-2024: no cubren casos históricos (coches compartidos de los 50, temporadas sin constructores antes de 1958) ni el contrato real frente a la base publicada.	Tests de contrato/propiedades con <code>Schemathesis</code> sobre <code>openapi.json</code> contra la base publicada en el <code>job api-image</code> ; <i>snapshot tests</i> de respuestas clave.	Detecta respuestas que no cumplen el esquema (p. ej. <code>None</code> en un campo no opcional) con datos reales.

#	Debilidad observada	Propuesta concreta	Justificación
A9	/docs y /openapi.json públicos, CORS abierto solo a la web; sin cabeceras de seguridad.	Mantener /docs (útil para divulgación) pero añadir X-Content-Type-Options: nosniff y documentar la política de uso; fijar la imagen base por digest.	Endurecimiento básico con coste nulo.

5.7.2 Web

#	Debilidad	Propuesta	Justificación
W1	Región de funciones en iad1 (EE. UU.) mientras la API está en Frankfurt: cada llamada no cacheada cruza el Atlántico dos veces; /es hace 5-6 llamadas y tarda 1,3-1,5 s de TTFB.	Fijar la región de funciones en fra1 (ajuste del proyecto en Vercel o vercel.json con "regions": ["fra1"]).	Cambio de una línea que reduce la latencia de cada fallo de caché en ~100-200 ms por ida y vuelta.
W2	Ninguna página se cachea en el CDN (no-store, MISS): leer searchParams o parámetros dinámicos sin generateStaticParams fuerza el renderizado por petición pese a revalidate = 3600.	Separar las páginas sin filtros (fichas de piloto/constructor, temporadas, pestañas de carrera) para renderizarlas con ISR (generateStaticParams para temporadas recientes + dynamicParams), o activar Cache Components/PPR de Next 16 con "use cache" y cacheLife. Mover los filtros a componentes que no invaliden la página entera.	Los datos cambian semanalmente: servir HTML estático desde el CDN eliminaría casi toda la latencia y el consumo de funciones.
W3	HTML muy pesado (hasta 497 KB en pace, 414 KB en inicio) porque las series completas se serializan como props de componentes cliente además de las tablas alternativas.	Reducir los datos enviados a los gráficos (solo columnas necesarias, submuestreo), cargar las tablas <details> bajo demanda o generar en servidor los gráficos más simples como SVG.	Mejora LCP/INP en móvil y el consumo de datos.
W4	Invalidación de caché desacoplada: la web puede mostrar datos de hasta 1 h tras cambiar la API.	Usar next: { tags: ["f1-data"] } y un route handler POST /api/revalidate (con secreto) que llame a revalidateTag; invocarlo desde el pipeline tras A2.	Publicación de datos coherente en toda la cadena.
W5	next.config.ts vacío: X-Powered-By: Next.js expuesto, sin CSP ni Referrer-Policy/Permissions-Policy.	poweredByHeader: false y headers() con CSP estricta (ECharts en SVG no necesita unsafe-eval).	Endurecimiento básico; Vercel ya aporta HSTS.
W6	Resiliencia ante la API: apiGet no tiene timeout ni reintento; un cold start largo acaba en error.tsx.	AbortSignal.timeout() + un reintento con espera; mensaje de «despertando el servicio» en error.tsx.	Complementa A1.
W7	SEO/i18n: no se declaran alternates.languages (hreflang) ni sitemap.ts/robots.ts; la elección de idioma no se recuerda (solo Accept-Language).	generateMetadata con alternates, app/sitemap.ts, cookie NEXT_LOCALE leída en proxy.ts.	Indexación correcta de las dos versiones y mejor experiencia.
W8	Sin pruebas unitarias de src/lib (formato de tiempos, driverCode, parseYear) ni visual regression.	Vitest para lib/ y snapshots visuales de Playwright para los gráficos.	Errores de formato de tiempos son difíciles de detectar con axe/e2e.

5.7.3 CI/CD

#	Debilidad	Propuesta	Justificación
C1	Acciones sobre Node 20 en retirada: <code>actions/upload-artifact@v4</code> , <code>astral-sh/setup-uv@v6</code> , <code>actions/upload-pages-artifact@v3</code> , <code>actions/deploy-pages@v4</code> (GitHub ya avisa de la deprecación del <i>runtime</i> Node 20).	Actualizar a las versiones mayores con Node 24 (<code>upload-artifact@v5+</code> , <code>setup-uv@v7</code> , etc.) y fijar por SHA.	Evita que los workflows empiecen a fallar cuando GitHub retire Node 20.
C2	Sin Dependabot/Renovate ni auditoria de dependencias (<code>uv.lock</code> , <code>package-lock.json</code> , acciones, imagen Docker).	<code>.github/dependabot.yml</code> con ecosistemas <code>uv</code> , <code>npm</code> , <code>github-actions</code> y <code>docker</code> , agrupando actualizaciones menores; <code>npm audit/pip-audit</code> en CI; CodeQL.	Mantenimiento de seguridad automatizado.
C3	Deriva de herramientas: pre-commit fija <code>ruff 0.8.4</code> pero la CI usa <code>ruff 0.16.9</code> del lock.	Hook local que ejecute <code>uv run ruff</code> , o actualizar <code>rev</code> con <code>pre-commit autoupdate</code> (Dependabot no lo cubre).	Lo que pasa en local debe pasar en CI.
C4	Despliegues sin puerta de calidad: Render y Vercel despliegan en el push aunque la CI falle (el 28-09 hubo dos pushes con la CI en rojo; con la configuración actual nada impide que se desplieguen).	Render: <code>autoDeploy: false + deploy hook</code> llamado desde un job <code>deploy</code> con <code>needs: [python, api-image]</code> ; Vercel: <code>Required checks/«Ignored Build Step»</code> o despliegue con <code>vercel deploy --prebuilt</code> desde Actions.	Evita publicar una versión rota.
C5	Doble construcción de la imagen (CI y Render) y ninguna se versiona.	Publicar la imagen en GHCR desde CI (etiqueta = SHA) y desplegar esa imagen en Render (<code>image:</code> en lugar de <code>dockerfilePath</code>).	Lo probado es exactamente lo desplegado; despliegues más rápidos.
C6	Las pruebas e2e dependen de la <code>release data-latest</code> viva (no reproducibles en el tiempo) y el pipeline no verifica el despliegue tras publicar.	E2E en PR contra la base fijada por <code>manifest</code> (guardar el SHA); tras publicar, un paso de <code>smoke test</code> contra la API/web de producción (<code>BASE_URL</code> , ya soportado por <code>playwright.config.ts</code>).	Separar fallos de código de cambios de datos y detectar regresiones en producción.
C7	Los workflows programados de un repo público se desactivan tras 60 días sin actividad (lo avisa el README).	Monitorizar con una alerta (p. ej. <code>/health → generated_at</code> con antigüedad > 8 días) o <code>keepalive</code> .	Evita que la plataforma deje de actualizarse en silencio.

5.7.4 Infraestructura

- **Punto único de fallo:** una única instancia gratuita (0,1 CPU/512 MB) sin réplicas. Alternativas de bajo coste: servir los Parquet/DuckDB estáticos y consultar con DuckDB-WASM, o desplegar la API en una plataforma con *scale-to-zero* rápido (Fly.io, Cloud Run) con la base dentro de la imagen.
- **Almacenamiento efímero:** `/data` se pierde en cada reinicio; un disco persistente o una capa de imagen con los datos evitaría descargas repetidas (y el consumo de ancho de banda de GitHub).
- **Latencia geográfica:** alinear Render (Frankfurt) y Vercel (`fra1`) y poner una CDN delante de la API (`A6`, `W1`, `W2`).
- **Seguridad de la cadena de suministro:** verificación SHA-256 de la base ya implementada (bien); faltaría firmar el manifiesto o usar *artifact attestations* de GitHub para que la API compruebe que la release la generó el workflow del repositorio.
- **Monitorización externa:** un comprobador de disponibilidad (UptimeRobot/Better Stack) sobre `/health` y la web, con alerta si `data.generated_at` supera 8 días.

5.8 Resumen

La aplicación sigue una arquitectura de tres capas muy desacoplada: los datos se publican como artefacto inmutable y verificado (release `data-latest`), la API es un servicio sin estado de solo lectura sobre DuckDB con intercambio en caliente de versiones y caché HTTP por versión, y la web es un cliente de servidor tipado extremo a extremo (OpenAPI → TypeScript, comprobado en CI) con un nivel de accesibilidad verificado automáticamente (axe WCAG 2.2 AA en 21 URL, tema oscuro, sin JS). Los puntos débiles son operativos más que de diseño: *cold start* y recursos del plan gratuito, ninguna capa de caché CDN efectiva (ni en la API ni en la web), funciones de Vercel en otra región que la API, despliegues no condicionados a la CI, acciones en Node 20 y ausencia de observabilidad y de actualización automática de dependencias.

CAPÍTULO 6

Arquitecturas y estructuras alternativas

Nota de trabajo para el informe de situación (29/09/2026). Recoge qué alternativas existen a la estructura y la arquitectura actuales de `f1-data-platform`, con fuentes, y propone qué adoptar, qué considerar y qué descartar **para este proyecto en concreto**: una sola persona, coste 0 €, fines de portfolio y TFG, datos que cambian una vez por semana.

Leyenda de la recomendación: **ADOPTAR** (merece la pena ya), **CONSIDERAR** (útil si se dan ciertas condiciones o como ampliación), **DESCARTAR** (no compensa aquí). Leyenda de esfuerzo: S (horas), M (1–3 días), L (una semana o más).

6.1 Resumen ejecutivo

1. **La arquitectura actual está bien planteada para su contexto.** Monorepo, uv, dbt-duckdb con capas medallion y modelo Kimball, snapshot semanal en GitHub Releases, API de solo lectura y web con ISR: es lo que recomiendan las guías de referencia para un proyecto analítico pequeño (véanse §2 y §3). No hay razones de peso para una reescritura.
2. **El punto débil es el serving:** la web depende de una API en el plan gratuito de Render (se duerme a los 15 min y tarda ~1 min en despertar) para servir datos que **solo cambian los lunes**. Es la pieza con más riesgo y la que más alternativas razonables tiene (§5).
3. **La alternativa de más valor por esfuerzo** es publicar desde el pipeline una «API estática» (JSON precalculado por recurso, o Parquet por tabla) en un CDN gratuito y hacer que Next.js lea de ahí; la API FastAPI se mantiene como servicio público y documentación (OpenAPI), pero deja de estar en el camino crítico de la web. Detalle en §5.3 y arquitectura B en §8.
4. **Mejoras de estructura baratas:** declarar como `sources` de dbt todas las entradas bronze (hoy solo F1DB lo está), subcarpetas en `intermediate/`, un `justfile` (o `Taskfile`) en la raíz como índice de comandos, y releases de datos inmutables con fecha además de `data-latest`.
5. **Orquestador (Dagster/Prefect/Airflow/Kestra):** CONSIDERAR solo como ejercicio de portfolio; para este volumen, GitHub Actions es suficiente. Hay que tener en cuenta que en julio de 2026 **Prefect compró Dagster Labs** (§4). Lo que sí se recomienda es disparar el pipeline **según el calendario de F1** en vez de un cron fijo de lunes.
6. **Frente:** mantener ECharts (con su módulo `aria` y los *decals*) y el `i18n` propio, que sigue la guía oficial de Next.js; Observable Plot es la alternativa más accesible si se rehace algún gráfico sencillo en SVG (§6).
7. **dbt v2** (motor Fusion en Rust, con adaptador DuckDB integrado) acaba de publicarse. El adaptador DuckDB figura aún como *beta*: CONSIDERAR la migración más adelante, no ahora (§2.3).

6.2 Punto de partida medido en el repositorio (28/09/2026)

Elemento	Situación actual
Repositorio	Monorepo: ingestion/, transform/, api/, web/, scripts/, tests/, docs/, .github/workflows/ (pipeline.yml, ci.yml), render.yaml
Python	Un único pyproject.toml (paquete f1-data-platform, wheel con ingestion y api en layout plano) y grupos de dependencias ingest, api, transform y dev; la imagen Docker de la API instala solo --group api
dbt	staging/<fuente>/stg_<fuente>__<entidad>.sql (f1db, fastf1, formula1db, ergast), intermediate/int_*.sql (10 modelos, sin subcarpetas), marts/core (5 dimensiones y 8 hechos) y marts/metrics (4 agg_*), quality/qa_* (7 modelos); esquemas silver, gold y quality; seeds de correcciones
Fuentes dbt	Solo F1DB está declarada en _f1db__sources.yml; los modelos stg_fastf1_*, stg_formula1db_* y stg_ergast_* leen Parquet directamente
Artefactos de datos	f1.duckdb 54,5 MB; gold-parquet.zip 28 MB (17 tablas; fact_laptimes 12,6 MB y 1,27 M filas, fact_quali_telemetry 14,1 MB y 0,52 M filas; el resto < 0,4 MB cada una); bronze.tar.gz 51,7 MB. Nota: el tamaño real del DuckDB es ~55 MB, no ~200 MB
Orquestación	pipeline.yml: cron 0 6 * * 1 + workflow_dispatch con temporadas y force_f1db; restaura bronze de la release, ingesta F1DB y FastF1, dbt build, publica en data-latest con --clobber
API	FastAPI + DuckDB solo lectura; descarga f1.duckdb de la release al arrancar y cada 6 h; ETag por versión de datos; Render free (Frankfurt)
Web	Next.js 16.3, App Router, src/app/[lang]/... (16 page.tsx), lib/api/client.ts con fetch(..., { next: { revalidate: 3600 } }), ECharts 6 + d3-geo, diccionarios JSON propios (getDictionary), Playwright + axe; 8 páginas usan searchParams (filtros como formularios GET)

6.3 Estructura del repositorio

6.3.1 Monorepo frente a multirrepo

Opción	Ventajas	Inconvenientes	Coste	Esfuerzo	Riesgo	Recomendación
Monorepo actual	Un solo sitio para el TFG; cambios atómicos entre contrato OpenAPI, API y web (npm run gen:api); una CI; un README que cuenta la historia completa	CI y despliegues deben filtrar por rutas (ya se hace: buildFilter en render.yaml, Root Directory web en Vercel)	0 €	—	Bajo	ADOPTAR (mantener)
Multirrepo (f1-data, f1-api, f1-web)	Permisos y versiones independientes; cada repo más pequeño	Coordinar versiones del contrato entre repos; tres CI; se diluye la narrativa del portfolio; el cron del repo de datos se desactiva a los 60 días sin commits si es público	0 €	M	Medio	DESCARTAR

Para una persona y un producto con un contrato interno (OpenAPI) que cambia a la vez en API y web, el monorepo es la opción natural; el problema típico de los monorepos (tiempos de CI, propiedad de código) no aparece a esta escala.

6.3.2 Layout de Python y uv

Situación: paquetes `ingestion/` y `api/` en la raíz (*flat layout*), un único proyecto `uv` con grupos de dependencias. Es una solución deliberada y funciona: la imagen de la API solo instala `duckdb`, `fastapi` y `uvicorn`.

Alternativas:

- **src-layout** (`src/f1_platform/ingestion`, `src/f1_platform/api`). Evita importar por accidente el código del directorio de trabajo en lugar del paquete instalado y es lo que genera `uv init --package`. Aquí el beneficio es pequeño porque no se publica en PyPI. → **CONSIDERAR** (S) solo si se reorganiza por otro motivo.
- **uv workspaces**: cada componente con su `pyproject.toml` (p. ej. `packages/ingestion`, `apps/api`, y quizá un `packages/core` con utilidades comunes), un único `uv.lock` y `uv run --package api ...`. La documentación de `uv` lo recomienda para «varios paquetes interrelacionados en un mismo repositorio» y lo desaconseja cuando los miembros tienen requisitos en conflicto o se quiere un entorno virtual separado por miembro; además impone un único `requires-python` para todo el *workspace* (`uv`: [Using workspaces](#)).
 - Ventajas: dependencias declaradas por componente (la API no «ve» `pandas` ni `FastF1` ni por error), versiones separadas, Docker más limpio (`uv sync --package api`).
 - Inconvenientes: hoy los grupos ya dan el aislamiento que importa (imagen ligera); hay que mover imports, rutas de tests, `Dockerfile`, workflows y README.
 - Coste 0 €; esfuerzo M; riesgo bajo-medio (rotura de rutas).
 - → **CONSIDERAR**. Tiene sentido si entra la fase 7 (analítica avanzada/ML con dependencias pesadas como `scikit-learn` o `statsmodels`), que merecería su propio miembro.
- **Proyectos uv totalmente independientes** (un lock por carpeta). → **DESCARTAR**: tres locks que mantener sin beneficio.

Referencias adicionales sobre *workspaces*: [pydevtools: monorepo con uv workspaces](#).

6.3.3 Estructura del proyecto dbt frente a la guía de dbt Labs

Guía de referencia: [How we structure our dbt projects](#) (capas staging → intermediate → marts; de datos «con forma de la fuente» a datos «con forma del negocio»).

Recomendación de dbt Labs	Proyecto actual	Valoración
Staging en carpetas por sistema de origen, <code>stg_[fuente]__[entidad]s</code> con doble guion bajo (staging)	Exactamente así (<code>staging/f1db/stg_f1db__races.sql...</code>)	Cumple
Staging materializado como vista , sin <i>joins</i> ni agregaciones; <i>base models</i> solo si hace falta unir	Vistas; <code>+materialized: view</code>	Cumple
<code>Un_[dir]__sources.yml</code> por carpeta de fuente	Solo <code>f1db</code> ; <code>FastF1</code> , <code>formula1db</code> y <code>Ergast</code> leen Parquet a mano	Mejorable
Intermediate agrupado en subcarpetas por área de negocio, nombres <code>int_[entidad]s_[verbo]s</code>	10 modelos planos (<code>int_laptimes</code> , <code>int_lap_numbering...</code>)	Aceptable; mejorable con subcarpetas

Recomendación de dbt Labs	Proyecto actual	Valoración
Marts por entidad, anchos y desnormalizados, en carpetas por área si hay más de ~10 (marts)	Esquema en constelación Kimball (<code>dim_*</code> , <code>fact_*</code>) + <code>agg_*</code>	Desviación justificada (reproduce el modelo del TFG y Power BI necesita estrella); dbt Labs también documenta Kimball (Building a Kimball dimensional model with dbt)
YAML de documentación por carpeta/modelo	<code>_core__models.yml</code> , <code>_quality__models.yml</code> , <code>_seeds.yml</code>	Casi completo (falta <code>metrics/</code> e <code>intermediate/</code>)

Propuestas concretas:

1. **Declarar todas las entradas bronze como sources** (`_fastf1__sources.yml`, `_formula1db__sources.yml`, `_ergast__sources.yml`) con `meta.external_location` apuntando al Parquet (patrón soportado por dbt-duckdb, [README de dbt-duckdb](#)). Ganancias: linaje completo en dbt docs desde el bronze, pruebas de `source freshness` (p. ej. «la última carrera cargada no tiene más de 10 días en temporada»), selección `dbt build -s source:fastf1+`. Coste 0 €, esfuerzo S, riesgo bajo. → **ADOPTAR**.
2. **Subcarpetas en `intermediate/`**: `laps/` (`formula1db`, `fastf1`, `ergast`, alineación, numeración, completitud, fusión), `race_data/` (correcciones, dorsales), `tyres/`. Esfuerzo S. → **ADOPTAR**.
3. **Exportar el gold a Parquet desde dbt** con la materialización `external` de dbt-duckdb en vez de hacerlo en `snapshot pack` (mismo README). Así el Parquet es un producto de dbt, con contratos y tests. Esfuerzo S–M. → **CONSIDERAR** (sobre todo si se adopta la arquitectura B).
4. **Marts «de servicio» anchos (OBT)** solo para la web, p. ej. `mart_web__race_lap_chart` o `mart_web__driver_profile`, sin tocar la estrella del TFG. Permiten que la API o los JSON estáticos sean un `select * where race_id = ?`. → **CONSIDERAR** junto con §5.3.
5. **dbt v2 / Fusion**. dbt Labs (ya fusionada con Fivetran desde el 1/6/2026) ha publicado dbt v2 sobre el motor Fusion en Rust, con Apache 2.0 y con **adaptador DuckDB integrado** ([DuckDB: DuckDB now ships inside dbt v2; dbt Core v2 is here](#)). La guía de migración marca DuckDB como «CLI only, beta» y enumera cambios incompatibles (anclas YAML, claves YAML desconocidas que pasan a ser error, `config.get()` sobre `meta`, eliminación de `--models`, tests unitarios primero...) ([Upgrading to v2](#)). dbt Labs afirma que v1 seguirá soportado sin fecha de fin. → **CONSIDERAR** a medio plazo: primero `dbt parse --use-v2-parser` en 1.12 para ver qué rompe; migrar cuando el adaptador DuckDB salga de beta. Esfuerzo M; riesgo medio.
6. **SQLMesh** como alternativa a dbt: Fivetran lo compró (Tobiko, 2025) y lo cedió a la Linux Foundation en marzo de 2026 ([The New Stack](#)). Aporta planes/entornos virtuales y detección de cambios, pero no compensa reescribir ~50 modelos y la memoria del TFG ya habla de dbt. → **DESCARTAR**.

6.3.4 Estructura del proyecto Next.js

La documentación oficial ([Project structure](#), carpeta `src`) admite tres estrategias: todo fuera de `app/` (en `src/components`, `src/lib`), todo colocado dentro de cada segmento de ruta, o por funcionalidad; propone carpetas privadas `_carpeta` para lo que no es ruta. El proyecto sigue la primera: `src/app/[lang]/...`, `src/components/charts`, `src/lib/api`, `src/dictionaries`. Es válida y coherente.

Mejora opcional: colocar los componentes que solo usa una ruta junto a ella (`app/[lang]/races/[id]/tyres/_components/stint-chart.tsx`) y dejar en `src/components` solo los compartidos (`chart-figure`, `echart`). Esfuerzo S; beneficio: localizar rápido el código de cada página del TFG. → **CONSIDERAR** (cosmético).

6.3.5 Herramientas de monorepo y ejecutores de tareas

Hoy los comandos están repartidos entre `uv run f1-ingest ...`, `cd transform && dbt ...` con `DBT_PROFILES_DIR`, `npm` en `web/` y el `README`.

Herramienta	Qué aporta	Encaje aquí	Recomendación
just (justfile)	Ejecutor de comandos (no sistema de <i>build</i>), sintaxis sencilla, multiplataforma, recetas con cualquier intérprete (comparativa Make/Task/just)	Ideal para <code>just ingest</code> , <code>just build</code> , <code>just api</code> , <code>just web</code> , <code>just e2e</code> , <code>just snapshot</code> ; funciona en Windows (el autor trabaja en Windows)	ADOPTAR (S)
Taskfile (go-task)	YAML, dependencias entre tareas y <i>checksums</i> de ficheros para saltarse pasos ya hechos; <i>includes</i> por paquete (Taskfile)	Igual de válido que just; más verboso	Alternativa a just
Makefile	Universal en Linux/macOS	Tabs, peor en Windows	DESCARTAR frente a just/Task
Turborepo	Caché de tareas y grafo de paquetes; está incorporando Python con <code>uv</code> como lenguaje nativo (RFC Native Python support)	Un solo paquete JS; caché poco útil	DESCARTAR
Nx	Monorepo políglota con <i>plugins</i> (Python/uv comunitarios), <i>affected</i> , caché distribuida (Nx vs Turborepo)	Sobredimensionado para 4 componentes y una persona	DESCARTAR

6.4 Arquitectura de datos

6.4.1 Medallion frente a otras organizaciones y modelos

El proyecto combina dos ideas ortogonales: **medallion** (capas por grado de calidad: *bronze* crudo, *silver* validado, *gold* enriquecido, según la definición de [Databricks](#)) y, dentro del *gold*, un **modelo dimensional Kimball** (constelación de hechos). Además mapea las capas de *dbt*: *bronze* = Parquet de ingesta, *silver* = *staging* + *intermediate*, *gold* = *marts*.

Alternativa	Ventajas	Inconvenientes	Encaje	Recomendación
Medallion + Kimball (actual)	Vocabulario conocido; el <i>bronze</i> permite reprocesar; la estrella es lo que espera Power BI y lo que describe la memoria (Fig. 5.7)	Nombres « <i>bronze/silver/gold</i> » y « <i>staging/marts</i> » conviven: hay que explicar el mapeo	Óptimo	ADOPTAR (mantener) y documentar el mapeo
Kimball «puro» sin capas explícitas	Menos jerga	Se pierde la trazabilidad <i>bronze</i> → <i>gold</i> que justifica las correcciones de fuentes	Peor	DESCARTAR
Data Vault 2.0 (hubs, links, satellites)	Auditoría e historización, pensado para muchas fuentes cambiantes y entornos regulados (comparativa Kimball vs DV)	Duplica el número de modelos; exige una capa dimensional encima para consumir	Excesivo (4 fuentes, 1 persona)	DESCARTAR
One Big Table (OBT)	Consultas sin <i>joins</i> , ideal para servir a una web o a DuckDB-WASM; <i>dbt Labs</i> recomienda <i>marts</i> anchos	Redundancia; peor para Power BI; se pierde el paralelismo con el TFG	Útil como capa de servicio adicional	CONSIDERAR (<i>marts mart_web_*</i> , §2.3)

6.4.2 Formatos de almacenamiento y de tabla

Formato	Situación en 2026	Ventajas aquí	Inconvenientes aquí	Recomendación
Fichero DuckDB (f1.duckdb)	Actual; se puede abrir en remoto con ATTACH 'https://...' en solo lectura desde DuckDB 1.1 (DuckDB over HTTPS/S3)	Un solo fichero, 55 MB; la API lo abre directamente	Formato binario ligado a la versión de DuckDB; no es interoperable fuera de DuckDB	ADOPTAR (mantener)
Parquet por tabla	Actual (gold-parquet.zip)	Interoperable (Power BI, pandas, Polars, DuckDB-WASM); lectura parcial con peticiones <i>Range</i>	Dentro de un zip no se puede leer por <i>Range</i> ; habría que publicarlos sueltos	ADOPTAR : publicar además los Parquet sin comprimir en zip , uno por tabla, y particionar fact_laptimes/fact_quali_telemetry por temporada
DuckLake 1.0	Estable desde abril de 2026 (DuckDB 1.5.2); catálogo en SQL (SQLite, Postgres o DuckDB), <i>snapshots</i> y <i>time travel</i> , <i>data inlining</i> ; admite «un lakehouse de solo lectura sin autenticación» con almacenamiento y un endpoint HTTPS público (DuckLake 1.0; extensión)	Versionado de datos nativo (cada ejecución semanal sería un <i>snapshot</i> consultable); encaja con DuckDB y con dbt v2	Requiere almacenamiento de objetos con URLs estables (no encaja con <i>-clobber</i> en Releases); más piezas que explicar	CONSIDERAR (arquitectura C)
Apache Iceberg	DuckDB ya escribe en Iceberg vía catálogo REST (Writes in DuckDB-Iceberg)	Estándar de la industria, buen punto para el CV	Necesita catálogo (servicio); sin valor práctico con 55 MB	DESCARTAR
Delta Lake	Extensión de DuckDB con escritura y <i>time travel</i> desde mayo de 2026 (Delta grows up)	Igual que Iceberg	Igual que Iceberg	DESCARTAR

6.4.3 Dónde publicar los artefactos de datos

Opción	Límites gratuitos (2026)	Ventajas	Inconvenientes	Recomendación
GitHub Releases (actual)	Ficheros < 2 GiB, hasta 1000 assets por release, sin límite de tamaño total ni de ancho de banda (About releases)	Gratis, junto al código, CLI gh, notas de release como registro	No envía cabeceras CORS : el navegador no puede leer los assets (la descarga es un 302 a <code>release-assets.githubusercontent.com</code> sin Access-Control-Allow-Origin) (issue de ejemplo ; explicación). Sin lecturas parciales útiles para DuckDB-WASM	ADOPTAR (mantener) como archivo y entrada del pipeline; añadir releases inmutables con fecha (§3.4)

Opción	Límites gratuitos (2026)	Ventajas	Inconvenientes	Recomendación
Cloudflare R2	10 GB-mes, 1 M operaciones clase A y 10 M clase B al mes, egress gratis (R2)	CORS configurable, <i>Range</i> , compatible S3 (DuckDB <code>https</code>); ideal para Parquet que lea el navegador	Requiere cuenta Cloudflare y tarjeta; el dominio <code>r2.dev</code> tiene <i>rate limiting</i> y no está pensado para producción: hace falta dominio propio (límites <code>r2.dev</code> ; ejemplo <code>DuckDB-WASM + R2</code>)	CONSIDERAR (arquitectura C, o B si se quiere Parquet en el navegador)
Hugging Face Datasets	Repos públicos generosos (orientativo: hasta 300 GB por repo sin pedir más) (Storage limits)	Visibilidad en la comunidad de datos, <i>dataset viewer</i> y SQL en la web, <code>hf://</code> desde DuckDB (HF + DuckDB); versionado git por <i>commit</i>	Las descargas redirigen al puente Xet, que falla el <i>preflight</i> CORS con <i>*Range*</i> y bloquea DuckDB-WASM (issue #7931); licencias: los datos de <code>formula1db.com</code> tienen permiso «para divulgación», conviene confirmarlo antes de redistribuir en otra plataforma	CONSIDERAR como escaparate del dataset gold (no como origen de la web)
MotherDuck	Plan gratuito: 10 GB, 10 horas de cómputo al mes, 3 usuarios (pricing)	DuckDB gestionado; <i>dbt</i> puede escribir directamente; cliente WASM propio	Dependencia de un proveedor y de un <i>token</i> en la web/API; el plan gratuito cambió en 2025-2026 (desapareció el plan Lite)	DESCARTAR como pieza central; CONSIDERAR solo para experimentar
GitHub Pages / Cloudflare Pages (sitio estático con los datos)	Pages: sitio ≤ 1 GB, ~100 GB/mes de ancho de banda «blando» (límites); Cloudflare Pages: ancho de banda ilimitado, 500 <i>builds</i> /mes, 20 000 ficheros por despliegue (resumen de límites)	CDN, CORS sencillo, sin cuentas nuevas en el caso de GitHub Pages (ya usado para <i>dbt docs</i>)	Límite de ficheros en Cloudflare Pages; tamaño en GitHub Pages	CONSIDERAR para la «API estática» (§5.3)

6.4.4 Versionado de datos

Hoy `data-latest` se sobrescribe cada semana (`--clobber`): no se puede volver a la versión de hace dos lunes ni reproducir un resultado antiguo, aunque `manifest.json` guarda los SHA-256.

Opción	Esfuerzo	Recomendación
Releases inmutables con fecha (<code>data-2026-09-28</code>) + <code>data-latest</code> como puntero móvil; conservar las N últimas (p. ej. 12) con un paso de limpieza. La API ya valida el SHA del manifest	S	ADOPTAR
Registrar en <code>manifest.json</code> el SHA del commit de código y la versión de <i>dbt</i> /DuckDB	S	ADOPTAR
<i>Snapshots</i> de DuckLake (versión y <i>time travel</i> nativos)	M	CONSIDERAR (arquitectura C)

Opción	Esfuerzo	Recomendación
DVC o lakeFS (versionado tipo git para datos) (comparativa lakeFS)	M	DESCARTAR: pensado para datasets grandes o ML; aquí basta con releases fechadas
Snapshots SCD tipo 2 de dbt (<code>dbt snapshot</code>) sobre F1DB para ver cómo cambian los datos oficiales	M	CONSIDERAR: tendría interés analítico (auditar correcciones de F1DB)

6.5 Orquestación y observabilidad

6.5.1 GitHub Actions frente a orquestadores

Limitaciones reales del cron de GitHub Actions, según la documentación oficial: los eventos `schedule` pueden retrasarse con carga alta, sobre todo al principio de cada hora, y **en un repositorio público los workflows programados se desactivan tras 60 días sin actividad** ([Events that trigger workflows](#)). Un análisis práctico concluye que Actions basta para una persona o un equipo pequeño con un `dbt build` periódico y sin dependencias externas complejas, y que conviene pasar a un orquestador cuando se necesita observabilidad por modelo, *catch-up* de ejecuciones perdidas, reintentos finos o mezclar dbt con otras tareas ([comparativa de orquestadores para dbt](#)).

Opción	Ventajas	Inconvenientes	Coste	Esfuerzo	Riesgo	Recomendación
GitHub Actions (actual)	Cero infraestructura, logs y artefactos incluidos, <code>workflow_dispatch</code> con parámetros ya implementado	Sin grafo por activo ni reintento por modelo; cron impreciso; desactivación a los 60 días	0 € (minutos ilimitados en repos públicos)	—	Bajo	ADOPTAR (mantener)
Dagster (OSS) + <code>dagster-dbt</code>	Modelo por activos: cada modelo dbt es un activo con linaje, particiones (temporada/carrera), <i>sensors</i> , <i>backfills</i> desde la UI; es el que mejor integra dbt (Dagster & dbt); ejemplo F1 con Dagster + dbt + DuckDB: frizzleqq/f1-dbt-duckdb	Necesita dónde ejecutarse: Dagster+ ya no incluye créditos en Solo/Starter desde mayo de 2026 (Solo: 10 \$/mes + 0,04 \$/crédito) (cambios de precios); autoalojado exige un servidor encendido o ejecutarlo dentro de Actions (<code>dagster job execute</code>), lo que anula parte de la ventaja. Prefect compró Dagster Labs en julio de 2026 ; ambos productos siguen, pero hay incertidumbre de hoja de ruta (anuncio)	0 € autoalojado en Actions; >0 € gestionado	L	Medio	CONSIDERAR solo como ejercicio de portfolio (definir activos y particiones por carrera y ejecutarlos desde Actions)
Prefect	Python puro, reintentos y mapeo dinámico; bueno para la ingesta (API FastF1 con límite de 500 peticiones/h)	Menos integrado con dbt que Dagster	Plan gratuito limitado	L	Medio	DESCARTAR frente a Dagster para este caso
Apache Airflow	Estándar de la industria; ejemplo F1 con Airflow + MinIO + Postgres + dbt: m1nk12/f1-data-pipeline	Pesado (planificador, base de datos, workers); sin alojamiento gratuito	>0 € o PC encendido	L	Alto	DESCARTAR
Kestra	Declarativo en YAML, <i>plugin</i> dbt, vista Gantt de modelos (Kestra + dbt)	Servidor Java autoalojado	>0 € o PC encendido	L	Medio	DESCARTAR

6.5.2 Disparar el pipeline según el calendario de F1

El cron fijo del lunes a las 06:00 UTC tiene tres problemas: se ejecuta en semanas sin carrera, llega tarde cuando hay dos carreras seguidas o una carrera en sábado (Las Vegas, Baréin/Arabia en algunos años) y no reintenta si FastF1 aún no ha publicado los datos.

Alternativa sin orquestador («sensor de pobre») — **ADOPTAR** (S–M, 0 €, riesgo bajo):

1. Cron **diario** (o cada 6 h) muy barato que solo ejecuta un paso de decisión en Python: `fastf1.get_event_schedule(año)` da `Session5DateUtc` (hora de la carrera) de cada evento (**FastF1 events**). Si hay una carrera terminada hace más de N horas (p. ej. 3–6 h) y `manifest.json` de `data-latest` no la incluye, lanza el pipeline completo (`gh workflow run pipeline.yml` o `repository_dispatch`); si no, termina en segundos.
2. Si FastF1 aún no tiene datos, el siguiente disparo lo reintenta solo (la ingesta ya es incremental e idempotente).
3. Mantener `workflow_dispatch` con temporadas/rondas como mecanismo de **backfill**; añadir una matriz por temporada si alguna vez hay que recargar 2018–2025 (cuidado con el límite de 500 peticiones/h de FastF1: mejor en serie con `max-parallel: 1`).
4. Contra la desactivación a los 60 días: el propio pipeline podría hacer un *commit* periódico (p. ej. actualizar un `docs/data-status.md`), o usar una acción *keepalive* (**gh-action-keepalive**). Crear releases **no** cuenta como actividad.

Con Dagster esto sería un *sensor* nativo sobre el calendario y una partición por carrera; es la versión «de libro» del mismo mecanismo.

6.5.3 Observabilidad

Opción	Qué aporta	Esfuerzo	Recomendación
Publicar <code>run_results.json</code> y <code>manifest.json</code> de dbt en la release y resumir tiempos/tests en <code>GITHUB_STEP_SUMMARY</code>	Historial de ejecuciones y duración por modelo sin herramientas nuevas	S	ADOPTAR
<code>dbt_source_freshness/dbt_utils.recency</code> sobre las fuentes (requiere §2.3.1)	Alerta si FastF1 o F1DB dejan de actualizarse (recency)	S	ADOPTAR
Aviso por correo/issue automático si falla el pipeline (Actions ya notifica al autor; se puede abrir un <i>issue</i> con <code>gh issue create</code>)	Visibilidad pública del estado	S	CONSIDERAR
Elementary OSS (paquete dbt + CLI con informe HTML estático, detección de anomalías) (repo)	Informe de observabilidad publicable en GitHub Pages junto a <code>dbt docs</code>	M	CONSIDERAR : comprobar primero el soporte del adaptador DuckDB (no confirmado en las fuentes consultadas)
<code>dbt_artifacts</code> (Brooklyn Data)	Modelos sobre metadatos de ejecución	M	DESCARTAR: no figura soporte DuckDB (repo)
La página <code>/quality</code> de la web mostrando también la <i>historia</i> de <code>qa_summary</code> por release	Observabilidad de cara al público; buen material para el TFG	M	CONSIDERAR (requiere releases fechadas, §3.4)

6.6 Serving: cómo llegan los datos a la web

6.6.1 Situación y problema

Cadena actual: GitHub Release → API FastAPI en Render (descarga el DuckDB al arrancar) → Next.js en Vercel (ISR de 1 h). Datos de Render free: el servicio **se duerme tras 15 minutos** sin peticiones, **tarda alrededor de un minuto** en volver, 750 h/mes, disco efímero y «Render puede reiniciar el servicio en cualquier momento» ([Render: Deploy for free](#)). El README lo mitiga con la caché de ISR, pero cualquier página no visitada en la última hora (la mayoría de las ~1 172 carreras × 7 subpáginas × 2 idiomas) paga el arranque en frío. Además, cada arranque vuelve a descargar 55 MB de GitHub.

Vercel Hobby: 1 M invocaciones de funciones, 4 h de CPU activa, 100 GB de transferencia, 1 M *edge requests*/mes, uso **no comercial** ([Vercel Hobby](#)). Es holgado para este tráfico.

6.6.2 Alojjar la misma API en otro sitio

Servicio	Plan gratuito 2026	Arranque en frío	Encaje	Recomendación
Render (actual)	750 h/mes, duerme a los 15 min	~1 min	Funciona; experiencia lenta en páginas frías	Mantener mientras no se cambie la arquitectura
Google Cloud Run	2 M peticiones, 180 000 vCPU-s y 360 000 GiB-s al mes (facturación por petición) (pricing)	Segundos (imagen pequeña); <code>min-instances=0</code> gratis, >0 se cobra	Mejor arranque que Render; la descarga de 55 MB en cada arranque sigue ahí (se puede meter el DuckDB en la imagen o leerlo de GCS)	CONSIDERAR si se mantiene API en el camino crítico; exige tarjeta y cuenta de facturación
Koyeb	Instancia gratuita que escala a cero tras 1 h sin tráfico; <i>deep sleep</i> de 1–5 s al despertar (scale-to-zero)	1–5 s	Muy buen encaje para una API pequeña	CONSIDERAR (la mejor sustitución directa de Render encontrada)
Hugging Face Spaces (Docker, CPU basic)	Gratis; se duerme tras 48 h sin actividad, 30–60 s al despertar (spaces)	Raro (48 h) pero lento	Curioso para portfolio de datos; no es su uso previsto	CONSIDERAR solo como réplica
Fly.io	Sin plan gratuito para cuentas nuevas desde oct. 2024 (solo prueba) (resumen)	—	Rompe el requisito de 0 €	DESCARTAR
Cloudflare Workers + D1	Workers: 100 000 peticiones/día, 10 ms de CPU por petición, 128 MB de memoria (pricing); D1 (SQLite): 5 GB, 5 M filas leídas/día, límites diarios aplicados desde el 1/9/2026 (D1)	Casi nulo	DuckDB no cabe en ese presupuesto de CPU/memoria; habría que portar la API a SQLite/TypeScript y cargar 1,8 M filas en D1 cada semana	DESCARTAR

6.6.3 Web estática o «API estática» (sin servidor en el camino crítico)

Idea: como los datos cambian una vez por semana, el pipeline puede **precalcular las respuestas** una vez y publicarlas como ficheros; la web solo lee ficheros de un CDN.

Tres variantes:

(a) **Exportación estática completa de Next.js** (`output: 'export'`). Según la documentación, no admite ISR, rutas dinámicas sin `generateStaticParams`, `cookies`, `headers`, `redirects`, `proxy` ni lectura de la petición ([Static exports](#)). Consecuencias aquí: habría que generar **todas** las páginas en cada *build* ($\approx 1\,172$ carreras × 7 subpáginas × 2 idiomas $\approx 16\,400$ HTML, más pilotos y constructores: por encima del límite de 20 000 ficheros de Cloudflare Pages y con *builds* largos), y los **8 filtros por `searchParams`** dejarían de

funcionar sin JavaScript (hoy son formularios GET que funcionan sin JS, un punto de accesibilidad del proyecto). → **DESCARTAR** la exportación completa.

(b) «API estática» + Next.js con ISR (recomendada). El pipeline genera, tras `dbt build`, un árbol de JSON con la misma forma que las respuestas de la API (reutilizando las consultas de los *routers* y los esquemas Pydantic, p. ej. `static-api/v1/races/1100/laps.json`), y lo publica en un CDN con CORS (GitHub Pages del propio repo, Cloudflare Pages/R2). `lib/api/client.ts` pasa a leer de `F1_DATA_BASE_URL` en vez de la API; ISR y `revalidate` se mantienen (o se revalida bajo demanda con un *webhook* de despliegue al acabar el pipeline).

- Ventajas: desaparece el arranque en frío y la dependencia de Render para la web; el contrato OpenAPI y los tipos TypeScript no cambian; los filtros siguen siendo formularios en el servidor (Vercel); se puede seguir desplegando la API FastAPI para quien quiera consultas libres.
- Inconvenientes: muchos ficheros (del orden de 10 000); parámetros libres (`drivers=...` en `/laps`) hay que resolverlos en la web filtrando el JSON completo de la carrera, o precalcular solo las combinaciones que usa la web; tamaño total estimado de cientos de MB sin comprimir (1,27 M vueltas + 0,52 M puntos de telemetría), que cabe en GitHub Pages (≤ 1 GB) o R2.
- Coste 0 €; esfuerzo M; riesgo bajo (la API sigue como respaldo).
- → **ADOPTAR** (es la base de la arquitectura B).

(c) Next.js lee Parquet/DuckDB en el servidor durante ISR (DuckDB para Node en la función de Vercel). Elimina la API, pero el binario nativo de DuckDB en funciones *serverless* y la descarga del fichero en cada arranque en frío de la función lo hacen frágil. → **DESCARTAR** frente a (b).

6.6.4 DuckDB-WASM en el navegador

DuckDB-WASM ejecuta DuckDB en el navegador y lee Parquet remoto con peticiones HTTP *Range* (solo el pie del fichero y las columnas/row groups necesarios) ([artículo VLDB 2022](#); [cliente WASM](#)). El paquete pesa unos ~3 MB comprimidos más extensiones, con arranque de 150–300 ms en frío ([comparativa 2026](#)). El servidor de los datos debe permitir CORS con *Range* ([troubleshooting](#)): **GitHub Releases no sirve y Hugging Face tampoco** hoy (§3.3); sí R2 con dominio propio, Cloudflare Pages o GitHub Pages.

- Ventajas: exploración libre (SQL, filtros arbitrarios, comparar pilotos entre temporadas) sin servidor; muy vistoso para portfolio; mismos Parquet que Power BI.
- Inconvenientes: el renderizado pasa al cliente (se pierde parte de RSC, SEO y la robustez sin JS que exige el proyecto: los gráficos accesibles con tabla y resumen se generan hoy en el servidor); 3+ MB de WASM y decenas de MB de Parquet para móviles; hay que repartir `fact_laptimes/fact_quali_telemetry` por temporada o carrera para que las lecturas sean pequeñas.
- Coste 0 €; esfuerzo M–L; riesgo medio.
- → **CONSIDERAR** como **complemento**: una página «Laboratorio de datos» (consultas SQL y descarga) que carga DuckDB-WASM solo en ella, sin sustituir las páginas del TFG.

6.6.5 BI-as-code como alternativa o complemento a Next.js

Herramienta	Qué es (2026)	Encaje	Recomendación
Evidence	SQL + Markdown → sitio estático; históricamente usa DuckDB-WASM en el navegador (glosario MotherDuck). En agosto de 2026 publicó Evidence Core (nuevo framework abierto, consultas en vivo, métricas en YAML) y la plataforma Evidence Studio (Evidence Core)	Ideal para un informe de calidad de datos o un anexo del TFG con pocas horas de trabajo; referencia de uso con deportes: mdsinabox / nba-monte-carlo	CONSIDERAR como complemento (p. ej. reports/ publicado en GitHub Pages); DESCARTAR como sustituto de la web (menos control de accesibilidad e i18n, framework en transición)
Observable Framework	Generador de sitios estáticos con <i>data loaders</i> en cualquier lenguaje ejecutados en el <i>build</i> y DuckDB en el cliente (data loaders ; DuckDB)	Encaja con Python + dbt; gráficos con Observable Plot (accesibles)	CONSIDERAR para prototipos de gráficos nuevos; DESCARTAR como sustituto
Rill	BI sobre DuckDB/ClickHouse con capa semántica en YAML (repo)	Orientado a <i>dashboards</i> de exploración con servidor	DESCARTAR
Streamlit (Community Cloud)	Apps en Python; se duermen tras 12 h sin tráfico, ~1 GB de RAM (gestión de apps)	Rápido para prototipos analíticos (fase 7: degradación de neumáticos)	CONSIDERAR solo para prototipos internos

La web Next.js ya cubre todas las páginas del TFG con un nivel de accesibilidad (WCAG 2.2 AA, axe en CI, funcionamiento sin JS) que ninguna de estas herramientas da de serie; sustituirla sería retroceder.

6.7 Frontend

6.7.1 Librería de gráficos

Un análisis comparativo de 2026 resume: ninguna librería es accesible «por defecto»; Vega-Lite destaca en jerarquía semántica para lectores de pantalla ([description](#)), Plotly es la única con navegación por teclado entre marcas de serie, Observable Plot genera etiquetas ARIA por marca y usa una paleta probada para daltonismo, **ECharts es la de mejor rendimiento con muchos datos** pero su accesibilidad depende de activar el módulo *aria*, y D3 no trae nada ([Accessible data-visualisation tooling in 2026](#)).

Librería	A favor (F1)	En contra	Recomendación
ECharts 6 (actual)	Canvas rápido para 1 000+ vueltas × 20 pilotos y telemetría; <i>aria.show</i> genera descripciones automáticas y <i>aria.decal</i> añade texturas además del color (ECharts ARIA); ya integrada con tabla de datos y resumen textual	Sin navegación por teclado entre puntos (issue #18585); canvas opaco para lectores de pantalla	ADOPTAR (mantener) ; revisar que <i>aria</i> y <i>decal</i> estén activos donde el color distingue series; valorar <code>renderer: 'svg'</code> en gráficos pequeños
Observable Plot	SVG, ARIA por marca (<i>ariaLabel</i> , <i>ariaDescription</i>) (Plot accessibility); gramática concisa; se puede renderizar en el servidor (SVG en RSC, sin JS en el cliente)	Menos interactividad (zoom, <i>brush</i>); SVG con 20 000+ puntos es pesado	CONSIDERAR para gráficos estáticos sencillos (barras de récords, puntos por temporada, H2H)
Vega-Lite	Especificación declarativa en JSON (se podría generar desde Python o dbt); <i>description</i> para lectores de pantalla	Bundle grande; teclado limitado (issue de accesibilidad)	DESCARTAR
D3 / visx	Control total (ya se usa <i>d3-geo</i> para el mapa)	Todo el trabajo de accesibilidad e interacción es propio	Mantener D3 solo para el mapa

Librería	A favor (F1)	En contra	Recomendación
Plotly.js	Navegación por teclado entre marcas de serie	Bundle muy grande, estética menos cuidada	DESCARTAR

6.7.2 Internacionalización

La web usa diccionarios JSON y `getDictionary(lang)` en `src/lib/i18n.ts`, que es exactamente el patrón de la guía oficial ([Next.js i18n](#)). La alternativa principal es `next-intl`, que añade plurales e interpolación ICU, formato de fechas y números y enrutado por idioma con *proxy* ([next-intl App Router](#)). Con dos idiomas y textos mayoritariamente fijos, el patrón actual basta; `Intl.NumberFormat`, `Intl.Date`, `Intl.PluralRules` e `Intl.DateTimeFormat` nativos cubren el formato. → **Mantener** (ADOPTAR); **CONSIDERAR** `next-intl` solo si se añaden idiomas o muchos textos con plurales.

6.8 Proyectos de referencia

Proyecto	Pila	Estructura / idea aprovechable
frizzleqq/f1-dbt-duckdb	Ergast → Dagster → dbt → DuckDB/MotherDuck	Carpeta dbt/ + paquete Dagster (<code>foneplatform/</code>) + Makefile; <i>staging</i> como <i>multi-asset</i> de Dagster. Referencia directa si se prueba Dagster
memadore/duck-f1	FastF1/Ergast → Parquet + DuckDB con dbt	<code>src/duck_f1/</code> (<code>src-layout</code>), <code>tasks/</code> , <code>.devcontainer/</code> , <code>SQLFluff</code> ; se distribuye como dataset DuckDB/Parquet
m1nk12/f1-data-pipeline	FastF1 + Jolpica → MinIO (bronze) → Postgres (silver/gold) con Airflow y dbt	Medallion «clásico» con almacenamiento de objetos; validación con Pydantic antes de silver. Muestra el coste operativo de Airflow + Docker Compose
matt-ohrenberger/f1-telemetry-platform	APIs públicas → MotherDuck → dbt	Ejemplo de almacén gestionado en MotherDuck
colin-k-rogers/formula-1-data-analysis	OpenF1 → MotherDuck (Flights) → dbt → Dive	Orquestación y visualización dentro de MotherDuck
matsonj/nba-monte-carlo («MDS in a box»)	dlt → dbt-duckdb → Parquet → Evidence estático en Netlify, make + GitHub Actions diario	El referente de «plataforma de datos gratis con DuckDB»: componentes intercambiables (<code>transform/</code> , <code>evidence/</code> , <code>dlt/</code>), Parquet como contrato entre capas, web estática (artículo de DuckDB)
Ddscully/dlt-dbt-duckdb-evidence	dlt → DuckDB → dbt → Polars → Evidence, orquestado por Dagster	Almacén reconstruido en cada <i>push</i> y publicado mensualmente
InfuseAI/awesome-public-dbt-projects	Lista de proyectos dbt públicos	Para comparar convenciones de carpetas

Conclusiones: ningún proyecto F1 público encontrado combina conciliación multifuente con umbrales de calidad, API documentada y web accesible bilingüe; el patrón más repetido en proyectos de bajo coste es «DuckDB + Parquet como contrato + sitio estático». Los que usan Airflow o Dagster lo hacen sobre todo como demostración de herramientas, no por necesidad.

6.9 Arquitecturas objetivo

6.9.1 A · Evolución mínima (endurecer lo que hay)

- Estructura: `justfile`; `sources` dbt para todo el bronze; subcarpetas en `intermediate/`.
- Datos: releases fechadas inmutables + `data-latest`; Parquet sueltos por tabla (particionados).
- Orquestación: disparo diario ligero guiado por el calendario de FastF1; `source freshness`; resumen de dbt en el `step summary`; `keepalive` contra los 60 días.
- Serving: API FastAPI igual; opcionalmente mover de Render a Koyeb (escala a cero tras 1 h, 1–5 s de arranque) o meter el DuckDB en la imagen para no descargarlo en cada arranque.
- Web: sin cambios.

6.9.2 B · API estática + web con ISR (recomendada)

- Todo lo de A.
- Nuevo paso del pipeline tras `dbt build`: `f1-ingest export-static` genera JSON con la forma de la API (reutilizando consultas y esquemas de `api/app`) y `marts mart_web__*` si hace falta.
- Publicación en un CDN con CORS: **GitHub Pages** del repo (0 cuentas nuevas; ≤ 1 GB) o Cloudflare Pages/R2.
- La web lee de `F1_DATA_BASE_URL`; revalidación bajo demanda al acabar el pipeline (o `revalidate` semanal) en vez de cada hora.
- La API FastAPI sigue desplegada como servicio público/documentación, fuera del camino crítico.
- Opcional: página «Laboratorio» con DuckDB-WASM sobre los Parquet del mismo CDN.

6.9.3 C · Orquestador + almacenamiento de objetos (lakehouse ligero)

- Dagster (activos = modelos dbt, particiones por carrera, *sensor* por calendario, *backfills* desde la UI) ejecutado dentro de GitHub Actions o en local.
- Almacenamiento en **Cloudflare R2** (bronze y gold en Parquet) con **DuckLake** como catálogo (fichero SQLite/DuckDB publicado) → versiones y *time travel* nativos.
- API y web leen de R2 (API: `ATTACH/read_parquet` remoto; web: API estática o DuckDB-WASM).
- Posible migración a dbt v2 cuando el adaptador DuckDB sea estable.

6.9.4 Comparación

Criterio	Actual	A · Evolución mínima	B · API estática	C · Orquestador + objeto
Coste mensual	0 €	0 €	0 €	0 € (R2 pide tarjeta; Dagster+ gestionado costaría)
Esfuerzo de migración	—	S–M (2–3 días)	M (1–2 semanas)	L (3–5 semanas)
Arranque en frío visible en la web	Sí (~1 min en páginas frías)	Sí (menor con Koyeb: 1–5 s)	No	No (si se sirve estático)
Piezas con servidor en el camino crítico	Render + Vercel	Render/Koyeb + Vercel	Solo Vercel	Solo Vercel (+ R2)
Versionado de datos	No (se sobrescribe)	Releases fechadas	Releases fechadas	<i>Snapshots</i> DuckLake

Criterio	Actual	A · Evolución mínima	B · API estática	C · Orquestador + objeto
Observabilidad	Logs de Actions	+ freshness, resumen dbt	Igual que A	UI de Dagster por activo
Riesgo técnico	Bajo	Bajo	Bajo-medio	Medio-alto (más proveedores, Dagster en transición tras la compra)
Valor para el TFG / portfolio	Alto	Alto (más robusto)	Muy alto (argumento de diseño claro: datos semanales → serving estático)	Alto en CV, pero más difícil de justificar por necesidad real
Coherencia con la memoria del TFG	Total	Total	Total (la API sigue existiendo)	Media (nuevas herramientas que explicar)
Recomendación	—	ADOPTAR ya	ADOPTAR a continuación	CONSIDERAR como línea futura

6.9.5 Hoja de ruta sugerida

1. Semana 1 (A, bajo riesgo)

- `justfile` en la raíz con las recetas del README (`ingest`, `build`, `test`, `api`, `web`, `e2e`, `snapshot`).
- `_fastf1__sources.yml`, `_formula1db__sources.yml`, `_ergast__sources.yml` con `external_location` y `freshness`; subcarpetas en `intermediate/`; YAML de `metrics/`.
- `pipeline.yml`: publicar también `data-AAAA-MM-DD immutable`; añadir SHA de commit y versiones al `manifest.json`; subir `run_results.json`; resumen en `GITHUB_STEP_SUMMARY`.
- Parquet gold sueltos (sin zip) y particionados por temporada en las dos tablas grandes.

2. Semana 2 (A)

- Workflow `trigger.yml` diario que consulta el calendario de FastF1 y el manifest y lanza el pipeline solo cuando hay una carrera nueva; *keepalive* contra la desactivación a 60 días.

3. Semanas 3–4 (B)

- `export-static` reutilizando las consultas de la API; tests que comparan JSON estático con la respuesta de la API para una muestra de recursos.
- Publicación en GitHub Pages (o Cloudflare Pages) y cambio de `client.ts` a `F1_DATA_BASE_URL`, con la API como respaldo configurable.
- Revalidación bajo demanda desde el pipeline; comprobar con Playwright + axe contra el despliegue.

4. Después (opcional)

- Página «Laboratorio» con DuckDB-WASM sobre los Parquet del CDN.
- Informe de calidad con Evidence u Observable Framework en GitHub Pages.
- `uv workspaces` si entra la fase 7 (ML) con dependencias pesadas.
- Prueba de dbt v2 (`dbt parse --use-v2-parser`) y, más adelante, explorar la arquitectura C (Dagster + DuckLake en R2) como rama experimental.

6.10 Fuentes principales (agrupadas)

- Estructura: [dbt: How we structure our dbt projects](#) · [staging](#) · [marts](#) · [Kimball con dbt](#) · [uv workspaces](#) · [Next.js project structure](#) · [just/Task/Make](#) · [Nx vs Turborepo](#) · [Turborepo Python RFC](#)

- **dbt y ecosistema:** [dbt Core v2](#) · [Upgrading to v2](#) · [DuckDB en dbt v2](#) · [dbt-duckdb](#) · [SQLMesh en la Linux Foundation](#)
- **Arquitectura y formatos:** [Medallion \(Databricks\)](#) · [DuckLake 1.0](#) · [Iceberg writes](#) · [Delta](#) · [ATTACH por HTTPS](#)
- **Almacenamiento:** [GitHub Releases](#) · [CORS en release assets](#) · [Cloudflare R2](#) · [HF storage limits](#) · [HF + DuckDB](#) · [HF CORS/Range](#) · [MotherDuck pricing](#) · [GitHub Pages limits](#)
- **Orquestación:** [GitHub Actions: schedule](#) · [Dagster + dbt](#) · [Prefect compra Dagster](#) · [Precios Dagster+ 2026](#) · [Kestra + dbt](#) · [Comparativa para dbt](#) · [FastF1 event schedule](#) · [Elementary](#)
- **Serving:** [Render free](#) · [Vercel Hobby](#) · [Cloud Run pricing](#) · [Koyeb scale-to-zero](#) · [Cloudflare Workers pricing](#) · [D1 pricing](#) · [Next.js static exports](#) · [DuckDB-Wasm \(VLDB\)](#) · [Evidence Core](#) · [Observable Framework](#) · [Streamlit Community Cloud](#)
- **Frontend:** [ECharts ARIA](#) · [Observable Plot accessibility](#) · [Accesibilidad de librerías 2026](#) · [Next.js i18n](#) · [next-intl](#)

Advertencia sobre las fuentes: varias cifras de planes gratuitos proceden de agregadores (costbench, dev.to, blogs) además de la documentación oficial, y cambian con frecuencia; antes de decidir conviene verificarlas en la página oficial de cada proveedor. Las noticias muy recientes (dbt v2 GA con DuckDB en beta, compra de Dagster por Prefect, Evidence Core) son de junio a septiembre de 2026.

Parte III

Arquitectura de datos y código

Síntesis de la parte III

Las capas y cómo se alimentan

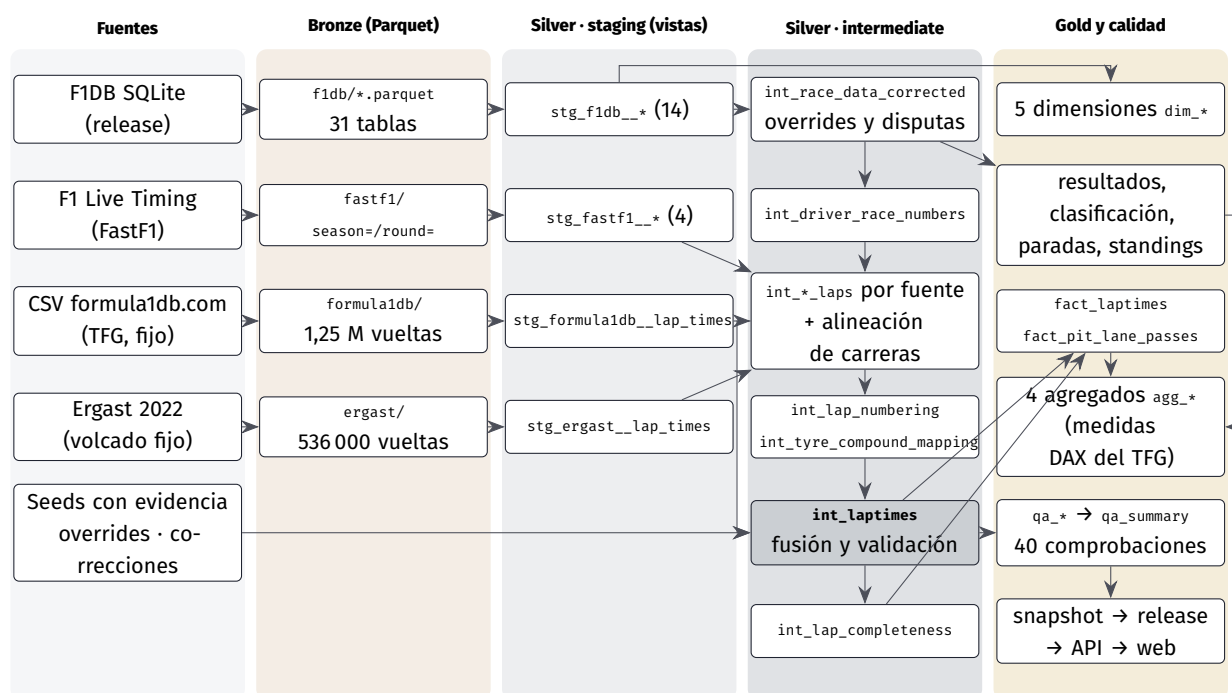


Figura 4: Linaje simplificado de las capas. La conciliación entre fuentes ocurre en intermediate; gold solo modela. Las seeds alimentan también `int_race_data_corrected`, y el bronze de la release alimenta la siguiente ejecución del pipeline.

- **Bronze** guarda cada fuente tal cual en Parquet, con particiones por temporada y ronda para FastF1, de forma idempotente: repetir una carga reescribe el fichero de esa carrera y nunca duplica filas. Solo F1DB pasa antes por `data/raw` (el SQLite de la release).
- **Silver · staging** son vistas que renombran, tipan y normalizan cada fuente (milisegundos, nombres, dorsales), sin mezclarlas.
- **Silver · intermediate** es donde vive la lógica de conciliación: correcciones y overrides con evidencia sobre F1DB, cruce de pilotos por dorsal y nombre normalizado, realineación de carreras mal etiquetadas, numeración de vueltas, mapeo de compuestos y la fusión de las tres fuentes de vueltas con su `validation_status`.
- **Gold** replica y amplía el modelo en constelación del TFG (figura 5), y **quality** publica 40 comprobaciones con umbrales que bloquean la publicación si fallan.

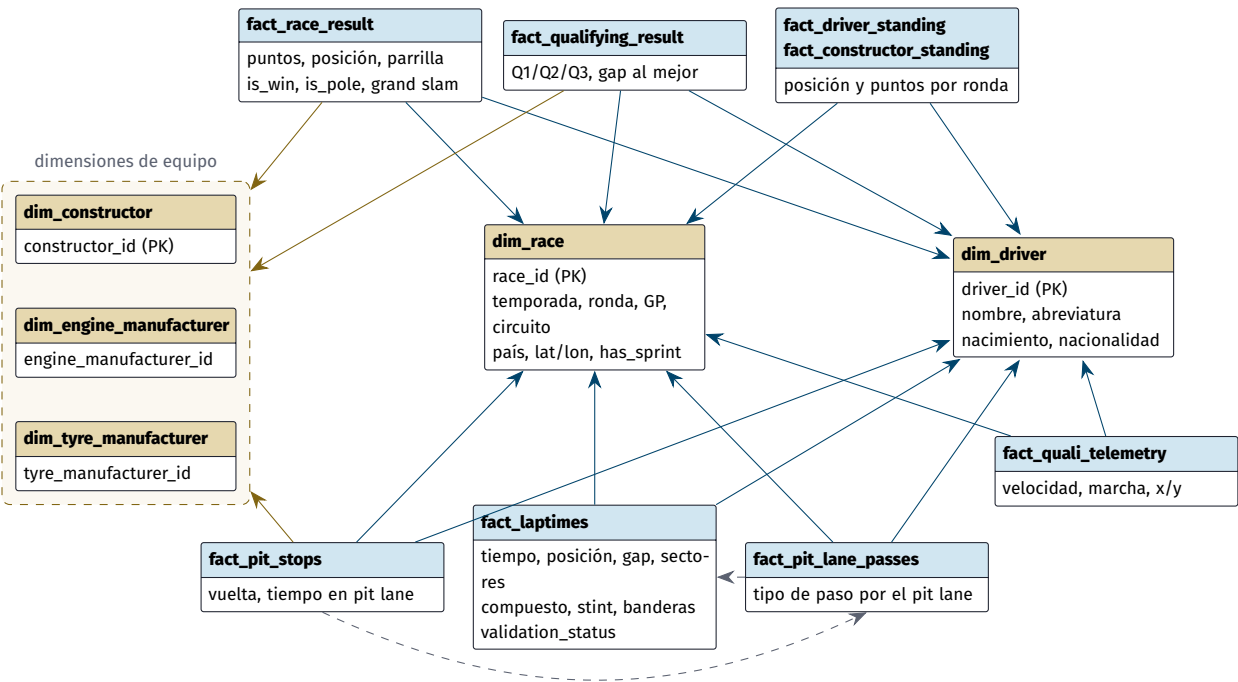


Figura 5: Esquema en constelación de la capa gold (relaciones N:1 hacia las dimensiones). En discontinuo, relaciones entre hechos por clave de grano, sin FK. fact_constructor_standing se relaciona además con constructor y motor. Los cuatro agregados agg_* se calculan sobre estos hechos.

Qué mejorar en datos y código

Área	Mejoras con más valor	Esfuerzo
Modelo	driver_number en las respuestas de vueltas y stints (coches compartidos); dorsales solo de sesiones de carrera (294 dorsales duplicados 2010–2013); desempate determinista en int_lap_numbering; tipos HUGEINT a BIGINT; borrar int_formula1db_laps_validated; revisar la rama de relleno Ergast (0 filas)	S–M
Pruebas dbt	Unit tests de int_laptimes, int_lap_numbering y parse_duration_ms; tests de clave y relación en paradas, standings, agregados y telemetría	M
Ingesta	Extracción atómica y reintentos en F1DB; recargar las últimas carreras de FastF1; segundo disparo si F1DB publica tarde	S
API	Arranque con la última copia si falla GitHub; /health que consulte la base e informe del último refresco; límite de peticiones	S–M
Web	Timeout y reintento en fetch; cabeceras de seguridad (CSP); abreviaturas desde abbreviation; ECharts con carga diferida	S
CI/CD	Dependabot; acciones fijadas a SHA y fuera de Node 20; prueba de humo antes de publicar; releases fechadas; pre-commit alineado con uv.lock	S

El estado de partida es bueno: ruff, las 40 pruebas de Python, eslint, la comprobación de tipos y npm audit pasan limpios, no hay inyección SQL (toda entrada va parametrizada) y todos los endpoints responden en menos de 60 ms contra la base local.

CAPÍTULO 7

Arquitectura de datos: capas, modelo y linaje

Nota de trabajo para el informe de situación. Estado verificado el 29/09/2026 sobre el repositorio (HEAD = cceddd3), el almacén local data/gold/f1.duckdb (último dbt build correcto el 28/09/2026 a las 19:18 UTC, 21,7 s) y el snapshot dist/ (manifiesto del 28/09/2026 19:23 UTC). Todas las cifras proceden de consultas de solo lectura sobre esos ficheros. Los bloques diagrama están pensados para convertirse después en TikZ.

7.1 Visión general

La plataforma sigue una **arquitectura medallón** (bronze → silver → gold) implementada con Python (ingesta), DuckDB como motor y almacén analítico y dbt-core 1.12.5 + dbt-duckdb como herramienta de transformación y pruebas. Sobre la capa gold se añaden un esquema quality (conciliación entre fuentes) y un proceso de exportación («snapshot») que publica los datos en la GitHub Release data-latest, de la que se alimenta la API FastAPI (y, a través de ella, la web Next.js).

Capa	Ubicación física	Formato	Quién la escribe	Contenido
raw	data/raw/f1db/ <tag>/f1db.db	SQLite	f1db_loader. download_ sqlite	Base de datos F1DB descargada (73,7 MB, v2026.15.1)
caché	data/cache/fastf1/	caché HTTP de FastF1	biblioteca fastf1	Respuestas de la API de F1 Live Timing (≈4,5 GB, no se publica)
bronze	data/bronze/{f1db, fastf1,formula1db, ergast}/	Parquet + _metadata. json	f1-ingest	Copia fiel de cada fuente, sin reglas de negocio (≈68 MB)
silver	esquema silver de data/gold/f1. duckdb	vistas (staging) y tablas (intermediate, seeds)	dbt	Tipado, renombrado, claves comunes, fusión y corrección de fuentes
gold	esquema gold del mismo fichero	tablas	dbt	Modelo dimensional en constelación (5 dimensiones, 8 hechos, 4 agregados)
quality	esquema quality	tablas	dbt	6 modelos de contraste entre fuentes + qa_summary (40 comprobaciones)
distribución	dist/ → Release data- latest	.tar.gz, .duckdb, . zip, .json	f1-ingest snapshot pack	Entrada de la próxima ejecución y datos que sirve la API

El almacén de trabajo (data/gold/f1.duckdb, 238 MB) contiene los tres esquemas: silver (20 vistas + 14 tablas), gold (17 tablas) y quality (7 tablas). La base que se publica (dist/f1.duckdb, 54,5 MB) solo contiene gold.* y quality.qa_summary.

7.1.1 Inventario del proyecto dbt (según transform/target/manifest.json)

Tipo de nodo	Nº	Detalle
Sources	22 tablas	f1db (13), fastf1 (4), formula1db (2), ergast (3)
Modelos staging	20	14 f1db, 4 fastf1, 1 formula1db, 1 ergast (vistas en <i>silver</i>)
Modelos intermediate	10	int_*
Modelos marts	17	13 en marts/core (5 dim + 8 fact) y 4 en marts/metrics (agg_*)
Modelos quality	7	6 qa_* + qa_summary
Seeds	3	race_data_overrides, race_data_corrections, lap_corrections
Tests de datos	50	13 not_null, 10 relationships, 9 accepted_values, 6 unique, 5 unique_combination (genérico propio), 7 singulares
Unit tests	4	en models/quality/_quality__models.yml
Macros	3	generate_schema_name, normalize_name, parse_duration_ms

Nota de recuento: en *silver* hay exactamente 20 vistas stg_* (14 de F1DB, 4 de FastF1, 1 de formula1db y 1 de Ergast); el total de modelos es 20 + 10 + 17 + 7 = 54, que coincide con los 54 model: success de run_results.json.

Resultado del último `dbt build(transform/target/run_results.json)`: 54 modelos success, 3 seeds success, 50 tests pass (incluidos los 3 de severidad warn, sin avisos) y 4 unit tests pass.

7.2 Datos en bruto y capa BRONZE (ingestion/)

El paquete `ingestion` sustituye a los scripts manuales del TFG (descarga manual de F1DB, Selenium contra formula1db.com, `fastf1-main.py` y `fastf1-telemetry.py` que escribían CSV en modo *append*). Se instala como *script* de consola `f1-ingest = "ingestion.cli:main"` (`pyproject.toml`) y sus dependencias van en el grupo `ingest` (`fastf1>=3.4`, `pandas>=2.2`, `pyarrow>=17`, `requests>=2.32`); la base común del proyecto es solo `duckdb>=1.1`.

Principios comunes a todos los cargadores:

- **Bronze = copia fiel:** no se aplican reglas de negocio; solo cambios de formato (SQLite/CSV → Parquet, `timedelta` → milisegundos enteros, nombres de columna en `snake_case` para los CSV del TFG, eliminación de duplicados exactos).
- **Idempotencia:** volver a ejecutar una carga reemplaza ficheros, nunca añade filas.
- **Escritura atómica:** se escribe en un temporal (`*.tmp`) y se renombra (`os.replace` o renombrado de directorio), de modo que *bronze* nunca queda a medio escribir.
- **Metadatos:** cada fuente de carga completa deja un `_metadata.json` con `loaded_at` (UTC) y el recuento de filas por tabla.

7.2.1 config.py — rutas y constantes

Constante	Valor	Uso
PROJECT_ROOT	raíz del repositorio	base de rutas
DATA_DIR	\$F1_DATA_DIR o <raíz>/data	permite redirigir los datos (tests, CI)
RAW_DIR, BRONZE_DIR, CACHE_DIR	data/raw, data/bronze, data/cache	capas físicas

Constante	Valor	Uso
F1DB_REPO, F1DB_ASSET	f1db/f1db, f1db-sqlite.zip	release de GitHub que se descarga
FASTF1_FIRST_SEASON	2018	primera temporada con cronometraje completo en FastF1
LEGACY_CSV_DIR	<raíz>/../FORMULA 1 DB	CSV del scraping del TFG (fuera del repo)

7.2.2 io.py — utilidades de escritura

Función	Qué hace
write_parquet(df, path)	Crea el directorio, escribe path.parquet.tmp con df.to_parquet(index=False) y lo mueve con os.replace (atómico). Garantiza que recargar una carrera sobrescribe su fichero.
write_metadata(directory, **fields)	Escribe _metadata.json con loaded_at (ISO, UTC, precisión de segundos) más los campos recibidos (release, origen, recuentos).
read_metadata(directory)	Lee _metadata.json o devuelve {} si no existe (lo usa f1db_loader para decidir si hay release nueva y snapshot para el manifiesto).

7.2.3 f1db_loader.py — F1DB (fuente principal de resultados)

F1DB es una base de datos abierta (CC BY 4.0) que se publica en releases de GitHub tras cada Gran Premio.

Función	Qué hace
latest_release()	GET https://api.github.com/repos/f1db/f1db/releases/latest (con Authorization: Bearer \$GITHUB_TOKEN si existe, para no agotar el límite anónimo). Devuelve tag, published_at y la URL del asset f1db-sqlite.zip.
download_sqlite(release)	Si data/raw/f1db/<tag>/f1db.db ya existe no descarga nada (idempotencia por versión). Si no, descarga el zip en <i>streaming</i> (bloques de 1 MiB, timeout 120 s), extrae f1db.db y borra el zip.
export_tables(db_path, out_dir)	Abre DuckDB en memoria, INSTALL/LOAD sqlite, adjunta la base en READ_ONLY y, para cada tabla (no las vistas, que F1DB deriva de race_data), ejecuta COPY (SELECT * FROM src.<tabla>) TO '<tabla>.parquet' (FORMAT parquet), respetando los tipos declarados en SQLite. Escribe en f1db.tmp/ y después sustituye el directorio completo (reemplazo atómico). Devuelve el recuento de filas por tabla.
load(force=False)	Orquesta: si el release de _metadata.json coincide con el último tag y no hay --force, termina sin hacer nada; si no, descarga, exporta y escribe los metadatos (release, published_at, tables).

Ficheros producidos: data/raw/f1db/v2026.15.1/f1db.db (73,7 MB) y data/bronze/f1db/*.parquet (31 tablas, 5,1 MB en total) + _metadata.json (release = v2026.15.1, publicada el 27/09/2026).
Tablas y filas:

Tabla	Filas	Tabla	Filas
race_data (71 columnas, 4,4 MB)	187 675	season_entrant_driver	3 882
race_driver_standing	21 541	season_driver	3 416
race_constructor_standing	10 654	season_entrant_chassis	2 292

Tabla	Filas	Tabla	Filas
race (47 columnas)	1 172	season_entrant_engine	2 027
driver (32 columnas)	917	season_entrant_tyre_manufacturer	1 955
constructor (22 columnas)	187	season_entrant_constructor	1 925
season_driver_standing	1 681	season_entrant	1 799
season_constructor_standing	721	chassis	1 153
country	249	season_constructor	1 079
circuit	78	entrant	830
engine_manufacturer	78	season_engine_manufacturer	560
grand_prix	54	engine	424
tyre_manufacturer	9	constructor_chronology	217
season	77	circuit_layout	160
continent	7	season_tyre_manufacturer	160
driver_family_relationship	86		

Solo 13 de las 31 tablas se declaran como *sources* de dbt (§2.2); el resto se conserva en bronce por completitud (p. ej. chasis, inscripciones por temporada), pero hoy no alimenta el modelo.

7.2.4 fastf1_loader.py — FastF1 (cronometraje 2018+ y telemetría)

Carga **incremental por carrera** desde la API de F1 Live Timing a través de la biblioteca *fastf1* (con caché local en `data/cache/fastf1`).

Elemento	Qué hace
LoadSummary	<i>Dataclass</i> con listas loaded, skipped, failed y la marca <code>rate_limited</code> .
race_path(kind, season, round)	Ruta de partición estilo Hive: <code>data/bronze/fastf1/<kind>/season=YYYY/round=RR.parquet</code> (<code>kind</code> ∈ laps, results, quali_results, quali_telemetry). Un fichero por carrera.
timedeltas_to_millis(df)	Convierte cada columna <code>timedelta64</code> en <code><col>_ms</code> (Int64, redondeado) y elimina la original (Parquet/DuckDB-friendly).
downsample_by_distance(tel, step=10)	Conserva la primera muestra de cada tramo de 10 m (<code>Distance // 10</code>), para no repetir los 364 MB del <code>quali_laps.csv</code> del TFG.
_with_keys(df, event, season)	Inserta al principio <code>season</code> , <code>round</code> y <code>event_name</code> .
_load_race(event, season)	Sesión R: <code>session.load(laps=True, telemetry=False, weather=False, messages=False)</code> ; escribe laps y results.
_load_qualifying(event, season, telemetry)	Sesión Q: escribe <code>quali_results</code> (Q1/Q2/Q3). Con telemetría, para cada piloto toma <code>pick_fastest()</code> , obtiene <code>get_telemetry()</code> , submuestra y guarda <code>Distance</code> , <code>Speed</code> , <code>RPM</code> , <code>nGear</code> , <code>Throttle</code> , <code>Brake</code> , <code>DRS</code> , <code>X</code> , <code>Y</code> , <code>Z</code> + <code>Driver</code> , <code>DriverNumber</code> , <code>Compound</code> , <code>LapTime_ms</code> . Un piloto sin datos solo genera un <i>warning</i> .
completed_races(season)	Calendario sin tests (<code>get_event_schedule</code>) filtrado a carreras cuya hora de inicio (<code>Session5DateUtc</code>) + 6 h ya ha pasado (margen para que el cronometraje oficial esté publicado).

Elemento	Qué hace
<code>load_season(season, rounds, telemetry, force)</code>	Rechaza temporadas < 2018; habilita la caché; por cada carrera calcula qué kinds faltan (o todos con <code>force</code>) y solo descarga lo pendiente ; captura <code>RateLimitExceededError</code> (límite de 500 peticiones/hora) y marca <code>rate_limited</code> para que la próxima ejecución continúe donde se quedó; cualquier otro error se anota en <code>failed</code> sin detener la temporada.

Ficheros producidos (45 MB en total):

Directorio	Ficheros	Cobertura	Filas	Columnas
<code>fastf1/laps/</code>	187	2018 (20) · 2019 (21) · 2020 (17) · 2021–2023 (22 c/u) · 2024 (24) · 2025 (24) · 2026 (15)	205 718	34
<code>fastf1/results/</code>	187	ídem	3 768	25
<code>fastf1/quali_results/</code>	187	ídem	3 769	25
<code>fastf1/quali_telemetry/</code>	63	2024 (24) · 2025 (24) · 2026 (15)	518 699	17

FastF1 no escribe `_metadata.json`: la cobertura se deduce de las particiones (así lo hace `snapshot.summarize`).

7.2.5 `legacy_loader.py` — `formula1db.com` (scraping del TFG, carga única)

Los scripts Selenium del TFG se retiran, pero sus CSV (tiempos por vuelta 1950–2024, que ninguna otra fuente abierta cubre antes de 1996) se conservan.

Función	Qué hace
<code>snake_case(name)</code>	Aplica RENAMES (" <code>+-</code> " → <code>positions_change</code> , " <code>No.</code> " → <code>car_number</code>) y normaliza a minúsculas con <code>_</code> .
<code>clean(df)</code>	Renombra columnas y elimina duplicados exactos (los CSV se generaron con <code>mode="a"</code>); devuelve cuántos se quitaron.
<code>load(source_dir)</code>	Lee <code>f1_lap_times.csv</code> , <code>f1_race_entries.csv</code> y <code>f1_drivers.csv</code> todo como texto (<code>dtype=str</code> , <code>keep_default_na=False</code>) para no perder formatos como <code>1:39.019</code> o <code>-</code> ; escribe <code>lap_times</code> , <code>race_entries</code> , <code>drivers</code> y <code>_metadata.json</code> .

Fichero bronze	Filas	Columnas	Tamaño	Duplicados eliminados
<code>formula1db/lap_times.parquet</code>	1 252 550	17 (<code>season</code> , <code>round</code> , <code>car_number</code> , <code>driver_name</code> , <code>lap</code> , <code>rank</code> , <code>positions_change</code> , <code>time</code> , <code>gap</code> , <code>interval</code> , <code>tyre</code> , <code>tyre_age</code> , <code>status</code> , <code>lap_time</code> , <code>sector_1..3</code>)	14,8 MB	0
<code>formula1db/race_entries.parquet</code>	28 558	11	0,18 MB	0
<code>formula1db/drivers.parquet</code>	1 051	6	0,06 MB	0

Cobertura de `lap_times`: 1950–2024, 1 125 carreras (etiquetadas por temporada y ronda).

7.2.6 ergast_loader.py — Ergast (volcado de octubre de 2022, solo validación)

Ergast dejó de actualizarse; se usa como **tercera fuente independiente** de vueltas (1996–2022). `load(zip_path)` abre `../ERGAST API/f1db_csv.zip`, lee `racas`, `results`, `drivers`, `lap_times`, `pit_stops`, `qualifying` con `na_values=["\\N"]`, escribe un Parquet por tabla y el `_metadata.json`.

Fichero	Filas	Columnas
ergast/lap_times.parquet	535 748	6
ergast/results.parquet	25 800	18
ergast/racas.parquet	1 079	18
ergast/pit_stops.parquet	9 559	7
ergast/qualifying.parquet	9 535	9
ergast/drivers.parquet	855	9

Solo `racas`, `results` y `lap_times` se declaran como `sources`.

7.2.7 snapshot.py — empaquetado y restauración

Resuelve un problema de despliegue: GitHub Actions no tiene los CSV del TFG ni el histórico de FastF1, así que cada ejecución **parte del último snapshot publicado** y publica el siguiente.

Función	Qué hace
<code>sha256(path)</code>	Hash por bloques de 1 MiB (integridad en el manifiesto).
<code>pack_bronze(bronze_dir, out)</code>	Exige que existan las fuentes estáticas (<code>STATIC_SOURCES = ("formula1db", "ergast")</code>); sin ellas <code>dbt build</code> no puede ejecutarse) y crea <code>bronze.tar.gz</code> con rutas <code>bronze/...</code> , omitiendo <code>*.tmp</code> ; escritura atómica.
<code>restore_bronze(archive, data_dir)</code>	Verifica que todos los miembros empiezan por <code>bronze/</code> , extrae con <code>filter="data"</code> (rechaza rutas absolutas, enlaces y escapes del directorio) y crea <code>data/gold/</code> (<code>dbt</code> crea la base pero no su carpeta). Devuelve las fuentes restauradas.
<code>build_api_database(warehouse, out)</code>	Crea una DuckDB nueva, adjunta el almacén en <code>read_only</code> y copia todas las tablas de gold y quality.qa_summary con <code>CREATE TABLE ... AS SELECT; CHECKPOINT</code> final. Resultado compacto (54,5 MB frente a 238 MB).
<code>export_gold_parquet(database, out)</code>	Exporta cada tabla <code>gold</code> a Parquet con compresión ZSTD y las mete en <code>gold-parquet.zip</code> (modo <code>ZIP_STORED</code> , porque Parquet ya va comprimido). Pensado para Power BI u otras herramientas.
<code>summarize(database, bronze_dir)</code>	Datos del manifiesto: release de F1DB, carreras de FastF1 por temporada (recuento de particiones), última carrera completada (<code>gold.dim_race where is_completed order by race_date desc</code>) y recuento de estados de <code>quality.qa_summary</code> .
<code>pack(out_dir, ...)</code>	Genera los tres ficheros y <code>manifest.json</code> (fecha, resumen, filas por tabla, bytes y SHA-256 de cada fichero). Falla si no existe <code>data/gold/f1.duckdb</code> .
<code>release_notes(manifest)</code>	Texto Markdown de la release (última carrera, versión F1DB, cobertura FastF1, estados de calidad, licencias).

Snapshot actual (`dist/`, 28/09/2026): `bronze.tar.gz` 51,7 MB, `f1.duckdb` 54,5 MB, `gold-parquet.zip` 28,1 MB, `manifest.json` 1,6 KB; última carrera 2026 R15 (Azerbaián); calidad: 23 PASS y 17 INFO.

7.2.8 cli.py — la CLI f1-ingest

argparse con subcomandos; las importaciones son diferidas (FastF1 tarda en importarse) y la salida se fuerza a UTF-8 (la consola de Windows usa cp1252).

Comando	Efecto	Código de salida
f1-ingest f1db [--force]	f1db_loader.load; imprime los metadatos	0
f1-ingest fastf1 --season 2025 2026 [--round 3 4] [--telemetry] [--force]	load_season por temporada; resumen «cargadas / ya existentes / con error»; si hay <i>rate limit</i> se detiene y avisa de que la carga es reanudable	1 si alguna carrera falla
f1-ingest legacy [--path DIR]	legacy_loader.load (carga única)	0
f1-ingest ergast [--path ZIP]	ergast_loader.load (carga única)	0
f1-ingest snapshot pack [--out dist]	snapshot.pack; imprime tamaños y hashes	0
f1-ingest snapshot restore <bronze.tar.gz>	restore_bronze	0
f1-ingest snapshot notes <manifest.json>	release_notes a stdout	0

Pruebas unitarias en tests/test_ingestion.py (conversión de timedelta, submuestreo, particionado, idempotencia de write_parquet, limpieza de duplicados) y tests/test_snapshot.py (ida y vuelta pack/restore, contenido de la base de la API, fuentes estáticas obligatorias, rechazo de archivos ajenos, notas de la release).

7.2.9 Resumen de fuentes

Fuente	Tipo de acceso	Frecuencia	Cobertura	Papel en el modelo
F1DB	Release GitHub (SQLite)	tras cada GP	1950–2026	Fuente de verdad de resultados, clasificaciones, campeonatos, paradas y dimensiones
FastF1	API Live Timing (biblioteca)	semanal, incremental	2018–2026	Vueltas 2025–2026 y pilotos ausentes; enriquece vueltas (stint, vida del neumático, speed trap, estado de pista); telemetría de clasificación (2024+); validación
formula1db.com	CSV del scraping del TFG	carga única	1950–2024	Fuente principal de tiempos por vuelta
Ergast	Volcado CSV oct. 2022	carga única	1996–2022	Validación de vueltas y relleno puntual marcado

7.3 Proyecto dbt (transform/) y capa SILVER

7.3.1 Configuración

- **dbt_project.yml**: proyecto f1, perfil f1. Materializaciones y esquemas por carpeta:

Carpeta	Materialización	Esquema
models/staging	view	silver

Carpeta	Materialización	Esquema
models/intermediate	table	silver
models/marts	table	gold
models/quality	table	quality
seeds	tabla (seed)	silver

- **profiles.yml:** un único target dev, type: duckdb, path: {{ env_var('F1_DUCKDB_PATH', ' ../data/gold/f1.duckdb') }}, threads: 4. Se invoca siempre con `--profiles-dir .` desde `transform/`.
- **Rutas de bronze:** las *sources* usan `meta.external_location` (funcionalidad de dbt-duckdb) con {{ env_var('F1_BRONZE_DIR', ' ../data/bronze') }}; así las vistas de staging leen los Parquet directamente, sin copiarlos al almacén. Consecuencia práctica: las vistas `stg_*` guardan rutas relativas y solo se pueden consultar con el directorio de trabajo en **transform/** (fuera de él fallan con «No files found»).

7.3.2 Sources (models/staging/f1db/_f1db__sources.yml)

Source	Tablas declaradas	Ubicación
f1db	race, race_data, race_driver_standing, race_constructor_standing, season_driver_standing, season_constructor_standing, driver, constructor, engine_manufacturer, tyre_manufacturer, grand_prix, circuit, country	../f1db/{name}.parquet
fastf1	laps, results, quali_results, quali_telemetry	read_parquet(' ../fastf1/<t>*/*.parquet', hive_partitioning = false, union_by_name = true)
formula1db	lap_times, race_entries	../formula1db/{name}.parquet
ergast	races, results, lap_times	../ergast/{name}.parquet

En FastF1 se desactiva `hive_partitioning` porque `season` y `round` ya van dentro del fichero, y `union_by_name` tolera cambios de esquema entre temporadas. `formula1db.race_entries` está declarada pero ningún modelo la usa.

7.3.3 Macros (transform/macros/)

Macro	Propósito	Implementación
<code>generate_schema_name</code>	Usar el esquema personalizado tal cual (<i>silver</i> , <i>gold</i> , <i>quality</i>) en lugar del prefijo por defecto <code><target>_<schema></code>	devuelve <code>custom_schema_name</code> o <code>target.schema</code>
<code>normalize_name(column)</code>	Nombre de persona comparable entre fuentes (p. ej. «José Froilán González» → «jose froilan gonzalez»)	quita «Jr.», cambia - por espacio, <code>strip_accents</code> , <code>lower</code> , elimina todo lo que no sea <code>[a-z]</code> , <code>trim</code>
<code>parse_duration_ms(column)</code>	Tiempos en texto (1:39.019, 42.414, 1:02:03.5) → milisegundos <code>bigint</code> ; <code>NULL</code> para - o vacío	valida con <code>regexp_full_match('\d+(:\d{1,2})*(\.\d+)?')</code> , divide por : y acumula en base 60 con <code>list_reduce</code>

7.3.4 Seeds (transform/seeds/, esquema silver)

Las seeds son el mecanismo de **corrección manual auditable**: cada fila lleva la decisión de la revisión de divergencias de 2026 (docs/revision_divergencias/INFORME.md) y su evidencia, y solo se aplica si el dato de origen sigue siendo el revisado (autodesactivación cuando la fuente lo corrige).

Seed	Filas	Columnas	Para qué sirve
race_data_corrections	4	race_id, session_type, match_driver_id, match_driver_number, match_position_display_order, set_driver_id, set_driver_number, note	Correcciones heredadas de Script-2.sql del TFG sobre race_data (piloto o dorsal erróneo) que F1DB aún no ha corregido y que Stats F1 sostiene (se retiraron Leoni 1978 y Mónaco 1975, decisión N3). Las columnas match_* son condiciones (NULL = comodín); set_* son los valores nuevos. Ej.: GP 331, Lammers #9 → #10; GP 14, el #50 de Trintignant → Jean Behra.
race_data_overrides	195	race_id, session_type, position_display_order, driver_id, column_name, f1db_value, corrected_value, action, decision, evidence	Correcciones celda a celda con evidencia (FIA, Stats F1, formula1db, Ergast). action = set sustituye el valor; action = flag lo conserva y lo marca como disputado. Generada por scripts/generate_override_seed.py a partir de las hojas CSV de docs/revision_divergencias. Tests: accepted_values en column_name (10 columnas permitidas) y action, y unique_combination(race_id, session_type, position_display_order, column_name).
lap_corrections	1	race_id, driver_id, lap_number, column_name, original_value, corrected_value, decision, evidence	Correcciones de vueltas concretas de formula1db.com (decisión A2). Hoy: GP 1124, Russell, vuelta 1: 120 631 → 115 631 ms (formula1db sumó a la vuelta 1 la sanción de 5 s). Si una carrera tiene alguna corrección, sus posiciones por vuelta se recalculan.

Desglose de race_data_overrides por decisión:

Decisión	Acción	Columna	Celdas
A3 (vueltas completadas históricas)	set / flag	race_laps	72 / 2
A4 (vuelta de la vuelta rápida)	set	fastest_lap_lap	97
A5 (parrillas)	set	race_grid_position_number (5), race_grid_position_text (5), race_positions_gained (4)	14
N1 (orden de clasificación)	set	position_display_order, position_number, position_text, race_positions_gained	8
N2 (tiempo de Q3)	set	qualifying_q3_millis, qualifying_gap_millis	2

No existe una seed de «mapeos» (compuestos, pilotos...): esas equivalencias se **calculan** en modelos (int_tyre_compound_mapping, int_driver_race_numbers, int_formula1db_race_alignment).

7.3.5 Staging (models/staging/<fuente>/stg_<fuente>__<entidad>.sql, vistas en silver)

Función: una vista por tabla de origen, con **renombrado a snake_case** y a **claves comunes** (id → driver_id, year → season, type → session_type), **casts** (:integer, :boolean) y decodificación de formatos, sin joins de negocio (salvo el de Ergast para obtener temporada, ronda y dorsal).

Modelo	Filas	Col.	Origen	Transformación relevante
stg_f1db__races	1 172	21	race	id→race_id, year→season, date→race_date, time→race_time_utc; has_sprint = sprint_race_date is not null; booleanos de «decider»
stg_f1db__race_data	187 675	42	race_data	Tabla ancha de F1DB (una fila por piloto × sesión × carrera, distinguida por type → session_type); selecciona 42 de 71 columnas; casts booleanos (race_shared_car, race_pole_position, race_fastest_lap, race_driver_of_the_day, race_grand_slam)
stg_f1db__drivers	917	13	driver	datos personales + claves de país
stg_f1db__constructors	187	4	constructor	
stg_f1db__engine_manufacturers	78	3	engine_manufacturer	
stg_f1db__tyre_manufacturers	9	3	tyre_manufacturer	
stg_f1db__circuits	78	10	circuit	length → circuit_length_km, type → circuit_type
stg_f1db__grands_prix	54	6	grand_prix	
stg_f1db__countries	249	6	country	alpha2_code, alpha3_code, demonym, continent_id
stg_f1db__race_driver_standings	21 541	8	race_driver_standing	
stg_f1db__race_constructor_standings	10 654	9	race_constructor_standing	
stg_f1db__season_driver_standings	1 681	7	season_driver_standing	year → season
stg_f1db__season_constructor_standings	721	8	season_constructor_standing	
stg_f1db__official_totals	1 104	10	driver ∪ constructor	Totales precalculados por F1DB (total_race_wins, total_podiums...) con entity_type; solo se usan en tests de paridad, nunca como métrica
stg_fastf1__laps	205 718	24	fastf1.laps	upper(Compound); is_pit_in_lap/out desde PitInTime_ms/PitOutTime_ms; decodifica TrackStatus (2 = amarilla, 4 = SC, 5 = roja, 6/7 = VSC); is_deleted, is_accurate, speed_trap_kmh
stg_fastf1__results	3 768	12	fastf1.results	team_color = '#' TeamColor, headshot_url, status, points
stg_fastf1__quali_results	3 769	8	fastf1.quali_results	q1_ms, q2_ms, q3_ms
stg_fastf1__quali_telemetry	518 699	15	fastf1.quali_telemetry	distance_m, speed_kmh, rpm, gear, throttle_pct, is_braking, drs, x, y (descarta Z)

Modelo	Filas	Col.	Origen	Transformación relevante
stg_formula1db__lap_times	1 226 299	23	formula1db.lap_times	Descarta filas que no son vueltas (Grid, Start, A; 26 251 filas); parse_duration_ms para lap_time, time, gap, interval, sectores; gap_to_leader_laps con regex \+(\d+) Lap; decodifica neumático (H/M/S/SS/US/HS/I/W → HARD...WET; C0-C6 se conservan) y status (Pit, Out Lap, Yellow Flag, Safety Car, Virtual Safety Car, Red Flag, Lap Time Deleted) en booleanos
stg_ergast__lap_times	535 748	6	ergast.lap_times JOIN races JOIN results	Añade season, round y el driver_number de la carrera (Ergast identifica al piloto por su id interno)

7.3.6 Intermediate (models/intermediate/int_*.sql, tablas en silver)

Es la capa donde vive la **lógica de conciliación de fuentes**. Orden de dependencias:

— descripción del diagrama —

```
# Grafo de dependencias de intermediate (arista = "alimenta a")
stg_f1db__race_data + seed race_data_corrections + seed race_data_overrides -> int_race_data_corrected
int_race_data_corrected + stg_f1db__drivers -> int_driver_race_numbers
stg_formula1db__lap_times + int_race_data_corrected + stg_f1db__races -> int_formula1db_race_alignment
stg_formula1db__lap_times + int_formula1db_race_alignment + int_driver_race_numbers -> int_formula1db_laps
stg_fastf1__laps + stg_f1db__races + int_driver_race_numbers -> int_fastf1_laps
stg_ergast__lap_times + stg_f1db__races -> int_ergast_laps
int_formula1db_laps + int_race_data_corrected -> int_lap_numbering
int_formula1db_laps + int_fastf1_laps -> int_tyre_compound_mapping
int_formula1db_laps + int_fastf1_laps + int_ergast_laps + int_lap_numbering + int_tyre_compound_mapping
+ int_race_data_corrected + int_driver_race_numbers + seed lap_corrections -> int_laptimes
int_laptimes + int_race_data_corrected -> int_lap_completeness
```

int_race_data_corrected — 187 675 filas × 46 columnas

Granularidad: la de race_data (carrera × sesión × position_display_order). Es la **única entrada de F1DB para los hechos** de resultados, clasificación y paradas.

1. **Correcciones heredadas** (legacy_corrected): LEFT JOIN race_data_corrections con condiciones comodín (match_* is null or match_* = valor); sustituye driver_id y driver_number con SELECT * REPLACE (coalesce(set_..., original)), conserva original_driver_id y marca is_corrected (7 filas afectadas).
2. **Overrides con evidencia**: applicable_overrides une la seed con las filas por (race_id, session_type, position_display_order, driver_id) y **solo acepta la celda si el valor actual de F1DB (casteado a texto) coincide con f1db_value**; la comparación se genera con un bucle Jinja sobre el diccionario override_columns (10 columnas con su tipo).
3. pivoted agrupa por fila y obtiene override_<col> (valor set), overridden_columns (lista de columnas con set) y disputed_columns (lista con flag).
4. Resultado final: REPLACE (coalesce(override_<col>::<tipo>, <col>)) as <col>) para las 10 columnas y las listas overridden_columns / disputed_columns (vacías por defecto). Filas con override: 177; con columnas disputadas: 2.

Tipos de sesión presentes: RACE_RESULT 27 621, QUALIFYING_RESULT 27 040, STARTING_GRID_POSITION 25 858, PIT_STOP 22 535, FASTEST_LAP 17 170, entrenamientos libres (FP1-FP4) 41 334, QUALIFYING_1/2_RESULT 15 248, WARMING_UP_RESULT 7 683, DRIVER_OF_THE_DAY_RESULT 894, PRE_QUALIFYING_RESULT 647, SPRINT_RACE_RESULT 590, SPRINT_STARTING_GRID_POSITION 589, SPRINT_QUALIFYING_RESULT 466. Gold solo usa seis de ellas.

int_driver_race_numbers — 28 332 × 6

SELECT DISTINCT race_id, driver_number, driver_id de int_race_data_corrected (dorsal no nulo) con driver_name, normalized_name y normalized_last_name (macro normalize_name). Es la **tabla puente** que traduce el dorsal de FastF1/formula1db/Ergast al driver_id de F1DB.

int_formula1db_race_alignment — 1 125 × 8

Asigna cada carrera de formula1db.com a una carrera de F1DB **por su contenido** y no por la ronda etiquetada (el script de actualización del TFG fijaba round_race a mano y desde 2023-R7 los datos quedaron desplazados una ronda). Firma de una carrera = vueltas completadas por dorsal (max(lap_number)); se cuenta cuántos dorsales coinciden con race_laps oficiales para cada carrera candidata de la misma temporada y se elige la de más coincidencias (QUALIFY row_number() ... order by matching_drivers desc, (candidate_round = labelled_round) desc); en empate se conserva la ronda etiquetada. Salidas: labelled_round, aligned_round, race_id, is_realigned, legacy_drivers, matching_drivers, match_pct. Resultado: **17 carreras realineadas**, todas en 2023 (rondas etiquetadas 7–23 → 6–22).

int_formula1db_laps — 1226 299 × 24

Vueltas de formula1db con race_id (vía la realineación) y driver_id. El piloto se resuelve **una vez por (carrera, dorsal, nombre)** con candidatos por dorsal, nombre normalizado o apellido, y un nivel de coincidencia (driver_match_level): 1 = dorsal y nombre (1 213 900 vueltas), 2 = dorsal y apellido (8 314), 3 = nombre (1 885), 4 = apellido único en la carrera (0), 5 = solo dorsal (2 200). Se queda el mejor nivel con QUALIFY row_number(). Esto resuelve los **coches compartidos de los años 50**, donde el dorsal es del coche y no del piloto (unit test test_shared_car_laps_follow_driver_name). Añade stint = NULL y source = 'formula1db'.

int_fastf1_laps — 205 718 × 24

Vueltas FastF1 con race_id (join por season, round) y driver_id (por dorsal). Calcula gap_to_leader_ms = session_time_ms - min(session_time_ms) over (partition by race_id, lap_number). source = 'fastf1'.

int_ergast_laps — 535 748 × 5

Vueltas de Ergast con race_id de F1DB (join por season, round): race_id, driver_number, lap_number, position, lap_time_ms.

int_lap_numbering — 638 × 7 (decisión N5)

Detecta, por carrera, si formula1db.com numera las vueltas igual que F1DB (problema típico en carreras relanzadas o en dos partes). Estima el desfase (vuelta formula1db – vuelta F1DB) con dos evidencias independientes:

- fastest_lap_offsets: para cada vuelta rápida oficial (session_type = 'FASTEST_LAP'), la vuelta de formula1db del mismo piloto con el mismo tiempo (±1 ms) más cercana (arg_min).
- pit_stop_offsets: para cada parada de F1DB (PIT_STOP), la vuelta de entrada a boxes de formula1db más cercana.

Cada evidencia necesita ≥ 3 casos (HAVING sum(cases) >= 3); se toma el desfase dominante y su proporción. Estados (numbering_status):

Estado	Regla	Carreras
consistent	todas las evidencias dan desfase 0	634

Estado	Regla	Carreras
pit_lap_convention	vuelta rápida con desfase 0 y paradas de F1DB una vuelta después que la entrada a boxes ($\geq 80\%$): convención, no error (España 1994)	1
renumbered	un único desfase $\neq 0$ en todas las evidencias con $\geq 80\%$ de los casos y sin superar las vueltas oficiales $\rightarrow$ se aplica <code>applied_lap_shift = -desfase</code>	0
inconsistent	las evidencias no cuadran (San Marino 1994 y Japón 1994 por las paradas de F1DB, con desfases de +4 y +13; Bélgica 2001 por sus vueltas rápidas, desfase -4) $\rightarrow$ se conserva la numeración y se marca	3

Las carreras sin evidencia suficiente no aparecen (en `int_laptimes` se etiquetan `no_evidence`).

int_tyre_compound_mapping — 288 × 6 (decisión N6)

Equivalencia por carrera entre el compuesto real de Pirelli C0–C6 (`formula1db`, 2019–2023) y el relativo HARD/MEDIUM/SOFT (FastF1). Cuenta pares de vueltas comunes, toma el relativo más frecuente (`arg_max`) por compuesto real y valida la coherencia con ventanas: cada compuesto real tiene un relativo distinto (`count(*) over (partition by race_id, relative_compound) = 1`) y, al ordenar por compuesto real, la «blandura» nunca retrocede (`lag`). `is_consistent` se evalúa por carrera con `bool_and(...)` over (`partition by race_id`). Resultado: 101 carreras, todas coherentes.

int_laptimes — 1 270 260 × 33 (decisiones A1, A2, A6, N5, N6, N8)

Modelo central de vueltas fusionadas. Pasos (CTE):

1. `official`: vueltas oficiales, clasificado, ganador y coche compartido por (carrera, piloto).
2. `count_back_races` (A6): carreras en las que alguna fuente registra al ganador más allá de las vueltas oficiales (bandera a cuadros anticipada o resultado tomado una vuelta antes: China 2014, Canadá 2018, Japón 2019, Bélgica 2021, Japón 2022), excluyendo coches compartidos.
3. `formula1db`: aplica `applied_lap_shift` (conserva `original_lap_number` y `lap_numbering_status`, `no_evidence` si no hay fila) y une `lap_corrections` solo si el valor original coincide.
4. `formula1db_corrected_time / formula1db_repositioned`: en carreras con corrección, recalcula el tiempo acumulado (`sum(...)` over (`partition by race_id, driver_number order by lap_number rows between unbounded preceding and current row`)), la diferencia con el líder y la posición (`rank()` over (`partition by race_id, lap_number order by corrected_race_time_ms`)).
5. `compared / resolved`: LEFT JOIN con FastF1 y Ergast por (`race_id, driver_number, lap_number`); regla de mayoría (A2): si FastF1 y Ergast coinciden entre sí (± 1 ms) y ambos discrepan de `formula1db` (vuelta > 1), se toma el tiempo de FastF1; igual para la posición.
6. `formula1db_final`: tiempo y posición resueltos; enriquecimiento con FastF1 (`stint, tyre_age, laps` con prioridad FastF1, `speed_trap_kmh, is_accurate`, banderas de pista); `validation_status, confirmed_by` (lista `ergast/fastf1`), `corrections` (lista `lap_time_ms:seed, lap_time_ms:majority, position:recalculated, position:majority`) y valores originales.
7. `fastf1_fill`: vueltas FastF1 de carreras sin `formula1db` o de pilotos ausentes en ella (con vueltas oficiales > 0); `validation_status = 'single_source'`, `lap_numbering_status = 'not_applicable'`.
8. `ergast_fill` (N8): vueltas sueltas que faltan en `formula1db` dentro de las vueltas oficiales, `source = 'ergast'` (hoy 0 filas: no hay huecos que rellenar).

9. `UNION ALL BY NAME` de las tres ramas, cálculo de `tyre_compound_pirelli` (C0–C6, o nombre comercial hasta 2018, o `INTERMEDIATE/WET`) y `tyre_compound_relative` (vía el mapeo coherente de 2019–2023, o directamente `HARD/MEDIUM/SOFT` desde 2019) y **borrado A6** de las vueltas de clasificados posteriores al final oficial en las `count_back_races`.

Estados de `validation_status`:

Estado	Regla	Vueltas
<code>confirmed</code>	Ergast o FastF1 dan el mismo tiempo (± 1 ms)	587 453
<code>timing_convention</code>	diferencia < 1 s, vuelta 1 o vuelta > 300 s (bandera roja): cada fuente reparte el tiempo distinto	443
<code>disputed</code>	otra fuente discrepa ≥ 1 s sin mayoría: se conserva <code>formula1db</code>	305
<code>corrected</code>	seed con evidencia o mayoría de las otras dos fuentes	1
<code>single_source</code>	sin otra fuente con la que contrastar (antes de 1996, o fuente FastF1)	682 058

Correcciones aplicadas: `position:recalculated` 185 vueltas (carrera de Russell), `position:majority` 49, `lap_time_ms:seed` 1. Fuente final: `formula1db` 1 226 272 vueltas (1 125 carreras, 1950–2024), `fastf1` 43 988 (40 carreras: 2025–2026 completas y un piloto suelto de 2024).

int_lap_completeness — 25 752 × 12

Concilia, por (carrera, piloto), las vueltas registradas con las oficiales de F1DB. Excluye a quien no tomó la salida (DNQ, DNPQ, DNP, EX, DNS sin vueltas); `FULL OUTER JOIN` entre oficial y registrado; fuente principal de la carrera con `mode(source)`. Calcula `missing_laps`, `laps_beyond_official` y `status`:

Estado	Significado	Pilotos-carrera
<code>ok</code>	todas las vueltas oficiales, sin huecos (se tolera una vuelta extra: la de abandono)	25 339
<code>disqualified</code>	DSQ: F1DB no computa sus vueltas	160
<code>shared_car</code>	coche compartido: la fuente no separa vueltas por piloto	117
<code>missing_laps</code>	faltan vueltas o hay huecos	116
<code>no_laps</code>	la carrera tiene vueltas pero el piloto no	11
<code>extra_laps</code>	más de una vuelta por encima de las oficiales	9

Observación: en `silver` existe además la tabla `int_formula1db_laps_validated` (1 226 299 × 22) **sin modelo en el proyecto**: es un residuo de una versión anterior que dbt no elimina. Conviene borrarla (ver §7).

7.4 Capa GOLD: modelo dimensional

7.4.1 Enfoque

Gold mantiene el diseño del TFG (Kimball, **esquema de constelación**: varias tablas de hechos que comparten dimensiones conformadas) y lo amplía. Rasgos de diseño:

- **Claves naturales** de F1DB como claves de dimensión: `driver_id`, `constructor_id`, `engine_manufacturer_id`, `tyre_manufacturer_id` (texto tipo *slug*, p. ej. lewis-hamilton) y `race_id` (entero). No hay claves sustitutas.
- **Dimensiones desnormalizadas** (sin copo de nieve), como en el TFG: el país se incrusta en cada dimensión; temporada, circuito y Gran Premio se incrustan en `dim_race`.

- **Dimensiones tipo 1** (sobrescritura completa en cada dbt build); no hay historización.
- **Atributos degenerados** en los hechos (session_type, position_text, reason_retired, driver_number, source, validation_status...), como decidió el TFG para no crear hasta seis dimensiones de una sola columna.
- Hechos con **metadatos de linaje y calidad por fila**: is_corrected, overridden_columns, disputed_columns, source, validation_status, confirmed_by, corrections, original_*, lap_numbering_status, quality_status.
- Capa de **métricas precalculadas** (agg_*) que sustituye a parte de las medidas DAX del TFG.

7.4.2 Inventario de tablas gold (DuckDB, 28/09/2026)

Tabla	Tipo	Filas	Col.	Granularidad	Clave (natural / compuesta)
dim_driver	dimensión	917	15	piloto	driver_id
dim_constructor	dimensión	187	6	constructor	constructor_id
dim_engine_manufacturer	dimensión	78	5	motorista	engine_manufacturer_id
dim_tyre_manufacturer	dimensión	9	5	proveedor de neumáticos	tyre_manufacturer_id
dim_race	dimensión	1 172	38	Gran Premio (1950–2026; 8 aún sin disputar)	race_id
fact_race_result	hecho transaccional	28 211	32	piloto × carrera × sesión (RACE 27 621, SPRINT 590)	(race_id, session_type, position_display_order)
fact_qualifying_result	hecho transaccional	27 506	19	piloto × carrera × sesión (QUALIFYING 27 040, SPRINT_QUALIFYING 466)	(race_id, session_type, position_display_order)
fact_pit_stops	hecho transaccional	22 535	11	parada real (F1DB; 1994–2026, 612 carreras)	(race_id, driver_id, stop_number) (no testeada)
fact_pit_lane_passes	hecho transaccional	24 271	7	entrada al pit lane registrada en las vueltas	(race_id, driver_id, driver_number, lap_number)
fact_laptimes	hecho transaccional	1 270 260	35	vuelta × piloto × carrera (1 164 carreras)	(race_id, driver_id, driver_number, lap_number)
fact_quali_telemetry	hecho transaccional	518 699	15	muestra cada 10 m de la vuelta rápida de clasificación (63 carreras, 2024–2026)	sin clave declarada (de facto race_id, driver_id, distance_m)
fact_driver_standing	hecho <i>snapshot</i> periódico	21 541	8	piloto × carrera (clasificación tras cada GP)	(race_id, driver_id) (no testeada)
fact_constructor_standing	hecho <i>snapshot</i> periódico	10 654	9	constructor (+motor) × carrera	(race_id, position_display_order) (no testeada)
agg_driver_career	agregado	917	23	piloto	driver_id
agg_constructor_career	agregado	187	16	constructor	constructor_id
agg_driver_season	agregado	1 681	15	piloto × temporada	(season, driver_id)
agg_teammate_h2h	agregado	13 530	13	piloto × compañero × constructor × temporada	(season, constructor_id, driver_id, teammate_id)

7.4.3 Dimensiones

Dimensión	Construcción
dim_driver	stg_f1db__drivers con dos LEFT JOIN a stg_f1db__countries: rol «país de nacimiento» (country_of_birth_name) y rol «nacionalidad» (nationality_country, nationality_demonym, nationality_alpha2). Es un <i>role-playing</i> de la dimensión país resuelto por desnormalización.
dim_constructor	stg_f1db__constructors LEFT JOIN países → country_constructor, demonym_constructor, alpha2_constructor.
dim_engine_manufacturer	Ídem con sufijo _engine_manufacturer.
dim_tyre_manufacturer	Ídem con sufijo _tyre_manufacturer.
dim_race	stg_f1db__races LEFT JOIN circuitos LEFT JOIN país del circuito LEFT JOIN Grandes Premios LEFT JOIN país del GP. Añade is_completed (existe algún RACE_RESULT: 1 164 de 1 172) e is_season_final_race (round = max(round) over (partition by season)). Como contiene season y race_date, hace también de dimensión temporal de grano carrera.

7.4.4 Hechos

- **fact_race_result:** int_race_data_corrected filtrado a RACE_RESULT y SPRINT_RACE_RESULT (renombrados RACE/SPRINT). Medidas: laps, time_ms, time_penalty_ms, gap_ms, gap_laps, interval_ms, points (coalesce(..., 0)), grid_position, positions_gained, pit_stops. Indicadores: is_win (position_number = 1), is_podium (<= 3), is_pole_position, is_fastest_lap, is_driver_of_the_day, is_grand_slam, is_shared_car. Linaje: is_corrected, overridden_columns, disputed_columns. A diferencia del TFG, la parrilla y la vuelta rápida ya vienen en la fila RACE_RESULT de F1DB, así que desaparecen los *merges* de Power Query con fact_starting_grid_position y fact_fastest_lap.
- **fact_qualifying_result:** QUALIFYING_RESULT y SPRINT_QUALIFYING_RESULT; best_time_ms = coalesce(least(q1, q2, q3), qualifying_time_millis) (cubre los formatos antiguos de sesión única); gap_to_pole_pct = round(100 · (best / min(best) over (partition by race_id, session_type) - 1), 3); is_pole; overridden_columns.
- **fact_pit_stops:** session_type = 'PIT_STOP'; stop_number, lap_number, time_ms y time_seconds (medida derivada del TFG). Solo paradas reales.
- **fact_pit_lane_passes** (nuevo, decisión N4): todas las vueltas con is_pit_in_lap de int_laptimes emparejadas con la parada de F1DB de la misma vuelta (corrigiendo pit_stop_offset en carreras inconsistent/pit_lap_convention y quedándose con una parada por vuelta con QUALIFY row_number()). Tipificación pass_type, en orden de evaluación: | pass_type | Regla | Filas | |—|—|—:| | pit_stop | coincide con una parada de F1DB | 22 485 | | unclassified | carrera sin paradas en F1DB (antes de 1994) | 1 118 | | red_flag | entrada con bandera roja | 62 | | retirement | no clasificado y entra en su última vuelta | 351 | | safety_car | bajo SC/VSC sin parada registrada | 114 | | penalty_or_other | resto (drive-through, stop-and-go...) | 141 |
- **fact_laptimes** (el fact_gp_laptimes del TFG): int_laptimes LEFT JOIN int_lap_completeness. Añade is_incomplete_lap (vuelta posterior a las oficiales: la de abandono) y quality_status (ok 1 249 767 vueltas, disqualified 6 860, shared_car 6 831, missing_laps 6 229, extra_laps 573; unmatched_driver si no hay conciliación, hoy 0). Distribución de lap_numbering_status: consistent 693 360, no_evidence 529 556, not_applicable 43 988, inconsistent 2 269, pit_lap_convention 1 087. Compuesto relativo conocido en 191 209 vueltas (HARD 77 192, MEDIUM 67 713, SOFT 28 696, INTERMEDIATE 13 935, WET 3 673).
- **fact_quali_telemetry** (nuevo): stg_fastf1__quali_telemetry con race_id y driver_id (vía int_driver_race_numbers; 0 filas sin piloto).

- **fact_driver_standing** y **fact_constructor_standing**: SELECT * de las vistas de staging de clasificaciones tras cada carrera (puntos y posición acumulados, positions_gained, championship_won).

7.4.5 Métricas (marts/metrics)

Modelo	Entradas	Lógica	Métricas
agg_driver_career	fact_race_result, stg_f1db__season_driver_standings, dim_driver	Agregados por piloto con count(distinct race_id) filter (where ...) (evita dobles conteos en coches compartidos); sprints aparte; campeonatos desde la clasificación final	championships, best_championship_position, first/last_season, race_entries, race_starts (excluye DNQ/DNPQ/DNP/EX/DNS), wins, podiums, pole_positions, fastest_laps, grand_slams, driver_of_the_day, sprint_wins, points (carrera + sprint), championship_points, best_race_result, laps_completed, win_rate_pct, podium_rate_pct
agg_constructor_career	fact_race_result, stg_f1db__season_constructor_standings, dim_constructor, dim_race	Criterios de F1DB: victorias por carrera, podios por coche (count(distinct (race_id, position_number))), vueltas rápidas por piloto acreditado	championships, best_championship_position, first/last_season, race_entries, wins, podiums, podium_races, one_two_finishes, pole_positions, fastest_laps, race_points
agg_driver_season	stg_f1db__season_driver_standings, fact_race_result, dim_race, dim_driver, dim_constructor	Posición y puntos finales de F1DB + recuentos de la temporada; equipo principal = mode(constructor_id)	championship_position(text), points, championship_won, races, wins, podiums, pole_positions, fastest_laps, best_result
agg_teammate_h2h	fact_race_result, fact_qualifying_result, dim_race	Self-join por carrera y constructor (sin coches compartidos); «por delante» según position_display_order en carrera y en clasificación	races_together, race_ahead(_pct), qualifyings_together, quali_ahead(_pct), points, teammate_points, points_difference

La paridad de agg_driver_career y agg_constructor_career con los totales oficiales de F1DB se verifica con tests singulares (§4.3).

7.4.6 Diagramas del modelo (para TikZ)

Constelación completa (cada hecho forma una estrella; las dimensiones son compartidas):

— descripción del diagrama —

```
# Esquema de constelación gold. Aristas = FK -> PK, cardinalidad N:1
dimensiones: dim_race[race_id], dim_driver[driver_id], dim_constructor[constructor_id],
             dim_engine_manufacturer[engine_manufacturer_id], dim_tyre_manufacturer[tyre_manufacturer_id]
hechos: fact_race_result, fact_qualifying_result, fact_pit_stops, fact_pit_lane_passes,
        fact_laptimes, fact_quali_telemetry, fact_driver_standing, fact_constructor_standing
aristas:
  fact_race_result      -> dim_race, dim_driver, dim_constructor, dim_engine_manufacturer,
    ↪ dim_tyre_manufacturer
  fact_qualifying_result -> dim_race, dim_driver, dim_constructor, dim_engine_manufacturer,
    ↪ dim_tyre_manufacturer
  fact_pit_stops        -> dim_race, dim_driver, dim_constructor, dim_engine_manufacturer,
    ↪ dim_tyre_manufacturer
```

```

fact_pit_lane_passes    -> dim_race, dim_driver
fact_laptimes           -> dim_race, dim_driver
fact_quali_telemetry    -> dim_race, dim_driver
fact_driver_standing    -> dim_race, dim_driver
fact_constructor_standing -> dim_race, dim_constructor, dim_engine_manufacturer
relaciones entre hechos (misma clave de grano, sin FK; línea discontinua):
fact_pit_lane_passes (race_id, driver_id, stop_number) ~ fact_pit_stops (race_id, driver_id, stop_number)
fact_pit_lane_passes (race_id, driver_id, driver_number, lap_number) ~ fact_laptimes (misma clave)
disposición sugerida: dim_race en el centro; dim_driver a la derecha; dim_constructor,
dim_engine_manufacturer y dim_tyre_manufacturer a la izquierda; hechos de resultados arriba,
hechos de vueltas y boxes abajo, standings a los lados.

```

Estrella principal (resultados):

descripción del diagrama

```

centro: fact_race_result
clave: race_id, session_type, position_display_order
medidas: points, laps, time_ms, gap_ms, grid_position, positions_gained, pit_stops
indicadores: is_win, is_podium, is_pole_position, is_fastest_lap, is_grand_slam
linaje: is_corrected, overridden_columns, disputed_columns
radios (N:1): dim_race, dim_driver, dim_constructor, dim_engine_manufacturer, dim_tyre_manufacturer

```

Estrella de vueltas:

descripción del diagrama

```

centro: fact_laptimes
clave: race_id, driver_id, driver_number, lap_number
medidas: lap_time_ms, position, gap_to_leader_ms, sector_1_ms, sector_2_ms, sector_3_ms, tyre_age_laps,
↳ speed_trap_kmh
atributos: stint, tyre_compound, tyre_compound_pirelli, tyre_compound_relative, is_pit_in_lap,
is_pit_out_lap, is_yellow_flag, is_safety_car, is_virtual_safety_car, is_red_flag
calidad: source, validation_status, confirmed_by, corrections, lap_numbering_status, quality_status
radios (N:1): dim_race, dim_driver
satélites: fact_pit_lane_passes (misma clave de vuelta), fact_quali_telemetry (race_id, driver_id)

```

Capa de métricas (dependencias de construcción, no FK):

descripción del diagrama

```

fact_race_result + stg_f1db__season_driver_standings + dim_driver -> agg_driver_career
fact_race_result + stg_f1db__season_constructor_standings + dim_constructor + dim_race ->
↳ agg_constructor_career
stg_f1db__season_driver_standings + fact_race_result + dim_race + dim_driver + dim_constructor ->
↳ agg_driver_season
fact_race_result + fact_qualifying_result + dim_race -> agg_teammate_h2h

```

7.4.7 Comparación con el modelo del TFG (Fig. 5.7, secciones 5.1–5.3)

En el TFG las dimensiones de análisis eran Carreras (agregado: temporada), Pilotos, Constructores, Fabricantes de motores y Fabricantes de neumáticos (Tabla 5.1). La Fig. 5.7 («Diseño final esquema de constelación») tenía 5 dimensiones y 6 hechos. La ETL se hacía en Power Query (ODBC contra la SQLite de F1DB y CSV de formula1db.com en SharePoint), las medidas en DAX y la actualización era manual (nueva descarga de F1DB, scripts Selenium y republicación o puerta de enlace local).

Aspecto	TFG (Power BI)	f1-data-platform
Dimensiones	dim_driver, dim_constructor, dim_engine_manufacturer, dim_tyre_manufacturer, dim_race (temporada, GP y circuito desnormalizados; Figs. 5.5–5.6)	Las mismas 5, con los mismos nombres y el mismo criterio de desnormalización; se añaden alpha2/alpha3 (banderas en la web), is_completed, is_season_final_race, has_sprint y los booleanos de «decider»

Aspecto	TFG (Power BI)	f1-data-platform
Hechos	fact_race_result, fact_qualifying_result, fact_pit_stops, fact_race_driver_standing, fact_race_constructor_standing, fact_gp_laptimes (6)	8: se renombran fact_driver_standing, fact_constructor_standing y fact_laptimes; se añaden fact_pit_lane_passes y fact_quali_telemetry
Tiempos por vuelta	Solo formula1db.com	Fusión formula1db + FastF1 + Ergast con estado de validación, correcciones y linaje por fila
Resultados	Merges con fact_starting_grid_position y fact_fastest_lap en Power Query; columnas condicionales de penalización	Directamente de race_data (fila RACE_RESULT), con correcciones auditables (seeds)
Medidas	DAX (victorias, podios, compañeros..., §5.3.3)	Columnas derivadas en los hechos (is_win, is_podium, gap_to_pole_pct, time_seconds) + tablas agg_* precalculadas y verificadas contra F1DB
Tipos	Conversión manual en Power Query (puntos → comas, enteros/decimales)	Tipos de SQLite preservados en Parquet; casts en staging
Atributos de texto	En los hechos (status, tyre, position_text, type...)	Mismo criterio (dimensiones degeneradas)
Calidad	Revisión manual (Script-2.sql)	Esquema quality (40 comprobaciones con umbral), 50 tests y 4 unit tests
Actualización	Manual	Semanal con GitHub Actions, incremental y reproducible

7.5 Capa de CALIDAD

La calidad se trabaja en tres niveles: (1) **modelos de contraste** entre fuentes (esquema quality), resumidos en qa_summary; (2) **tests de datos de dbt** (genéricos y singulares) que se ejecutan en cada dbt build e impiden publicar si fallan; (3) **unit tests de dbt** sobre la lógica de conciliación más delicada.

7.5.1 Modelos qa_* (esquema quality, tablas)

Modelo	Filas	Qué compara	Emparejamiento	Indicadores
qa_formula1db_vs_fastf1_laps	161 475	Vueltas de formula1db frente a FastF1 en las carreras que cubren ambas	(race_id, driver_number, lap_number)	is_matched, is_time_match (± 1 ms, excluye la vuelta 1: cada fuente la cronometra desde un punto distinto), is_position_match, is_tyre_match (solo compuestos comparables SOFT/MEDIUM/HARD/INTERMEDIATE/WET)
qa_formula1db_vs_ergast_laps	535 760	formula1db frente a Ergast (1996–2022)	ídem	is_matched, is_time_match, is_position_match

Modelo	Filas	Qué compara	Emparejamiento	Indicadores
qa_fastf1_vs_f1db_results	3 768	Resultados de carrera FastF1 frente a F1DB	(race_id, driver_number)	is_position_match (solo clasificados), is_points_match, is_grid_match (excluye salidas desde el pit lane)
qa_fastf1_vs_f1db_qualifying	8 326	Q1/Q2/Q3 FastF1 frente a F1DB	(race_id, driver_number) + CROSS JOIN LATERAL (VALUES ...) para desplegar las tres sesiones	is_matched, is_time_match
qa_pit_stops	22 535	Cada parada de F1DB frente a las entradas a boxes de las vueltas	fact_pit_stops LEFT JOIN fact_pit_lane_passes (pass_type = 'pit_stop') por (race_id, driver_id, stop_number)	is_match, fuente dominante de la carrera
qa_fastest_laps	14 005	Mejor vuelta calculada (sin vueltas borradas) frente a la vuelta rápida oficial de F1DB	(race_id, driver_id)	difference_ms, is_match (±1 ms)

7.5.2 qa_summary (40 comprobaciones)

Una fila por comprobación con check_id, description, compared, matched, min_match_pct, match_pct y status: INFO si no tiene umbral, NO_DATA si no hay casos, PASS si match_pct ≥ min_match_pct y FAIL en otro caso. Tests: unique y not_null en check_id, accepted_values en status. Resultado actual: 23 PASS, 17 INFO, 0 FAIL, 0 NO_DATA.

check_id	Comparados	Coinciden	%	Umbral	Estado
formula1db_vs_fastf1_lap_time	156 003	154 910	99,30	98	PASS
formula1db_vs_fastf1_position	161 367	159 817	99,04	98	PASS
formula1db_vs_fastf1_tyre	43 019	42 611	99,05	98	PASS
formula1db_vs_ergast_coverage	535 760	535 542	99,96	98	PASS
formula1db_vs_ergast_lap_time	525 436	524 725	99,86	98	PASS
formula1db_vs_ergast_position	535 542	534 945	99,89	98	PASS
fastf1_vs_f1db_race_position	3 260	3 260	100,00	100	PASS
fastf1_vs_f1db_points	3 768	3 768	100,00	100	PASS
fastf1_vs_f1db_grid	3 646	3 616	99,18	99	PASS
fastf1_vs_f1db_qualifying	8 263	8 255	99,90	99	PASS
f1db_pit_stops_in_laps_fastf1	1 238	1 238	100,00	99	PASS
f1db_pit_stops_in_laps_formula1db	21 297	21 256	99,81	99	PASS
fastest_lap_fastf1	779	772	99,10	99	PASS
fastest_lap_formula1db	13 226	13 123	99,22	95	PASS
lap_completeness_fastf1	794	794	100,00	99	PASS
lap_completeness_formula1db	24 681	24 545	99,45	97	PASS
lap_driver_attribution	1 270 260	1 270 260	100,00	99,9	PASS
tyre_mapping_consistent	101	101	100,00	100	PASS
f1db_overrides_A3 / A4 / A5 / N1 / N2	74 / 97 / 14 / 8 / 2	todas	100,00	100	PASS (5)
formula1db_race_alignment	1 125	1 108	98,49	—	INFO
lap_numbering_consistent / inconsistent / pit_lap_convention	638	634 / 3 / 1	—	—	INFO (3)
lap_source_formula1db / fastf1	1 270 260	1 226 272 / 43 988	96,54 / 3,46	—	INFO (2)
lap_time_confirmed / single_source / timing_convention / disputed / corrected	1 270 260	587 453 / 682 058 / 443 / 305 / 1	—	—	INFO (5)
pit_lane_pass_<tipo> (6 tipos)	24 271	ver §3.4	—	—	INFO (6)

Los umbrales se fijaron por tipo de comparación: 100 % donde la semántica es idéntica (puntos, posición de clasificados, overrides aplicados, mapeo de neumáticos), 98–99 % en vueltas y tiempos, y 95–97 %

donde el histórico tiene limitaciones conocidas (vuelta rápida y completitud en formula1db).

7.5.3 Tests de dbt

Genéricos (declarados en `_core__models.yml`, `_quality__models.yml` y `_seeds.yml`; 43 en total):

Test	Nº	Dónde
unique	6	PK de las 5 dimensiones; qa_summary.check_id
not_null	13	PK de dimensiones, dim_race.season, driver_id de fact_race_result, fact_qualifying_result, fact_laptimes, fact_quali_telemetry, pass_type, validation_status, check_id
relationships	10	Integridad referencial de fact_race_result con las 5 dimensiones; driver_id de fact_qualifying_result, fact_pit_stops, fact_laptimes, fact_driver_standing; constructor_id de fact_constructor_standing
accepted_values	9	fact_race_result.session_type (RACE, SPRINT), fact_pit_lane_passes.pass_type, fact_laptimes.source, .quality_status, .validation_status, qa_summary.status, seed race_data_overrides.column_name y .action, seed lap_corrections.column_name
unique_combination (propio, tests/generic/unique_combination.sql)	5	Clave compuesta de fact_race_result, fact_qualifying_result, fact_pit_lane_passes, fact_laptimes y de la seed race_data_overrides

Singulares (transform/tests/, 7):

Test	Severidad	Qué valida
assert_driver_totals_match_f1db	error	Paridad de agg_driver_career con los totales oficiales de F1DB (campeonatos, inscripciones, salidas, victorias, podios, poles, vueltas rápidas, grand slams), excluyendo los pilotos afectados por race_data_corrections
assert_constructor_totals_match_f1db	error	Ídem para agg_constructor_career
assert_one_winning_car_per_race	error	Exactamente un coche ganador (count(distinct driver_number) = 1) por carrera y sesión (los coches compartidos pueden repartir la victoria entre pilotos)
assert_quality_thresholds	error	Ninguna fila de qa_summary en FAIL (es el «interruptor» que bloquea la publicación)
assert_standings_points_match_results	warn	Desde 1991, suma de puntos de carrera + sprint = puntos de la clasificación final
assert_corrections_still_apply	warn	Cada fila de race_data_corrections sigue encajando con F1DB (si no, F1DB lo corrigió y puede retirarse)
assert_overrides_still_apply	warn	Cada override sigue aplicándose (piloto y valor original sin cambios)

Unit tests (4, en `_quality__models.yml`, con datos de entrada simulados con given/expect): test_shared_car_laps_follow_driver_name (int_formula1db_laps: Farina/Bonetto 1951), test_realigns_race_labelled_with_wrong_round y test_keeps_labelled_round_on_tie (int_formula1db_race_

alignment: Mónaco 2023 etiquetado como R7; empate) y test_lap_completeness_statuses (int_lap_completeness: vuelta de abandono, abandono en la vuelta 1, descalificado y huecos).

Resultado actual: 50/50 tests pass (ningún aviso de los 3 warn) y 4/4 unit tests pass. Además, las pruebas de Python (pytest) cubren la ingesta y el snapshot, y la CI ejecuta dbt parse y dbt build completos contra el último snapshot publicado.

7.6 Linaje completo y pipeline

7.6.1 Linaje de extremo a extremo

descripción del diagrama

```
# Linaje (izquierda -> derecha). Agrupar por columnas: Fuente | Raw | Bronze | Staging | Intermediate |
↳ Gold | Quality | Distribución | Consumo
Fuente: GitHub f1db/f1db (release) | API F1 Live Timing (FastF1) | CSV scraping formula1db.com (TFG)
↳ | f1db_csv.zip Ergast 2022
Raw: data/raw/f1db/<tag>/f1db.db (SQLite) [solo F1DB; FastF1 usa data/cache/fastf1]
Bronze: bronze/f1db/*.parquet (31) |
↳ bronze/fastf1/{laps,results,quali_results,quali_telemetry}/season=YYYY/round=RR.parquet
  | bronze/formula1db/{lap_times,race_entries,drivers}.parquet | bronze/ergast/*.parquet (6)
Staging (vistas silver): stg_f1db_* (14) | stg_fastf1_* (4) | stg_formula1db_lap_times |
↳ stg_ergast_lap_times
Seeds (silver): race_data_corrections, race_data_overrides, lap_corrections
Intermediate (tablas silver): int_race_data_corrected -> int_driver_race_numbers ->
↳ {int_formula1db_race_alignment -> int_formula1db_laps, int_fastf1_laps, int_ergast_laps}
  -> {int_lap_numbering, int_tyre_compound_mapping} -> int_laptimes -> int_lap_completeness
Gold: dim_* (desde stg_f1db_*) ; fact_race_result/fact_qualifying_result/fact_pit_stops (desde
↳ int_race_data_corrected)
  ; fact_laptimes (int_laptimes + int_lap_completeness) ; fact_pit_lane_passes (int_laptimes +
  ↳ int_lap_numbering + int_race_data_corrected)
  ; fact_quali_telemetry (stg_fastf1_quali_telemetry + int_driver_race_numbers) ; fact_*_standing
  ↳ (stg_f1db_race_*_standings)
  ; agg_* (hechos + dims + stg_f1db_season_*_standings)
Quality: qa_* (int_* y fact_*) -> qa_summary -> test_assert_quality_thresholds
Distribución (f1-ingest snapshot pack): bronze.tar.gz ; f1.duckdb (gold.* + quality.qa_summary) ;
↳ gold-parquet.zip ; manifest.json -> GitHub Release data-latest
Consumo: API FastAPI (Render) descarga f1.duckdb de la release y lo sirve en solo lectura -> web Next.js
↳ (Vercel)
  ; Power BI / otras herramientas -> gold-parquet.zip
Ciclo: bronze.tar.gz de la release -> (restore) -> bronze de la siguiente ejecución
```

Qué ocurre en cada fase:

Fase	Operaciones	Artefacto
Fuente → raw	Descarga versionada por tag (F1DB); caché HTTP (FastF1)	f1db.db, data/cache
raw/fuente → bronze	Conversión a Parquet sin reglas de negocio, timedelta → ms, snake_case, deduplicación exacta, partición por temporada/ronda, escritura atómica, _metadata.json	data/bronze/**
bronze → staging	Lectura directa de Parquet (external_location), renombrado, casts, parseo de tiempos y códigos (neumático, estado de pista)	vistas silver.stg_*
staging → intermediate	Correcciones (seeds), claves comunes (dorsal → driver_id, temporada/ronda → race_id), realineación de carreras, numeración de vueltas, mapeo de compuestos, fusión de vueltas con validación y linaje, conciliación de completitud	tablas silver.int_*

Fase	Operaciones	Artefacto
intermediate → gold	Modelo dimensional (dimensiones desnormalizadas, hechos por tipo de sesión), indicadores derivados, métricas agregadas	tablas gold.*
gold/intermediate → quality	Contrastes entre fuentes y resumen con umbrales; tests	tablas quality.*, run_results.json
gold → distribución	Base compacta para la API, Parquet ZSTD, manifiesto con hashes, notas de release	dist/* → Release data-latest
distribución → API	La API (config F1_DATA_REPO) descarga manifest.json y f1.duckdb, guarda cada versión como f1-<sha256[:12]>.duckdb, la abre read_only y comprueba cada 6 h (F1_API_REFRESH_HOURS) si hay una nueva	data/api o /data en el contenedor

La API consulta 16 de las 17 tablas gold (todas salvo dim_tyre_manufacturer) y quality.qa_summary (endpoint de estado de calidad).

7.6.2 Pipeline .github/workflows/pipeline.yml paso a paso

Disparadores: `schedule` los **lunes a las 06:00 UTC** (los datos de cronometraje del GP del domingo ya están publicados) y `workflow_dispatch` con entradas `seasons` (temporadas de FastF1) y `force_f1db`.
 permissions: contents: write; concurrency: data-pipeline sin cancelación (nunca dos ejecuciones escribiendo la release a la vez); variables `DATA_TAG = data-latest` y `GH_TOKEN`. Job pipeline en ubuntu-latest, timeout-minutes: 120:

#	Paso	Comando / lógica	Propósito
1	Checkout	actions/checkout@v5	Código
2	uv	astral-sh/setup-uv@v6 con caché	Gestor de entornos
3	Dependencias	uv sync --locked	Entorno reproducible (uv.lock)
4	Restaurar snapshot	gh release download data-latest --pattern bronze.tar.gz → f1-ingest snapshot restore; si no existe la release, error explicativo (el snapshot inicial debe publicarse desde el equipo del autor, que tiene los CSV del TFG)	Recuperar el bronce completo (incluidas las fuentes estáticas)
5	Ingesta F1DB	f1-ingest f1db [--force]	Nueva release de F1DB si la hay (idempotente)
6	Ingesta FastF1	temporadas = entrada o año actual (en enero también el anterior); validación con regex <code>^[0-9]{4}([0-9]{4})*\$</code> ; f1-ingest fastf1 --season \$SEASONS --telemetry; continue-on-error: true	Carga incremental de las carreras nuevas; una carrera sin datos no bloquea la publicación
7	dbt build	uv run dbt build --profiles-dir . en transform/	Seeds, modelos, tests y unit tests en orden de DAG; si falla un test de severidad error (p. ej. assert_quality_thresholds) el pipeline se detiene y no publica
8	Snapshot	f1-ingest snapshot pack --out dist; snapshot notes → dist/notes.md y al GITHUB_STEP_SUMMARY	Generar artefactos y resumen

#	Paso	Comando / lógica	Propósito
9	Publicar	crea la release si no existe (--latest=false); sube f1.duckdb, gold-parquet.zip, manifest.json con --clobber y bronze.tar.gz el último (es la entrada de la próxima ejecución); actualiza las notas	Distribución
10	Aviso FastF1	si el paso 6 falló: ::warning::	Visibilidad de carreras pendientes
11-12	Docs dbt (opcional, vars. PUBLISH_DOCS == 'true')	dbt docs generate --static → site/index.html → upload-pages-artifact	Catálogo y linaje navegables
13	Logs (si falla)	upload-artifact de transform/logs/ y run_results.json	Diagnóstico
job docs	needs: pipeline, deploy-pages@v4	Publicación en GitHub Pages	

El workflow `ci.yml` complementa al pipeline en cada *push*/PR: `ruff` + `pytest`, `dbt parse` y `dbt build` sobre el último snapshot, construcción y prueba de humo de la imagen Docker de la API y pruebas de la web (tipos OpenAPI, Playwright + axe).

— descripción del diagrama —

```
# Flujo del pipeline semanal (nodos = pasos; aristas = orden)
cron lunes 06:00 UTC | workflow_dispatch -> checkout + uv sync
-> restaurar bronze.tar.gz (release data-latest)
-> f1-ingest f1db -> f1-ingest fastf1 --telemetry (continue-on-error)
-> dbt build (seeds, modelos, tests) --[fallo]--> subir logs, fin sin publicar
-> snapshot pack + notes
-> gh release upload (f1.duckdb, gold-parquet.zip, manifest.json, bronze.tar.gz al final)
-> [opcional] dbt docs -> GitHub Pages
release data-latest -> API (Render, refresco cada 6 h) -> web (Vercel)
release data-latest (bronze.tar.gz) -> siguiente ejecución (ciclo)
```

7.7 Diccionario de datos de la capa gold (resumen)

Convenciones: `*_id` = clave natural de F1DB; `*_ms` = milisegundos (BIGINT); `is_*` = booleano; listas `VARCHAR[]`. Tipos tal como están en DuckDB.

7.7.1 Dimensiones

dim_driver (917 × 15) — PK `driver_id`. `driver_id` VARCHAR · `name`, `first_name`, `last_name`, `full_name` VARCHAR · `abbreviation` VARCHAR (código de 3 letras) · `permanent_number` VARCHAR · `gender` VARCHAR · `date_of_birth`, `date_of_death` DATE · `place_of_birth` VARCHAR · `country_of_birth` VARCHAR · `nationality_country`, `nationality_demonym`, `nationality_alpha2` VARCHAR.

dim_constructor (187 × 6) — PK `constructor_id`. `constructor_id`, `name`, `full_name`, `country_constructor`, `demonym_constructor`, `alpha2_constructor` (todas VARCHAR).

dim_engine_manufacturer (78 × 5) — PK `engine_manufacturer_id`. `engine_manufacturer_id`, `name`, `country_engine_manufacturer`, `demonym_engine_manufacturer`, `alpha2_engine_manufacturer`.

dim_tyre_manufacturer (9 × 5) — PK `tyre_manufacturer_id`. `tyre_manufacturer_id`, `name`, `country_tyre_manufacturer`, `demonym_tyre_manufacturer`, `alpha2_tyre_manufacturer`.

dim_race (1 172 × 38) — PK `race_id`. Carrera: `race_id` BIGINT, `season` BIGINT, `round` BIGINT, `race_date` DATE, `race_time_utc` VARCHAR, `official_name`, `qualifying_format`, `sprint_qualifying`

format VARCHAR, has_sprint BOOLEAN, circuit_type, direction VARCHAR, course_length DOUBLE, turns BIGINT, laps BIGINT, distance DOUBLE, scheduled_laps BIGINT, scheduled_distance DOUBLE, drivers_championship_decider, constructors_championship_decider, is_completed, is_season_final_race BOOLEAN. Circuito: circuit_id, circuit_name, circuit_full_name, circuit_place_name, circuit_latitude/circuit_longitude DOUBLE, circuit_country, circuit_demonym, circuit_alpha2, circuit_alpha3. Gran Premio: grand_prix_id, grand_prix_name, grand_prix_full_name, grand_prix_short_name, grand_prix_abbreviation, grand_prix_country, grand_prix_demonym.

7.7.2 Hechos

fact_race_result (28 211 × 32) — clave (race_id, session_type, position_display_order).

Columna	Tipo	Descripción
race_id	BIGINT	FK dim_race
session_type	VARCHAR	RACE / SPRINT
position_display_order	BIGINT	orden de presentación (incluye no clasificados)
position_number / position_text	BIGINT / VARCHAR	posición o motivo (DNF, DSQ, DNQ, NC...)
driver_number	VARCHAR	dorsal
driver_id, constructor_id, engine_manufacturer_id, tyre_manufacturer_id	VARCHAR	FK a dimensiones
is_shared_car	BOOLEAN	coche compartido (años 50)
laps, time_ms, time_penalty_ms, gap_ms, gap_laps, interval_ms	BIGINT	vueltas y tiempos
reason_retired	VARCHAR	motivo de abandono
points	DOUBLE	puntos (0 si nulo)
grid_position / grid_position_text / positions_gained / pit_stops	BIGINT / VARCHAR / BIGINT / BIGINT	parrilla, ganancia y nº de paradas
is_pole_position, is_fastest_lap, is_driver_of_the_day, is_grand_slam, is_win, is_podium	BOOLEAN	indicadores
is_corrected	BOOLEAN	fila corregida por race_data_corrections
overridden_columns, disputed_columns	VARCHAR[]	columnas corregidas / disputadas (race_data_overrides)

fact_qualifying_result (27 506 × 19) — clave (race_id, session_type, position_display_order). session_type (QUALIFYING/SPRINT_QUALIFYING), posición (position_display_order, position_number, position_text), driver_number, 4 FK, time_ms, q1_ms, q2_ms, q3_ms, laps, best_time_ms, gap_to_pole_pct DOUBLE, is_pole, overridden_columns.

fact_pit_stops (22 535 × 11) — grano: parada. race_id, position_display_order, driver_number, 4 FK, stop_number, lap_number, time_ms BIGINT, time_seconds DOUBLE.

fact_pit_lane_passes (24 271 × 7) — clave (race_id, driver_id, driver_number, lap_number). race_id, driver_id, driver_number, lap_number, stop_number (si pass_type = pit_stop), pass_type (6 valores, §3.4), source.

fact_laptimes (1 270 260 × 35) — clave (race_id, driver_id, driver_number, lap_number).

Grupo	Columnas
Claves	race_id BIGINT, driver_id VARCHAR, driver_number VARCHAR, lap_number BIGINT

Grupo	Columnas
Medidas	position BIGINT, lap_time_ms BIGINT, gap_to_leader_ms HUGEINT, sector_1_ms/sector_2_ms/sector_3_ms BIGINT, speed_trap_kmh DOUBLE
Neumático	stint INTEGER, tyre_compound (tal como lo publica la fuente), tyre_compound_pirelli (C0-C6 o comercial), tyre_compound_relative (HARD/MEDIUM/SOFT/INTERMEDIATE/WET desde 2019), tyre_age_laps (FastF1 si existe), formula1db_tyre_age_laps
Estado de pista	is_pit_in_lap, is_pit_out_lap, is_yellow_flag, is_safety_car, is_virtual_safety_car, is_red_flag, is_deleted, is_accurate
Linaje y calidad	source (formula1db/fastf1/ergast), validation_status (5 valores), confirmed_by VARCHAR[], corrections VARCHAR[], original_lap_time_ms, original_position, original_lap_number, lap_numbering_status (5 valores), is_incomplete_lap, quality_status (5 valores + unmatched_driver)

fact_quali_telemetry (518 699 × 15) — grano: muestra cada 10 m. race_id, driver_id, driver_code, driver_number, tyre_compound, lap_time_ms, distance_m, speed_kmh, rpm DOUBLE, gear BIGINT, throttle_pct DOUBLE, is_braking BOOLEAN, drs BIGINT, x, y DOUBLE.

fact_driver_standing (21 541 × 8) — grano: piloto tras cada carrera. race_id, position_display_order, position_number, position_text, driver_id, points DOUBLE, positions_gained, championship_won.

fact_constructor_standing (10 654 × 9) — grano: constructor (+motor) tras cada carrera. Ídem con constructor_id y engine_manufacturer_id.

7.7.3 Agregados

agg_driver_career (917 × 23): driver_id, name, nationality_country, nationality_alpha2, championships, best_championship_position, first_season, last_season, race_entries, race_starts, wins, podiums, pole_positions, fastest_laps, grand_slams, driver_of_the_day, sprint_wins, points DOUBLE, championship_points DOUBLE, best_race_result, laps_completed HUGEINT, win_rate_pct, podium_rate_pct DOUBLE.

agg_constructor_career (187 × 16): constructor_id, name, country_constructor, alpha2_constructor, championships, best_championship_position, first_season, last_season, race_entries, wins, podiums, podium_races, one_two_finishes, pole_positions, fastest_laps, race_points DOUBLE.

agg_driver_season (1 681 × 15): season, driver_id, driver_name, constructor_id, constructor_name, championship_position, championship_position_text, points, championship_won, races, wins, podiums, pole_positions, fastest_laps, best_result.

agg_teammate_h2h (13 530 × 13): season, constructor_id, driver_id, teammate_id, races_together, race_ahead, race_ahead_pct, qualifyings_together, quali_ahead, quali_ahead_pct, points, teammate_points, points_difference.

7.8 Propuestas de mejora

Ordenadas por relación beneficio/esfuerzo. Cada una indica el problema observado y la justificación.

7.8.1 Higiene y robustez inmediata (esfuerzo bajo)

1. Eliminar la tabla huérfana **silver.int_formula1db_laps_validated** (1,2 M filas, sin modelo). dbt no borra relaciones que ya no gestiona; ocupa espacio en el almacén y confunde al explorar. Añadir un on-run-end o una macro de limpieza de relaciones no gestionadas.

2. **Rutas absolutas o PRAGMA/dbt-duckdb external_root en las sources:** las vistas `stg_*` guardan `../data/bronze/...` y solo funcionan si el directorio de trabajo es `transform/` (comprobado: fuera de él fallan). Resolver la ruta con `env_var` absoluta o materializar las vistas que se consulten fuera de dbt.
3. **Tests de clave que faltan:** `unique_combination` para `fact_pit_stops` (`race_id`, `driver_id`, `stop_number`), `fact_driver_standing` (`race_id`, `driver_id`), `fact_constructor_standing` (`race_id`, `constructor_id`, `engine_manufacturer_id`), las cuatro tablas `agg_*` y `fact_quali_telemetry`; `relationships` para `race_id` de todos los hechos (hoy solo se prueba en `fact_race_result`) y para `constructor_id/engine_manufacturer_id` de `fact_qualifying_result` y `fact_pit_stops`. `accepted_values` para `fact_qualifying_result.session_type` y `fact_laptimes.lap_numbering_status`.
4. **Documentar marts/metrics y staging en YAML** (hoy no tienen `.yaml`): descripciones de columnas para que dbt docs y la API tengan un catálogo completo.
5. **Tipos:** `gap_to_leader_ms` y `laps_completed` salen como `HUGEINT` (efecto de sumas y restas de ventanetas); convertir a `BIGINT`. `race_time_utc` es `VARCHAR`; `points DOUBLE` podría ser `DECIMAL(6, 2)` (medios puntos históricos).
6. **Declarar fuentes no usadas o retirarlas:** `formula1db.race_entries` está declarada y no se usa; 18 tablas de F1DB y 3 de Ergast están en bronze sin source. O se modelan (chasis, inscripciones) o se documenta que se conservan solo por completitud.

7.8.2 Gobierno del modelo (esfuerzo medio)

1. **Contratos de modelo dbt** (config: `contract: {enforced: true}`) en los 17 modelos gold, con `data_type` y `constraints` (`primary_key`, `not_null`). Gold es la interfaz pública de la API y del zip de Parquet: un contrato impide que un cambio de tipo o el borrado de una columna llegue a producción sin que falle el build. Complementar con **versiones de modelo** (`versions:`) si se rompe la interfaz.
2. **Freshness de sources:** `loaded_at_field` a partir de `_metadata.json` (o una columna `_loaded_at` que la ingesta añada al Parquet) y `freshness: warn_after: 8 days` para F1DB y FastF1; así el pipeline avisa si la ingesta lleva una semana sin datos nuevos.
3. **Nomenclatura según la guía de estilo de dbt Labs:**
 - Separar los YAML por carpeta y tipo (`_f1db__models.yaml`, `_fastf1__sources.yaml`...): hoy todas las sources están en `staging/f1db/_f1db__sources.yaml`.
 - Prefijo por fuente también en los intermediate (`int_formula1db__laps_aligned`, `int_laps__merged`) y verbo en el nombre (`int_race_data__corrected`).
 - Plural consistente en marts (`fact_pit_stops` frente a `fact_race_result`); dbt Labs recomienda nombres de entidad en plural (`fct_race_results`, `dim_drivers`) o, si se mantiene la convención del TFG, aplicarla sin excepciones.
 - Mover los unit tests de `_quality__models.yaml` al YAML del modelo que prueban.
4. **Exposures** (`exposures:`) para la API, la web y el zip de Parquet: el DAG de dbt docs mostraría qué endpoints dependen de cada tabla y el análisis de impacto sería automático.
5. **Tests de rango y consistencia adicionales** (paquete `dbt_utils/dbt_expectations`): `lap_time_ms` entre 30 s y 3 h salvo bandera roja; `sector_1+2+3 ≈ lap_time_ms (±5 ms)`; `points ≥ 0`; `gap_to_pole_pct ≥ 0`; recuento de vueltas por carrera `≤ dim_race.laps + 1`; monotonía de `race_time_ms`; `expression_is_true` para `is_podium → position_number ≤ 3`. Pasar `assert_standings_points_match_results` a error si lleva tiempo sin avisos.

6. **Seed de mapeos explícita** para códigos de neumático de formula1db (H, SS, US...) y códigos de Track Status, hoy embebidos en CASE dentro del staging: una seed documentada es más fácil de revisar y de testear (accepted_values sobre el compuesto).

7.8.3 Evolución del modelo dimensional (esfuerzo medio-alto)

1. **Dimensión de sesión (dim_session)**: hoy session_type es un atributo degenerado con vocabularios distintos por hecho (RACE/SPRINT, QUALIFYING/SPRINT_QUALIFYING) y la telemetría/vueltas no indican sesión. Una dimensión (race_id, session_type) con fecha y hora de sesión, formato (Q1-Q3, sesión única, sprint shootout) y estado permitiría unir hechos de distintas sesiones y preparar la ingesta de libres y sprint de FastF1.
2. **Dimensión de fecha/temporada (dim_date / dim_season)**: dim_race hace de dimensión temporal de grano carrera, pero no hay atributos de temporada (reglamento de puntos, n° de carreras, era técnica, campeón) ni calendario. dim_season (77 filas en F1DB season) facilitaría análisis por eras y normalizar puntos entre sistemas de puntuación.
3. **Dimensión de estado/resultado (dim_result_status)** a partir de position_text y reason_retired: agrupar DNF mecánico/accidente/DSQ permite análisis de fiabilidad sin expresiones regulares en la web. Revisa la decisión del TFG de mantenerlos degenerados, que tenía sentido en Power BI (tamaño del modelo) pero no en DuckDB.
4. **SCD tipo 2 para pertenencia a equipo y datos cambiantes**: la afiliación piloto-equipo ya está en los hechos (correcto), pero atributos como el nombre comercial del constructor (constructor_chronology de F1DB), el motor por temporada o los datos del piloto cambian con el tiempo y hoy se sobrescriben. Opciones: dbt_snapshot (estrategia check) sobre las dimensiones para conservar el historial entre releases de F1DB, o construir SCD2 a partir de las tablas season_entrant_* y constructor_chronology que ya están en bronze.
5. **Claves sustitutas (dbt_utils.generate_surrogate_key)** para los hechos con clave compuesta (race_result_sk, lap_sk...) y, si se adopta SCD2, para las dimensiones (driver_sk + valid_from/valid_to). Mejora los joins en herramientas BI (Power BI necesita relaciones de una columna) y desacopla el modelo de los slugs de F1DB, que pueden cambiar.
6. **Hecho de puente / factless para coches compartidos (bridge_shared_car)**: hoy se resuelven con is_shared_car y count(distinct ...); una tabla puente coche-pilotos haría explícita la semántica.
7. **Hecho de clasificación final de temporada (fact_season_driver_standing, fact_season_constructor_standing)**: los agregados leen stg_f1db__season_*_standings directamente desde staging, lo que rompe la regla de dbt de que marts dependan de intermediate/marts y deja esos datos fuera de gold (la API no puede exponerlos sin pasar por agg_driver_season).

7.8.4 Rendimiento y operación

1. **Materialización incremental** de int_laptimes, fact_laptimes, fact_quali_telemetry y stg_fastf1_* (estrategia delete+insert por race_id): el histórico 1950–2024 de formula1db y Ergast no cambia; solo cambian las carreras nuevas de FastF1 y las correcciones. Con is_incremental() y un --full-refresh periódico (o cuando cambien seeds), el build semanal dejaría de reprocesar 1,27 M de vueltas. Hoy el build completo tarda ≈22 s, así que el beneficio es más de escalabilidad (p. ej. si se incorporan libres o telemetría de carrera) que inmediato.
2. **Particionado en la exportación**: gold-parquet.zip contiene un fichero por tabla; para fact_laptimes y fact_quali_telemetry convendría PARTITION_BY (season) (Hive), lo que permite a Power BI/DuckDB/Polars leer solo las temporadas necesarias y hacer cargas incrementales. Aplicar lo mismo en bronze a FastF1 ya se hace (temporada/ronda).

3. **Metadatos de carga en bronze:** añadir `_loaded_at`, `_source_file` y `_source_version` como columnas en cada Parquet (hoy solo en `_metadata.json`, y FastF1 no tiene), para trazabilidad fila a fila y para la *freshness* de dbt.
4. **Snapshot incremental de bronze:** `bronze.tar.gz` (51,7 MB) se reconstruye y sube entero cada semana; separar las fuentes estáticas (formula1db, Ergast: nunca cambian) de las dinámicas reduciría transferencia y riesgo.
5. **Validación del esquema de FastF1** en la ingesta: `union_by_name = true` tolera cambios de columnas entre versiones de la biblioteca sin avisar; un test de columnas esperadas (`dbt_expectations.expect_table_columns_to_contain_set`) sobre las sources detectaría cambios de la API.
6. **Semantic layer / métricas dbt:** las métricas de `agg_*` (victorias, podios, % por delante) podrían declararse como `metrics` de MetricFlow sobre los hechos para servirlos con distintos cortes (por temporada, circuito, equipo) sin multiplicar tablas agregadas; el proyecto ya genera `semantic_manifest.json` vacío.

7.8.5 Resumen de prioridades

Prioridad	Propuestas
Alta (rápidas, reducen riesgo)	1, 2, 3, 5, 7
Media (gobierno y documentación)	4, 8, 9, 10, 11, 12, 19, 22
Evolución funcional	13, 14, 15, 16, 17, 18, 25
Escalabilidad	20, 21, 23, 24

CAPÍTULO 8

Revisión de código e infraestructura

Revisión de solo lectura del repositorio `f1-data-platform` a 29/09/2026 (commit `cceddd3`, rama `main`). No se ha modificado ningún fichero salvo esta nota.

8.1 Método y estado de partida

Qué se ha hecho:

- Lectura completa de `ingestion/`, `api/`, `scripts/`, `tests/`, `api/tests/`, los workflows, `render.yaml`, `api/Dockerfile`, los modelos dbt intermedios y *marts*, macros, seeds y YAML, y los ficheros clave de `web/` (cliente de la API, `proxy.ts`, layout, páginas de carrera, gráficos).
- Ejecución de linters y pruebas:
 - `ruff check .` y `ruff format --check .`: sin errores (ruff 0.16.9, 50 ficheros).
 - `pytest`: **40 pruebas superadas en 2,3 s**.
 - `npm run lint` (ESLint) y `tsc --noEmit`: sin errores.
 - `npm audit --omit=dev`: **0 vulnerabilidades**. `npm outdated`: eslint 9→10, typescript 5.9→7.0, react/react-dom 19.2.8→19.3.0, @types/node 20→26.
- Medición de los endpoints con `TestClient` contra `dist/f1.duckdb` (la base de datos que se publica), 5 repeticiones, mediana (ver §4).
- Consultas de integridad contra `data/gold/f1.duckdb` (solo lectura): unicidad de claves, cruces por dorsal, estados de numeración y relaciones huérfanas.
- Tiempo de `dbt build` según `transform/target/run_results.json`: **21,7 s** en total (`int_laptimes` 6,7 s, `int_formula1db_laps` 4,2 s, `fact_laptimes` 2,6 s).

Puntos fuertes que conviene conservar: consultas siempre parametrizadas (las pocas interpolaciones usan listas cerradas); escritura atómica de Parquet; `tarfile` con `filter="data"`; descarga verificada por SHA-256 y versionada por nombre; ETag + Cache-Control; GZip; CORS restringido a GET; usuario no root y HEALTHCHECK en la imagen; contrato OpenAPI comprobado en la CI (`git diff --exit-code`); pruebas de accesibilidad con axe; seeds de correcciones que se desactivan solas cuando F1DB corrige el dato.

Leyenda: Esfuerzo S (< 2 h), M (medio día a 2 días), L (> 2 días). Prioridad alta, media o baja.

8.2 Hallazgos por área

8.2.1 Ingesta (ingestion/)

ING-1. Extracción de F1DB no atómica. `ingestion/f1db_loader.py`:42-59. Si `zf.extract("f1db.db", ...)` se interrumpe, queda un `f1db.db` truncado y en la siguiente ejecución `db_path.exists()` (línea 45) lo da por bueno para siempre (el directorio lleva el tag de la release). *Impacto*: bronze construido

sobre una SQLite corrupta; el error aparece más tarde y es críptico. *Propuesta:* extraer a un temporal y renombrar; comprobar la integridad.

```
tmp = target_dir / "f1db.db.part"
with zipfile.ZipFile(zip_path) as zf, zf.open("f1db.db") as src, open(tmp, "wb") as dst:
    shutil.copyfileobj(src, dst, 1 << 20)
with sqlite3.connect(tmp) as con:
    assert con.execute("pragma quick_check").fetchone()[0] == "ok"
os.replace(tmp, db_path)
```

Esfuerzo S · Prioridad media.

ING-2. Descargas sin reintentos. ingestion/f1db_loader.py:25-39,51-55. requests.get sin Retry; un 502 de GitHub hace fallar el pipeline entero. next(a for a in release["assets"] ...) (línea 34) lanza StopIteration si falta el asset, con un mensaje ininteligible. *Propuesta:* requests.Session con HTTPAdapter(max_retries=Retry(total=5, backoff_factor=2, status_forcelist=[429, 500, 502, 503, 504])) y next(..., None) con error explícito. Esfuerzo S · Prioridad media.

ING-3. Sustitución del directorio bronze de F1DB no atómica. ingestion/f1db_loader.py:84-86. rmtree(out_dir) y después rename(tmp_dir): entre ambos no hay directorio, y en Windows el rename falla si algo tiene abierto un Parquet (DuckDB, Power BI). *Propuesta:* out_dir → out_dir.old, tmp_dir → out_dir, borrar .old; si el segundo paso falla, restaurar. Esfuerzo S · Prioridad baja.

ING-4. Carreras de FastF1 «congeladas» tras la primera carga. ingestion/fastf1_loader.py:153-157. Una carrera se da por cargada si existe su Parquet. Con 6 h de margen (línea 121) FastF1 a veces publica datos incompletos que no se vuelven a pedir salvo con --force. Además _load_race escribe laps y results, pero solo se comprueba laps: si falta results nunca se regenera. *Propuesta:* recargar siempre las dos últimas carreras disputadas (o las que tengan menos vueltas que el resultado oficial de F1DB) y añadir results a kinds. Esfuerzo S · Prioridad media.

ING-5. Errores sin traza. ingestion/fastf1_loader.py:170-172 usa log.error("...%s", exc); usar log.exception para conservar la traza en los logs de Actions. Esfuerzo S · Prioridad baja.

ING-6. Desfase entre FastF1 y F1DB los lunes. pipeline.yml:181 y transform/models/marts/core/dim_race.sql:10-14. El pipeline corre una sola vez (lunes 06:00 UTC). Si F1DB aún no ha publicado la release con el GP del domingo, entran las vueltas de FastF1 (fastf1_fill) pero la carrera queda is_completed = false y la web la muestra como «próxima» durante una semana. *Propuesta:* un segundo cron ligero (p. ej. diario) que solo compare el tag de F1DB y el calendario de FastF1 y lance el pipeline si hay algo nuevo; o repetir el martes y el miércoles. Esfuerzo S · Prioridad media.

ING-7. Rutas interpoladas en SQL. ingestion/f1db_loader.py:70, ingestion/snapshot.py:75, 114, scripts/make_api_fixture.py:38, api/tests/conftest.py:22. Las rutas se meten entre comillas simples: una ruta con ' rompe la sentencia (no es un riesgo de seguridad, las rutas son locales). *Propuesta:* un helper sql_literal(path) = "'" + path.as_posix().replace("'", "'") + "'. Esfuerzo S · Prioridad baja.

ING-8. Restos al fallar la exportación. ingestion/snapshot.py:100-122: si falla un COPY, queda el directorio .d a medias. Usar tempfile.TemporaryDirectory(dir=out.parent). Esfuerzo S · Prioridad baja.

ING-9. Rutas fijas al equipo del autor. ingestion/config.py:20 y ingestion/ergast_loader.py:20 apuntan a PROJECT_ROOT.parent / Está documentado, pero convendría leerlas de variables de entorno (F1_LEGACY_CSV_DIR, F1_ERGAST_ZIP). Esfuerzo S · Prioridad baja.

8.2.2 dbt y modelo de datos (transform/)

DBT-1. Grano de las vueltas y consumidores que lo ignoran. Datos + api/app/routers/races.py:99-156. fact_laptimes tiene 85 filas repetidas en (race_id, driver_id, lap_number): pilotos que condujeron dos coches en la misma carrera (Harry Schell 1955, Bob Scott y Jimmy Davies en Indianápolis 1954, Stirling Moss 1961...). El test de unicidad incluye driver_number, así que el modelo es coherente, pero:

- Lap y Stint (api/app/schemas.py:163-192) no devuelven driver_number, así que el cliente no puede separar los dos coches;

- `/stints` particiona la ventana por `driver_id` (`rices.py:139,143`): mezcla las vueltas de los dos coches y calcula tramos erróneos;
- la web agrupa por `driver_id` (`web/src/app/[lang]/rices/[id]/lap-chart/page.tsx:28,37`) y la última fila pisa a la primera. *Propuesta:* añadir `driver_number` a `Lap` y `Stint`, particionar `/stints` por (`driver_id`, `driver_number`) y agrupar en la web por esa pareja. Esfuerzo S-M · Prioridad media.

DBT-2. Dorsales tomados de todas las sesiones. `transform/models/intermediate/int_driver_race_numbers.sql:3-12`. El modelo usa `race_data` completo, incluidos los libres. Resultado: **294 parejas (carrera, dorsal) con dos pilotos**, casi todas de 2010-2013 (pilotos de pruebas del viernes con el dorsal del titular; p. ej. carrera 897, #23: Chilton y Rodolfo González). Hoy no causa daño porque `FastF1` empieza en 2018 y `ergast_fill` filtra por el resultado oficial, pero `int_fastf1_laps.sql:35-37` y `fact_quali_telemetry.sql:12-14` cruzan solo por (`race_id`, `driver_number`): si vuelve a pasar en la era `FastF1`, las vueltas se duplican sin aviso. *Propuesta:* limitar a `RACE_RESULT`, `QUALIFYING_RESULT` y `SPRINT_*`, o priorizar con `qualify row_number() over (partition by race_id, driver_number order by <sesión de carrera primero>) = 1` en una vista para `FastF1`; añadir `unique_combination [race_id, driver_number, lap_number]` a `int_fastf1_laps` y [`race_id`, `driver_id`, `distance_m`] a `fact_quali_telemetry`. Esfuerzo S · Prioridad media.

DBT-3. Empates no deterministas y estado consistent demasiado laxo. `transform/models/intermediate/int_lap_numbering.sql:402-405,419,441,480`. `arg_min/arg_max` no desempatan (un desfase de +1 y otro de -1 con la misma distancia, o dos desfases con el mismo número de casos), así que la salida puede variar entre ejecuciones. Además `consistent` solo exige `distinct_offsets = 1` and `common_offset = 0`, sin `min_share >= 0.8`: hay **2 carreras marcadas consistent con una cuota dominante < 0,8**. Solo afecta a la etiqueta (`applied_lap_shift` sí exige 0,8), pero la etiqueta se publica en `fact_laptimes`. *Propuesta:* `arg_max(lap_offset, (cases, -abs(lap_offset)))` y exigir `min_share >= 0.8` en `consistent` (lo que no llegue, `inconsistent`). Esfuerzo S · Prioridad baja-media.

DBT-4. Corrección de tiempos acumulados por dorsal. `int_laptimes.sql:87-91`. La suma acumulada va por (`race_id`, `driver_number`); en coches compartidos mezcla pilotos. Hoy solo hay una carrera con corrección (2024), pero la regla es general. Particionar por (`race_id`, `driver_id`, `driver_number`). Esfuerzo S · Prioridad baja.

DBT-5. Vueltas sin tiempo en formula1db que las otras dos fuentes sí tienen. `int_laptimes.sql:139-143,158-162`. Si `lap_time_ms` es nulo y `FastF1` y `Ergast` coinciden, la mayoría no se aplica (`abs(null)`) y la vuelta queda sin tiempo y `single_source`. Añadir la rama «rellenada por mayoría». Esfuerzo S · Prioridad baja.

DBT-6. Equipo principal no determinista. `transform/models/marts/metrics/agg_driver_season.sql:20`. `mode(constructor_id)` no desempata cuando un piloto corre el mismo número de carreras con dos equipos. Usar `arg_max(constructor_id, (carreras, última_ronda))`. Esfuerzo S · Prioridad baja.

DBT-7. Definiciones de récords duplicadas entre dbt y la API. `api/app/routers/rankings.py:39-77,93-133` frente a `agg_driver_career.sql` y `agg_constructor_career.sql`. Hoy coinciden (comprobado: los podios por coche dan lo mismo con `race_id || driver_number` y con (`race_id`, `position_number`)), pero hay dos implementaciones de las mismas reglas (inscripciones, salidas, podios por coche, vueltas rápidas por piloto). *Propuesta:* modelos `agg_driver_season_stats` y `agg_constructor_season_stats` (una fila por entidad y temporada, con las mismas definiciones), y que `/rankings` solo haga `sum(...)` where `season between ? and ?`. Los agregados de carrera deportiva serían la suma sin filtro. Una definición, consultas más simples y más rápidas. Esfuerzo M · Prioridad media.

DBT-8. Cobertura de pruebas de dbt desigual.

- Los 10 modelos intermedios no tienen YAML (ni descripciones ni tests); staging tampoco.
- Solo hay 2 `unit_tests` (`int_formula1db_laps`, `int_formula1db_race_alignment`, en `models/quality/_quality__models.yml:33`). La lógica más delicada no tiene pruebas unitarias: la mayoría y el recálculo de posiciones de `int_laptimes`, la regla A6, `int_lap_numbering`, `int_tyre_compound_mapping` y la macro `parse_duration_ms`.

- Faltan tests de grano en `fact_pit_stops`, `fact_quali_telemetry`, `fact_driver_standing` y `fact_constructor_standing`.
- Las *sources* no tienen *freshness* (se podría avisar si F1DB lleva más de N días sin actualizarse durante la temporada). *Propuesta*: un `unit_test` de 4-6 filas por regla (el formato ya se usa en el proyecto) y `unique_combination` en los hechos que faltan. Esfuerzo M · Prioridad **alta** (es la parte con más lógica y menos red).

DBT-9. Relación huérfana en el almacén local. `silver.int_formula1db_laps_validated` (1,2 M filas) existe en `data/gold/f1.duckdb` pero no corresponde a ningún modelo: es un resto de una versión anterior. No llega a la API (`build_api_database` copia solo `gold` y `qa_summary`), pero ocupa espacio y confunde. Borrarla y añadir un script que liste relaciones sin nodo dbt. Esfuerzo S · Prioridad baja.

DBT-10. Materializaciones y almacenamiento. El build tarda 22 s, así que los modelos incrementales no compensan. `int_laptimes` y `fact_laptimes` duplican 1,27 M filas; `int_laptimes` podría ser `view/ephemeral` (o `fact_laptimes` absorberla). Ordenar físicamente `fact_laptimes` por `race_id` apenas mejora (medido: 10,7 → 9,7 ms en la consulta de `/laps`), así que no es prioritario. Esfuerzo S · Prioridad baja.

DBT-11. Tipos. `dim_race.race_time_utc` es `VARCHAR` y nulo en muchas carreras antiguas; exponer un `race_start_utc` `TIMESTAMP` calculado facilitaría calendarios y zonas horarias en la web. Esfuerzo S · Prioridad baja.

DBT-12. Estilo SQL sin comprobar. No hay `sqlfluff` (u otro linter SQL) ni en pre-commit ni en la CI; con 60 modelos convendría fijar el estilo (dialecto `duckdb`, plantilla `dbt`). Esfuerzo S · Prioridad baja.

8.2.3 API (api/)

API-1. Arranque en frío frágil y lento en Render. `api/app/main.py:56-62`, `api/app/release.py:93-118`, `render.yaml:8`, `api/Dockerfile:24-29`. El plan gratuito duerme la API a los 15 min y su disco es efímero: cada arranque descarga de nuevo 54 MB y calcula su SHA-256 antes de servir la primera petición. El README habla de unos 30 s, pero la descarga los alarga. Si GitHub falla justo entonces, `resolve_database` lanza una excepción y la API no arranca (bucle de reinicios). *Propuesta*: (a) si falla la descarga y hay alguna copia en `data_dir`, arrancar con ella y reintentar en segundo plano; (b) disco persistente o plan de pago cuando haya tráfico real; (c) en la web, `timeout` y reintento (WEB-2) para no depender del arranque. Esfuerzo M · Prioridad alta.

API-2. Descarga de la release sin reintentos. `api/app/release.py:38-50,104-116`. No hay reintentos ni *backoff*. El temporal tiene nombre fijo (`f1-<sha>.download`): dos procesos (varios *workers* de *uvicorn*) escribirían el mismo fichero. La descarga no es reanudable. *Propuesta*: 3 intentos con espera exponencial y `tempfile.NamedTemporaryFile(dir=data_dir, delete=False)`. Esfuerzo S · Prioridad media.

API-3. Publicación no atómica de la release. `.github/workflows/pipeline.yml:267-268`. `gh release upload --clobber` borra y vuelve a subir cada asset. Mientras tanto la API puede leer el manifiesto nuevo con la base antigua (se detecta por el SHA y se reintenta 6 h después) o no encontrar el asset (404). *Propuesta*: subir `manifest.json` en un último comando aparte y, mejor aún, releases inmutables por versión (`data-2026-09-28-<sha>`) con `data-latest` apuntando a la última (ver CI-6). Esfuerzo S-M · Prioridad alta.

API-4. Cierre de la conexión antigua por tiempo, no por uso. `api/app/database.py:76-94`, `api/app/main.py:44-47`. Tras el cambio de versión, la conexión anterior se cierra a los 120 s aunque haya consultas en curso; una consulta que llegue tarde daría `Connection already closed (500)`. Hoy es improbable (todas las consultas tardan menos de 60 ms), pero la garantía no existe. *Propuesta*: contador de uso por conexión (un *context manager* que incremente y decremente) y cerrar cuando llegue a 0. Esfuerzo S · Prioridad baja.

API-5. ETag calculada antes de la consulta. `api/app/cache.py:32-40`. Si el cambio de versión cae entre el cálculo y la consulta, una respuesta con datos nuevos sale con la ETag antigua y se cachea (600 s + `stale-while-revalidate` de 1 día). Guardar la versión en `request.state` y usar esa misma conexión en la consulta. Esfuerzo S · Prioridad baja.

API-6. Validación de parámetros mejorable. Comprobado con peticiones reales:

- LIKE sin escapar: `/drivers?search=%25%25` y `search=__` devuelven todos los pilotos (`drivers.py:40`, `constructors.py:29`). Usar `contains(strip_accents(lower(c.name)), strip_accents(lower(?)))`.
- `season_from > season_to` devuelve `[]` en vez de 422 (`rankings.py:33-34`, `circuits.py:14-15`).
- `?drivers=,,,` produce una lista vacía y responde `[]` (`deps.py:21-28`): devolver `None` o 422.
- Límites incoherentes: `/drivers` admite `limit ≤ 1000`, `/constructors ≤ 500`.
- Temporadas sin rango (`seasons.py:65`): añadir `Path(ge=1950, le=2100)`. Esfuerzo S · Prioridad media-baja.

API-7. ORDER BY interpolado y rango sin empates. `api/app/routers/rankings.py:74,79,130,135`. `order by {order_by}` es seguro porque `Literal` valida el valor, pero depende de esa validación; mapear a columnas con un diccionario, como hace `records.py:13-32`. `rank` es el índice de la fila, así que no respeta empates, a diferencia de `/records`, que usa `rank()`. Esfuerzo S · Prioridad baja.

API-8. Rendimiento y tamaño de las respuestas. Ver §4. Todo responde en menos de 60 ms en local, pero:

- `/races/{id}/laps` pesa 455 KB en Baréin 2024 (1 129 filas) y **1,9 MB en Indianápolis 1954** (4 828 filas); con gzip, 27 KB y 19 KB.
- La mayor parte del tiempo es validación y serialización de Pydantic (la consulta de `/laps` tarda unos 10 ms y el endpoint 34 ms). *Propuesta:* formato columnar opcional (`?format=columns`, listas paralelas como ya hace `/telemetry`) y selección de columnas (`fields=`); `ORJSONResponse` para listas grandes. Esfuerzo M · Prioridad baja.

API-9. BaseHTTPMiddleware. `api/app/cache.py:22`. Añade coste por petición y tiene limitaciones conocidas (tareas en segundo plano, *streaming*). Reescribirlo como middleware ASGI puro (unas 30 líneas). Esfuerzo S · Prioridad baja.

API-10. Sin límite de peticiones ni cabeceras de seguridad. `api/app/main.py:90-98`, `api/Dockerfile:36`. La API es pública; en Render free (0,1 CPU) una ráfaga de `/races/{id}/laps` la satura. No envía `X-Content-Type-Options: nosniff` ni `Referrer-Policy`. `--forwarded-allow-ips='*'` hace que `X-Forwarded-For` sea falsificable: hay que tenerlo en cuenta antes de limitar por IP. *Propuesta:* `slowapi` (p. ej. 120 peticiones/min por IP) tomando la IP de la cabecera que fija Render, o un CDN delante (Cloudflare) aprovechando que las respuestas ya son cacheables; cabeceras básicas en un middleware. Esfuerzo S-M · Prioridad media.

API-11. Observabilidad mínima. `api/app/main.py:49-50,136`. Solo hay `logging.basicConfig` y el log de accesos de `uvicorn`. Si el refresco falla una y otra vez, solo queda un `log.exception` que nadie ve. `/health` (`main.py:102-115`) no toca la base de datos. *Propuesta:* `/health` con `select 1, last_refresh_at` y `last_refresh_error`; logs en JSON con id de petición y latencia; monitor externo (UptimeRobot o un workflow programado que haga `curl /health` y abra un *issue* si falla). Esfuerzo S-M · Prioridad media.

API-12. Configuración sin validar. `api/app/config.py:38,42`: un valor no numérico en `F1_API_REFRESH_HOURS` rompe el arranque con un `ValueError` poco claro. `pydantic-settings` valida y documenta. Esfuerzo S · Prioridad baja.

API-13. Detalles menores. `database.py:96-98`: `query_one` hace `fetchall`; `races.py:225-226` reordena en Python lo que puede ordenar SQL (`order by list_position(?, driver_id)`); versión fija "1.0.0" en `main.py:80` (tomarla de `importlib.metadata`). Esfuerzo S · Prioridad baja.

Seguridad: **no se han encontrado inyecciones SQL**. Todas las entradas del usuario van como parámetros `?`; las interpolaciones (`rankings.py`, `records.py`, `seasons.py:186-205`) usan valores de listas cerradas validadas por FastAPI.

8.2.4 Web(web/)

WEB-1. Sin cabeceras de seguridad. `web/next.config.ts:1-7` está vacío. No hay CSP, X-Content-Type-Options, Referrer-Policy, Permissions-Policy ni frame-ancestors, y se envía X-Powered-By: Next.js. Una CSP con *nonce* obliga a renderizar en dinámico y rompe el ISR, así que conviene una CSP estática.

```
const csp = [
  "default-src 'self'", "script-src 'self' 'unsafe-inline'", "style-src 'self'
  ↪ 'unsafe-inline'",
  "img-src 'self' data:", "font-src 'self'", "connect-src 'self'",
  "frame-ancestors 'none'", "object-src 'none'", "base-uri 'self'", "form-action 'self'",
].join("; ");
const nextConfig: NextConfig = {
  poweredByHeader: false,
  async headers() {
    return [{ source: "/*", headers: [
      { key: "Content-Security-Policy", value: csp },
      { key: "X-Content-Type-Options", value: "nosniff" },
      { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
      { key: "Permissions-Policy", value: "camera=(), microphone=(), geolocation=()" },
    ]}];
  },
};
```

Esfuerzo S · Prioridad alta (mucho valor por poco trabajo).

WEB-2. `fetch` sin timeout ni reintento. `web/src/lib/api/client.ts:33-38`. Con la API dormida, la función de servidor espera hasta agotar su límite de duración y acaba en `error.tsx`. *Propuesta:* `signal: AbortSignal.timeout(20_000)` y un reintento con espera corta para 502/503/504 y errores de red. Esfuerzo S · Prioridad alta.

WEB-3. Dirección de la API por defecto en producción. `client.ts:7`. Si falta `F1_API_URL` en Vercel, la web intenta `127.0.0.1:8000` y falla en silencio. Lanzar un error en el arranque si `process.env.VERCEL_ENV === "production"` y no está definida. Esfuerzo S · Prioridad baja.

WEB-4. Códigos de piloto ambiguos. `web/src/lib/format.ts:40-45` deduce el código de las tres primeras letras del apellido del identificador. En la misma carrera, «michael-schumacher» y «ralf-schumacher» dan los dos SCH (1997-2006), y «nico-rosberg» y «pedro-de-la-rosa» los dos ROS (2010-2012). Afecta al gráfico de posiciones y a la tabla de ritmo (`lap-chart/page.tsx:42`, `pace/page.tsx:135,190`). *Propuesta:* exponer `abbreviation` de `dim_driver` (F1DB tiene MSC/RSC) en `RaceResult` y usarla, con la heurística solo como último recurso. Esfuerzo S · Prioridad media.

WEB-5. Tamaño del bundle. `.next/static` ocupa 1,6 MB; el chunk mayor, 663 KB (220 KB con gzip), es ECharts (núcleo, 4 tipos de gráfico, 6 componentes y el renderizador SVG, `web/src/components/charts/echart.tsx:3-28`) y se carga en 10 páginas. *Propuesta:* medir con el analizador de bundles; cargar EChart con `next/dynamic` cuando entre en pantalla (la tabla alternativa ya da el contenido sin JavaScript); registrar cada tipo de gráfico solo en el componente que lo usa. Esfuerzo M · Prioridad media.

WEB-6. El gráfico se recrea en cada cambio. `web/src/components/charts/echart.tsx:133-144`. Cada cambio de build (p. ej. marcar una casilla en `LapTimesChart`) hace `dispose()` + `init()`: se pierde el zoom y se repite la animación. Mantener la instancia en un ref y llamar a `chart.setOption(option, { notMerge: true })`. Esfuerzo S · Prioridad baja.

WEB-7. Lógica en las páginas y sin pruebas unitarias. Las funciones estadísticas `density` y `quantile` están en `web/src/app/[lang]/races/[id]/pace/page.tsx:11-30`; `format.ts` y `proxy.ts:6-17` (negociación de idioma; `Number(q)` puede dar NaN y desordenar la lista) tampoco tienen pruebas. Solo hay 8 pruebas Playwright (navegación y axe). *Propuesta:* mover las funciones a `lib/stats.ts` y añadir Vitest para las funciones puras. Esfuerzo S-M · Prioridad media.

WEB-8. Dependencias. `@types/node ^20` mientras la CI usa Node 24 (`ci.yml:125`): alinear a ^24 y declarar engines. Planificar eslint 10 y TypeScript 7 (cambio mayor). Esfuerzo S · Prioridad baja.

Accesibilidad: no se han visto problemas claros. Hay enlace para saltar al contenido, `role="img"` con tabla alternativa en cada gráfico, paleta Okabe-Ito con tipo de línea y marcador, `prefers-reduced-`

motion, fechas con T12:00:00Z para evitar el día anterior por zona horaria, y pruebas axe en claro y oscuro.

8.2.5 CI/CD e infraestructura

CI-1. Acciones sin fijar a SHA. ci.yml:23-24,67,123,168, pipeline.yml:210,212,284,290,311. Todas van por etiqueta mayor (@v5, @v6, @v4...). El pipeline tiene contents: write: una acción comprometida podría reescribir las releases de datos. *Propuesta:* fijar a SHA con comentario de versión (uses: actions/checkout@<sha> # v5.0.0) y dejar que Dependabot las actualice. Esfuerzo S · Prioridad alta.

CI-2. Acciones en Node 20. actions/upload-artifact@v4 (ci.yml:67,168, pipeline.yml:290), actions/upload-pages-artifact@v3 (usa upload-artifact@v4 por dentro, pipeline.yml:284), actions/deploy-pages@v4 (pipeline.yml:311) y, según su versión, astral-sh/setup-uv@v6 usan Node 20. GitHub lo ha retirado como runtime por defecto en 2026: primero avisos y después fallos. Subir a las mayores con Node 24 (hay que comprobar la última en cada repositorio). checkout@v5 y setup-node@v5 ya usan Node 24. Esfuerzo S · Prioridad alta.

CI-3. Sin Dependabot ni Renovate. .github/ solo contiene workflows/.

```
# .github/dependabot.yml
version: 2
updates:
  - { package-ecosystem: github-actions, directory: "/", schedule: { interval: weekly } }
  - { package-ecosystem: uv, directory: "/", schedule: { interval: weekly } }
  - { package-ecosystem: npm, directory: "/web", schedule: { interval: weekly },
      groups: { minor: { update-types: [minor, patch] } } }
  - { package-ecosystem: docker, directory: "/api", schedule: { interval: monthly } }
```

Esfuerzo S · Prioridad alta.

CI-4. Permisos de escritura en todo el job del pipeline. pipeline.yml:193-194. El job que ejecuta código de terceros (FastF1, pandas, dbt) tiene contents: write. *Propuesta:* dividir en build (solo lectura, sube dist/ como artefacto) y publish (contents: write, solo gh release). Esfuerzo M · Prioridad media.

CI-5. Se publica sin probar el snapshot. pipeline.yml:255-269. dbt build valida los datos, pero la API no se arranca contra el dist/f1.duckdb nuevo antes de publicarlo; la CI sí lo hace, pero sobre lo que ya está publicado. *Propuesta:* antes de gh release upload, F1_API_DB_PATH=dist/f1.duckdb pytest api/tests/smoke con unas cuantas pruebas de contrato (temporadas, última carrera, /races/{id}/laps no vacío, /quality sin FAIL). Esfuerzo S · Prioridad alta.

CI-6. Sin copias de seguridad de los datos. pipeline.yml:267-268 y ingestion/snapshot.py:41-53. data-latest se sobrescribe cada semana (--clobber) y bronze.tar.gz es la **única copia en la nube** de formula1db.com y Ergast, datos que no se pueden volver a obtener. pack_bronze exige que existan (bien), pero un snapshot publicado con datos malos no tiene marcha atrás. *Propuesta:* (1) una release inmutable bronze-static-v1 con las fuentes estáticas, que no cambian nunca; (2) releases fechadas con retención de las 8 últimas (un paso que borre las antiguas); (3) documentar la restauración y la vuelta atrás (DOC-2). Esfuerzo S-M · Prioridad alta.

CI-7. pre-commit desfasado. .pre-commit-config.yml:3,11. ruff-pre-commit v0.8.4 frente a ruff 0.16.9 en uv.lock (lo que usa la CI), y pre-commit-hooks v5.0.0. El formateo local y el de la CI pueden diferir. pre-commit autoupdate, o un hook local uv run ruff. Esfuerzo S · Prioridad media.

CI-8. Imagen Docker. api/Dockerfile:5,7,36. python:3.12-slim y ghcr.io/astral-sh/uv:0.12 sin digest; el binario de uv (unos 35 MB) se queda en la imagen final; --forwarded-allow-ips='*' (ver API-10); no se analiza la imagen en la CI. *Propuesta:* construcción en dos etapas (builder con uv y --mount=type=cache, final solo con .venv y código), imágenes fijadas por digest (Dependabot las mantiene) y Trivy en el job api-image. Ya está bien: usuario no root, HEALTHCHECK, capa de dependencias reutilizable. Esfuerzo S-M · Prioridad media.

CI-9. Huecos de la CI. Sin cobertura (pytest-cov) ni comprobación de tipos en Python (mypy o pyright); Playwright instala Chromium en cada ejecución sin caché (ci.yml:163); el job dbt descarga 52 MB de bronze en cada PR (cachear por el SHA del manifiesto); los cambios que solo tocan docs/ lanzan todos los jobs (paths-ignore). Esfuerzo S · Prioridad baja.

CI-10. render.yaml. buildFilter (render.yaml:15-19) no incluye ingestion/__init__.py, que el Dockerfile copia (Dockerfile:19). Es trivial, pero conviene alinear los dos o eliminar la copia si no hace falta. Esfuerzo S · Prioridad baja.

8.2.6 Pruebas

Estado: 40 pruebas de Python (tests/ 10, api/tests/ 30) que cubren bien la API, la caché, el CORS, la descarga verificada y el snapshot. Qué **no** está probado:

Componente	Qué falta	Propuesta	Esfuerzo
f1db_loader	export_tables, download_sqlite, load (idempotencia por tag)	SQLite de 2 tablas creada con sqlite3 en tmp_path; latest_release con monkeypatch	S
fastf1_loader. load_season	omitir lo ya cargado, force, rate_limited, failed	monkeypatch de completed_races y load_race	S
ergast_loader, cli.main	parseo y códigos de salida	zip en memoria; main([...])	S
main.refresh_ periodically	cambio de versión, error de red, cierre de la conexión antigua	refresh_hours diminuto y Fake Release (ya existe)	S
Database	consultas concurrentes durante el cambio de versión	ThreadPoolExecutor con consultas y swap a la vez	S
dbt intermedios	reglas de int_laptimes, int_lap_numbering, int_tyre_compound_mapping, parse_duration_ms	unit_tests (DBT-8)	M
Web	funciones puras y componentes	Vitest (WEB-7)	S-M
Pipeline	snapshot antes de publicar	pruebas de humo (CI-5)	S

8.2.7 Documentación

DOC-1. No hay fichero LICENSE. El repositorio es público y redistribuye datos (F1DB con CC BY 4.0, que exige atribución; los datos de formula1db.com con permiso). Añadir LICENSE para el código (MIT o Apache-2.0) y un DATA_LICENSE/NOTICE con las licencias y atribuciones de cada fuente. Esfuerzo S · Prioridad alta.

DOC-2. Falta un runbook de operación. Cómo volver a una versión anterior de los datos, qué hacer si el pipeline falla a mitad, cómo republicar el bronze estático, cómo rotar F1_GITHUB_TOKEN, cómo forzar la recarga en Render. Esfuerzo S · Prioridad media.

DOC-3. README. La cifra de unos 30 s de arranque en frío (README.md:203-205) no cuenta la descarga de 54 MB. Esfuerzo S · Prioridad baja.

DOC-4. Linaje incompleto en dbt docs. Los modelos intermedios solo se documentan con comentarios Jinja; pasarlos a YAML y declarar exposures para la API y la web completaría el catálogo que publica el pipeline. Esfuerzo S · Prioridad baja.

DOC-5. Scripts que dependen del directorio de trabajo. scripts/generate_override_seed.py:19-23 se ejecuta al importarse y exige estar en transform/; export_openapi.py:14 y make_api_fixture.py:16-17, en la raíz. Usar Path(__file__).resolve() y if __name__ == "__main__". Esfuerzo S · Prioridad baja.

8.3 Dependencias

- Python: las dependencias se declaran con cotas mínimas (>=) y se fijan en uv.lock (se instala con --locked), lo cual es correcto. Versiones instaladas: duckdb 1.5.5, fastapi 0.141.1, starlette 1.7.0, pydantic 2.13.5, dbt-core 1.12.5.
- httpx2 (pyproject.toml:33) es legítima: la exige starlette.testclient en Starlette 1.x. Pero su versión se publicó el 23/09/2026: conviene que Dependabot vigile estos paquetes nuevos.

- `requires-python = ">=3.12,<3.13"`: valorar 3.13, porque todas las dependencias lo admiten.
- Web: 0 vulnerabilidades; desactualizadas solo en versiones mayores de herramientas (§1).

8.4 Mediciones de la API (TestClient, `dist/f1.duckdb`, mediana de 5)

Endpoint	Tiempo	Tamaño	Con gzip	Filas
<code>/seasons</code>	8,9 ms	12,6 KB	1,3 KB	77
<code>/seasons/2024/standings/drivers</code>	21,9 ms	4,3 KB	1,0 KB	24
<code>/seasons/2024/standings/progression?top=20</code>	10,9 ms	58,7 KB	3,6 KB	464
<code>/races/1102/laps</code> (Baréin 2024)	34,5 ms	454,6 KB	26,8 KB	1 129
<code>/races/34/laps</code> (Indianápolis 1954)	59,7 ms	1 946,8 KB	19,4 KB	4 828
<code>/races/1102/telemetry</code> (4 pilotos)	22,7 ms	116,1 KB	45,0 KB	4
<code>/races/1102/stints</code>	14,8 ms	7,3 KB	0,8 KB	63
<code>/drivers?limit=1000</code>	16,0 ms	156,3 KB	19,9 KB	917
<code>/drivers/lewis-hamilton/results</code>	10,8 ms	74,4 KB	4,9 KB	424
<code>/rankings/drivers?limit=500</code>	41,6 ms	106,4 KB	14,1 KB	500
<code>/rankings/constructors?limit=500</code>	25,5 ms	35,2 KB	4,2 KB	186
<code>/circuits</code>	11,5 ms	17,6 KB	3,8 KB	78
resto (<code>/races/{id}</code> , <code>/records</code> , <code>/quality...</code>)	4-10 ms	< 10 KB	< 2 KB	—

Conclusión: el rendimiento de DuckDB no es un problema. El cuello de botella real es el arranque en frío (API-1) y, en menor medida, el tamaño de `/laps` sin comprimir y la serialización (API-8).

8.5 Top 15 de mejoras (ordenadas por valor/esfuerzo)

#	Mejora	Ref.	Esfuerzo	Prioridad
1	Dependabot (acciones, uv, npm, docker) y acciones fijadas a SHA	CI-3, CI-1	S	alta
2	Subir las acciones que aún usan Node 20	CI-2	S	alta
3	Cabeceras de seguridad en la web (CSP estática, nosniff, Referrer-Policy) y <code>poweredByHeader: false</code>	WEB-1	S	alta
4	Timeout y reintento en el <code>fetch</code> de la web	WEB-2	S	alta
5	Prueba de humo de la API contra el snapshot nuevo antes de publicarlo	CI-5	S	alta
6	Copia inmutable del bronze estático y releases fechadas con retención; manifiesto subido al final	CI-6, API-3	S-M	alta
7	LICENSE y atribución de las fuentes de datos	DOC-1	S	alta
8	Arranque de la API con la última copia si falla GitHub; reintentos en la descarga	API-1, API-2	M	alta
9	Códigos de piloto desde <code>abbreviation</code> y <code>driver_number</code> en <code>Lap/Stint</code> (corrige SCH/SCH y los stints de coches compartidos)	WEB-4, DBT-1	S-M	media
10	Dorsales solo de sesiones de carrera y tests de grano en los cruces por dorsal	DBT-2, DBT-8	S	media
11	<code>unit_tests</code> de dbt para <code>int_laptimes</code> , <code>int_lap_numbering</code> y <code>parse_duration_ms</code>	DBT-8	M	alta
12	Extracción atómica y reintentos en la ingesta de F1DB; recarga de las últimas carreras de FastF1	ING-1, ING-2, ING-4	S	media
13	Segundo disparo del pipeline cuando F1DB publica tarde	ING-6	S	media

#	Mejora	Ref.	Esfuerzo	Prioridad
14	<code>/health</code> que consulte la base de datos e informe del último refresco, más un monitor externo	API-11	S-M	media
15	pre-commit alineado con el ruff de uv, lock y un linter SQL	CI-7, DBT-12	S	media

Siguientes candidatos: separar permisos del pipeline (CI-4), límite de peticiones (API-10), ECharts con carga diferida (WEB-5), definiciones de récords únicas en dbt (DBT-7), imagen Docker en dos etapas con Trivy (CI-8) y runbook de operación (DOC-2).

Parte IV

Visualizaciones y récords

Síntesis de la parte IV

Veredicto sobre los gráficos actuales

La web tiene 17 gráficos o mapas, 15 tarjetas KPI y unas 25 tablas. Todos van con título, resumen en texto y tabla de datos, una base de accesibilidad mejor que la de casi todas las webs de F1 revisadas. El problema no es la calidad de cada gráfico sino el conjunto: algunos se repiten, otros tienen errores de lectura y faltan las preguntas que más se hace un aficionado.

Gráfico	Veredicto	Motivo y acción
Mapa de GP por país o circuito (5.23)	útil	Buena portada; añadir el calendario país × temporada (P19)
Evolución del campeonato	muy útil	Arreglar las etiquetas solapadas; añadir la diferencia con el líder y la eliminación matemática (P1)
Tarjetas de temporada (5.26)	poco útil	Repiten la tabla; sustituir por KPIs de relato (P11): cuándo se decidió, margen, dominio
Clasificación, % sobre el mejor (5.27)	útil	Aclarar que la referencia es el mejor tiempo de cualquier segmento y no la pole; mostrar segmentos y cortes
Vuelta a vuelta, posiciones (5.28)	muy útil	Mantener; añadir franjas SC/VSC y el race trace (P1b) como vista hermana
Estrategia de neumáticos (5.29)	muy útil	Mantener
Matriz de compuestos (5.29)	redundante	Dice lo mismo que los stints: pasa a ser su tabla alternativa o un mapa de calor de ritmo (P3)
Tiempos por vuelta en líneas (5.30)	poco útil con 20 pilotos	Limitarlo a 2–4 pilotos seleccionados y dejar el violín como vista principal
Violines de ritmo (5.31)	útil	Redondear ticks, agrandar el texto, diferenciar pilotos más allá del color; partir por compuesto (P4)
Paradas por constructor (5.32)	útil con matices	Es tiempo en el pit lane, no la parada; el eje en 0 aplana las diferencias
Compañeros: 3 gráficos + tablas (5.33)	redundante	Fusionar en un único gráfico divergente (P12) y una tabla
Puntos por temporada	útil	Eje continuo de años (hoy desaparecen 2007–2009 de Schumacher); sin barras antes de 1958 en constructores
Títulos en récords (5.24)	redundante	Repite una columna del ranking: sustituir por una línea de tiempo de títulos (P16)
Telemetría y trazado	útil	Pasa a la pestaña de clasificación; añadir delta y minisectores (P6); formato español en ejes

Qué añadir: lista combinada

Cruzando el catálogo de 21 webs y proyectos externos con las 26 propuestas propias, estas son las diez ideas con mejor relación entre valor y esfuerzo. Todas son factibles con los datos actuales.

#	Idea	Por qué	Esf.
1	Race trace: diferencia con el líder vuelta a vuelta (P1b)	La principal carencia; se reconstruye exacto desde 1996 y casi ninguna web gratuita lo ofrece para carreras antiguas	S–M
2	Franjas SC/VSC/bandera roja en los gráficos por vuelta	Explican saltos de ritmo y paradas; dato disponible desde 1993	S
3	Matriz de resultados de temporada estilo Wikipedia	Formato conocido por todos los aficionados; muy compacto	S
4	Récords ampliados: rachas, hat-tricks, vueltas y km lideradas, edades, remontadas, conversión de pole	35 de 37 récords catalogados se calculan hoy	S–M
5	Diferencia con el líder del campeonato y eliminación matemática (P1)	Responde «¿cuándo se decidió el título?»; hay 76 carreras marcadas como decisivas sin usar	S–M
6	Página de circuito (P24)	El hueco más grande: 78 circuitos con ganadores, récord de vuelta y % de victorias desde la pole	M
7	Comparador libre de dos pilotos (P25)	Pregunta más habitual; hoy solo se compara con compañeros	M
8	Parrilla → vuelta 1 → llegada (P9) y valoración de la salida	Explica la carrera en un gráfico sencillo	S
9	Campeonatos con otros sistemas de puntos (P14)	Comparación de eras; muy compartible	S–M
10	Detector de undercut y overcut (P5)	Inédito como gráfico automático: 204 intentos y 49 logrados en 2024	M

Corrección al capítulo 10

El catálogo externo marca el race trace como factible «desde 1950». Comprobado en la base: antes de 1990 no hay tiempos por vuelta (0–0,8 % por década) ni diferencias con el líder (0,2–1,4 %). El race trace es factible desde 1990 (96,5 % de tiempos en los 90); antes solo se puede dibujar el lap chart de posiciones. El diferencial real frente a otras webs es, por tanto, el lap chart desde 1950 y el race trace desde 1990.

Entre las ideas más originales, sin equivalente en las webs revisadas, están la red de compañeros con «grados de separación» (de Fangio a Antonelli en 8 saltos, P15), la «ghost race» animada y accesible (P8), el streamgraph de victorias por constructor desde 1950 (P18) y el «ADN» del piloto (P13).

Navegación propuesta



Figura 6: Navegación propuesta. En verde, las secciones nuevas; Calidad pasa al pie de página.



Figura 7: Temporada 2021: tarjetas que repiten la tabla y etiquetas solapadas al final de la evolución del campeonato.



Figura 8: Ficha de Michael Schumacher: los años sin carrera (2007–2009) desaparecen del eje de temporadas.

CAPÍTULO 9

Evaluación de nuestros gráficos y propuestas nuevas

Estado: COMPLETA (29/09/2026).

9.1 Resumen

- La web tiene 17 gráficos o mapas en 16 entradas de inventario (13 ECharts, 1 violín SVG propio, 1 mapa mundial d3, 1 trazado SVG y 1 matriz-tabla), 15 tarjetas KPI y unas 25 tablas. Todos los gráficos van dentro de ChartFigure, con título, resumen en texto y la tabla de datos desplegable. La base de accesibilidad es buena y mejor que la de la mayoría de webs de F1.
- Réplica del TFG: las páginas de las figuras 5.23-5.33 están todas (mapa, récords de pilotos y constructores, temporada, clasificación, vuelta a vuelta, neumáticos, tiempos, ritmo, boxes y detalle de piloto). Además hay extras: evolución del campeonato, telemetría con trazado, fichas de constructor y calidad de datos.
- Problemas principales:
 - Color sin identidad: la paleta Okabe-Ito de 8 colores se repite en carreras de 20 pilotos. Los compañeros no comparten color de equipo y en el violín dos pilotos distintos salen con el mismo color sin otra diferencia.
 - Solapamientos: stints y matriz de neumáticos cuentan lo mismo. Hay tres gráficos y dos tablas de compañeros con la misma información. Las tarjetas de temporada repiten la tabla.
 - Errores de lectura concretos: la «referencia» de clasificación no es la pole. La duración de parada es el tiempo en el pit lane, no la parada. Las temporadas sin carrera desaparecen del eje. Ferrari tiene barras a 0 en 1950-57. Las etiquetas del final de la evolución del campeonato se solapan. «1,000 m» aparece con el formato inglés.
 - Huecos de relato: no hay página de circuito, ni «gap al líder», ni comparador libre de pilotos, ni fiabilidad o edades. Tampoco se ve cuándo se decidió el título.
- Propongo 26 visualizaciones nuevas (P1-P26, más la P1b), todas comprobadas contra data/gold/f1.duckdb, y una navegación con dos entradas nuevas: Circuitos y Comparar. Además, Historia agruparía Récords y las vistas por eras.

Fuentes: código de web/src/app/[lang]/** y web/src/components/** (sin modificar), la web desplegada (navegada el 29/09/2026 a 640 px y a 375 px de ancho), el TFG (texto de las figuras 5.23-5.33 extraído con pdftotext, porque el PDF no deja leerlo directamente) y data/gold/f1.duckdb en modo de solo lectura.

Capturas en docs/informe_situacion/figuras/:

Fichero	Contenido
web_lap_chart_bahrain2024.png	Posiciones vuelta a vuelta, Bahrain 2024

Fichero	Contenido
web_tiempos_vuelta_bahrain2024.png	Tiempos por vuelta (20 pilotos)
web_violin_bahrain2024.png	Violines de ritmo
web_stints_bahrain2024.png	Estrategia de neumáticos
web_quali_gap_bahrain2024.png	% sobre el mejor tiempo (etiquetas solapadas)
web_telemetria_bahrain2024.png	Velocidad, acelerador y freno
web_temporada2021_kpi_progresion.png	Tarjetas KPI y evolución del campeonato de 2021 (etiquetas solapadas)
web_piloto_schumacher_kpi_puntos.png	KPIs y puntos por temporada (los años 2007-2009 desaparecen del eje)
web_piloto_schumacher_companeros.png	Gráficos de compañeros

9.2 Método

1. Lectura completa de las 17 páginas y de los 17 componentes visuales: tipos de marca, escalas, colores, datos de entrada y textos alternativos.
2. Navegación por la web desplegada: /es, /es/seasons/2021, /es/races/1102 (Bahrain 2024) y sus pestañas, /es/drivers/michael-schumacher, /es/constructors/ferrari y /es/records. Se probó a 640 px y a 375 px de ancho.
3. Consultas de cobertura en DuckDB para saber qué gráficos se pueden dibujar y desde qué año (sección 5.0).

9.3 Inventario completo

Leyenda de accesibilidad:

- F = ChartFigure: título, resumen textual y tabla desplegable.
- img = contenedor role="img" con aria-label.
- T = tabla semántica: caption, scope y región desplazable con el teclado.

9.3.1 Gráficos y mapas

#	Ruta	Componente	Tipo	Datos (endpoint)	Pregunta que responde	Interacción	Accesibilidad	TFG
G1	/ (#mapa)	WorldMap (d3-geo, SVG en servidor, sin JS)	Mapa de símbolos proporcionales (área $\propto \sqrt{GP}$), proyección Equal Earth	/circuits?season=2023; from&season_to; en la vista por país, centroide ponderado por carreras	¿Dónde se ha corrido la F1 y cuánto?	Por país o por circuito (enlaces); rango de temporadas (formulario GET); <title> en cada círculo	<title>/<desc> figcaption y tabla por país o circuito	523

#	Ruta	Componente	Tipo	Datos (endpoint)	Pregunta que responde	Interacción	Accesibilidad	TFG
G2	/seasons/[year]	Progression Chart	Líneas de puntos acumulados por ronda (top 10), con etiqueta al final	/seasons/{y}/standings/progression?top=10	¿Cómo se fraguó el campeonato?	Tooltip por eje; resaltado de la serie al pasar	F, img, tabla piloto × ronda	Nuevo
G3	/races/[id]/qualifying	BarChart vertical	Barras del % sobre el mejor tiempo de la sesión; la mejor en morado	/races/{id}/qualifying?session=>gap_to_pole_pct	¿A cuánto quedó cada piloto? ¿Qué equipo del mejor tiempo?	Pilotos o constructores; Q o Q-sprint	F, img	5.27
G4	/races/[id]/lap-chart	LapChart	Bump chart de posición por vuelta, con parrilla en la vuelta 0 y código del piloto al final	/races/{id}/laps + /results (parrilla)	¿Cómo evolucionó el orden en carrera?	Resaltado al pasar; tooltip por punto	F, img, tabla salida/llegada/-mejor/vueltas lideradas	5.28
G5	/races/[id]/tyres	Stint Chart	Barras horizontales apiladas por stint; color Pirelli + letra	/races/{id}/stints	¿Qué estrategia hizo cada uno?	Tooltip	F, img, leyenda con letra, tabla	5.29
G6	/races/[id]/tyres (#matriz)	Compound Matrix	Tabla-mapa de calor piloto × vuelta (compuesto; vuelta de entrada recuadrada)	/races/{id}/laps	¿Qué neumático llevaba en cada vuelta y cuándo paró?	title por celda; desplazamiento horizontal	T con texto sr-only por celda	5.29 (matriz)
G7	/races/[id]/pace (#tiempos)	LapTimes Chart	Líneas de tiempo por vuelta; 2 dataZoom (vueltas y tiempos)	/races/{id}/laps	¿Cómo evolucionó el ritmo de cada piloto?	Casillas por piloto, Todos/Ninguno, ocultar vueltas neutralizadas, zoom	F, img, tabla vuelta × piloto	5.30
G8	/races/[id]/pace (#ritmo)	Violin Chart (SVG propio, KDE de Silverman)	Violín + caja P25-P75 + mediana por piloto, ordenado por mediana	Laps filtrados: sin vuelta 1, boxes, SC/VSC/roja ni vueltas >150 % de la mediana; mín. 5 vueltas	¿Quién tuvo mejor ritmo de verdad y lo mantuvo?	Casillas por piloto; <title> por violín	F, img, tabla mediana/mejor/P25-P75/n°	5.31
G9	/races/[id]/pitstops	BarChart vertical	Barras del tiempo medio por equipo (paradas ≤ 60 s); la mejor en morado	/races/{id}/pitstops	¿Qué equipo perdió menos tiempo en boxes?	Tooltip	F, img	5.32
G10	/races/[id]/telemetry (desde 2024)	Telemetry Chart	3 paneles con el eje X compartido: velocidad, acelerador y freno frente a la distancia	/races/{id}/telemetry?driver=a,b (vuelta más rápida de clasificación)	¿Dónde gana tiempo un piloto frente a otro?	Selector A/B (formulario GET); eje enlazado	F, img, tabla tiempo/v. máx./v. mín./ % a fondo	Nuevo
G11	/races/[id]/telemetry	TrackMap	Trazado x/y coloreado por velocidad (azul→naranja), solo del piloto A	Igual que G10	¿Cómo es el circuito y dónde es rápido o lento?	Ninguna	<title><description> Nueva leyenda con aria-hidden	Nuevo
G12	/drivers/[id]	Season Points Chart	Barras de puntos por temporada; título en morado y posición final encima	/drivers/{id}/seasons	¿Cómo fue su carrera deportiva año a año?	Tooltip	F, img, tabla temporada/equipo/-pos./pts/V/P/poles	5.33 (parcial)
G13	/drivers/[id]	Teammate Points Chart	Barras agrupadas: puntos propios frente a los de sus compañeros (trama diagonal)	/drivers/{id}/teammates	¿Sumó más que sus compañeros?	Leyenda y tooltip	F, img, textura además del color	5.33
G14	/drivers/[id]	Teammate Share Chart ×2	Barras apiladas al 100 %: % por delante en clasificación y en carrera	Igual	¿Batió a sus compañeros en clasificación y en carrera?	Leyenda y tooltip	F, img, textura	5.33

#	Ruta	Componente	Tipo	Datos (endpoint)	Pregunta que responde	Interacción	Accesibilidad	TFG
G15/	constructors/[id]	Seasons Chart	Igual que G12	/constructors/{id}/seasons	Evolución del equipo	Tooltip	F, img	Nuevo
G16/	records	BarChart horizontal	Títulos por piloto o constructor en el rango de años	/rankings/{entity}?orderby=championships	¿Quién tiene más títulos en esa época?	Entidad y rango de años	F, img	5.24-5.25

9.3.2 Tarjetas KPI

Ruta	Componente	Contenido	TFG
/seasons/[year]	Highlight ×3	Campeón (o líder), equipo campeón (o líder) y victorias del campeón	5.26 (tarjetas de campeón y equipo)
/drivers/[id]	Stat ×6	Títulos, victorias, podios, poles, vueltas rápidas y salidas	5.33 (tarjetas superiores)
/constructors/[id]	Stat ×6	Títulos, victorias, podios, poles, dobles y carreras	Nuevo
/quality	Párrafo resumen	«N controles superados, M fallidos»	Nuevo

9.3.3 Tablas

Ruta	Tabla	Observación
/	Top 10 de la última carrera; top 5 de pilotos y de constructores	Puerta de entrada; repite la página de temporada
/seasons	Tarjetas por década con los campeones	Es una lista de enlaces, sin datos
/seasons/[year]	Pilotos, constructores y calendario con ganador	Réplica de la figura 5.26
/races/[id]	Resultado de carrera o sprint	Pills de vuelta rápida y dato corregido
/races/[id]/qualifying	Q1/Q2/Q3 con el mejor segmento en morado; %	La columna de % repite G3
/races/[id]/pitstops	Lista de paradas; pasos por el pit lane que no son paradas	La segunda tabla es original y valiosa (SC, bandera roja, sanciones)
/drivers, /constructors	Top 50 por victorias o búsqueda	Sin orden por columnas (a diferencia de /records)
/drivers/[id]	Detalle de compañeros (temporada × compañero)	Más la tabla por temporada, repetida en los 3 gráficos
/records	Ranking ordenable (7 métricas) con rango de años	Réplica de las figuras 5.24-5.25
/quality	Controles con umbral y controles informativos	Página original del proyecto

9.4 Evaluación crítica

9.4.1 Problemas transversales

1. Color y paleta.

- `LIGHT_SERIES` tiene 8 colores. A partir del 9.º piloto el color se repite y cambia el trazo (discontinuo o punteado), lo que funciona en líneas finas.
- En el **violín** (G8) no hay trazo que valga: Hülkenberg y Piastrri salen del mismo marrón, y Stroll y Pérez del mismo naranja (captura `web_violin_bahrain2024.png`).
- No se usa el color del equipo. El aficionado espera Ferrari en rojo y los dos compañeros con el mismo tono. **Dato que falta:** no hay colores de equipo en `gold.stg_fastf1__results.team_color` existe, pero su vista apunta a un bronze que no está disponible (da error al consultarla), y además solo cubre 2018+. Propuesta: `seed_constructor_colors.csv` (id, temporada desde/hasta, hex claro y oscuro) y usar variantes claras y oscuras para distinguir a los compañeros, manteniendo el trazo o la forma.

2. Ejes de temporadas como categorías. `SeasonPointsChart` y `TeammatePointsChart` usan `type: "category"`, así que las temporadas sin carrera desaparecen. En Schumacher, 2006 va pegado a 2010 y parece que no hubo retirada. Debe ser un eje de valor (años) o marcar los huecos.

3. Puntos entre eras. Los gráficos de puntos por temporada (G12, G15) mezclan sistemas de puntuación: 9 puntos por victoria en 1950 frente a 25 hoy, y ahora más carreras. Los 148 puntos de Schumacher en 2004 parecen poco frente a los 575 de Verstappen en 2023. Hace falta una métrica normalizada (sección 5, P14).

4. Base del eje en cero donde no aporta.

- En la media de paradas (G9) las barras van de 24,1 a 27,2 s y el eje de 0 a 30: todas parecen iguales. En barras hay que empezar en 0, así que es mejor un gráfico de puntos o *dot plot* con el eje ajustado, o mostrar la diferencia con el mejor.
- En telemetría (G10) la velocidad empieza en 0 km/h y la mitad inferior del panel queda vacía.

5. Formato numérico en ECharts. Los ejes usan el formato por defecto de ECharts y aparece «1,000 m» en la página española. Los tooltips sí usan `toLocaleString`. Falta un `axisLabel.formatter` con `Intl.NumberFormat(locale)`.

6. Etiquetas que se solapan.

- `ProgressionChart`: `labelLayout.moveOverlap` no se aplica a `endLabel`. En 2021 «Max Verstappen» y «Lewis Hamilton» se pisan, y «Carlos Sainz Jr./Lando Norris/Charles Leclerc» no se leen.
- Barras verticales de G3 con 20 pilotos: los valores «1,78 %», «1,79 %»... chocan entre sí y los nombres van girados 40°.

7. Móvil (375 px).

- El lap chart se comprime en ~250 px de ancho y los intercambios en boxes forman una madeja ilegible.
- Las pestañas de carrera y la navegación principal se desplazan en horizontal con una barra visible (se ve «Vuelta a vuelta» y la siguiente pestaña cortada).
- El violín obliga a desplazarse en horizontal.
- El mapa del inicio funciona bien porque es un SVG que se escala.

8. Sin enlaces entre gráficos. Ningún gráfico enlaza a la ficha del piloto o de la carrera (ECharts permite `on('click')`) ni recuerda la selección de pilotos entre pestañas. Las casillas de G7 y G8 son independientes: el aficionado marca 3 pilotos en tiempos y tiene que volver a marcarlos en el violín.

9. **Accesibilidad.** Lo correcto: figura, resumen, tabla, letras en compuestos, texturas en compañeros y `prefers-reduced-motion`. Mejorable:

- `aria.enabled: false` es deliberado (la tabla sustituye al gráfico), pero el tooltip no es accesible con el teclado. No hay forma de «recorrer» la serie.
- El texto del violín es de 10 px dentro de un `viewBox` que se escala hacia abajo: en la práctica queda por debajo de 8 px (captura).
- El resumen de G16 dice «Lewis Hamilton suma más títulos: 7» cuando empata con Schumacher. Los resúmenes deberían detectar empates.

10. **Rendimiento y carga.** Cada pestaña de carrera vuelve a pedir `/laps` (unas 1 200 filas) y `/results`. Hay `cache` por petición, pero no entre pestañas. Es aceptable, aunque con `Render free` la primera visita tarda.

9.4.2 Evaluación de cada elemento

#	Valoración	Puntos fuertes	Problemas concretos	Recomendación
G1 Mapa	Útil	Sin JS; rango de años; 2 vistas; tabla	Europa es una mancha de círculos que se solapan; el centroide por país sitúa a EE. UU. en medio del país y no en sus circuitos; no muestra el tiempo	Añadir un zoom a Europa (inserto) o una vista de «calendario por país × década» (P9); color por última temporada (vigente o histórico)
G2 Evolución campeonato	Muy útil	Cuenta la temporada; tabla completa	Etiquetas solapadas; 10 líneas con colores repetidos; los puntos absolutos esconden la diferencia en la lucha por el título	Añadir el modo «diferencia con el líder» (P1), resaltar por defecto los 2-3 aspirantes, marcar la ronda en que se decidió (<code>drivers_championship_decider</code>)
G3 % sobre el mejor tiempo	Útil, con un fallo de relato	Métrica fiel al TFG; modo por constructores	(1) La referencia es el mejor tiempo de cualquier segmento: en Bahrain 2024 la «referencia» es Leclerc (Q2, 1:29.165) y Verstappen, que hizo la pole, aparece a +0,016 %; el resumen dice «Leclerc marcó la referencia», lo que confunde. (2) Mezcla segmentos con distinta evolución de pista: Stroll (eliminado en Q2) queda por delante de Tsunoda porque su Q1 fue mejor. (3) Barras verticales con 20 nombres girados y etiquetas solapadas	Barras horizontales (como G16); nombrar la métrica «mejor vuelta de la sesión»; opción «por segmento» (Q1/Q2/Q3 por separado); marcar la línea de corte de Q1 y Q2

#	Valoración	Puntos fuertes	Problemas concretos	Recomendación
G4 Lap chart	Muy útil	Etiqueta directa; parrilla en la vuelta 0; tabla con vueltas lideradas	Madeja en las ventanas de paradas (vueltas 10-17 y 30-38 en Bahrain); con 20 pilotos no se sigue a nadie salvo al que se resalta; no marca SC, abandonos ni paradas; datos desde 1950, pero en los años 50 solo el 56 % de las posiciones por vuelta tienen dato, así que las líneas salen rotas	Resaltar por defecto el top 3 y atenuar al resto; marcar las paradas con puntos y el SC/VSC con bandas; ofrecer al lado un «race trace» (P1b), que responde mejor a la pregunta «quién ganó por estrategia»
G5 Stints	Muy útil	Claro y fiel a la convención de la TV; letra en cada tramo; el duro con contorno	Eje hasta 60 cuando la carrera tiene 57 vueltas; no distingue neumático nuevo o usado (el 20 % de los stints de 2024 salen con más de 1 vuelta de uso, según <code>tyre_age_laps</code>); el primer stint de 1 vuelta (Hülkenberg) sale sin letra	max = vueltas; trama o transparencia para los usados; tooltip con la edad inicial
G6 Matriz	Poco útil (repetida)	Réplica fiel del TFG; accesible	Da la misma información que G5 en 20 × 57 celdas; obliga a desplazarse en horizontal; el recuadro de la vuelta de entrada no añade nada a los cortes de G5	Fusionarla con G5 como su «tabla de datos», o reutilizar la forma de la matriz para algo nuevo: mapa de calor del tiempo por vuelta respecto a la mediana o de la posición (P3)
G7 Tiempos por vuelta	Útil para 2-4 pilotos, poco útil con 20	Zoom doble (fiel al TFG); ocultar neutralizadas activado si hace falta; tabla completa	Con 20 líneas y 8 colores es ilegible (captura); las vueltas de boxes cortan las líneas; el eje de tiempos no es invertido (en F1 «más rápido» se suele dibujar arriba o se dibuja el delta)	Por defecto, solo el top 3 o los dos del mismo equipo; añadir la vista «delta con un piloto de referencia»; opción de corregir el combustible (P2)
G8 Violín	Muy útil	Filtro de vueltas cuidado (incluye la heurística del 150 %); orden por mediana; tabla	Colores repetidos sin otra diferencia; texto diminuto; ticks con valores extraños (1:41.470, 1:39.698...) en lugar de valores redondos; la cola de VER hasta 1:32.6 (vuelta rápida con blandos nuevos) alarga mucho la escala	Ticks redondos (cada 0,5 s o 1 s); color de equipo; nombres horizontales más grandes; opción de dividir el violín por compuesto (mitad izquierda y derecha, <i>split violin</i>) (P4)
G9 Media de paradas	Poco útil tal como está	Réplica del TFG; umbral de 60 s	La «duración» es el tiempo en el pit lane (mediana 2020s: 23,9 s), no la parada estática (unos 2-3 s), y el título no lo aclara; el eje en 0 aplana las diferencias; la media esconde las paradas malas	Renombrarlo a «Tiempo en el pit lane»; usar <i>strip plot</i> (una marca por parada) con la mediana; dato que falta: la parada estática (DHL Fastest Pit Stop o documentos de la FIA)

#	Valoración	Puntos fuertes	Problemas concretos	Recomendación
G10 Telemetría	Útil (con un fallo de relato)	Tres paneles enlazados; selector A/B sin JS	Es la vuelta rápida de clasificación dentro de una pestaña de carrera y el título no lo dice; no hay delta de tiempo acumulado (lo más informativo en una comparación de telemetría); rpm, gear y drs están al 100 % en gold pero no se usan; velocidad desde 0; el eje Y del freno sin sentido; «1,000 m»	Titular «Vuelta rápida de clasificación»; añadir el panel de delta (se integra con distancia y velocidad) y el de marchas; ajustar el eje de velocidad; ver P6 y P7
G11 Trazado	Útil (atractivo)	Proporciones reales; escala continua	Solo el piloto A; no enseña la comparación que se acaba de elegir; la escala azul→naranja no marca dónde está la curva lenta	«Dominio por minisectores» A frente a B (P6) y el número de curva o la marcha en el trazado
G12 Puntos por temporada (piloto)	Útil	Título en morado; posición encima	Eje de categorías (se pierden los huecos); puntos no comparables entre eras; la barra no dice con qué equipo corrió	Eje temporal; alternar puntos, posición final y % de puntos posibles; color por equipo (P13)
G13 Puntos frente a compañeros	Poco útil (repetido)	Textura accesible	Repite lo que ya dicen G14 y la tabla; al sumar a varios compañeros en una temporada (1994: Verstappen + Lehto + Herbert) se mezclan duelos distintos	Fusionar con G14 en un solo gráfico de barras divergentes por temporada y compañero (P12)
G14 H2H clasificación y carrera (×2)	Útil	Porcentajes claros	Dos gráficos casi iguales uno al lado del otro; el apilado al 100 % gasta la mitad de la tinta en el «complemento»	Un único gráfico divergente: clasificación a la izquierda y carrera a la derecha, o 2 puntos por temporada
G15 Puntos por temporada (constructor)	Poco útil en equipos históricos	—	Ferrari 1950-57: el campeonato de constructores empezó en 1958, así que salen barras a 0 (el points ?? 0 del código) con 3-7 victorias por temporada; puntos no comparables	Para antes de 1958, mostrar victorias o puntos de sus pilotos o un marcador «sin campeonato»; métrica normalizada
G16 Títulos	Útil	Horizontal, legible	Repite la columna «Títulos» de la tabla ordenable de al lado; el resumen no detecta el empate	Sustituirlo por algo que la tabla no dé: «títulos a lo largo del tiempo» (línea de tiempo por piloto, P16)
KPI temporada	Poco útil	Réplica fiel del TFG	El campeón sale 3 veces (tarjeta, pill en la tabla y resumen de G2); «victorias del campeón» es un dato pobre	Sustituirlas por KPIs de relato: margen del título, ronda de la decisión, n° de ganadores distintos, dominio del equipo (% de victorias)

#	Valoración	Puntos fuertes	Problemas concretos	Recomendación
KPI piloto y constructor	Útil	Resumen rápido	Sin contexto (¿91 victorias es mucho?)	Añadir el percentil o el puesto histórico («2.º de todos los tiempos») o el porcentaje (<code>win_rate_pct</code> y <code>podium_rate_pct</code> existen en <code>agg_driver_career</code>)
Tabla de pasos por el pit lane /quality	Muy útil y original Útil para el TFG y la transparencia	Distingue parada, SC, bandera roja y abandono —	Poco visible, al final de la pestaña Solo tablas	Integrarla como marcas en G4 y G5 Opcional: gráfico de barras de % de coincidencia con la línea del umbral

9.4.3 Repeticiones y solapamientos (qué fusionar o eliminar)

Solapamiento	Elementos	Propuesta
Estrategia de neumáticos	G5 stints + G6 matriz + tabla de stints	Dejar G5. La matriz pasa a ser su tabla alternativa o se reutiliza como mapa de calor del ritmo (P3)
Ritmo de carrera	G4 posiciones + G7 tiempos + G8 violín	Cada uno responde a algo distinto, pero G7 con 20 pilotos no responde a nada. Propuesta: pestaña «Carrera» con G4 y el race trace (P1b); pestaña «Ritmo» con el violín y G7 limitado a 2-4 pilotos o en modo delta
Compañeros de equipo	G13 + G14 ×2 + tabla por temporada (repetida 3 veces como tabla de cada figura) + tabla de detalle	Un único gráfico divergente (P12) + una tabla de detalle
Campeón de la temporada	3 tarjetas + pill de la tabla + resumen de G2 + tarjeta en /seasons	Tarjetas de relato (ver KPI temporada)
% en clasificación	G3 + columna % de la tabla de clasificación	Aceptable (la tabla es la referencia), pero G3 debería aportar algo más (segmentos, cortes)
Títulos	G16 + columna «Títulos» del ranking	Sustituir G16 (P16)
Inicio frente a temporada	Top 5 de pilotos y constructores del inicio = tabla de la temporada actual	Aceptable como portada; mejor un «mini G2» (la lucha por el título en una línea)
Puntos por temporada	G12 (piloto) y G15 (constructor) con el mismo componente	Correcto (reutilización); arreglar los ejes y la normalización
Salida y llegada	Tabla de G4 (salida, llegada) y tabla de resultado (parrilla, posición)	Aceptable

9.4.4 Huecos de relato (preguntas típicas de un aficionado sin respuesta)

1. «¿Por cuánto ganó y cuándo se decidió la carrera?» No hay diferencias en el tiempo. `gap_to_leader_ms` está al 84-100 % desde 2022, y el tiempo acumulado ($\Sigma \text{lap_time_ms}$) lo reconstruye exactamente desde 1996: en Bahrain 2024, vuelta 30, el cálculo coincide con la fuente al centésimo.
2. «¿Hubo adelantamientos? ¿Fue una carrera entretenida?» No hay índice de cambios de posición ni de cambios de líder (calculable: media de cambios de líder por carrera de 4,0 en los 50, 1,8 en los 70 y 2,7 en los 2020).
3. «¿Qué pasa en este circuito?» No hay página de circuito, aunque `dim_race` tiene 78 circuitos con

lat/lon, tipo, sentido, longitud y curvas. No se ven ganadores por circuito, conversión de la pole en victoria (Yas Marina 70,6 %, Catalunya 69,4 %...), probabilidad de SC ni récord de vuelta.

4. «¿Quién es mejor: X o Y?» No hay un comparador libre de pilotos. Solo se compara con los compañeros o, en telemetría, en una sesión.
5. «¿Cuándo se decidió el título y quién podía ganarlo todavía?» Hay 76 carreras marcadas como `drivers_championship_decider`, pero no se usan. Tampoco se calcula la eliminación matemática.
6. **Fiabilidad y abandonos por época.** `reason_retired` tiene datos de todas las décadas. Los clasificados pasan del 41 % en los 80 al 87 % en los 2020, y la parte mecánica de los abandonos baja del 86 % al 55 %.
7. **Edades y carreras deportivas.** Ganador más joven (Verstappen, 18,6 años) y edad media de la parrilla (35,1 en los 50 frente a 28,4 en los 2020). `date_of_birth` está al 100 %.
8. **Nacionalidades y «victorias en casa».** Hay 95 victorias en casa de 1 167.
9. **Equipos y motores.** No se ve el motor por equipo y temporada, aunque `engine_manufacturer_id` está en los resultados. Tampoco hay una línea de tiempo de las alineaciones.
10. **Sprint.** Hay resultado de sprint, pero no vueltas (ver nota 03).
11. **Datos que no tenemos.** Meteorología, mensajes de dirección de carrera (solo hay banderas por vuelta), parada estática y adelantamientos oficiales.

9.5 Propuestas nuevas

9.5.1 Qué se puede dibujar y desde cuándo (comprobado en `data/gold/f1.duckdb`)

Dato	Tabla y columna	Cobertura
Posición por vuelta	<code>fact_laptimes.position</code>	Las 1 164 carreras disputadas desde 1950 (56 % de las filas con dato en los 50, 95 % en los 60, 100 % desde los 70)
Tiempo por vuelta	<code>fact_laptimes.lap_time_ms</code>	0 % antes de 1990, 96,5 % en los 90, ~100 % desde 2000 (desde 1996 en la práctica)
Gap al líder	<code>gap_to_leader_ms</code>	60-87 % por temporada desde 1996, 100 % en 2025-26; reconstruible exactamente con $\sum \text{lap_time_ms}$ (comprobado en Bahrain 2024, vuelta 30: Pérez +15,68 s, Sainz +18,20 s en ambos cálculos)
Compuesto y edad del neumático	<code>tyre_compound</code> , <code>tyre_age_laps</code>	Desde 2011 (100 %); compuesto Pirelli C1-C5 desde 2019
Sectores y speed trap	<code>sector_*_ms</code> , <code>speed_trap_kmh</code>	Desde 2018 (98 %)
Banderas por vuelta	<code>is_safety_car</code> , <code>is_yellow_flag</code> , <code>is_virtual_safety_car</code> , <code>is_red_flag</code>	SC desde los 90 (1 922 vueltas), VSC desde 2015, bandera roja desde 2000
Paradas	<code>fact_pit_stops.time_ms</code>	Desde 1994 (612 carreras); es el tiempo en el pit lane (mediana de 30,0 s en los 90 y 23,9 s en los 2020)

Dato	Tabla y columna	Cobertura
Pasos por el pit lane	fact_pit_lane_passes.pass_type	pit_stop 22 485, unclassified 1 118, retirement 351, penalty_or_other 141, safety_car 114, red_flag 62
Telemetría de clasificación	fact_quali_telemetry	2024-2026 (63 carreras); ~450 puntos por vuelta; velocidad, rpm, marcha, acelerador, freno, DRS y x/y al 100 %
Clasificación del campeonato por ronda	fact_driver_standing, fact_constructor_standing	Todas las rondas desde 1950 (1950: 108/108 filas con posición)
Resultado	fact_race_result	Parrilla ~90 %; reason_retired con 198 valores distintos; positions_gained; is_grand_slam (71); piloto del día (2016+, 228)
Q1/Q2/Q3	fact_qualifying_result	Desde 2006 (40 % de los 2000, ~99 % desde 2010); mejor tiempo desde 1950 (96 %)
Carrera	dim_race	78 circuitos con lat/lon, tipo (ROAD/RACE/STREET), sentido, longitud, curvas; drivers_championship_decider (76 carreras)
Piloto	dim_driver	date_of_birth 100 % (917 pilotos), nacionalidad alpha2
Compañeros	agg_teammate_h2h	13 530 filas; grafo de 4 componentes conexas, con 810 pilotos en la mayor

Datos que faltan para algunas propuestas:

- **Colores de equipo por temporada:** hace falta un seed. La vista `stg_fastf1__results.team_color` apunta a un bronze que no está disponible y solo cubriría 2018+.
- **Clasificación de `reason_retired`** en mecánico, accidente u otro: seed de unas 200 filas.
- **Parada estática** (solo tenemos el tiempo en el pit lane).
- **Meteorología.**
- **Mensajes de dirección de carrera** (solo hay banderas por vuelta).
- **Vueltas de sprint** (ver nota 03).
- **Adelantamientos oficiales:** los cambios de posición son un proxy.

Esfuerzo: S = menos de 1 día (reutiliza un endpoint o componente existente); M = 1-3 días (endpoint o modelo dbt nuevo); L = más de 3 días.

9.5.2 Carrera (pestañas de /races/[id])

P1b. Race trace: diferencia con el líder vuelta a vuelta. *Esfuerzo S-M.*

- **Pregunta:** ¿por cuánto iba ganando, cuándo se abrió o cerró la carrera y qué hizo el SC con las diferencias?
- **Boceto:**
 - Eje X: vuelta. Eje Y: segundos detrás del líder, invertido, con el líder en 0 arriba.
 - Una línea por piloto; las vueltas bajo SC/VSC como bandas grises de fondo y las paradas como puntos huecos.
 - Etiqueta directa al final. Por defecto se resalta el top 5 y el resto va en gris al 25 %.

- Variante: «delta con un piloto elegido» (la referencia horizontal es ese piloto).
- Datos: `fact_laptimes` (Σ `lap_time_ms` por piloto y vuelta; en cada vuelta, referencia = mínimo acumulado), `is_safety_car/is_virtual_safety_car`, `is_pit_in_lap`. Desde 1996.
- Dónde: pestaña «Vuelta a vuelta», junto a G4, con un conmutador «posiciones | diferencias».
- Accesibilidad: resumen generado («X abrió N s en M vueltas; el SC de la vuelta K dejó la ventaja en 1 s») y tabla vuelta $\times$ piloto.

P2. Degradación por stint. *Esfuerzo M.*

- Pregunta: ¿quién cuidó mejor los neumáticos? ¿Cuánto perdía por vuelta cada compuesto?
- Boceto:
 - (a) *Dot plot*: una fila por piloto; cada stint es un punto en X = pendiente en ms/vuelta, con color y letra del compuesto.
 - (b) Al pulsar un piloto, dispersión de tiempo corregido frente a la edad del neumático, con su recta de regresión por stint.
 - Corrección de combustible: restar la tendencia común de la carrera (mediana por vuelta de todos los pilotos) o un coeficiente fijo documentado como estimación.
- Datos: `regr_slope(lap_time_ms, tyre_age_laps)` por (piloto, stint) sobre vueltas limpias (sin vuelta 1, boxes ni SC/VSC; mínimo 8 vueltas). Probado en Bahrain 2024: pendientes de 6 a 123 ms/vuelta (Leclerc: 84 ms/vuelta con blando y 6 con el primer duro). La dispersión muestra que la corrección de combustible es imprescindible antes de publicarlo. Desde 2011.
- Dónde: pestaña «Neumáticos» o «Estrategia», **en lugar de la matriz G6.**
- Accesibilidad: tabla piloto/stint/compuesto/vueltas/pendiente; resumen «el que menos degradó con duro fue X».

P3. Mapa de calor del ritmo (reutiliza la forma de la matriz). *Esfuerzo S.*

- Pregunta: ¿en qué fase de la carrera fue rápido cada piloto?
- Boceto:
 - Filas = pilotos (orden de llegada); columnas = vueltas.
 - Color divergente = diferencia con la mediana de la vuelta, con los rápidos en azul y los lentos en naranja (apto para daltonismo).
 - Vueltas neutralizadas rayadas, paradas recuadradas y la letra del compuesto opcional.
- Datos: los mismos `laps` que usa hoy G6.
- Dónde: sustituye a G6.
- Accesibilidad: sigue siendo una `<table>` con el valor en texto `sr-only`, igual que la matriz actual.

P4. Violín partido por compuesto. *Esfuerzo S.*

- Pregunta: ¿el ritmo de X era mejor con medio o con duro?
- Boceto: mitad izquierda del violín con el compuesto 1 y mitad derecha con el compuesto 2, en colores Pirelli con contorno; ticks redondos.
- Datos: los de G8 más `tyre_compound`.
- Dónde: opción de G8.
- Accesibilidad: la tabla gana una columna por compuesto.

P5. Detector de undercut y overcut. *Esfuerzo M.* No lo he visto como gráfico automático en otras webs de aficionados.

- Pregunta: ¿quién ganó o perdió posiciones por la estrategia de paradas?
- Boceto:
 - Lista de «duelos de estrategia»; cada duelo es un mini *slope chart* con dos puntos (antes y después) y dos líneas (A y B).
 - Etiqueta: «Norris paró 2 vueltas antes que Leclerc → undercut logrado (+1 posición)».
 - Verde u óxido según el resultado, con icono ✓/✗ para no depender del color.
- Datos:
 - Heurística: A y B van consecutivos en pista la vuelta anterior a la parada de A; B para entre 1 y 3 vueltas después; se comparan las posiciones en la vuelta $1b+2$.
 - Resultado en 2024: **204 intentos, 49 undercuts logrados**.
 - Mejora: pedir que la diferencia previa sea menor de 3 s (con `gap_to_leader_ms` o el acumulado) y descartar las paradas bajo SC.
 - Desde 1996 (las paradas empiezan en 1994).
- Dónde: pestaña «Boxes» o «Estrategia» (le da el relato que ahora le falta).
- Accesibilidad: es una lista de frases; el gráfico solo ilustra.

P6. Delta de tiempo y dominio por minisectores (telemetría). *Esfuerzo M.*

- Pregunta: ¿en qué curvas gana tiempo A sobre B?
- Boceto:
 - (a) Panel nuevo encima de la velocidad: delta acumulado (s) frente a la distancia, con el cero en el centro; por encima, A es más lento.
 - (b) Trazado dividido en unos 25 minisectores, cada uno con el color del piloto más rápido en él y patrón o letra A/B para no depender del color. Es el gráfico más conocido de FastF1, pero no hay una web en español que lo ofrezca navegable.
- Datos: `fact_quali telemetry` (distancia, velocidad, x, y). El tiempo acumulado se obtiene con $t = \sum \Delta d/v$ y se interpola a una rejilla común de distancia. 2024+.
- Dónde: telemetría, que debería pasar a la pestaña Clasificación (sección 6).
- Accesibilidad: tabla por minisector (distancia, piloto más rápido, diferencia).

P7. Mapa de marchas y DRS en el trazado. *Esfuerzo S.*

- Pregunta: ¿cómo se conduce este circuito?
- Boceto: trazado coloreado por marcha (paleta ordinal de 8 pasos con el número en las zonas largas), tramos con DRS abierto en trazo grueso y RPM en el tooltip.
- Datos: `gear`, `drs`, `rpm` (100 %, sin usar hoy). 2024+.
- Dónde: telemetría y página de circuito (P24).

P8. «Ghost race» animada. *Esfuerzo L.* Inédita en formato accesible.

- Pregunta: ¿cómo se vería la carrera desde arriba?
- Boceto:

- Un punto por coche (código y color de equipo) sobre la silueta del circuito, o sobre una recta horizontal que representa una vuelta cuando no hay trazado.
- Posición = fracción de vuelta según el tiempo acumulado interpolado.
- Controles: play/pausa, velocidad $\times 10/\times 30$ y deslizador de vuelta. Bandas SC.
- Datos: Σ lap_time_ms (desde 1996); silueta de fact_quali_telemetry x/y (2024+) o, antes, una línea neutra.
- Accesibilidad:
 - Con prefers-reduced-motion se muestra en su lugar una serie estática de mini-gráficos (vueltas 1, 10, 20...).
 - El deslizador es un `<input type="range">` con aria-valuetext («Vuelta 23: 1.º VER, 2.º PER a 4,1 s...»).
 - Nunca arranca sola.

P9. Parrilla → vuelta 1 → llegada (*slope chart*). Esfuerzo S.

- Pregunta: ¿quién ganó posiciones en la salida y quién en la carrera?
- Boceto: 3 columnas (parrilla, fin de la vuelta 1 y llegada); una línea por piloto, verde si gana y óxido si pierde, con grosor o flecha como segunda señal; los abandonos terminan en una columna «Ab.».
- Datos: grid_position, fact_laptimes.position de la vuelta 1 y position_number. Desde 1950.
- Dónde: pestaña «Resultado», encima de la tabla.
- Extra histórico: ranking de «mejores salidas» (mínimo de 100 carreras: Jochen Mass +1,56 posiciones de media en la vuelta 1, Brundle +1,52, Stroll +1,34).

9.5.3 Temporada (/seasons/[year])

P1. Diferencia con el líder del campeonato y eliminación matemática. Esfuerzo S-M.

- Pregunta: ¿cuándo se decidió el título y quién seguía con opciones?
- Boceto:
 - Eje X: ronda. Eje Y: puntos detrás del líder, invertido.
 - Banda sombreada con los «puntos que quedan en juego» (restantes $\times$ máximo por carrera; en gold, el máximo es 9 en 1950 y 1990, 25 en 2010 y 2025 y 26 en 2019 por la vuelta rápida, más el sprint).
 - Cuando la línea de un piloto sale de la banda, un ✖ indica que queda eliminado. Una línea vertical marca la carrera drivers_championship_decider.
 - Etiquetas directas con separación real (hay que resolver el solape de G2).
- Datos: fact_driver_standing (todas las rondas desde 1950), fact_race_result.points para el máximo por carrera y dim_race.drivers_championship_decider. Aviso: las reglas de «mejores N resultados» anteriores a 1991 hacen que la eliminación sea aproximada y hay que indicarlo.
- Dónde: modo alternativo de G2 (conmutador «puntos | diferencia | posición»).
- Accesibilidad: resumen «El título se decidió en la ronda N (GP de X); Y quedó eliminado en la ronda M».

P10. Bump chart de posiciones en el campeonato. Esfuerzo S.

- Pregunta: ¿cómo cambió el orden del campeonato ronda a ronda?

- Boceto: como G4, pero por ronda y con `position_number` de la clasificación del campeonato.
- Datos: `fact_driver_standing.position_number`.
- Dónde: tercera opción del conmutador de G2. Reutiliza LapChart.

P11. KPIs de relato de la temporada. *Esfuerzo S.* Sustituyen a las tarjetas actuales.

- Contenido: margen final del título, ronda en que se decidió, nº de ganadores distintos, % de victorias del mejor equipo y, en temporadas en curso, «carreras restantes / puntos en juego».
- Datos: `standings`, `is_win` y `drivers_championship_decider`.

9.5.4 Piloto y constructor

P12. H2H divergente con los compañeros (fusiona G13 y G14). *Esfuerzo S.*

- Pregunta: ¿batió a sus compañeros, y a cuál?
- Boceto:
 - Una fila por temporada y compañero; barra divergente desde el 50 %: a la derecha, % de clasificaciones ganadas; a la izquierda, perdidas.
 - Un segundo marcador (rombo) con el % en carrera.
 - Etiqueta con el nombre del compañero y color del equipo.
 - Se ve de un vistazo, por ejemplo, el 2010-2012 de Schumacher frente a Rosberg.
- Datos: `agg_teammate_h2h` (ya en `/drivers/{id}/teammates`).
- Accesibilidad: una única tabla (hoy la misma tabla aparece 3 veces).

P13. «ADN» o carrera deportiva del piloto. *Esfuerzo M.*

- Pregunta: ¿qué tipo de piloto es: poleman, de domingo, fiable, remontador?
- Boceto:
 - (a) Franja temporal (*swimlane*) sobre un eje de años continuo: las temporadas coloreadas por equipo y un punto con la posición final (eje invertido). Se ven los huecos, como la retirada de Schumacher de 2007 a 2009.
 - (b) «Huella» con 6 barras de percentil frente a los pilotos de su época (no radar, que distorsiona las áreas): % de victorias, % de podios, % de poles, H2H en clasificación, posiciones ganadas de media y % de abandonos. Cada barra lleva la cifra real.
- Datos: `agg_driver_career` (`win_rate_pct`, `podium_rate_pct`, `pole_positions`, `race_starts`), `agg_teammate_h2h`, `fact_race_result.positions_gained` y `reason_retired`. Para los percentiles, pilotos con más de 30 salidas en la misma década.
- Accesibilidad: tabla métrica/valor/percentil.

P14. Comparación de eras normalizada. *Esfuerzo S.*

- Pregunta: ¿quién habría sumado más con el sistema de puntos actual?
- Boceto: en `/records` y en la ficha, conmutador «puntos oficiales | sistema 2010 (25-18-15-12-10-8-6-4-2-1) | puntos por carrera».
- Datos: se recalcula desde `fact_race_result.position_number`. Comprobado (mínimo de 50 carreras), en puntos 2010 por carrera: Fangio 15,05, Verstappen 13,86, Hamilton 13,85, M. Schumacher 12,63, Prost 12,26, Clark 11,34, Senna 11,61 y Stewart 11,09.
- Dónde: `/records` (nueva métrica ordenable) y G12 (eje alternativo).

- Accesibilidad: es una columna más de la tabla.

P15. Red de compañeros y «grados de separación». *Esfuerzo M.* Inédito.

- Pregunta: ¿cómo se conecta Fangio con Antonelli?
- Boceto:
 - (a) Buscador de dos pilotos → camino mínimo dibujado como cadena horizontal de «fichas» (piloto → equipo y año → piloto...).
 - (b) Grafo de fuerzas opcional del vecindario a 2 saltos (no los 810 nodos a la vez).
- Datos: aristas de `agg_teammate_h2h`. BFS en Python sobre gold: **Fangio → Jo Bonnier → Rolf Stommelen → Riccardo Patrese → Michael Schumacher → Nico Rosberg → Lewis Hamilton → George Russell → Kimi Antonelli** (8 saltos). 4 componentes conexas; la mayor tiene 810 pilotos. Con más de 900 nodos, el BFS se calcula en milisegundos en la API (Python) o se precalcula. La consulta recursiva en SQL explotaba combinatoriamente y no conviene.
- Accesibilidad: el camino es una lista ordenada de texto; el grafo es decorativo, con alternativa textual.

P16. Línea de tiempo de títulos (sustituye a G16). *Esfuerzo S.*

- Pregunta: ¿quién dominó cada época?
- Boceto:
 - Eje X: temporada 1950-2026. Una fila por campeón, ordenada por su primer título.
 - Un cuadrado por título con el color del equipo y una línea fina que une su primer y último título.
 - Filtro por rango (el que ya existe).
- Datos: `agg_driver_season.championship_won` (o `/seasons`).
- Accesibilidad: tabla campeón → años.

P17. Alineación y motor del constructor. *Esfuerzo S.*

- Pregunta: ¿quién pilotó para el equipo y con qué motor?
- Boceto: *swimlane* por temporada con una fila por piloto (barra por temporada); banda superior coloreada por motor.
- Datos: `/constructors/{id}/seasons` (ya trae los pilotos) y `fact_race_result.engine_manufacturer_id` (22 fabricantes con victorias).
- Arreglo relacionado: en G15, antes de 1958 no dibujar barras a 0, sino un marcador «sin campeonato de constructores» o las victorias.

9.5.5 Historia (vistas agregadas nuevas)

P18. Streamgraph de victorias por constructor, 1950-2026. *Esfuerzo S-M.*

- Pregunta: ¿cómo han cambiado las eras de dominio?
- Boceto:
 - Eje X: temporada. El grosor de cada capa es el % de victorias de la temporada (normalizado, para que las temporadas de 7 y de 24 carreras pesen igual).
 - 10 equipos con más victorias en color y el resto en «otros» gris; etiquetas dentro de las capas; conmutador «constructores | motores».

- Línea superpuesta con el «índice de dominio» (% de victorias del mejor equipo de cada año).
- Datos: `fact_race_result.is_win × constructor_id/engine_manufacturer_id` (38 constructores y 22 motores con victorias; p. ej., 2025: McLaren 14, Red Bull 8, Mercedes 2).
- Accesibilidad: tabla temporada × equipo y resumen por eras.
- Nota: el streamgraph centrado es difícil de leer con precisión; la alternativa es un área apilada al 100 %.

P19. Calendario mundial: país × temporada. *Esfuerzo S.*

- Pregunta: ¿cuándo entró y salió la F1 de cada país?
- Boceto:
 - Rejilla con filas = países, agrupados por continente y ordenados por primer GP, y columnas = temporadas.
 - Celda rellena con el n° de GP de ese año (1, 2 o 3) y el circuito en el tooltip.
 - Parecido a un gráfico de contribuciones de GitHub.
- Datos: `dim_race(circuit_country/grand_prix_country, season, circuit_name)`. El continente habría que añadirlo a `gold: stg_f1db__countries.continent_id` existe, pero la vista depende del bronze.
- Dónde: inicio, como segunda vista del mapa («mapa | calendario»). Responde a lo que el mapa no muestra: el tiempo.
- Accesibilidad: es una tabla país × década con recuento.

P20. Fiabilidad a lo largo de la historia. *Esfuerzo M* (por el seed).

- Pregunta: ¿son los coches más fiables? ¿Se abandona más por accidentes?
- Boceto: área apilada al 100 % por temporada con clasificados, abandono mecánico y abandono por accidente u otro, con anotaciones. Solo se clasificaba el 41 % de los participantes en los 80 y el 87 % en los 2020.
- Datos: `reason_retired` (198 valores) → seed `retirement_categories.csv`. Con una clasificación provisional, la parte mecánica de los abandonos baja del 86 % (años 60) al 55 % (2020s) y la de accidentes sube del 12 % al 44 %.
- Dónde: Historia. Versión por equipo en la ficha del constructor.

P21. Mapa de calor de posiciones de llegada. *Esfuerzo S.*

- Pregunta: ¿cómo terminaba de verdad un piloto: siempre en puntos o todo o nada?
- Boceto:
 - En la ficha: filas = temporadas; columnas = posición de llegada 1...20 + «Ab.».
 - Intensidad secuencial = n° de veces; la columna 1 lleva el borde morado.
 - Versión global en Historia: filas = décadas.
- Datos: `fact_race_result.position_number` y `reason_retired`. Desde 1950.
- Accesibilidad: es una `<table>` con los recuentos en texto.

P22. Edad y relevo generacional. *Esfuerzo S.*

- Pregunta: ¿los pilotos son cada vez más jóvenes?

- Boceto: *beeswarm* por temporada con la edad de cada ganador el día de la victoria; línea con la edad media de la parrilla (35,1 años en los 50 → 28,4 en los 2020); anotaciones para el más joven (Verstappen, 18,62 años en 2016; le sigue Antonelli con 19,55 en 2026) y el mayor.
- Datos: `dim_driver.date_of_birth` (100 %) y `dim_race.race_date`.

P23. Margen de victoria e «índice de emoción». *Esfuerzo M.*

- Pregunta: ¿las carreras son hoy más ajustadas? ¿Cuáles fueron las más disputadas?
- Boceto:
 - (a) *Strip plot* del margen del ganador por carrera (escala logarítmica) con la mediana móvil: 27,6 s en los 50, 11,2 s en los 90 y 6,2 s en los 2020.
 - (b) Tabla «carreras más disputadas» con un índice compuesto de cambios de líder, cambios de posición en pista (sin la vuelta 1 ni vueltas de boxes) y margen final.
 - El índice es un **proxy** y hay que explicarlo: sin datos oficiales de adelantamientos, un cambio de posición no es siempre un adelantamiento.
- Datos: `gap_ms` de P2 y posición por vuelta. Cambios de líder por carrera: media de 4,0 en los 50, 1,8 en los 70, 3,6 en los 2000 y 2,7 en los 2020.
- Accesibilidad: el ranking es una tabla.

9.5.6 Entidades nuevas

P24. Página de circuito. *Esfuerzo M.* Es el hueco más grande.

- Pregunta: ¿qué pasa en este circuito?
- Contenido:
 - Ficha: tipo, sentido, longitud, curvas y temporadas.
 - Trazado de P7 si hay telemetría.
 - Ganadores por año (tira de cuadrados con el color del equipo) y conversión de pole en victoria (mínimo de 15 carreras: Yas Marina 70,6 %, Catalunya 69,4 %, Marina Bay 68,8 %, Shanghái 63,2 %).
 - % de vueltas bajo SC desde 2000 (Marina Bay 9,8 %, Baku 9,1 %, Melbourne 8,5 %, Interlagos 7,8 %).
 - Récord de vuelta en carrera por trazado: hay que agrupar por `course_length`, porque los trazados cambian.
- Datos: `dim_race`, `fact_race_result` y `fact_laptimes`. Endpoint nuevo `/circuits/{id}`; la ruta `/circuits` ya existe para el mapa.
- Dónde: los nombres de circuito de las tablas y del mapa pasan a ser enlaces.

P25. Comparador libre de dos pilotos. *Esfuerzo M.*

- Pregunta: ¿quién es mejor, X o Y?
- Boceto:
 - (a) Victorias acumuladas frente al n° de carrera, o frente a la edad (dos líneas con etiqueta directa).
 - (b) KPIs enfrentados en espejo (barras a izquierda y derecha).
 - (c) Si coincidieron, H2H en carreras compartidas (quién terminó delante).

- (d) Normalización de P14.
- Datos: fact_race_result, agg_driver_career y dim_driver.
- Dónde: /compare?a=8b=, enlazado desde cada ficha («comparar con...»).
- Accesibilidad: formulario GET sin JS (como la telemetría) y una tabla comparativa.

P26. Victorias en casa. Esfuerzo S.

- Pregunta: ¿rinden más los pilotos en su país?
- Boceto: barras por nacionalidad con victorias en casa y fuera (95 de 1 167 victorias fueron en casa, comparando nationality_alpha2 con circuit_alpha2).
- Dónde: Historia o la página de circuito.

9.5.7 Prioridad sugerida

Prioridad	Propuestas	Motivo
1 (arreglos baratos)	Ejes temporales en G12, G13 y G15; ticks y texto del violín; formato numérico local; G3 horizontal y con la métrica bien nombrada; seed de colores de equipo; etiquetas de G2; títulos de G9 («tiempo en el pit lane») y G10 («vuelta de clasificación»); barras a 0 de G15; empates en los resúmenes	Errores de lectura con impacto directo y poco coste
2 (relato de carrera)	P1b race trace, P3 en lugar de la matriz, P6 delta y minisectores, P9 parrilla→llegada, P12 H2H divergente	Responden a las preguntas más frecuentes y usan datos que ya sirve la API
3 (entidades)	P24 circuito, P25 comparador, P1 eliminación matemática	Huecos de relato grandes
4 (historia)	P18 streamgraph, P19 calendario, P14 eras, P16 títulos, P20 fiabilidad, P22 edades, P21 mapa de calor	Nueva sección «Historia»
5 (singulares)	P5 undercut, P2 degradación, P15 red de compañeros, P8 ghost race, P23 índice de emoción	Diferenciadores; más esfuerzo o una heurística que hay que validar

9.6 Reorganización de la navegación

Situación actual

- Navegación: Temporadas · Pilotos · Constructores · Récords · Calidad.
- Pestañas de carrera: Resultado · Clasificación · Vuelta a vuelta · Neumáticos · Ritmo · Boxes · Telemetría. Son 7 y en móvil se desplazan en horizontal.

Propuesta

Inicio	última carrera + mini «diferencia con el líder» (P1) + mapa calendario (G1/P19)
Temporadas	/seasons/[year]: KPIs de relato (P11) · campeonato [puntos diferencia
↔ posiciones]	· clasificaciones · calendario
↳ Carrera	/races/[id], 5 pestañas:
	1. Resultado (parrilla→V1→llegada P9 + tabla)

	2. Clasificación (G3 horizontal y por segmento + tabla + telemetría G10/G11 ↪ con P6 y P7 desde 2024)
	3. Carrera (G4 posiciones P1b diferencias; P8 ghost race opcional)
	4. Ritmo (violín G8/P4 + mapa de calor P3 + tiempos G7 limitado a ↪ 2-4 pilotos)
	5. Estrategia (stints G5 + degradación P2 + undercuts P5 + paradas G9 + ↪ pasos por el pit lane)
Pilotos	ficha: KPIs con contexto · ADN P13 · puntos normalizables P14 · H2H divergente ↪ P12 · posiciones P21
Constructores	ficha: KPIs · temporadas (arreglo pre-1958) · alineación y motor P17 · ↪ fiabilidad del equipo P20
Circuitos	NUEVA (P24)
Comparar	NUEVA (P25 + grados de separación P15)
Historia	Récords (tabla ordenable + P14) · títulos P16 · streamgraph P18 · fiabilidad ↪ P20 · edades P22 · margen P23 · en casa P26
Datos	Calidad: enlace en el pie de página (es transparencia, no navegación principal)

Motivos

1. **La telemetría pasa a Clasificación**, porque es la vuelta de clasificación. Así desaparece el malentendido de verla en una pestaña de carrera.
2. **Neumáticos y Boxes se fusionan en «Estrategia»**: la parada y el compuesto son la misma decisión, y el detector de undercut necesita las dos cosas.
3. **«Ritmo» deja de ser un muro de 20 líneas**: el violín es la vista principal y el detalle vuelta a vuelta es secundario.
4. **5 pestañas en lugar de 7**, que caben en 375 px sin desplazamiento.
5. **Estado compartido entre pestañas**: los pilotos seleccionados en la URL (`?d=VER,NOR`) se mantienen en Carrera, Ritmo y Estrategia. Hoy cada gráfico tiene sus propias casillas.
6. **Récords se convierte en «Historia»**, con subsecciones. Es donde viven las vistas por eras.
7. **Circuitos y Comparar cubren los dos huecos de entidad más evidentes**. Con 7 entradas en la navegación principal hay que revisar el menú en móvil (hoy se desplaza en horizontal).
8. **Enlaces cruzados**: al pulsar una línea o barra (ECharts `on('click')`) se abre la ficha del piloto, del equipo o de la carrera, y los nombres de circuito de las tablas enlazan a P24.

9.7 Notas para el informe LaTeX

Correspondencia con el TFG

Figura del TFG	Elemento de la web
5.23	G1
5.24-5.25	G16 y la tabla de récords
5.26	KPIs de temporada y tablas
5.27	G3
5.28	G4
5.29	G5 y G6
5.30	G7
5.31	G8
5.32	G9
5.33	KPIs del piloto, G12, G13 y G14

Figura del TFG	Elemento de la web
----------------	--------------------

Extras sin equivalente en el TFG: G2, G10, G11, G15, la tabla de pasos por el pit lane y /quality.

Mejoras frente al TFG que se pueden defender

- Accesibilidad: resumen y tabla en cada gráfico.
- Bilingüe.
- Sin licencia ni cuenta de Power BI.
- Filtro de vueltas neutralizadas más riguroso: incluye la bandera roja y la heurística del 150 %.
- Letras en los compuestos y texturas en los compañeros.
- Paleta apta para daltonismo.
- Todas las páginas funcionan sin JS salvo los gráficos, con formularios GET.

Limitaciones heredadas del TFG que conviene reconocer

- La métrica de clasificación mezcla segmentos.
- El «tiempo de parada» es en realidad el tiempo en el pit lane.
- Los puntos no están normalizados entre eras.

Capturas: tomadas el 29/09/2026 a 640 px de ancho CSS con escala de dispositivo 1,25 (PNG de 480 a 1 280 px de ancho). El recorte de la captura del navegador integrado impedía hacerlas a más ancho.

CAPÍTULO 10

Gráficos y récords de otras webs y proyectos

Fecha de la investigación: 2026-09-29. Objetivo: catalogar qué gráficos, estadísticas y récords ofrecen otras webs, apps y proyectos de F1, contrastarlos con los datos que ya tiene `f1-data-platform` y proponer una lista priorizada de ideas para la web (<https://f1-data-platform.vercel.app/es>).

10.1 Metodología y limitaciones

- Búsquedas web y lectura de páginas públicas. Las descripciones de gráficos están redactadas con palabras propias; no se reproducen textos.
- **formula1db.com** responde 403 (Cloudflare) y su robots.txt prohíbe el scraping: **no se ha accedido a su contenido**. Lo que se dice de ella procede solo de los **títulos y URL indexados por el buscador** (que revelan su estructura de secciones y récords).
- **statsf1.com** devuelve error 500 a los clientes automáticos; su catálogo se reconstruye igualmente a partir de títulos indexados.
- Kaggle, Reddit, X/Instagram y Tableau Public requieren JS o sesión: se usan resultados de búsqueda y páginas de descripción, no el contenido interactivo.
- La disponibilidad de datos se ha comprobado con consultas `read_only` sobre `data/gold/f1.duckdb` (ver §1). Consultas de prueba: vueltas lideradas por piloto (Hamilton 5 529, Schumacher 5 111), victoria desde la posición de parrilla más retrasada (Watson, Long Beach 1983, desde la 22.^a), ganador más joven (Verstappen, 18,6 años) y grand slams (Clark 8). Es decir, los récords de este tipo **se pueden calcular ya**.

10.2 Qué datos tenemos (gold)

Dato	Tabla	Cobertura
Resultados de carrera y sprint: posición, parrilla, puntos, pole, vuelta rápida (VR), piloto del día, grand slam, posiciones ganadas, motivo de abandono, número de paradas	<code>fact_race_result</code>	1950–2026
Clasificación: Q1/Q2/Q3, mejor tiempo, % sobre la pole	<code>fact_qualifying_result</code>	1950–2026 (Q1–Q3 desde 2006)
Posición y tiempo de cada vuelta, gap al líder	<code>fact_laptimes</code>	1950–2026 (1 164 de 1 172 carreras; gap en 1 141)
Sectores S1/S2/S3 y speed trap por vuelta	<code>fact_laptimes</code>	2018–2026 (187 carreras)
Compuesto, stint y edad del neumático	<code>fact_laptimes</code>	2011–2026 (325 carreras)
Banderas por vuelta (SC, VSC, amarilla, roja), vuelta borrada	<code>fact_laptimes</code>	SC desde 1993 (268 carreras con SC)

Dato	Tabla	Cobertura
Paradas con duración	fact_pit_stops	1994–2026 (612 carreras)
Entradas y salidas del pit lane	fact_pit_lane_passes	parcial
Telemetría de la vuelta rápida de clasificación (velocidad, rpm, marcha, acelerador, freno, DRS, x/y)	fact_quali_telemetry	2024–2026 (63 carreras)
Clasificación del campeonato tras cada carrera (pilotos y constructores)	fact_driver_standing, fact_constructor_standing	1950–2026
Agregados de carrera deportiva, temporada y duelos entre compañeros	agg_*	completo
Dimensiones: piloto (nacimiento, fallecimiento, nacionalidad), circuito (coordenadas, tipo, sentido, longitud, curvas), GP, motor, neumático	dim_*	completo

No tenemos: telemetría de carrera (solo la vuelta de clasificación), clima y temperatura de pista, adelantamientos oficiales en pista (solo se pueden **aproximar** con cambios de posición vuelta a vuelta), minisectores, posiciones GPS en carrera, libres, radio, mensajes de dirección de carrera, penalizaciones como entidad, puntos del carné de la FIA, presupuestos o costes de daños.

10.3 Catálogo por fuente

Leyenda de la columna «¿Tenemos?»: ✓ sí, con los datos actuales · ● parcial (solo algunas temporadas o con aproximación) · ✗ no. Originalidad y valor para el público general: escala de 1 a 3.

10.3.1 formula1db.com (solo estructura indexada)

URL: <https://www.formula1db.com/record> · <https://www.formula1db.com/races> · <https://www.formula1db.com/seasons>

Los títulos indexados muestran una **matriz de récords muy sistemática**: `record/{driver|team|constructor}/{estadística}/{race}/{tipo}/{ámbito}`, donde:

- estadística: win, podium, point, pole-to-win, lead (vueltas lideradas), finish, start, hat-trick, grand-slam, combined-grand-slam (equipo)...
- tipo: most (total), ratio (porcentaje), consecutive (racha), age/youngest/oldest, chronicle (evolución histórica de quién tenía el récord);
- ámbito: all (carrera deportiva), season, circuit, team.

Por carrera: resultado, lap chart, tiempos por vuelta, VR, estadísticas de neumáticos y de paradas. Por temporada: vueltas lideradas y paradas.

Elemento	Pregunta	¿Tenemos?	Orig.	Valor
Récords con filtro de ámbito (total, temporada, circuito, equipo) y de tipo (total, %, racha, edad)	¿Quién es el mejor en X, en una temporada o en un circuito?	✓	2	3
«Chronicle»: evolución del poseedor de cada récord	¿Cómo ha pasado el récord de victorias de Fangio a Hamilton?	✓	3	3
% de puntos obtenidos sobre el máximo posible	¿Quién aprovechó más su coche?	✓	2	2
Pole-to-win por circuito	¿Dónde es más decisiva la pole?	✓	2	2

Elemento	Pregunta	¿Tenemos?	Orig.	Valor
Vueltas lideradas por temporada o carrera y su %	¿Quién dominó realmente la carrera?	✓	1	3
Estadísticas de neumáticos y paradas por carrera y temporada	¿Qué compuesto funcionó? ¿Qué equipo para más rápido?	✓ (2011+/1994+)	1	2

10.3.2 statsf1.com

URL: <https://www.statsf1.com/en/statistiques/pilote.aspx> · <https://www.statsf1.com/en/statistiques/constructeur.aspx> · <https://www.statsf1.com/en/statistiques/moteur.aspx>

Estadísticas para **pilotos, constructores, motores, neumáticos, naciones y circuitos**, con subcategorías: número de GP, cronología, edad, parrilla, victorias, poles, VR, puntos (incluido «con más constructores distintos»), podios, **vueltas en cabeza** (total, por GP, en un año, consecutivas, GP liderados en un año), **km en cabeza**, y un bloque «misc» con **hat trick** y **grand chelem**. En cada GP hay «lap by lap» y resúmenes de vueltas en cabeza por temporada.

Elemento	Pregunta	¿Tenemos?	Orig.	Valor
Km en cabeza (vueltas lideradas × longitud del circuito)	¿Quién ha liderado más distancia?	✓ (course_length)	3	2
Estadísticas por motor y por marca de neumáticos	¿Qué motor ha ganado más?	✓ (engine_manufacturer_id, tyre_manufacturer_id)	2	2
Estadísticas por nación (pilotos y GP)	¿Qué país tiene más victorias?	✓	1	3
Tabla «vuelta a vuelta» con el orden en cada vuelta	¿Quién iba delante en la vuelta N?	✓	1	2
Puntos con más constructores distintos	¿Quién ha puntuado con más equipos?	✓	3	1

10.3.3 FIA: documentos de cronometraje (PDF)

URL: <https://www.fia.com/eventtiming-information> · ejemplos: [Race Lap Chart](#), [Race History Chart](#), [Race Lap Analysis](#)

Documento	Qué muestra	¿Tenemos?	Valor para la web
Lap Chart	Matriz posición × vuelta con el número de coche	✓ (ya lo tenemos como lap chart de posiciones)	ya cubierto
History Chart	Por vuelta: coche, gap al líder y tiempo de vuelta, y «PIT» cuando para	✓ (gap_to_leader_ms, is_pit_in_lap)	base del race trace (gráfico de huecos), que nos falta
Lap Analysis	Tiempos de cada piloto vuelta a vuelta, con las vueltas de parada marcadas	✓	cubierto por los tiempos por vuelta
Pit Stop Summary	Parada, vuelta, hora, duración en el pit lane y total acumulado	✓ (1994+)	ya tenemos la media por constructor; falta la tabla y el ranking de paradas
Best Sector Times	Mejor S1/S2/S3 de cada piloto y vuelta ideal (suma de los mejores sectores)	✓ (2018+)	nuevo: vuelta ideal frente a vuelta real
Maximum Speeds / Speed Trap	Velocidad punta en la trampa y en los puntos intermedios	● (solo speed trap, 2018+)	nuevo: ranking de velocidad punta
Classification	Resultado oficial	✓	cubierto

10.3.4 formula1.com y F1 Insights powered by AWS

URL: <https://www.formula1.com/en/results/awards> · <https://aws.amazon.com/sports/f1/> · <https://corp.formula1.com/new-f1-insights-powered-by-aws-will-help-formula-1-fans-make-sense-of-split-second-decisions-on-the-track/>

Archivo de resultados de formula1.com: resultado de la carrera, parrilla de salida, clasificación, sprint, **vueltas rápidas** (ranking de la VR de cada piloto), **resumen de paradas**, y premios (**DHL Fastest Lap** y **DHL Fastest Pit Stop**, piloto del día).

F1 Insights (unos 22 gráficos de TV):

Gráfico	Qué responde	¿Tenemos?	Comentario
Track Dominance	¿Quién es más rápido en cada tramo del circuito?	● (telemetría de clasificación 2024+, x/y)	factible: trazado coloreado por el piloto más rápido en cada tramo
Car Performance Scores	Rendimiento en curva, en recta y de manejo por equipo	● (aproximable con el speed trap y los sectores 2018+)	una versión simplificada, «velocidad punta frente a tiempo en curva», es factible
Driver Season Performance (7 métricas: ritmo de clasificación, salidas, vuelta 1, ritmo de carrera, gestión de neumáticos, paradas, adelantamientos)	Perfil del piloto en una temporada	● (salidas y vuelta 1 con posiciones; ritmo con tiempos; neumáticos 2011+)	radar del piloto , muy vistoso
Battle Forecast	¿En cuántas vueltas alcanzará al de delante?	● (se puede reconstruir a posteriori con gaps)	es un gráfico en directo; poco sentido en un histórico
Pit Window, Predicted Pit Strategy, Undercut Threat, Pit Strategy Battle	Estrategia en directo	●	se puede hacer un análisis a posteriori de undercut y overcut con posiciones antes y después de la parada
Tyre Performance	Degradación y vida útil del neumático	✓ (2011+)	curva de degradación por compuesto
Time Lost, Pit Lane Performance	Tiempo perdido en la parada	✓ (1994+)	cubierto en parte
Braking Performance, Corner Analysis, Exit Speed	Frenada y curvas	● (telemetría de clasificación 2024+)	análisis de curva en la vuelta de pole
Close to the Wall, Car Development, Alternative Strategy, Projected Knockout Time	Varios	✗	sin datos
Fastest Driver (ranking desde 1983)	¿Quién es el más rápido de la historia a igualdad de coche?	✓ (clasificación frente al compañero, desde 1950)	ranking «a igualdad de coche» , de gran valor

10.3.5 F1 TV / Live Timing, MultiViewer y apps de cronometraje (Undercut, F1Dash, Formula Live Pulse, formula1dashboard)

URL: <https://multiviewer.app/> · <https://github.com/JustAman62/undercut-f1> · <https://f1dash.net/> · <https://www.f1livepulse.com/en/features/> · <https://app.formula1dashboard.com/>

Elementos comunes: torre de tiempos, driver tracker (mapa en vivo), página de stints de neumáticos, historial de tiempos por vuelta, **modo de comparación de gaps entre dos pilotos**, radio, dirección de carrera, gap al líder, delta frente a la media, calculadora de campeonato, **tracker de adelantamientos**, box plots de ritmo por equipo, «track DNA» y consistencia (formula1dashboard), costes de daños y elementos de unidad de potencia usados.

Elemento	¿Tenemos?	Valor
Gap entre dos pilotos elegidos, vuelta a vuelta	✓ (1950+)	3

Elemento	¿Tenemos?	Valor
Repetición animada de la carrera (posiciones o gaps vuelta a vuelta con play/pause)	✓	3
Tracker de adelantamientos (aproximado por cambios de posición fuera de las paradas)	●	2
Calculadora de «¿quién puede ganar aún el título?»	✓ (clasificación por carrera y calendario)	3
Consistencia (dispersión de las vueltas limpias)	✓	2
Radio, dirección de carrera, clima, elementos de la PU, costes de daños	✗	—

10.3.6 f1-tempo.com y GP Tempo

URL: <https://www.f1-tempo.com/> · <https://www.gp-tempo.com/> · [hilo en F1technical](#)

Superposición de telemetría de varias vueltas (velocidad, acelerador, freno, rpm, marcha, DRS), **mapa del circuito coloreado por el piloto más rápido en cada punto**, tiempos por vuelta por stint, dominio por sectores a lo largo de la temporada y **estado completo en la URL para compartir** cualquier comparativa.

- ¿Tenemos? ●: telemetría solo de la vuelta rápida de clasificación 2024–2026; sectores 2018+.
- Ideas: comparar la telemetría de 2 pilotos con una curva de **delta acumulado**; **mapa de dominio**; «**quién gana cada sector**» a lo largo de la temporada; compartir el estado mediante la URL, que es barato en Next.js con query params.

10.3.7 TracingInsights (web, GitHub, redes y Substack)

URL: <https://tracinginsights.com/> · <https://tracinginsights.com/analysis/lap-chart/> · <https://github.com/TracingInsights/2026> · <https://tracinginsights.substack.com>

El catálogo más amplio: ritmo de carrera, **ritmo en tandas largas**, **degradación de neumáticos** (con corrección de combustible), **race trace**, lap chart, cambios de posición, **tiempo en tráfico** («drivers in traffic»), **vueltas en el top 10**, **valoración de la salida**, vuelta ideal, **mapa de calor de tiempos por vuelta**, velocidad punta en la trampa, diagrama g-g y aceleraciones lateral y longitudinal, % de acelerador a fondo, adelantamientos, **contendientes al título**, puntos del carné, temperatura de pista. Herramientas: corrección de combustible, ocultar outliers y mostrar el estado de pista (SC/VSC) sobre los gráficos.

Elemento	¿Tenemos?	Valor
Race trace (gap a un ritmo de referencia)	✓	3
Degradación con corrección de combustible	✓ (2011+)	2
Mapa de calor de tiempos (piloto × vuelta)	✓	2
Vueltas en el top 10 y tiempo en tráfico (gap < 1 s al de delante)	✓ (con gaps)	2
Valoración de la salida (posiciones ganadas en la vuelta 1)	✓ (parrilla y posición en la vuelta 1)	3
Vuelta ideal	✓ (2018+)	2
Franjas SC/VSC sobre los gráficos de la carrera	✓ (1993+)	3
Diagrama g-g, aceleraciones, temperatura	✗/●	1

10.3.8 Galería de ejemplos de FastF1

URL: https://docs.fastf1.dev/gen_modules/examples_gallery/index.html

16 ejemplos: estilo por piloto, **trazado con curvas numeradas**, dispersión de tiempos por vuelta coloreada por compuesto, distribución de tiempos (violines), cambios de posición, **comparativa de ritmo por equipo** (box plot), estrategias de neumáticos, resumen de clasificación (barras de gap a la pole), «¿quién

puede ganar aún el mundial?», mapa de calor de posiciones del campeonato (piloto × ronda), resumen de temporada, velocidades superpuestas, marchas sobre el trazado, velocidad con curvas anotadas, velocidad sobre el mapa y trazado medio de velocidad entre temporadas.

- Casi todo es ✓ con nuestros datos (la telemetría, ●). Lo que nos falta: mapa de calor del campeonato, «¿quién puede ganar aún?», box plot por equipo, marchas y velocidad sobre el trazado, curvas numeradas y resumen de clasificación con barras de gap.

10.3.9 OpenF1

URL: <https://openf1.org/>

API con 18 endpoints: telemetría a unos 3,7 Hz, posición GPS, intervalos, vueltas con minisectores, paradas, stints, radio, clima, dirección de carrera, adelantamientos, parrilla y clasificación del campeonato. Proyectos de la comunidad: circuito de LED, visualización en VR y generador de comentarios con IA.

- Interesante como **fuentes futuras**: clima, adelantamientos y dirección de carrera desde 2023. Hoy ✗ para nosotros.

10.3.10 formula1points.com

URL: <https://www.formula1points.com/>

Simulador de campeonato, **conversor de sistemas de puntuación** («¿quién habría ganado con los puntos de 1991?»), comparativa de pilotos y de compañeros, estadísticas de todos los tiempos, rachas, outliers, **animaciones** (carreras de barras), juegos (tipo Wordle: Driverle, Teammatle...), creador de tier lists.

Elemento	¿Tenemos?	Orig.	Valor
Campeonatos recalculados con otros sistemas de puntos	✓ (resultados completos)	3	3
Bar chart race de victorias o puntos históricos	✓	2	3
Juego tipo Wordle de pilotos	✓ (dimensiones)	3	2 (engagement)

10.3.11 RaceFans (datos interactivos de cada GP)

URL: [2026 Azerbaijan GP interactive data](#)

Por carrera: lap chart con resaltado del piloto al hacer clic, **gráfico de huecos respecto al tiempo medio de vuelta del ganador** (race history clásico), tiempos por vuelta sin las vueltas muy lentas, tablas de vueltas rápidas (tiempo, gap, **velocidad media** y vuelta), estrategias de neumáticos y **tabla de paradas con gap a la más rápida**.

- Todo ✓. Lo único que nos falta es el **race history/gaps**, más la velocidad media de la VR (derivable de `course_length` y del tiempo).

10.3.12 The Race / F1MATHS / lapttime.me (análisis de ritmo)

URL: <https://www.f1technical.net/news/28301> · <https://lapttime.me/lapttime-me-f1-2026-fuel-adjusted-pace-analysis-miami/>

Ranking de ritmo de carrera por **mediana de vueltas limpias** y corrección de combustible, **pecking order** por equipos a lo largo de la temporada (gap medio al más rápido, en %).

- ✓ Con los tiempos por vuelta (filtrando SC, entradas y salidas de boxes y la vuelta 1). **Pecking order de temporada** (gap de clasificación % por equipo y carrera): ✓ con `gap_to_pole_pct`, que ya existe.

10.3.13 Autosport Forix / Motorsport Stats

URL: <https://www.forix.com/> · <http://www.forix.com/8w/6thgear/index.html>

Base de datos profesional: **gap charts** desde 2012, tiempos de vuelta con gráficos por piloto, «gap analysis». Da servicio de datos a medios.

- ✓ Ya lo cubrimos con más años (gaps desde 1950).

10.3.14 fermanpre/f1-stats (estilo Atlas F1) y otros «gap charts»

URL: <https://github.com/fermanpre/f1-stats> · <https://github.com/izmishi/f1-lap-chart> · <https://davidor.github.io/formula1-lap-charts/> · <https://f1-visualization.vercel.app/>

- **Gap chart en tiempo absoluto que incluye a los doblados**, ranking de vueltas de todo el campo (todas las vueltas ordenadas de la más rápida a la más lenta) y resúmenes por piloto.
- izmishi: gap respecto a un coche imaginario a ritmo constante, lo que evita que la línea del líder sea plana.
- f1-visualization: **repetición animada** del gap al líder con play/pause y selección de vuelta.
- Todo ✓.

10.3.15 ChicaneF1 y grandprix.com (enciclopedias)

URL: <https://www.chicanef1.com/> · <https://www.grandprix.com/encyclopedia>

Gráficos de puntos de la temporada, **matrices de participación** (piloto × carrera), perfiles de chasis, motores, diseñadores, patrocinadores y dorsales, carreras no puntuables (1945–1983) y buscador de récords.

- ✓ Matriz de participación y gráfico de puntos. ✗ Chasis, diseñadores y patrocinadores.

10.3.16 Wikipedia

URL: https://en.wikipedia.org/wiki/List_of_Formula_One_driver_records · https://en.wikipedia.org/wiki/List_of_Formula_One_race_records · [https://en.wikipedia.org/wiki/Grand_Slam_\(Formula_One\)](https://en.wikipedia.org/wiki/Grand_Slam_(Formula_One))

- **Matriz de resultados del campeonato** (piloto × GP) con el color de cada celda según el resultado: oro para la victoria, plata, bronce, verde si puntúa, azul si termina sin puntos, morado si no se clasifica, negro si es descalificado... con superíndices para la pole, la VR y la posición en el sprint. Es el formato más reconocible por el aficionado. ✓ con nuestros datos.
- Lista de récords de pilotos organizada en 12 bloques (ver §3); récords de carrera: abandonos, paradas, adelantamientos, **márgenes de victoria**, velocidades medias, duración, **safety cars**, **banderas rojas**, calendario, temperatura y asistencia.

10.3.17 Kaggle, Tableau Public, comunidad de Power BI, Observable

URL: <https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020> · <https://public.tableau.com/app/profile/sports.chord/viz/VisualHistoryofF1Tableau/VisualHistoryofFormula1Tableau> · <https://community.fabric.microsoft.com/t5/Data-Stories-Gallery/Formula-1-analysis-1950-2021/m-p/2052947> · <https://community.fabric.microsoft.com/t5/Data-Stories-Gallery/Power-BI-Animated-Bar-Chart-Race-for-Formula-1-season-2021/m-p/2256794> · <https://observablehq.com/@dis-2025-fall/f1-top-15-bump-chart>

Patrones repetidos: gantt de la historia de cada GP (qué años se celebró), mapa de circuitos (ya lo tenemos), distribución de carreras por mes, nacionalidades en gráfico de círculos, barras apiladas de motivos de abandono, **bar chart race**, **bump chart** (ranking a lo largo del tiempo, p. ej. ranking ELO de pilotos) y dashboard con 4 vistas (pilotos, equipos, temporadas y circuitos).

- ✓ Todo. De más valor: **gantt de la historia de los GP y los circuitos**, **evolución de los motivos de abandono (fiabilidad)**, **bump chart del campeonato** y **ranking ELO**.

10.3.18 Redes (F1Visualized, F1_charts, r/formula1, r/F1Technical)

URL: <https://x.com/f1visualized> · https://x.com/F1_charts · <https://f1bythenumbers.com/r-formula1-or-r-f1art/>

- F1Visualized (~120 mil seguidores en Instagram): resultados y estadísticas en **pixel art**; demuestra que la estética atrae más que el dato.
- En r/formula1 lo que más triunfa es el contenido visual; entre lo estadístico, las comparativas históricas simples (p. ej. «líder del campeonato tras N carreras en los últimos 10 años»).
- En r/F1Technical dominan la **degradación con corrección de combustible** (vida del neumático frente a tiempo corregido) y los cruces entre compuestos.

10.3.19 Modelos de valoración de pilotos (f1metrics, F1 Mathematical Model, F1-analysis)

URL: <https://f1metrics.wordpress.com/2019/11/22/the-f1metrics-top-100/> · <https://f1mathematicalmodel.com/2026/08/17/2026-mid-season-f1-driver-ratings/> · <https://f1-analysis.com/2025/12/09/f1-2025-mathematical-driver-rankings/>

Aíslan el efecto del coche comparando solo con el compañero de equipo y encadenan esos duelos entre temporadas para obtener un ranking histórico; corrigen por edad y experiencia.

- ✓ Tenemos `agg_teammate_h2h` (desde 1950). Un **ranking «a igualdad de coche»** sencillo (red de duelos entre compañeros) es muy original para una web pública.

10.3.20 Proyectos de GitHub parecidos

URL: <https://github.com/hugoogb/f1-tracker> · <https://github.com/gaurab6969/F1-ErgastEra-Archive> · <https://github.com/PeterCollavino7/f1-analysis> · <https://github.com/smit-sms/Formula-One-Data-Visualization-Dashboard>

f1-tracker (Next.js, FastAPI y PostgreSQL, desde 1950) es el más parecido a nuestro proyecto: progresión del campeonato, perfiles, comparativa de pilotos con **radar**, H2H de clasificación, posiciones vuelta a vuelta, estrategia de neumáticos, páginas de circuito con trazado SVG e historial, **récords**, **buscador universal** y colores de equipo.

- Nuestro diferencial: calidad de datos y validación entre fuentes, telemetría, violines de ritmo y matriz de compuestos. Nos falta: **páginas de circuito**, **radar**, **buscador** y **comparador libre entre dos pilotos cualesquiera**.

10.3.21 Prensa (FT, NYT, Guardian, Datawrapper)

No se han encontrado gráficos interactivos de F1 destacables de estos medios en la búsqueda. Lo que aparece son piezas sobre el dominio de una era (p. ej. el % de victorias de Mercedes en 2014 o de Red Bull en 2023; [Statista](#)) y rankings de temporadas más dominantes. Idea derivada: el «**índice de dominio**» por **temporada** (% de victorias o de puntos del mejor equipo), que ✓ podemos calcular.

10.4 Catálogo de récords y estadísticas curiosas

Todas se calculan con `fact_race_result`, `fact_laptimes`, `fact_qualifying_result`, `fact_*_standing`, `agg_*` y `dim_driver/dim_race`. «Ámbito» indica en qué variantes merece la pena ofrecerlas (C = carrera deportiva, T = temporada, Ci = circuito, E = equipo).

#	Récord / estadística	Definición	Fuente de inspiración	¿Calculable hoy?	Ámbito
1	Victorias, poles, VR, podios, puntos, salidas	totales	todas	✓ (ya en la web)	C/T/Ci/E
2	Porcentajes (victorias, poles, podios, puntos sobre el máximo posible)	con un mínimo de salidas	formula1db, Wikipedia	✓ (win_rate_pct ya existe)	C/T
3	Rachas : victorias, podios, poles, VR, carreras en los puntos, carreras terminadas y salidas consecutivas	secuencias	formula1db, Wikipedia	✓	C/Ci
4	Hat trick (pole, victoria y VR)		statsf1, formula1db	✓	C/T/E
5	Grand chelem (hat trick y liderar todas las vueltas)		statsf1, Wikipedia	✓ (grand_slams)	C/T/Ci
6	Victorias desde la pole y % de conversión de la pole		Wikipedia, formula1db	✓	C/T/Ci
7	Vueltas lideradas , % de vueltas lideradas, carreras lideradas, km en cabeza	con position = 1	statsf1, formula1db	✓ (1 164 carreras con vueltas)	C/T/Ci
8	Vueltas consecutivas en cabeza	secuencia a lo largo de varias carreras	statsf1	✓	C
9	Liderar vueltas sin ganar nunca / más vueltas lideradas sin victoria		Wikipedia	✓	C
10	Edad : más joven y más veterano en debutar, puntuar, subir al podio, ganar, lograr la pole o ser campeón	con date_of_birth	Wikipedia, formula1db	✓	C
11	Remontada máxima : victoria o podio desde la parrilla más retrasada; más posiciones ganadas en una carrera	grid_position, positions_gained	Wikipedia	✓	C/Ci
12	Carreras hasta la primera victoria, pole o podio; más carreras sin ganar		Wikipedia	✓	C
13	Más podios sin ganar; más puntos sin ser campeón		Wikipedia	✓	C
14	Compañeros derrotados : número de compañeros a los que se ganó en la temporada	con agg_teammate_h2h	f1metrics, formula1points	✓	C
15	Duelos en clasificación y carrera contra el compañero (% y rachas)		formula1points, f1-tracker	✓ (ya en la web)	T
16	Victorias con más equipos distintos, puntos con más constructores distintos		statsf1	✓	C
17	Márgenes de victoria : el más ajustado y el más amplio (tiempo y vueltas)	gap_ms y gap_laps del 2.º	Wikipedia (race records)	✓	Ci/T
18	Mayor diferencia en la pole (en %)	gap_to_pole_pct	Wikipedia	✓	T
19	Carreras con más y menos coches clasificados; motivos de abandono	reason_retired	Wikipedia	✓	T
20	Carreras con más safety cars o vueltas bajo SC; banderas rojas	con las banderas por vuelta	Wikipedia	● (1993+)	T/Ci
21	Parada más rápida por temporada y equipo; más paradas en una carrera	fact_pit_stops	DHL Fastest Pit Stop	✓ (1994+; la duración total en el pit lane no es el tiempo parado)	T/E
22	Más cambios de líder en una carrera	cambios de position = 1	the-race, statsf1	✓	Ci/T
23	Adelantamientos aproximados por carrera o piloto (cambios de posición fuera de las vueltas de parada, sin contar abandonos)	en fact_laptimes	OpenF1, TracingInsights	● (es aproximación: no cuenta adelantamientos intermedios)	Ci/T
24	Mejor salida (posiciones ganadas en la vuelta 1)	comparando la parrilla con la posición en la vuelta 1	AWS, TracingInsights	✓	C/T
25	Campeón decidido en la última carrera , en qué ronda se decidió y el título más ajustado	drivers_championship_decider, standings	Wikipedia	✓	T

#	Récord / estadística	Definición	Fuente de inspiración	¿Calculable hoy?	Ámbito
26	Líder del campeonato tras la carrera N frente al campeón final («¿el líder tras N carreras gana el título?»)	con los standings	r/formula1	✓	T
27	Índice de dominio (% de victorias o puntos del mejor equipo)		Statista, prensa	✓	T
28	Porcentaje de dobles (1-2) por equipo	one_two_finishes	formula1db (equipo)	✓	C/T
29	Victorias en casa (piloto que gana su GP nacional)	nacionalidad del piloto = país del GP	statsf1 (naciones)	✓	C
30	Estadísticas por motor y por neumático		statsf1	✓	C/T
31	Vuelta rápida más rápida por circuito y velocidad media	course_length / tiempo	RaceFans, Wikipedia	✓ (ojo: trazados distintos del mismo circuito)	Ci
32	Victorias distintas: más GP distintos o más circuitos distintos ganados		statsf1	✓	C
33	Temporadas consecutivas con victoria o con pole		Wikipedia	✓	C
34	Pilotos del día (desde 2016)	is_driver_of_the_day	formula1.com	✓	C/T
35	Récords de sprint (victorias, poles, puntos)	session_type = 'SPRINT'	Wikipedia	✓	C
36	«Chronicle»: cronología del poseedor de un récord	cálculo acumulado	formula1db	✓	C
37	Temperatura de pista, asistencia		Wikipedia	✗	—

10.5 Lista consolidada y deduplicada de ideas (ordenada por valor/esfuerzo)

Valor: interés para el público general y para la defensa del TFG (1 a 5). Esfuerzo: S = horas, M = 1–2 días, L = más de 3 días. «Hoy» indica si es factible con los datos actuales.

#	Idea	Tipo	Inspiración	Valor	Esfuerzo	Hoy
1	Race trace / gráfico de huecos (gap de cada piloto respecto al ritmo medio del ganador, con marcas de parada, SC/VSC y VR)	carrera	FIA History Chart, RaceFans, TracingInsights, izmishi	5	S	✓ 1950+
2	Franjas SC/VSC/bandera roja sombreadas en todos los gráficos por vuelta que ya existen	carrera	TracingInsights, The Field	4	S	✓ 1993+
3	Matriz de resultados de temporada estilo Wikipedia (piloto × GP coloreada por resultado, con superíndices de pole y VR)	temporada	Wikipedia, ChicaneF1	5	S	✓
4	Récords ampliados: rachas, hat tricks, victorias desde la pole y % de conversión, vueltas y km lideradas, edad (más joven y más veterano), remontadas, carreras hasta la primera victoria, márgenes de victoria. Con filtros de ámbito (carrera deportiva, temporada, circuito, equipo) y de tipo (total, %, racha, edad)	récords	formula1db, statsf1, Wikipedia	5	M	✓
5	Mapa de calor o bump chart del campeonato (posición en la clasificación × ronda) y «¿quién puede ganar aún?»	temporada	FastF1, Observable	4	S	✓
6	Tabla de paradas y ranking de la parada más rápida (por carrera y temporada, con gap a la más rápida)	carrera/temporada	FIA Pit Stop Summary, RaceFans, DHL	4	S	✓ 1994+

#	Idea	Tipo	Inspiración	Valor	Esfuerzo Hoy
7	Valoración de la salida (posiciones ganadas o perdidas en la vuelta 1, por carrera y temporada)	carrera/piloto	AWS, TracingInsights	4	S ✓
8	Vueltas lideradas por carrera (barra o anillo del % de vueltas en cabeza de cada piloto) y cambios de líder	carrera	statsf1, formula1db	4	S ✓
9	Comparador libre de dos pilotos (en una carrera: gap entre ambos vuelta a vuelta; en su carrera deportiva: estadísticas enfrentadas y radar)	pilotos	Pitwall, f1-tracker, Undercut	5	M ✓
10	Campeonatos con otros sistemas de puntos («¿quién habría ganado en 2008 con los puntos de 2010?»)	temporada	formula1points	5	M ✓
11	Mejores sectores y vuelta ideal frente a la vuelta real; ranking de speed trap	carrera	FIA Best Sector Times y Speed Trap	3	S ✓ 2018+
12	Mapa de dominio del circuito (trazado coloreado por el piloto más rápido en cada tramo) y delta acumulado entre dos vueltas de clasificación	carrera	AWS Track Dominance, F1 Tempo, GP Tempo	4	M ● 2024+
13	Curva de degradación por compuesto (edad del neumático frente al tiempo con corrección de combustible)	carrera	r/F1Technical, TracingInsights	3	M ✓ 2011+
14	Pecking order de la temporada (gap % a la pole o ritmo mediano por equipo y carrera, en líneas)	temporada	The Race, F1MATHS	4	S ✓
15	Páginas de circuito (historial de ganadores, récord de vuelta, % de victorias desde la pole, SC por edición, trazado)	circuito	f1-tracker, statsf1	4	M ✓
16	Repetición animada (play/pause) del lap chart o del race trace	carrera	f1-visualization, F1 TV	4	M ✓
17	Bar chart race histórico (victorias o títulos acumulados por piloto, equipo, motor o nación)	récords	Power BI, Tableau, formula1points	4	M ✓
18	Mapa de calor de tiempos por vuelta (piloto × vuelta) y consistencia (dispersión de las vueltas limpias)	carrera	TracingInsights, formula1dashboard	3	S ✓
19	«Chronicle» de récords (línea escalonada con quién tuvo el récord de victorias o poles en cada momento)	récords	formula1db	4	M ✓
20	Ranking «a igualdad de coche» (red de duelos entre compañeros, al estilo ELO o f1metrics simplificado)	pilotos	f1metrics, AWS Fastest Driver	5	L ✓
21	Radar de temporada del piloto (clasificación frente al compañero, salidas, ritmo de carrera, consistencia, paradas del equipo, adelantamientos aproximados)	pilotos	AWS Driver Season Performance, f1-tracker	4	M ●
22	Estadísticas por motor, neumático y nación , y victorias en casa	récords	statsf1	3	S ✓
23	Fiabilidad : evolución de los motivos de abandono por década o equipo	histórico	Tableau, Wikipedia	3	S ✓
24	Índice de dominio por temporada y ranking de las temporadas más dominantes	histórico	prensa, Statista	3	S ✓
25	Gantt de la historia de los GP y circuitos (qué años se celebró cada GP)	mapa/histórico	Tableau	3	S ✓
26	Undercut/overcut a posteriori (posiciones antes y después del ciclo de paradas entre rivales directos)	carrera	AWS Undercut Threat	3	M ✓ 1994+
27	Adelantamientos aproximados y tiempo en tráfico	carrera	OpenF1, TracingInsights	3	M ●
28	Estado de la vista en la URL para compartir cualquier comparativa	UX	GP Tempo	3	S ✓

#	Idea	Tipo	Inspiración	Valor	Esfuerzo Hoy
29	Buscador universal (pilotos, equipos, circuitos)	UX	f1-tracker	3	S ✓
30	Juego tipo Wordle o quiz de pilotos	engagement	formula1points	2	M ✓
31	Clima, temperatura de pista, radio, dirección de carrera, adelantamientos oficiales	varios	OpenF1, F1 TV	3	L ✗ (requeriría ingerir OpenF1, 2023+)
32	Diagrama g-g, telemetría de carrera, minisectores, Close to the Wall	telemetría	TracingInsights, AWS	2	L ✗

Top 5 recomendado a corto plazo (máximo valor y mínimo esfuerzo, todo factible hoy): (1) race trace, (2) franjas SC/VSC en los gráficos existentes, (3) matriz de resultados estilo Wikipedia, (4) récords ampliados (rachas, hat trick, vueltas lideradas, edad, remontadas), (5) mapa de calor del campeonato y «¿quién puede ganar aún?». Después: el comparador libre de pilotos y los campeonatos con otros sistemas de puntos, que son los más «virales».

Diferencial frente a la competencia: pocas webs gratuitas ofrecen **gaps y posiciones vuelta a vuelta desde 1950** (Pitwall compara desde 1996, f1-visualization desde 1996, Forix desde 2012 y TracingInsights desde 2018). Un race trace y una repetición animada para carreras de los años 50 a 90 serían una rareza atractiva, siempre avisando de la calidad del dato con la página de calidad que ya existe.

10.6 Fuentes consultadas

- formula1db.com (solo títulos indexados): <https://www.formula1db.com/record> · <https://www.formula1db.com/record/driver/lead/race/ratio/all/lap> · <https://www.formula1db.com/record/driver/hat-trick/race/chronicle> · <https://www.formula1db.com/record/driver/start/race/age/youngest> · <https://www.formula1db.com/races/2025-italian-grand-prix/stats/race/tyres> · <https://www.formula1db.com/seasons/2025/results?section=lead>
- statsf1.com (títulos indexados): <https://www.statsf1.com/en/statistiques/pilote.aspx> · <https://www.statsf1.com/en/statistiques/pilote/divers/hattrick.aspx> · <https://www.statsf1.com/fr/statistiques/pilote/divers/chelem.aspx> · <https://www.statsf1.com/fr/statistiques/pilote/entete/tour-consecutif.aspx> · <https://www.statsf1.com/en/statistiques/pilote/point/constructeur-different.aspx>
- FIA: <https://www.fia.com/eventtiming-information> y los PDF enlazados en §2.3
- formula1.com: <https://www.formula1.com/en/results/awards> · <https://www.formula1.com/en/results/2026/awards/fastest-pit-stops>
- AWS F1 Insights: <https://aws.amazon.com/sports/f1/> · <https://aws.amazon.com/blogs/machine-learning/accelerating-innovation-how-serverless-machine-learning-on-aws-powers-f1-insights/>
- F1 Tempo: <https://www.f1-tempo.com/> · GP Tempo: <https://www.gp-tempo.com/>
- TracingInsights: <https://tracinginsights.com/> · <https://tracinginsights.substack.com/p/ferrari-disaster-class-is-hard-compound>
- FastF1: https://docs.fastf1.dev/gen_modules/examples_gallery/index.html
- OpenF1: <https://openf1.org/>
- formula1points: <https://www.formula1points.com/>
- RaceFans: <https://www.racefans.net/2026/09/26/2026-azerbaijan-grand-prix-interactive-data-lap-charts-times-and-tyres/>
- The Field (guía de gráficos): <https://www.thefieldf1.com/charts>

- Pitwall: <https://pitwall.app/>
- Forix: <https://www.forix.com/>
- ChicaneF1: <https://www.chicanef1.com/> · [grandprix.com: https://www.grandprix.com/encyclopedia](https://www.grandprix.com/encyclopedia)
- Wikipedia: https://en.wikipedia.org/wiki/List_of_Formula_One_driver_records · https://en.wikipedia.org/wiki/List_of_Formula_One_race_records
- Apps: <https://github.com/JustAman62/undercut-f1> · <https://f1dash.net/> · <https://multiviewer.app/> · <https://www.f1livepulse.com/en/features/> · <https://app.formula1dashboard.com/>
- Proyectos: <https://github.com/hugoogb/f1-tracker> · <https://github.com/fermanpre/f1-stats> · <https://github.com/izmishi/f1-lap-chart> · <https://davidor.github.io/formula1-lap-charts/> · <https://f1-visualization.vercel.app/> · <https://github.com/PeterCollavino7/f1-analysis>
- Comunidades: <https://public.tableau.com/app/profile/sports.chord/viz/VisualHistoryofF1Tableau/VisualHistoryofFormula1Tableau> · <https://community.fabric.microsoft.com/t5/Data-Stories-Gallery/Formula-1-analysis-1950-2021/m-p/2052947> · <https://observablehq.com/@dis-2025-fall/f1-top-15-bump-chart> · <https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020> · <https://f1bythenumbers.com/r-formula1-or-r-flart/> · <https://x.com/f1visualized> · https://x.com/F1_charts
- Modelos de pilotos: <https://f1metrics.wordpress.com/2019/11/22/the-f1metrics-top-100/> · <https://f1mathematicalmodel.com/2026/08/17/2026-mid-season-f1-driver-ratings/>
- Ritmo: <https://www.f1technical.net/news/28301> · <https://laptime.me/laptime-me-f1-2026-fuel-adjusted-pace-analysis-miami/> · **Statista:** <https://www.statista.com/chart/18556/mercedes-f1-win-ratio>

CAPÍTULO 11

Hoja de ruta y preparación de la fase 7

11.1 Iteraciones propuestas

Iteración 1 · Endurecer (3–5 días)

Arreglos internos de datos (`has_sprint`, bandera roja en la vuelta correcta, tiempos por sectores, compuestos `NAN`, tipos, tabla huérfana). Licencia y atribuciones. Dependabot, acciones a SHA y fuera de Node 20, cabeceras de seguridad, timeout en `fetch`. Releases fechadas, copia del bronze estático y prueba de humo antes de publicar. Región `fra1` y caché de página en Vercel.

Iteración 2 · Corregir lo visible y probar (4–5 días)

Coches compartidos, abreviaturas únicas y errores de lectura de los gráficos. Unit tests de la fusión y la numeración de vueltas; tests de clave en todos los hechos.

Iteración 3 · Más datos (1–2 semanas)

FastF1 ampliado (sprint, meteorología, dirección de carrera, libres), Jolpica como segunda fuente, eventos históricos SC/VSC/rojas. Vueltas de sprint extremo a extremo.

Iteración 4 · Mejores visualizaciones (1–2 semanas)

Race trace, franjas de neutralización, matriz de resultados, récords ampliados, eliminación matemática, navegación de 5 pestañas. Después, página de circuito y comparador.

Iteración 5 · Arquitectura B (1–2 semanas)

Exportación estática, GitHub Pages y revalidación bajo demanda.

11.2 Qué necesita la fase 7 de los datos

La fase 7 prevé tres líneas de analítica avanzada. Cada una tiene requisitos de datos que conviene cubrir antes para no construir modelos sobre huecos:

Línea	Datos necesarios	Estado y acción
Degradación de neumáticos por compuesto y circuito	Vueltas limpias (sin SC, VSC, bandera roja, entrada ni salida de boxes), edad del neumático, compuesto real C1–C6, corrección por combustible, temperatura de pista	● Falta C1–C6 desde 2024 (Pirelli) y la meteorología (FastF1); la bandera roja mal situada contamina las vueltas limpias
Predicción del resultado con parrilla y ritmo	Parrilla, clasificación, ritmo de libres, histórico por circuito, fiabilidad	● Libres y clasificaciones antiguas están en F1DB sin exponer; hace falta categorizar <code>reason_retired</code>
Estilos de pilotaje con telemetría	Telemetría por vuelta de varios pilotos y sesiones	● Solo hay la vuelta de clasificación de 2024–2026; ampliar a 2018–2023 con FastF1

Antes de empezar la fase 7

Hacer las iteraciones 1 y 3 (datos) antes que la analítica. La degradación, que es el modelo más vistoso, depende de tres huecos que hoy están abiertos: compuesto real, meteorología y vueltas neutralizadas bien marcadas. Si la fase 7 incorpora dependencias pesadas (scikit-learn, notebooks), conviene separarla como paquete con `uv workspaces`.

11.3 Pendientes que siguen abiertos

- Casos de numeración aplazados: paradas de Japón y San Marino 1994 y vueltas rápidas de Bélgica 2001 ([docs/revision_divergencias/INFORME.md](#)).
- Publicar la propuesta de correcciones a F1DB ([docs/revision_divergencias/PROPUESTA_F1DB.md](#)).
- Documentación de dbt en GitHub Pages (`PUBLISH_DOCS=true`).
- Descargar los volcados de Jolpica y `f1resultsdatabase` solo con tu permiso explícito.

APÉNDICE A

Metodología del informe

El informe se elaboró la noche del 29 al 30 de septiembre de 2026 con nueve agentes de investigación en paralelo, cada uno con un encargo independiente y la instrucción de no modificar el código.

Nota	Encargo	Método
01	Inventario cuantificado de datos que faltan	Consultas SQL de solo lectura al DuckDB y a bronze
02	Fuentes externas y licencias	Búsqueda web y peticiones reales pequeñas a APIs públicas
03	Viabilidad de las vueltas de sprint	Carga real de los 29 sprints con FastF1 y comparación con OpenF1, Jolpica y F1DB
04	Arquitectura de datos	Lectura de <code>ingestion/</code> y <code>transform/</code> , manifiesto dbt y consultas
05	Arquitectura de aplicación, CI/CD y estructura	Lectura del código, <code>git ls-files</code> y medidas contra la API desplegada
06	Alternativas de arquitectura	Investigación web (unas 80 fuentes citadas)
07	Revisión de código e infraestructura	Lectura completa, linters, pruebas y medidas de rendimiento
08	Gráficos y récords externos	Revisión de 21 webs y proyectos y consultas de prueba
09	Evaluación de los gráficos propios	Navegación de la web desplegada, capturas y consultas

Los capítulos de detalle se generaron automáticamente desde las notas Markdown ([docs/informe_situacion/notas/](#)) con `md2tex.py`; las síntesis y los diagramas se redactaron a partir de ellas. Para regenerar el PDF:

```
cd docs/informe_situacion
for f in notas/*.md; do python md2tex.py "$f" "capitulos/$(basename "$f" .md).tex"
↪ --sin-titulo; done
latexmk -xelatex -outdir=build informe.tex
```

Restricciones cumplidas: no se accedió a [formula1db.com](#) ni se intentó sortear ninguna protección; no se descargaron volcados completos de terceros; los PDF de la FIA se usaron solo como evidencia; no se creó ninguna cuenta ni se usaron credenciales; y no se hizo ningún commit.